

—— // ZTMUX // ——

ztmux: tmux, Rewritten in Rust

The Superset – v3.7.21

THE SUPERSET

the server · the client · the grid · the command language

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTOPENSOURCE

Contents

ztmux — tmux, Rewritten in Rust	4
Copyright and Attribution	5
About This Edition	6
How to Read This Book	7
Before You Begin	8
Chapter 1 — The Pitch	9
Chapter 2 — How the Port Is Built	13
Chapter 3 — The Server	16
Chapter 4 — The Client	20
Chapter 5 — The Grid and Screen Model	24
Chapter 6 — The Input Parser	29
Chapter 7 — Layouts	34
Chapter 8 — The Command Language	38
Chapter 9 — Formats, Config, and Keys	43
Chapter 10 — The Terminal Back End	47
Chapter 11 — The Parity Suite: Correctness as Method	51
Chapter 12 — The Anti-Drift Gate	56
Chapter 13 — Beyond tmux: The Superset	60
Chapter 14 — The Road to “Done Right”: Shrinking <code>unsafe</code>	66
Chapter 15 — Status and the Honest Gaps	70
Chapter 16 — The Format Mini-Language in Depth	75
Chapter 17 — The Wire Protocol and <code>imsg</code> Framing	80
Chapter 18 — Copy Mode and the Grid Reader	83
Chapter 19 — Spawning Panes and Windows	85
Chapter 20 — The Status Line and Screen Redraw	88

Chapter 21 — Hooks, Alerts, and Notifications	90
Chapter 22 — Two Lifecycles: A Keystroke and a Pane	93
Chapter 23 — Performance and the Hot Paths	96
Chapter 24 — Colour: Names, Spaces, and the Nearest Match	99
Chapter 25 — UTF-8, Wide Characters, and Combining Sequences	101
Chapter 26 — Mouse Events and the Window Modes	103
Chapter 27 — Jobs, run-shell, and pipe-pane	105
Chapter 28 — Paste Buffers and the Environment	108
Chapter 29 — Design Decisions and Objections	111
Chapter 30 — Sessions, Groups, and the Window Index Space	114
Chapter 31 — Resize: From SIGWINCH to Reflow	117
Chapter 32 — Terminal Overrides and the Capability Correction Layer	120
Chapter 33 — The Options Table as a Data Structure	123
Chapter 34 — Overlays: Menus, Popups, and the Chooser Modes	126
Chapter 35 — Porting Idioms: A Catalogue of C-to-Rust Translations	129
Chapter 36 — Failure, Diagnosis, and the Logs	132
Chapter 37 — Bug-Compatibility: What the Port Deliberately Inherits	135
Chapter 38 — The Trust Boundary	138
Chapter 39 — Signals, and Why a Daemon Handles Them Twice	141
Chapter 40 — The Clock: Everything That Happens Without a Keystroke	144
Chapter 41 — Nesting: A Multiplexer Inside a Multiplexer	146
Chapter 42 — Control Mode as an API Surface	149
Chapter 43 — What the Server Forgets	152
Chapter 44 — Measuring the Port Without Lying About It	155
Chapter 45 — Exhaustion: Where the Program Draws Its Bounds	157
Chapter 46 — Decoding the Keyboard: the Other Parser	160
Chapter 47 — The Command Line: One Binary, Two Programs	164
Chapter 48 — Identity: Ids, Indices, and Names	168
Chapter 49 — The Platform Layer	171
Chapter 50 — Scripting the Server	174

Appendix A — The Command Reference	179
Appendix B — The Format Variable Reference	182
Appendix C — The Option Reference	185
Appendix D — The Parity Methodology and the Current Gap List	188
Appendix E — The Anti-Drift Gate and the Fake-Fn Categories	191
Appendix F — The Source Tree, Module by Module	193
Appendix G — Glossary	196
Appendix H — Building ztmux and the Vendored References	198
Appendix I — The Default Key Bindings	200
Appendix J — Format Recipes	202
Appendix K — The Audit Workflow	204
Appendix L — Coexisting with tmux and Operational Notes	206
Appendix M — A Reading Guide to the Source	208
Colophon	209
Reproducing the Book’s Claims	210
Also from MenkeTechnologies	211

ztmux – tmux, Rewritten in Rust

The Superset — the whole multiplexer, ported, verified, and taken past tmux.

server · client · grid · the input parser · layouts · the command language · formats · the terminal back end · the superset tree

MenkeTechnologies

Copyright and Attribution

ztmux — tmux, Rewritten in Rust: The Superset

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews and for the source excerpts identified below, which carry their own upstream licenses.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the tmux project, its maintainers, the Rust Foundation, or any other project named in these pages.

The software this book describes

ztmux is released under the MIT License, Copyright © 2026 MenkeTechnologies. The source is at github.com/MenkeTechnologies/ztmux.

The upstream this book is a port of

ztmux is a from-source port of **tmux**, and the tmux C sources are vendored in the ztmux repository under [vendor/tmux/](#) as a read-only, SHA-pinned copy. tmux is the work of its authors and contributors and is distributed under the ISC license. Its per-file notices read, in the form reproduced in the project's [COPYING](#):

Copyright (c) <author>

Permission to use, copy, modify, and distribute this software for any purpose with or without fee is hereby granted, provided that the above copyright notice and this permission notice appear in all copies.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF MIND, USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.

Short excerpts of tmux's C are quoted in this book for the purpose of commentary, criticism, and technical explanation — specifically, to show what a ported Rust function was validated against. Every such excerpt is cited to its file and function in [vendor/tmux/](#).

About This Edition

This edition corresponds to **ztmux v3.7.21**.

The version number tracks the tmux release ztmux targets for compatibility — upstream [tmux next-3.7](#) — not the book’s own revision and not a claim about ztmux’s maturity. Chapter 1 explains that convention; Chapter 15 states, without rounding up, what is finished and what is not.

Every figure in this book — parity counts, module counts, line counts — was measured against the source tree at this edition and is cited to the file that produces it, so a reader with the repository checked out can reproduce it rather than take it on faith. Software moves; the citations are what let you check whether a number still holds.

How to Read This Book

The book is written to be read front to back, and it is also written so that any chapter can be opened cold.

Chapters 1–2 state what ztmux is and how the port is built and kept honest. **Chapters 3–10** walk the program itself, in the order data moves through it: the server, the client, the grid and screen model, the input parser, layouts, the command language, formats and options, and the terminal back end. **Chapters 11–12** are the two instruments that make the port's claims checkable — the byte-for-byte parity suite and the anti-drift gate. **Chapter 13** is the superset: the extension tree that adds what tmux never had. **Chapters 14–15** are the remaining `unsafe` and an honest status report. **Chapters 16–50** go deep on one subsystem, decision, or lifecycle apiece. **Appendices A–M** are the reference matter: commands, format variables, options, the parity methodology, the module map, a glossary, build instructions, default bindings, and a reading guide to the source.

A reader who wants the argument rather than the machinery can read Chapters 1, 2, 11, 12, 13, and 15 and stop. A reader who wants to work on the port should read Chapter 2, then Appendix M, then the chapter covering whatever subsystem they are about to touch.

Code excerpts, file paths, and command examples are set in a monospaced face. Paths are relative to the ztmux repository root unless stated otherwise.

Before You Begin

This is the book about ztmux — the one you read in a chair, front to back, to understand not only *what* ztmux does but *why* it is shaped the way it is. It is not a man page. The tmux man page enumerates every command, option, and format, and you reach for it when you need one fact fast. This book is the other thing: the continuous account of the program — from the socket a client opens to the cells a pane paints — and of the architecture ztmux inherits, faithfully, from tmux itself.

ztmux is a from-source Rust port of [tmux](#). It is not a wrapper around the [tmux](#) binary and not a control-mode client — it is the tmux program itself, reimplemented in Rust: the server, the client, the grid/screen model, the input parser, the layout engine, the command language, the format system, and the terminal back end. It is the multiplexer, rebuilt — and it is a *superset*: a faithful tmux core with a second tree bolted alongside it that adds what tmux never had, without breaking the muscle memory of anyone who already lives in tmux. That second tree is not a plan. It is 119 modules and roughly twenty-nine thousand lines under [src/extensions/](#), walled off from the port's own correctness gate precisely because it is honestly *not* tmux. Chapter 13 is its chapter. The repository states the first half on its own front page: [\[TMUX, REWRITTEN IN RUST – DONE RIGHT\]](#), with the two-line creed that the rest of this book unpacks — “*Not a wrapper. Not control mode. The multiplexer itself,*” and “*Ported against the C, verified against the C — byte for byte.*”

This book is honest about that foundation, because the honesty is the point. ztmux stands on one reference, vendored as a plain committed copy so the repo is self-contained: [vendor/tmux/](#), the upstream **C sources**, pinned to a specific commit and never edited. Those sources are the source of truth every ported module is validated against — not a memory of how tmux behaves, not the man page, but the C itself, sitting in the same tree as the Rust that claims to reproduce it. [vendor/VENDOR.md](#) records the pinned commit and the license, and the tree is read-only by convention: nothing is developed in it, and the living port is the crate at the repository root.

The work this book describes is the port itself: reading a C module, writing the Rust that reproduces it, and proving the reproduction with the two instruments of Chapters 11 and 12 — a suite that diffs ztmux's output against the real tmux byte for byte, and a gate that fails the build when a function appears in [src/](#) with no counterpart upstream. On top of that floor sits the superset tree of Chapter 13. This is still-evolving software: the crate version is [v3.7.21](#) at the time of writing — a number that tracks the tmux release ztmux targets for compatibility ([tmux next-3.7](#)), not a claim about its own maturity — and where a subsystem is partial or a feature is still aspirational this book marks it rather than rounding up. ztmux is the work of MenkeTechnologies, built on the foundation of the tmux authors, and this book is grounded in that code.

Let's start.

Chapter 1 – The Pitch

A terminal multiplexer is, on paper, a solved problem: `tmux` has multiplexed sessions for two decades, and it is excellent. This chapter states what `ztmux` is for — not “`tmux` but faster,” but *tmux owned in Rust*: a memory-safe reimplementaion of the whole program that can be extended past the C original while staying bit-for-bit familiar to its users.

What `ztmux` is — the whole program, not a wrapper or control-mode client

There is a lazy way to build “a `tmux` in language X,” and `ztmux` is not it. The lazy way is to shell out to the real `tmux` binary and drive it — parse its output, send it commands, wrap its control mode (`tmux -CC`) — and call the wrapper a new multiplexer. That produces a client, not a multiplexer; it inherits every bug and every limit of the program it wraps, and it can never be a superset of `tmux` because it *is* `tmux` with a coat of paint. `ztmux` takes the hard road instead. The `ztmux` binary is a complete `tmux`: it contains the server that owns all session state, the client that attaches to it, the grid and screen model that every pane is drawn on, the VT input parser that turns a program’s bytes into cells, the layout engine that tiles panes, the command language that `bind-key` and `split-window` are written in, the `#{...}` format system, and the terminfo-driven terminal back end. Nothing is delegated to an external `tmux`; there is no external `tmux` in the picture at all.

The evidence for that claim is structural, not rhetorical. The source tree has a `src/ported/server.rs` that binds a Unix socket and runs an event loop; a `src/ported/client.rs` that connects to that socket and performs the attach handshake; a `src/ported/grid.rs` and `src/ported/screen.rs` that maintain the cell matrices; a `src/ported/input.rs` that implements Paul Williams’ DEC ANSI parser state machine; a `src/ported/layout.rs` with the pane-tiling tree; sixty-odd `src/ported/cmd_*.rs` files, one per `tmux` command; a `src/ported/format.rs` that is two hundred kilobytes of format-expansion logic; and a `src/ported/tty.rs` that drives the outside terminal through terminfo. Each of those is a transliteration of the corresponding `tmux` C module, function by function, name for name. A wrapper would have none of them; it would have a subprocess handle and a protocol parser. `ztmux` has the whole machine.

That distinction has a practical consequence you can see from the first command. Because `ztmux` is a server in its own right, it opens its *own* socket namespace — `ztmux-<uid>` rather than `tmux-<uid>` — so it does not collide with a running `tmux` on the same machine (`src/ported/tmux.rs`, `make_label`). You can run `tmux` and `ztmux` side by side, each with its own server, its own sessions, its own detach-survival, and neither knows the other exists. That is only possible because `ztmux` is not a client of `tmux`; it is a peer implementation of the same program.

There is a sibling project, `ztmux-core`, that *is* a native `tmux` client engine — a library that speaks the `tmux` wire protocol to talk to a running `tmux` server — and it is deliberately a different thing from this repository. Keeping the two separate is part of the honesty: `ztmux-core` is the client-of-`tmux` story, and `ztmux` (this book’s subject) is the whole-server-and-client story. When the README says “it is `tmux`, reimplemented,” it means the second thing, and the rest of this book is the detailed account of that reimplementaion.

Coexistence in practice: `ztmux` next to `tmux`

The claim that `ztmux` is a peer implementation rather than a client of `tmux` has a concrete, testable consequence you can see in one session. Because `ztmux` binds its own socket namespace (`ztmux-<uid>`, Chapter 3 and

Appendix L) rather than tmux's (`tmux-<uid>`), the two servers never touch. Start a tmux session and a ztmux session from the same shell and they are entirely independent: `tmux ls` lists the tmux server's sessions, `ztmux ls` lists ztmux's, each has its own detach-survival, its own options tree, its own key bindings, its own set of panes and running programs. Detach from one and the other is unaffected; kill one server and the other keeps running. There is no shared state because there is no shared server — ztmux is not talking to tmux at all. A wrapper could not do this: a wrapper *is* tmux with a coat of paint, so it would share tmux's socket, tmux's sessions, and tmux's bugs. The clean coexistence is the structural proof that ztmux contains its own server, and it is the first thing a skeptical user can check in ten seconds.

Port, not rewrite-from-spec: standing on the C source of truth

There are two ways to reimplement a program, and they produce very different artifacts. One is to read the documentation — the man pages, the protocol specs, the escape-sequence tables — and write a fresh implementation of what they describe. The other is to port: to take the original source, function by function, and translate it into the new language, keeping the same names, the same control flow, the same data structures, and the same defaults, then verify the translation against the original. ztmux is emphatically the second kind. It does not re-derive tmux's behavior from a specification of tmux; it translates tmux's C into Rust and checks the translation against the C.

This is not a stylistic preference; it is the only way to be *faithful* in the strong sense the project demands. tmux's real behavior is not fully captured by any document — it lives in the exact order of operations in `server_loop`, in the exact rounding of the even-horizontal layout, in the exact bytes `format_unescape` writes for a `#[l:...]` literal. A spec-driven rewrite would get the documented behavior right and quietly diverge on all the undocumented behavior that real scripts and real terminals depend on. A port cannot diverge silently, because every function carries its provenance: a Rust function in ztmux keeps its C name, and directly above it sits a doc comment citing the exact C file, line, and full signature it was ported from. Open `src/ported/grid.rs` and you find, verbatim:

```
/// C `vendor/tmux/grid.c:374`: `struct grid *grid_create(u_int sx, u_int sy, u_int
    hlimit)`
pub unsafe fn grid_create(sx: u32, sy: u32, hlimit: u32) -> *mut grid {
```

There are more than two thousand such citations across the tree — a line-level map back into upstream tmux, one per ported function. They are not decoration. They are what makes the port auditable: a reviewer can jump from any Rust function to the exact C lines it claims to implement and diff the two by eye, and an automated gate (Chapter 12) fails the build if a Rust function's name has no C counterpart at all. “Standing on the C source of truth” is therefore a literal description of the method: the truth is `vendor/tmux/`, checked into the repository, and every module is diffed against it rather than trusted.

Keeping the C in the tree is what makes that method durable rather than a habit. A port validated against a copy of tmux the author happened to have installed is validated against a moving target: distributions ship different releases, and behavior that is correct against one is a bug against another. Pinning the C to a commit inside the repository means the question “is this function faithful?” always has one answer, and that answer does not change when someone upgrades their system tmux. This is the same discipline the whole MenkeTechnologies stack uses when it builds on a foundation: name the foundation, pin it, keep it legible, and make the correspondence inspectable rather than asserted.

The superset: a faithful core, and the tree that grew on it

The word *superset* is a promise with two halves, and both halves have to hold or the word is dishonest. The first half is that ztmux must be a faithful tmux — every command, every format, every default behaving byte-for-byte

as tmux does — so that someone who has lived in tmux for a decade feels nothing change when they switch. The second half is that, being memory-safe Rust that MenkeTechnologies owns rather than frozen C it merely wraps, ztmux can grow features the C tmux does not have, without forking away from the familiar core. A superset is a strict extension: it does everything the original does, exactly, *and* more.

The reason the first half is non-negotiable is that a superset that breaks parity is not a superset — it is a fork with extra features, and forks lose. The value of being tmux-compatible is muscle memory: the config files, the keybindings, the scripts, the status-line format strings that people have accumulated over years all have to keep working unchanged. The moment `#{pane_current_command}` reports the wrong thing or an even-horizontal split rounds the wrong way, the compatibility promise is broken and the user is on their own. This is why the project spends so much of its energy on a parity suite (Chapter 11) that diffs ztmux against the real tmux byte-for-byte: the superset can only add things once the base is provably identical.

The reason the second half is possible at all is ownership. tmux is superb and it is also a mature C program with a stable feature surface; adding a deep new capability to it means convincing its maintainers and writing it in C against decades of existing code. ztmux is young, it is Rust, and it is the property of the same author who builds the rest of the stack. That means the natural extensions are not speculative — they are the integrations the surrounding MenkeTechnologies tools already want: tighter coupling with the `zterminal` emulator that already speaks a tmux wire protocol, with `zshrs` running inside the panes, with the shared GUI component library that the stack's other apps are built from. Chapter 13 is the honest ledger of which of those are shipped and which are roadmap.

The discipline is: parity first, superset second, and never confuse the two. Everything in Chapters 3 through 10 is the faithful core, documented as the port of a specific C module. Chapter 13 is the superset — the 119 modules that are shipped code, and, marked plainly as such, the stack integrations that are still roadmap. Keeping that line sharp is what lets the book claim faithfulness without overclaiming novelty, and it is the same line every book in this series draws between the foundation it inherits and the work it adds.

The creed, line by line

The repository front page carries a two-line creed, and because the rest of this book is an expansion of it, it is worth reading each clause deliberately. The first line — *“Not a wrapper. Not control mode. The multiplexer itself.”* — is three negations narrowing to a claim. “Not a wrapper” rejects the shell-out approach: ztmux does not drive an external `tmux` binary, parse its output, or send it commands; there is no external tmux in the picture (Chapter 1’s structural argument). “Not control mode” rejects the subtler version of the same shortcut: it is not a `tmux -CC` client either, not a program that speaks tmux’s control protocol to a real tmux server — that is `ztmux-core`’s job, deliberately kept a separate thing. “The multiplexer itself” is the positive claim the negations clear room for: ztmux contains the server, the client, the grid, the parser, the layout engine, the command language, the format system, and the terminal back end — the whole machine, not a view of someone else’s.

The second line — *“Ported against the C, verified against the C — byte for byte.”* — names the method twice, because the method has two halves. “Ported against the C” is the construction: every module is a transliteration of a specific tmux C module, function for function, name for name, with a citation to its origin (Chapter 2). “Verified against the C” is the proof: the parity suite runs the same inputs through the vendored C tmux and ztmux and compares (Chapter 11). “Byte for byte” is the *bar* of that proof — not “behaves reasonably,” not “matches the man page,” but produces the identical bytes, which is the only bar strict enough to guarantee the muscle-memory compatibility the superset rests on (Chapter 24). Read together, the two lines are the whole thesis: ztmux is the multiplexer itself (not a wrapper or a client), built by porting the C and proved by diffing against it to the byte. Everything in the chapters that follow is that thesis worked out in detail.

What v3.7.21 means — the version tracks the tmux target, not ztmux’s own maturity

ztmux’s crate version is `v3.7.21`, and that number means something specific you should hold onto: it tracks the *tmux release ztmux targets* for compatibility — the parity suite measures against `tmux next-3.7` — not ztmux’s own maturity on some independent scale. Read carelessly it would imply a decade-seasoned program; read correctly it says only that this ports the 3.7 line of tmux. Where the port actually stands is not in the version string at all — it is in the parity ledger and the staged roadmap below. The port is no sketch, and the parity suite is why: it first reached 122 of 122 cases byte-for-byte identical to the vendored tmux on both Linux and macOS, and then, having exhausted that first set, was *expanded* to 1115 cases spanning far more of the surface. That larger, adversarial target initially surfaced about twenty specific port gaps — which the port then closed, one by one, until the suite stood at **1115 of 1115 passing (100%), byte-for-byte identical to the vendored tmux on both Linux and macOS** (Chapter 11 and Appendix D walk the whole method). The README calls the result a daily driver. But none of that makes the surface uniform: some subsystems are complete and battle-tested, others are transliterated but lightly exercised, and a number of the configuration options that appear in the options table exist only so that parsing a real `.tmux.conf` does not error — they are accepted and stored but not yet wired to behavior. The Cargo manifest says as much in a comment: most of the feature flags exist “to make matching upstream’s configuration options easier,” with the notable exception that sixel graphics are actually implemented.

Reading against early software means trusting the parity claim exactly as far as it is measured and no further. When this book says a behavior is faithful, it means there is a parity case, or a direct line-for-line port with a C citation, behind the claim. When a subsystem is present but not yet proven, the book says “ported” rather than “verified,” and Chapter 15 keeps the running ledger of which is which. The version-string discipline extends into the artifacts themselves: the parity summary JSON records the exact reference it measured against (`tmux next-3.7`) and the count it achieved, so the claim is never a vibe — it is a number with a provenance.

The upside of early software is that the shape is still legible. A mature program accretes special cases until its architecture is only visible to people who have lived in it for years; a young port, especially one that keeps its C citations inline, wears its structure on the surface. That is the opportunity this book takes: because ztmux is still close to the C it came from and still small enough to hold in your head, the chapters that follow can walk the *actual mechanism* — the real function that binds the socket, the real state that survives a detach, the real loop that turns bytes into cells — rather than an idealized picture of it. The early-software caveat is real; it is also what makes the machine worth reading front to back.

Chapter 2 — How the Port Is Built

Before any subsystem, the method. ztmux stands on one vendored reference and a discipline: the upstream C is committed into the repository, and every ported module is checked against its counterpart in it. This chapter documents that machinery — it is what makes “faithful” a claim you can verify rather than trust.

The reference: `vendor/tmux/`, the C source of truth

Under `vendor/` sits a complete copy of the upstream tmux C source, committed as a plain read-only tree rather than a git submodule, SHA-pinned, and documented in `vendor/VENDOR.md`. It is the *source of truth*: the code every ported Rust module is diffed against, the code the parity suite builds and runs as its reference, and the code the anti-drift gate indexes to decide whether a Rust function name is real. The vendored tree is the `next-3.7` development line of tmux: its own `configure.ac` declares `AC_INIT([tmux], next-3.7)` and its `CHANGES` file opens with “CHANGES FROM 3.7 to 3.7a,” and the parity summary names the reference `tmux next-3.7`. Pinning to a specific upstream commit rather than “whatever tmux the system has” is essential, because tmux’s behavior drifts between versions and a parity suite measured against the wrong version would report false failures — a point Chapter 11 returns to.

`VENDOR.md` carries the pin metadata: the upstream URL, the commit SHA, the ISC license, the date the tree was vendored, and the refresh procedure — export a fresh `git archive` with no `.git`, rsync it over the vendored path, record the new `git rev-parse HEAD`. The file also states the rule that keeps the reference trustworthy: *do not develop in it*. The vendored tree is never patched to make a port pass. If ztmux and the C disagree, the C wins and the Rust changes.

Keeping the C in-repo, rather than as a submodule or a download step, is a deliberate self-containment choice. A clone of ztmux carries everything needed to build the port, build the reference, run the parity suite, and audit every function against its C origin, with no network access and no version ambiguity. The cost is repository size; the benefit is that “faithful to tmux” means faithful to a *specific, committed* tmux that ships with the code, not to whatever tmux a contributor happens to have installed.

The instruments that keep the port from inflating

The honesty problem for any port is *inflation* — presenting a thin translation as substantial work, or presenting invented code as ported code. ztmux answers it with two instruments that are mechanical rather than rhetorical, and both are cheap enough to run on every build.

The first is the per-function citation. A Rust function that claims to be a port carries, directly above it, the C file, line, and full signature it was ported from; there are 2,035 such citations across the tree. That is a line-level map back into upstream tmux, and it makes fidelity checkable one function at a time: jump to the cited C lines and read the two side by side.

The second is the anti-drift gate of Chapter 12 — a test that indexes every function name in `vendor/tmux/` and fails the build when a free function appears in the ported tree with no counterpart there. It is what stops the citation discipline from being merely aspirational. A port can always be padded with Rust-only “helpers” that exist in no upstream; this gate turns that padding into a red build. Original code is not forbidden — it is *relocated*, to the `src/extensions/` tree of Chapter 13, which the gate deliberately exempts because that code is honestly not tmux.

Between them the two answer different questions. The citations answer “where did this function come from?” — provenance, per function. The gate answers “is anything in the ported tree not from tmux at all?” — integrity, across the tree. Neither requires taking anything on faith, and the port report of the next section turns both into a number.

Validating a module against its C counterpart

Porting a single module is a repeatable procedure, and ztmux follows it the same way every time. Open the module’s C counterpart in `vendor/tmux/` and read it whole before writing anything. Translate function by function, keeping the C name unchanged — `grid_create` stays `grid_create`, `input_csi_dispatch` stays `input_csi_dispatch` — because the shared name is what lets everything downstream (citations, the anti-drift gate, the reviewer’s eye) line the Rust up against the C. Preserve the control flow, not just the behavior: where the C uses a `for` loop that increments on `continue`, the Rust loop must advance on every path too, and where it does not, subtle bugs follow (Chapter 11’s `format_unescape` crash is exactly this mistake). Then annotate each ported function with its C file, line, and full signature, and validate the result against the reference.

Validation happens at three levels, coarse to fine. The coarsest is compilation against the anti-drift gate: does every free function in the module correspond to a real function in the C source, or is it an invented helper inflating the apparent completeness? The middle level is the C-signature citation, generated by a script that records not just the C function’s name but its *full parameter list with names*, so a reviewer can diff the Rust parameters against the C parameters and catch a renamed, reordered, dropped, or invented argument. The finest level is the parity suite: feed the same input to the vendored tmux and to ztmux and compare the output byte-for-byte, so that any behavioral divergence — not just a structural one — shows up as a failing case.

The three levels catch different classes of error, and that is the point of having all three. The anti-drift gate catches *structural* dishonesty: functions that exist in Rust but not in tmux, which would otherwise let a port look more complete than it is. The signature citations catch *interface* drift: a function with the right name but the wrong arguments. The parity suite catches *behavioral* drift: a function with the right name and the right signature that nonetheless produces different bytes. A port that passes all three is faithful in the structural, the interface, and the behavioral sense simultaneously, and each gate is cheap enough to run in CI on every push.

What this procedure explicitly refuses is the shortcut of “close enough.” A port that renames functions for Rust idiom, or invents helpers to make a module read more naturally, or approximates a rounding rule because the difference “won’t matter,” breaks one of the three gates and fails the build. The friction is deliberate: it is much easier to keep a port faithful by refusing every small divergence than to let divergences accumulate and try to reconcile them later. The idiomatic-Rust cleanup — the part where the port *stops* looking like transliterated C — is a later, separate phase, and Chapter 14 explains why it deliberately comes after parity rather than before.

The port report and the doc-count / man-page gates

Beyond the per-function gates, ztmux tracks its progress in aggregate through a generated port report. A script (`scripts/gen_port_report.py`) walks the Rust source and the vendored C, matches functions across the two, and produces a report — published as `port_report.html` on the project’s documentation site — that says, in numbers, how much of tmux has been ported and how much remains. This is the “how complete is it?” question answered mechanically rather than by assertion, and it is the natural companion to the anti-drift gate: the gate rules out functions that came from nowhere, the report shows how much of the C surface is covered at all.

The report is generated from the same machinery that produces the C-signature citations, so the two stay consistent. The annotation script, `scripts/annotate_c_links.py`, walks the C sources to build a `name → (file, line, signature)` index; the port report reuses that index to compute coverage. Because both are derived from the vendored C rather than from a hand-maintained list, neither can drift out of sync with reality — refresh the vendored tmux, re-run the scripts, and both the citations and the report update to match.

Alongside the coverage report sit the *documentation* gates that the wider stack imposes on every project: a doc-count check and a man-page check that keep the project's own documentation from falling behind its code. These are the same gates the sibling projects use, and they exist because documentation that is not gated rots. A doc-count gate ensures the ratio of documented to undocumented public surface does not regress; a man-page gate ensures the reference material stays present and buildable. They do not measure correctness — that is the parity suite's job — but they measure whether the project is still legible, which for a port that sells itself on legibility is its own kind of correctness.

Taken together, the machinery of Chapter 2 forms a closed loop. The vendored C makes the reference fixed and inspectable; the C citations make each function's provenance legible; the anti-drift gate makes structural honesty enforceable; the parity suite makes behavioral fidelity measurable; and the port report and doc gates make the aggregate progress and the documentation legible. None of it depends on trusting the author. That is the whole design: a port whose faithfulness is a property you can check, not a claim you have to believe. With the method established, the rest of the book can walk the machine it produces — starting where tmux itself starts, with the server.

A guided tour of one ported function

To make “port, not rewrite” concrete, take a single function from `src/ported/grid.rs` and read it against its origin. The Rust is:

```
/// C `vendor/tmux/grid.c:374`: `struct grid *grid_create(u_int sx, u_int sy, u_int
    hlimit)`
pub unsafe fn grid_create(sx: u32, sy: u32, hlimit: u32) -> *mut grid {
```

Four things about this one line encode the whole method. The **name** is `grid_create`, identical to the C — not `create_grid`, not `Grid::new`, because the shared name is what lets the anti-drift gate match it to the C index, what lets the citation script find its origin, and what lets a reviewer line the two up. The **signature** takes `sx`, `sy`, `hlimit` in that order with those names, mirroring the C's `u_int sx`, `u_int sy`, `u_int hlimit`, so the citation's full-signature form lets a reviewer catch a reordered or renamed argument. The **citation** above it names the exact file and line (`grid.c:374`) and the full C signature, so the jump from Rust to C is mechanical. And the function is `pub unsafe fn` returning `*mut grid` — a raw pointer to a `#[repr(C)]` struct — because the port reproduces tmux's manual-memory model exactly (Chapter 14), which is what makes the behavior checkable against the C in the first place.

Read the body and the fidelity continues: it allocates the `grid` with the port's `xmalloc`, sets `sx/sy`, sets the `GRID_HISTORY` flag only when `hlimit` is non-zero (the “scrollback of zero keeps no history” rule of Chapter 5), allocates `sy` lines, and returns the pointer — the same order of operations as the C, so any divergence would be a divergence you could see by diffing the two side by side. This is what every one of the two-thousand-plus ported functions looks like: same name, same signature, same control flow, a citation to its origin, and a raw-pointer C-shaped body that the parity suite pins and a later idiomatic pass (Chapter 14) will eventually soften. One function shows the whole discipline in miniature.

Chapter 3 – The Server

tmux is a client/server program: a long-lived server owns all state, and clients attach to it. This chapter walks the ztmux server — its event loop, the socket it binds, the session/window/pane tree it owns, and how it survives every client detaching.

The client/server split and why it exists

The defining architectural decision of tmux — the one that makes it a multiplexer rather than a terminal — is that the state lives in a server, not in the client you type into. When you run `ztmux new-session`, the process you launched does not *become* your session; it connects to a server (starting one if none exists) and attaches to a session the server owns. Your terminal window is a thin client; the sessions, windows, panes, their scrollback, their running programs, and their layouts all live in the server process, which has no controlling terminal of its own and outlives every client that comes and goes. ztmux inherits this split exactly, because the split *is* the feature: it is what lets you detach from a session, close your laptop, and re-attach from another machine to find everything still running.

The split is visible in the source as two entry points that share a binary. `src/main.rs` sets up a custom global allocator (a thin shim over libc `malloc/free/calloc/realloc`, so allocation behaves exactly as the C's does) and calls `tmux_main`, which decides — based on the command and the environment — whether this invocation should act as a client connecting to a server, or should *be* the server. `src/ported/server.rs` is the server half; `src/ported/client.rs` is the client half (Chapter 4). They communicate over a Unix domain socket using an imsg-based protocol with a version number (`PROTOCOL_VERSION = 8`, in `src/ported/tmux_protocol.h.rs`) that both sides check, so a client and a server built from mismatched code refuse to talk rather than corrupt each other's state.

Why a separate server at all, rather than a library the client links? Because the server must survive the client. A library in the client's process dies when the client's terminal closes; a separate process with no controlling terminal does not. The server is deliberately daemonized — forked away from the launching terminal, its signals cleared and re-established — so that the death of a client (a closed window, a dropped SSH connection, a `kill` on the client process) is just a detach, not a loss. Everything the server owns keeps running in the forked daemon until the server itself is told to exit.

This is also why the server is the natural home for *all* shared behavior: key bindings, options, paste buffers, hooks, and the marked pane are single-copy state that any attached client can read and modify. When two clients attach to the same session, they are two windows onto one server-owned world; a `set-option` from one is immediately visible to the other because there is only one options tree, and it lives in the server. Chapter 4's multi-client story and Chapter 9's options story both rest on this: the client is a view, the server is the truth.

The event loop and the server socket

The server is, at bottom, one event loop. tmux is built on `libevent`, the C event-notification library, and for most of ztmux's life the port linked that library and called its C API through FFI. It no longer does: the loop is ztmux's own, written in Rust under `src/extensions/event_loop/`, and the crate links no C library beyond libc.

What did *not* change is the shape, because the shape is what the ported code is written against. There is still one implicit global base created by `event_init`. Events are still `struct event` values owned by the

caller and embedded by value in the structs that need them — several of them module-level `static mut` in `server.rs` (`SERVER_EV_ACCEPT` for incoming connections, `SERVER_EV_TIDY` for the hourly housekeeping timer) — and they are still registered by handing `event_set/event_add` an `unsafe extern "C-unwind" fn` pointer, exactly as C would. `evbuffer` byte queues and classic `bufferevents` are there too, with the same semantics the ported code depends on. Not one call site in `src/ported/` changed when the C library came out of the build; the names, signatures, and event values are the ones tmux uses, so the code still reads as the transliteration it is.

Underneath, the loop does what tmux asks libevent to do and no more. Readiness comes from `select` on macOS and `poll` everywhere else — precisely the backends tmux forces libevent onto by setting `EVENT_NOKQUEUE/EVENT_NOPOLL` in `osdep-darwin.c` and `EVENT_NOEPOLL` in `osdep-linux.c`, because `kqueue` and `epoll` do not work on the things a multiplexer watches: `ttys`, `pane ptys`, `/dev/null`. The descriptor sets are sized to the highest descriptor in use rather than capped at `FD_SETSIZE`, the way libevent's own select backend does it, so a server with many panes, jobs, and clients does not walk off the end of a fixed `fd_set`. Timers are absolute deadlines the loop sorts through to pick how long to sleep. Signals are delivered by the mechanism Chapter 39 describes. Everything libevent grew for its other users — rate limiting, OpenSSL `bufferevents`, DNS, HTTP, threading, multiple bases — is simply absent, because nothing in tmux asks for it.

The loop is entered through `proc_loop(SERVER_PROC, Some(server_loop))`. `proc_loop` runs the loop's dispatch, and `server_loop` (in `server.rs`) is the body called on each iteration: it stamps `CURRENT_TIME`, drains the command queue for the global queue and for every identified client until no work remains (Chapter 8's command queue is pumped here), runs `server_client_loop` to service clients, and then decides whether the server should exit — checking the `exit-empty` and `exit-unattached` options, the count of attached clients, whether any command is still waiting, and whether any background job is still running. Returning 1 from `server_loop` tells `proc_loop` to stop, at which point the server kills its jobs, saves the prompt history, and exits.

The socket the loop accepts on is created by `server_create_socket`. It fills an `AF_UNIX sockaddr_un`, copies the socket path into `sun_path` (guarding against `ENAMETOOLONG` for over-long paths), unlinks any stale socket at that path, creates the socket, sets a umask appropriate to whether this is the default socket, binds, and then calls `listen(fd, 128)` — a backlog of 128 pending connections — before switching the descriptor to non-blocking. The path itself comes from `make_label` in `src/ported/tmux.rs`, which builds `"{base}/ztmux-{uid}"`, creates that directory with `S_IRWXU` and the same owner-and-permission safety checks tmux performs (the directory must be owned by the caller and carry no world-accessible bits), and places the per-label socket inside it. With the default label `"default"`, the socket is `<dir>/ztmux-<uid>/default` — the `ztmux-` prefix being the one place the path deliberately differs from tmux, so the two servers never share a socket.

Incoming connections are handled by `server_accept`, the callback armed on `SERVER_EV_ACCEPT`. It re-arms itself, accepts the new descriptor, and — importantly — backs off gracefully under file-descriptor pressure: on `ENFILE` or `EMFILE` it disables accept for one second (via `server_add_accept` with a timeout) rather than spinning, and on the transient `EAGAIN/EINTR/ECONNABORTED` it simply returns. A successful accept creates a client for the new fd (`server_client_create`) and runs the access-control join (`server_acl_join`). This is the faithful tmux behavior: a server under fd exhaustion degrades instead of failing, and the loop keeps running.

The state tree: sessions, windows, panes

Everything the server owns hangs off a small number of global containers, declared as module-level statics in `server.rs` and initialized when the server starts. The connected clients live in a `CLIENTS` tail queue; sessions in a `SESSIONS` red-black tree; windows in a `WINDOWS` tree and, separately, all window panes in an `ALL_WINDOW_PANES` tree so any pane can be found by id. These are the same intrusive BSD data structures tmux uses — `TAILQ` for ordered lists, `RB` (red-black) trees for keyed lookup — ported as Rust functions (`tailq_foreach`, `rb_insert`, and the rest) that operate on the same `#[repr(C)]` node structs. The choice to keep the C data structures rather than reach for Rust's `Vec` and `BTreeMap` is faithfulness again: the iteration order,

the insertion semantics, and the pointer identity all have to match tmux, and the surest way to match them is to run the same algorithms.

The conceptual hierarchy is the familiar tmux one. A *session* is a collection of windows with one current window; a *window* is a collection of panes tiled by a layout with one active pane; a *pane* is a single pseudo-terminal running a program, backed by a screen, which is backed by a grid of cells. A client attaches to a session and sees its current window; switching windows, splitting panes, and resizing all mutate this tree, and because the tree lives in the server, the mutations persist across detach. Chapter 5 descends into the grid at the bottom of this hierarchy; Chapter 7 covers the layout that tiles the panes; here the point is that the server *owns* the whole tree and every client is a view onto it.

Two pieces of cross-cutting state deserve names because they recur. The *marked pane* — tmux’s `select-pane -m` target, used by commands like `join-pane` and `swap-pane` as an implicit source — is a single server-global `cmd_find_state` (`MARKED_PANE`), with a small API (`server_set_marked`, `server_clear_marked`, `server_is_marked`, `server_check_marked`) that ztmux ports directly. The *message log* — the server’s recent status messages, surfaced by `show-messages` — is a `MESSAGE_LOG` list trimmed to the `message-limit` option, appended to through a `server_add_message!` macro. Both are examples of the general rule: shared, single-copy state that only makes sense in one place, and that place is the server.

The pane at the leaf of the tree is where the server meets the outside world. Each pane owns a pseudo-terminal master fd, and the server watches those fds through the event loop: when a program running in a pane writes bytes, an event fires, the bytes are read and fed to the input parser (Chapter 6), which mutates the pane’s screen, which marks the affected cells dirty, which eventually produces output to every client viewing that pane. The child processes themselves are reaped by the server’s `SIGCHLD` handling (`server_child_signal` → `server_child_exited`/`server_child_stopped`), which marks a pane exited or continues a stopped one. The server is thus simultaneously the owner of the abstract session tree and the concrete manager of every child process and every fd — a single loop multiplexing all of it.

Finding anything by walking the trees

The global containers are not just storage; they are the answer to every “which object does this name mean?” question the server fields, and a couple of concrete lookups show how. When a command resolves a target like `work:2.1` (Chapter 8), `cmd_find` walks `SESSIONS` — the red-black tree keyed so a session can be found by name — to locate `work`, then walks that session’s window list to index 2, then that window’s pane list to pane 1. When a `SIGCHLD` arrives and the server must find *which pane* a dead child belonged to, it has the pid and walks `ALL_WINDOW_PANES` — the separate tree of every pane across every window — to find the pane whose process matches, then marks it dead. When a format expands `#{window_panes}`, it walks the current window’s pane list and counts. Every one of these is a walk over the same intrusive BSD structures tmux uses (`rb_find`, `tailq_foreach`), ported as functions over the same `#[repr(C)]` nodes, because the *order* these walks visit objects in — the iteration order of a red-black tree, the insertion semantics of a tail queue — is observable through formats like `list-windows -F` and must match tmux exactly. Reaching for Rust’s `BTreeMap` would have been more idiomatic and would have iterated in a subtly different order; the port keeps the algorithms so the observable order is tmux’s.

Server lifecycle: start, detach-survival, kill

The server comes to life in `server_start`, and the sequence is careful because it is forking a daemon. First it blocks all signals (`sigfillset` + `sigprocmask(SIG_BLOCK)`) so the fork happens in a known signal state, then calls `proc_fork_and_daemon` — the parent (the client that triggered the autostart) returns early with a descriptor, while the child continues as the server. The child installs a Rust panic hook that, on any panic, dumps a backtrace to a `server-panic-<pid>.txt` file — a small original touch that turns a Rust panic in the daemon into a diagnosable artifact rather than a silent death. It clears the inherited signal handlers, re-initializes the event loop in the new process (`event_reinit`, fatal if it fails, because a base carried across

a fork holds state that belongs to the parent), starts its own `tmuxproc`, and re-establishes signal handling with `server_signal` as the handler.

With the process set up, the server initializes the world: the RB and TAILQ trees for windows, panes, clients, sessions, and the message log; the key bindings (`key_bindings_init`); the input key tables (`input_key_build`). It creates the listening socket (optionally under systemd socket activation), creates the first client for the connecting descriptor, arms the hourly tidy timer (an `evtimer` set to 3600 seconds, whose callback `server_tidy_event` expires stale format jobs and, off macOS, calls `malloc_trim(0)` to return freed memory to the OS), initializes the access-control list, arms the accept event, and finally enters `proc_loop`. From that point the server is live: it accepts clients, services them, and owns state.

Detach-survival is not a feature the server implements so much as a consequence of the architecture, and the code makes the consequence explicit. When a client detaches, the server receives the detach and tears down that client — but the session the client was viewing is untouched, because the session belongs to the server, not the client. The decision to actually `exit` is made in `server_loop`, deliberately and conservatively: the server exits only when its policy options say it should (`exit-empty` when no sessions remain, `exit-unattached` when no client is attached) *and* nothing is still pending — no command waiting, no job running. Until all of those line up, the server keeps looping, which is exactly why your programs keep running after you close the window.

Killing the server is the deliberate counterpart to surviving a detach. A `SIGINT/SIGTERM` sets `SERVER_EXIT` and sends every client an exit message; a `kill-server` command walks the same path; and on exit the server runs its cleanup — `job_kill_all` to terminate background jobs, `status_prompt_save_history` to persist the command-prompt history — before calling `exit(0)`. The signal handler `server_signal` also wires the other operational signals faithfully: `SIGCHLD` reaps children, `SIGUSR1` recreates the socket (so a moved or deleted socket can be restored without losing the server), and `SIGUSR2` toggles logging. The lifecycle, end to end, is the tmux lifecycle: start as a daemon, own everything, survive every client, and exit only when asked or when policy says the work is done. With the server understood, the natural next question is the other half of the split — how a client finds this server, attaches to it, and becomes your window onto its state.

A walk through one `server_loop` iteration

It is worth slowing down on a single pass of `server_loop`, because the order of its steps is not arbitrary and the parity discipline reaches all the way down to it. The loop body runs, conceptually, in five stages, and each stage exists to leave the world consistent for the next. First it stamps `CURRENT_TIME` with `gettimeofday`, so every piece of work in this iteration — a format that expands `{t:...}`, an activity timer, a status redraw — sees one coherent notion of “now” rather than drifting by microseconds across the pass. Second, it drains command queues: the global queue first, then each identified client's queue, calling `cmdq_next` until no item is ready, so that a `bind-key` that ran three commands has all three applied before anything downstream observes the state. Third, `server_client_loop` walks every client, servicing pane output, recomputing which panes are visible, and flushing redraws. Fourth, alerts and hooks that the command work armed get their chance to fire. Only fifth, at the very end, does the loop ask “should the server exit?”

That final question is a conjunction, and getting the conjunction exactly right is why your programs survive a closed laptop lid. The server exits only if *all* of these hold: the relevant policy option says it may (`exit-empty` with no sessions, or `exit-unattached` with no attached client), *and* no command is still waiting in a queue, *and* no background job is still running, *and* the server is not in the middle of a config load. Miss any conjunct — treat “no clients attached” as sufficient on its own, say — and the server would exit out from under a detached session with a long-running build still going. The C computes this conjunction in a specific order with specific short-circuits, and ztmux ports the order, because “when exactly does the server decide it is done?” is observable behavior a parity `.sh` case can pin: detach from a session running `sleep 100`, confirm the server is still alive, re-attach, confirm the sleep is still there.

Chapter 4 – The Client

The client is the thin half: it connects to the server socket, forwards the terminal, relays keys, and renders what the server sends. This chapter covers the attach handshake, the terminal hand-off, and the client-side of the protocol.

Connecting to the server socket and the attach handshake

A client's first job is to find the server, and `client_connect` in `src/ported/client.rs` does it. It fills an `AF_UNIX sockaddr_un` with the socket path and calls `connect`. If the connection is refused or the path does not exist (`ECONNREFUSED/ENOENT`) and the caller passed the `STARTSERVER` flag — as an attaching command does — the client autostarts a server: it takes a lock file next to the socket ("`{path}.lock`", via `client_get_lock`, an exclusive non-blocking `flock` so two racing clients cannot both start a server) and calls the very `server_start` from Chapter 3, forking the daemon in the background before connecting to the socket it now provides. On a successful connect the descriptor is switched to non-blocking. This is the mechanism behind the everyday experience that `ztmux` “just works” whether or not a server is already running: the first client to arrive brings the server up.

Once connected, the client runs `client_main`, which chooses the message type for this invocation — `MSG_SHELL` for the shell-command path, `MSG_COMMAND` for a normal command — starts its own `tmuxproc`, sets up signal handling with `client_signal`, and registers the peer with `proc_add_peer`, wiring `client_dispatch` as the callback for messages arriving from the server. Then it performs the *identify* handshake, which is the heart of attaching: the client has to tell the server everything about the terminal it is bringing to the party, so the server can drive that terminal correctly.

`client_send_identify` sends that description as a sequence of `imsg` messages, one fact per message: `MSG_IDENTIFY_LONGFLAGS` (the 64-bit client flags), `MSG_IDENTIFY_TERM` (the `TERM` value), `MSG_IDENTIFY_FEATURES` (negotiated terminal features), `MSG_IDENTIFY_TTYNAME` (the tty path from `ttyname(STDIN_FILENO)`), `MSG_IDENTIFY_CWD` (the working directory), one `MSG_IDENTIFY_TERMINFO` per terminfo capability the client read, `MSG_IDENTIFY_STDIN` and `MSG_IDENTIFY_STDOUT` (which actually pass *duplicated file descriptors* across the socket, so the server can drive the client's real terminal), `MSG_IDENTIFY_CLIENTPID`, one `MSG_IDENTIFY_ENVIRON` per environment variable, and finally `MSG_IDENTIFY_DONE` to mark the end. The server assembles a client object from this stream. The attach completes when the server replies `MSG_READY`; the client's `client_dispatch_wait` handler sets `CLIENT_ATTACHED` and sends an initial `MSG_RESIZE`, and from that moment the client is a live window onto a server session.

Before any of that, the client also verifies it is speaking to a compatible server. On `MSG_VERSION` it checks the server's protocol version against its own `PROTOCOL_VERSION` (8), and on mismatch prints “protocol version mismatch (client {}, server {})” and refuses to proceed. This is the guard that keeps a client and a server built from different code from corrupting each other — the `imsg` protocol is an internal contract, not a stable ABI, and the version check enforces that both halves come from the same build.

Terminal hand-off: raw mode, size, and the render surface

For the server to draw a session into the client's terminal, the client must hand that terminal over in a known state, and `client_main` does the hand-off carefully. It reads the working directory, the `TERM` value, and the tty name; reads the terminal's terminfo capability list (`tty_term_read_list`, Chapter 10); closes the inherited pty-master descriptor it no longer needs; and frees the global options and environment it will not use as a client. Then, for the control-terminal path, it puts the terminal into raw mode — `cfmakeraw` as a base, then the specific tmux flag set: input flags cleared except a controlled `ICRNL/IXANY`, output `OPOST | ONLCR`, control `CREAD | CS8 | HUPCL`, and the canonical single-byte read discipline `VMIN = 1, VTIME = 0`. Raw mode is what lets tmux see every keystroke immediately and unprocessed, which is the precondition for a multiplexer that binds keys and forwards the rest verbatim.

The “render surface” on the client side is deliberately minimal, because the client does not own the grid — the server does. The client's terminal *is* the surface: the server computes what each pane should look like, and drives the client's real terminal through terminfo to make it look that way. The client hands over the raw terminal and the terminfo capabilities describing it; the server does the drawing. This is the opposite of an emulator like `zterminal`, which owns a GPU grid and paints pixels; a tmux client paints nothing itself — it lends its terminal to the server and relays events. Chapter 10's terminal back end is where the server-side drawing lives, and it drives *this* client's terminal through the descriptors the identify handshake passed across.

Terminal size is a first-class part of the hand-off and a recurring event afterward. The initial `MSG_RESIZE` the client sends on attach tells the server how big the terminal is; thereafter, every time the terminal is resized, the client's `SIGWINCH` handler sends a fresh `MSG_RESIZE`, and the server relays out the geometry so the layout engine can re-tile the panes to the new size. Size is negotiated, not assumed, and it is renegotiated continuously — which is why dragging a tmux window larger reflows the panes live.

The hand-off is reversible, and its reversibility is the whole point of a multiplexer. When the client detaches or dies, `client_signal` and the dispatch handlers restore the terminal and tear down the client, but they do *not* touch the server's grid — the session keeps its size and state until the next client attaches and hands over its own terminal. The terminal is borrowed, used, and returned; the session persists across the loan.

Keys in, cells out: the client relay loop

Once attached, the client runs a tight relay loop, and it is genuinely thin: keys go in, cells come out, and almost nothing is decided on the client side. Input from the client's terminal is read and forwarded to the server, which interprets it — against the current key table and bindings (Chapter 9) — and either runs a bound command or forwards the key on to the active pane's program. Output flows the other way: the server draws each pane and sends the client the terminal sequences to make its real terminal match, which the client writes out. The client is a conduit; the intelligence is in the server.

The messages that carry this traffic are dispatched by `client_dispatch`, which splits into two handlers depending on whether the client has attached yet. `client_dispatch_wait` handles the pre-attach messages — `MSG_VERSION`, `MSG_READY`, `MSG_SHELL`, the file read/write messages, and the various exit and detach messages that can arrive before attach completes. `client_dispatch_attached` handles the steady-state traffic — flag changes (`MSG_FLAGS`), detach requests (`MSG_DETACH/MSG_DETACHKILL`), execution requests (`MSG_EXEC`), suspend (`MSG_SUSPEND`, which stops the client with `SIGTSTP`), lock (`MSG_LOCK`, which runs the lock command and replies `MSG_UNLOCK`), and the exit and shutdown messages. The client's whole behavior is this dispatch table; the loop underneath it is the same one the server runs on.

Two of these paths are worth naming because they show how much the client deliberately does *not* do itself. `MSG_EXEC` is how the server tells the client to run a program in place of itself — used for things that must happen in the client's own process and terminal — and the client obeys via `client_exec`, which sets up the environment and `exec`s the shell. `MSG_LOCK` similarly has the client run a lock program locally. In both cases

the server decides *what* should happen and the client is the hands that carry it out in the right process context. The division is consistent: policy in the server, mechanism in the client.

Because the client is thin, its failure modes are simple and its exit reasons are enumerated precisely. The `client_exitreason` enum names every way a client can end — detached, detached-and-hung-up, lost-tty, terminated, lost-server, server-exited, exited, and so on — and the client prints the corresponding message on the way out. This is faithful tmux behavior and it is useful behavior: when a client goes away you get a specific reason, not a generic disconnect. The client does one job — relay a terminal to and from a server — and it reports exactly how that job ended.

Detach, re-attach, and multiple clients on one session

Detach is the multiplexer's signature move, and on the client side it is undramatic: the client receives (or initiates) a detach, restores the terminal, and exits, while the server keeps the session running. Re-attach is the mirror image: a new client connects, runs the identify handshake with *its* terminal's characteristics, and the server binds it to the session, resizing the session to the new terminal and redrawing it from the grid the server never stopped maintaining. Nothing about the session's programs, scrollbar, or layout was lost in between, because none of it ever lived in the client. Detach and re-attach are just the loan of the terminal ending and a new loan beginning.

Multiple clients on one session is the same mechanism run concurrently. Two clients can attach to the same session at once; both hand their terminals to the server, both receive the server's drawing of the same window, and both relay keys into the same panes. Because there is exactly one session tree in the server, the two clients are genuinely looking at the same thing — a keystroke or a pane switch from one is seen by the other, mediated by the server's single copy of the state. This is how tmux does pair programming and screen sharing without any special mode: it is just two clients viewing one server-owned session.

The interesting wrinkle multiple clients introduce is *size*, and tmux resolves it with an option `zterm ports faithfully`. When two attached terminals are different sizes, the session cannot be both sizes at once; the `window-size` option decides whether the session tracks the largest client, the smallest, the latest to attach, or a manual size, and the layout is computed against that. Each `MSG_RESIZE` from any client feeds this computation, and the server re-tiles accordingly. The client does not participate in the decision — it just reports its size and draws what the server sends — but the decision exists because clients can differ.

The through-line of the whole chapter is that the client is *defined by what it refuses to own*. It owns no session state, no grid, no layout, no options, no bindings; it owns a socket, a terminal in raw mode, and a dispatch table. That poverty is exactly what makes detach-survival, re-attach, and multi-client sharing fall out for free — there is nothing in the client to lose, so losing the client loses nothing. With both halves of the split now walked, the book descends into the thing every client is ultimately a window onto and every server ultimately maintains: the grid of cells at the bottom of the pane tree.

A worked example: two clients, two sizes

The `window-size` option earns a concrete walk, because it is where the multi-client story gets an arithmetic answer. Suppose client A attaches to session `work` from an 120×40 terminal, and then client B attaches to the same session from an 80×24 terminal. The session cannot be both sizes at once, so the layout has to be computed against *one* geometry, and `window-size` decides which:

- `largest` — the session sizes to 120×40, the union that fits the biggest client. Client B (80×24) sees only the top-left 80×24 of a larger virtual window; the `window_bigger` flag is set and `window_offset_x/window_offset_y` (Appendix B) track what portion B is scrolled to, so B can pan around the larger window.
- `smallest` — the session sizes to 80×24, the intersection every client can show fully. Client A (120×40) sees the 80×24 session centered or corner-aligned in its larger terminal, with the surplus area filled by `fill-character`. Nothing is clipped for anyone.

- `latest` — the session tracks whichever client most recently did something, so it follows the active user's terminal.
- `manual` — the size is fixed by `resize-window` and ignores client sizes entirely.

Every `MSG_RESIZE` from any client (Chapter 17) feeds this computation, and on each one the server recomputes the chosen geometry, re-tiles the layout (Chapter 7), reflows the grids (Chapter 5), and redraws each client to *its own* terminal. The client contributes only its size and its drawing surface; the decision is entirely server-side. This is the precise mechanism behind the everyday experience of two people sharing a tmux session from differently sized windows — and porting the option's four choices faithfully is what makes that experience identical under ztmux, down to which corner the smaller client anchors and what character fills the gap.

Exit reasons: how a client explains itself on the way out

A client can stop for a dozen different reasons, and lumping them together would make the program frustrating in exactly the moments when a user needs information — the moments when something ended unexpectedly. tmux therefore models the *reason* a client is exiting as a distinct piece of state, set at the point the decision is made and consulted once, at the very end, to decide what the client prints to the terminal it is about to give back. `client.rs` carries that enumeration and the port keeps it.

The reasons fall into three groups. There are the ordinary, expected endings: the session was detached, the client was told to detach and kill its session, the server shut down cleanly. There are the endings caused by state disappearing underneath the client: the session it was attached to was destroyed, or the server exited while it was attached. And there are the failures: the client could not create or connect to the socket, the protocol version did not match (Chapter 17), the server terminated abnormally, or the terminal could not be set up.

Why this deserves attention in a book about a port is that the message text and the exit status are both observable, and both are relied upon. A script that runs a command through the client and branches on the exit status is depending on the mapping from reason to status; a user who sees `[detached]` after pressing the detach binding is depending on the message. The reason enumeration is what makes both consistent: one decision point sets the reason, one place at the end renders it, and there is no scattered set of `printf`-and-exit calls each deciding for itself. Ported as-is, the client's departure messages match tmux's character for character, which is exactly the kind of unglamorous surface a user notices only when it is wrong.

The ordering at exit matters as much as the message. The client must restore the terminal it borrowed — leave the alternate screen if it entered one, restore the saved `termios` attributes, re-show the cursor, reset any modes it enabled — *before* printing its reason, because a message printed while the terminal is still in raw mode with the alternate screen active is a message the user will never see. That teardown is the mirror image of the hand-off described earlier in this chapter, and it runs on every path out, including the failure paths. A client that dies without restoring the terminal leaves the user typing blind into a shell with no echo, which is the single most user-hostile failure a terminal program can produce, and the reason the restore is unconditional rather than best-effort.

There is one asymmetry worth naming. Detaching is initiated by the *client* but decided by the *server*: the client sends the request, the server tears down its client object and sends back the message that ends the loop. The client does not simply exit on the keystroke. This matters because the server may have work to do first — running an appropriate hook, notifying other clients, deciding whether the session should now be destroyed under `destroy-unattached` — and a client that exited unilaterally would race all of it. The round trip costs nothing perceptible and keeps the server the sole authority on state, which is the same principle that shapes every other part of the split.

Chapter 5 – The Grid and Screen Model

Everything visible is a grid of cells. This chapter covers the data structure at the heart of the multiplexer: the grid, the screen, scrollback history, cell attributes and colors, and how panes are windows onto grids.

The grid: cells, lines, and scrollback history

At the very bottom of the pane tree is the `grid` — the two-dimensional array of character cells that a pane's program writes into and a client eventually sees. Its struct (defined in `src/lib.rs`, with the functions in `src/ported/grid.rs`) is small and telling: `sx` and `sy` are the visible columns and rows; `hsize`, `hlimit`, and `hscrolled` track the scrollback history — how many lines of history exist, the maximum allowed, and how far the viewer has scrolled up into it; `linedata` is a pointer to an array of `grid_lines`. The grid is created by `grid_create(sx, sy, hlimit)`, which allocates `sy` lines and sets the `GRID_HISTORY` flag only when `hlimit` is non-zero — a scrollback of zero is a grid that never keeps history, and the flag records that choice.

A `grid_line` is where the port's attention to memory layout shows. Each line holds `celldata` (a pointer to packed cell entries), `cellused` and `cellsize` (how many of the allocated cells are in use and how many are allocated), a separate `extddata/extdsize` pair for extended cells, a `flags` field, and a `time` stamp. Lines grow on demand: `grid_expand_line` snaps the allocation up through a sequence of sizes — a quarter of the screen width, then half, then the full width — so a line that fills up reallocates a bounded number of times rather than once per character. This is tmux's exact growth policy, ported rather than reinvented, because the allocation pattern is part of the behavior the parity suite pins.

Cells come in two physical forms for space, and understanding the split is the key to the whole grid. Most cells are *packed*: a `grid_cell_entry` whose union holds a compact `grid_cell_entry_data` of four bytes (`attr`, `fg`, `bg`, `data`) plus a flags byte — enough for an ASCII character with a 256-color palette entry and basic attributes. Cells that need more — a wide or combining Unicode character, a 24-bit RGB color, a non-default underscore color, or a hyperlink id — are *extended*: the packed entry stores an offset into the line's `extddata` array of `grid_extd_entry` values, which carry the full `utf8_char`, a 16-bit attribute field, and the wide color and link fields. `grid_need_extended_cell` decides which form a cell requires, and `grid_store_cell/grid_extended_cell` place it. The packed/extended split is how tmux keeps the common case cheap without capping the rich case, and ztmux ports the promotion rules exactly.

Scrollback is the history at the top of the grid, and it is managed by a small family of functions. `grid_scroll_history` allocates a fresh bottom line and grows `hsize` and `hscrolled`; `grid_scroll_history_region` does the same within a scroll region. The bound is enforced by `grid_collect_history`: once `hsize` reaches `hlimit`, it frees the oldest `hlimit / 10` lines (at least one) — the “drop the top ten percent” policy that amortizes the cost of trimming rather than freeing one line per scroll. The default `hlimit` comes from the `history-limit` session option, which defaults to 2000 lines (`src/ported/options_table.rs`) and, per its own help text, applies only to newly created panes when changed. Scrollback is thus a bounded ring of past lines, and everything copy mode navigates (below) is a walk through it.

A worked example: the ten-percent trim

The scrollback bound has a specific amortization policy, and watching it act explains why tmux does not simply free one line per scroll. Suppose a pane's `history-limit` is 2000 and its history is full — `hsize == 2000`. A new line scrolls off the top. A naive implementation would free exactly one line to make room, paying a free

(and the bookkeeping) on every scroll forever. `grid_collect_history` instead waits until `hsize` reaches `hlimit` and then frees the oldest `hlimit / 10` lines at once — 200 lines here, or at least one if the tenth is zero — so the pane now holds 1800 lines and will not trim again for another 200 scrolls. The cost of trimming is amortized across those 200 scrolls rather than paid on each one, which matters when a program is scrolling thousands of lines a second (a `yes` loop, a fast build log). This “drop the top ten percent” policy is tmux’s exact rule, ported rather than reinvented, because the allocation and free pattern is part of the behavior the parity discipline pins — and because getting it wrong (trimming one line at a time) would be a quiet performance regression invisible to a correctness test but obvious under a flood.

Cell attributes and color (`attributes.rs`)

A cell is not just a character; it is a character *styled*, and the style lives in the `grid_cell`’s `attr`, `fg`, `bg`, `us`, and `link` fields. The `attr` field is a 16-bit bitflag set (`grid_attr`, in `src/lib.rs`) with a flag per rendition: `GRID_ATTR_BRIGHT` (which is bold — tmux, like the SGR spec, treats bold and bright as one), `DIM`, `UNDERSCORE`, `BLINK`, `REVERSE`, `HIDDEN`, `ITALICS`, `CHARSET` (the ACS line-drawing charset), and — above the low byte, so a cell carrying them is forced extended — `STRIKETHROUGH`, `OVERLINE`, and the four additional underline styles `UNDERSCORE_2` through `UNDERSCORE_5` (double, curly, dotted, dashed). The `GRID_ATTR_ALL_UNDERSCORE` union groups the five underline flavors so code can ask “is this cell underlined in any style?” in one test.

Color is an `i32` with a tagging scheme, and it repays a close look because it encodes three color spaces in one field. The values 0–7 and 90–97 are direct ANSI palette indices; 8 is “default” (`COLOUR_DEFAULT` also treats 9 as default); an index or-ed with `COLOUR_FLAG_256` (`0x01000000`) is a 256-color palette entry; and a packed `(r << 16) | (g << 8) | b` or-ed with `COLOUR_FLAG_RGB` (`0x02000000`) is a 24-bit truecolor value. The `fg` and `bg` fields carry text and background color this way, and `us` carries the underscore color separately — which is why an underline can be a different color from the text it sits under, a feature modern terminals expose through SGR 58 and tmux supports. `src/ported/colour.rs` holds the pack/unpack helpers; the grid just stores the tagged integer.

The bridge between these numeric representations and the human-readable names that appear in config files and `#{...}` formats is `src/ported/attributes.rs`. Its `attributes_tostring` turns an `attr` bitset into a comma-separated string, and `attributes_fromstring` parses such a string back into flags, against a fifteen-entry table that aliases both “bright” and “bold” to `GRID_ATTR_BRIGHT` and names the exotic underlines (“double-underscore”, “curly-underscore”, “dotted-underscore”, “dashed-underscore”) and “acs” for the charset flag. This is the table that lets you write `bold,underline` in a style option and have it mean the right bits, and because it is a direct port of tmux’s table, the accepted names are exactly tmux’s.

The default cell ties the representation together. `GRID_DEFAULT_CELL` is a single space with empty attributes and `fg`, `bg`, and `us` all set to 8 (default) — the cell every fresh grid position starts as. There are a few other canonical cells: `GRID_PADDING_CELL`, a zero-width placeholder (marked with the `PADDING` grid flag) that sits in the second column of a wide character; and `GRID_CLEARED_CELL`, which records that a cell was explicitly cleared rather than merely never written. These canonical cells are what clearing, wide-character placement, and grid comparison (`grid_cells_equal`, `grid_cells_look_equal`) are defined against, and they are ported byte-identically because “cleared” versus “never written” is a distinction tmux’s redraw logic actually depends on.

Screen vs. grid: the visible window onto history

The `grid` is raw storage; the `screen` is the stateful view of it, and separating the two is what lets one grid support scrollbar, alternate screens, and cursor motion cleanly. The `screen` struct (`src/lib.rs`, functions in `src/ported/screen.rs`) owns a `grid` pointer plus everything about how a program is currently drawing into it: the cursor position (`cx`, `cy`), the cursor style and color, the scroll region (`rupper` and `rlower`, the top and bottom rows a scroll is confined to), the mode flags (`MODE_CURSOR`, `MODE_WRAP`, and the rest), the tab stops, the title stack, and the selection. Where the grid answers “what characters are stored?”, the screen answers “where is the cursor, what region scrolls, what mode are we in, and what is selected?”

`screen_init` creates the backing grid with `grid_create` and sets the initial mode to `MODE_CURSOR`; `screen_reinit` resets it, setting the scroll region to the full height and the mode to `MODE_CURSOR | MODE_WRAP`. Tab stops default to every eighth column (`screen_reset_tabs`), stored compactly as a bit string. Resize is where the screen/grid split earns its keep: `screen_resize` (and `screen_resize_cursor`, which takes a `reflow` flag) changes the visible dimensions and, when reflow is on, rewraps the grid's lines to the new width — the machinery in `grid.rs`'s `grid_reflow_*` functions — so that making a pane narrower re-wraps long lines instead of truncating them. The cursor is moved to stay sensible across the resize. This is subtle code and it is a faithful port, because reflow-on-resize is precisely the sort of behavior a spec would under-specify and a parity suite pins.

The *alternate screen* is the screen model's most visible trick and a clean example of why the split exists. Full-screen programs — an editor, a pager, `http` — switch to an alternate screen so that when they exit, the shell's scrollbar is exactly as they left it. `screen_alternate_on` saves the current cursor and, optionally, the whole grid into `saved_grid/saved_cx/saved_cy`; `screen_alternate_off` restores them. Because the alternate screen is a property of the `screen` (the view) rather than a second `grid`, entering and leaving it is a save/restore of view state, and the primary scrollbar is untouched underneath — which is exactly the behavior you rely on every time you quit `less` and your prompt is where you left it.

Panes are windows onto grids in the literal sense this chapter's title promises. Each pane owns a screen, and each screen owns a grid; the pane's program writes into the screen through the screen-write API (below), the screen mutates the grid, and the server draws the visible portion of the grid — offset by `hscrolled` when the viewer has scrolled up — into whatever region of the client's terminal the layout assigns the pane. The `screen_size_x/screen_size_y/screen_hsize/screen_hlimit` accessors are thin wrappers over the grid's dimensions, because the screen's size is the grid's visible size. A pane is thus three nested objects — pane, screen, grid — and the client sees a rectangle of the innermost one.

Copy mode and the selection over the grid

Copy mode is how you read and copy the scrollbar without a mouse, and it is implemented as a *mode* over the grid rather than a separate buffer — which is why it can scroll seamlessly from the visible screen up into history. When a pane enters copy mode, a virtual cursor detaches from the program's cursor and moves over the grid under keyboard control; the grid keeps updating underneath (the program is still running), but the copy-mode cursor navigates the frozen coordinate space of stored lines, including the scrollbar above the visible region. The `grid_reader` struct — a `gd` pointer plus a `cx/cy` position — is the virtual cursor's state, and `src/ported/grid_reader.rs` holds the motions that walk it word by word and line by line.

The selection is a region of the grid, and it lives on the screen as a `screen_sel`. Its fields are exactly what a selection needs: `sx/sy` and `ex/ey` for the start and end coordinates, a `rectangle` flag for block versus stream selection, the `modekeys` (emacs or vi) that govern the motions, and a `cell` template for how selected cells are drawn. `screen_set_selection` establishes it, `screen_clear_selection` and `screen_hide_selection` remove or hide it, and `screen_check_selection(s, px, py)` answers the question the renderer asks for every cell: “is this position selected?” — which is how the highlight follows the selection as it grows.

Copy itself is defined against the grid's string-extraction functions, and this is where color and attributes come back. `grid_string_cells` walks a run of cells and produces a string; with the right flags it can embed the SGR escape sequences that reproduce each cell's color and attributes, using the helper family `grid_string_cells_fg`, `grid_string_cells_bg` (which emit `38;5;n / 38;2;r;g;b` and the 30–37/90–97 forms), `grid_string_cells_us` (the 58-based underscore color), and `grid_string_cells_code` (a table mapping each `grid_attr` flag to its SGR code — bright to 1, italics to 3, and so on). This is why a copy-mode selection can preserve styling: the extractor can regenerate the escape sequences that produced it. Whether a copy keeps styling or strips to plain text is governed by the string flags (`grid_string_flags`: with-sequences, trim-spaces, used-only, and the rest).

The larger point copy mode makes is that the grid is not just a rendering buffer — it is a queryable, extractable, navigable data model, and every higher-level feature is expressed against it. Copy mode is a cursor and a selection over the grid; search in copy mode is a scan over the grid; the status line and `capture-pane` are

string extractions from the grid; even the layout engine ultimately exists to decide which rectangle of which grid goes where on screen. The grid is the substrate, and the rest of the multiplexer is operations on it. The next chapter climbs one level down the input path — to the parser that turns a program's raw byte stream into the grid mutations this chapter has been treating as given.

Reflow: rewrapping wrapped lines when a pane narrows

The screen/grid split earns its keep most dramatically on resize, and the hardest part of resize is *reflow* — rewrapping soft-wrapped lines to a new width instead of truncating them. The distinction the machinery turns on is a single per-line flag. When a program prints past the right margin and the line continues onto the next row, tmux does not store a newline; it stores the continuation and marks the first line `GRID_LINE_WRAPPED`. A real newline (the program pressed Enter, or `screen_write_linefeed` ran) leaves the flag clear. So the grid already records, line by line, which line breaks are *soft* (an artifact of the current width) and which are *hard* (structural). Reflow is the operation that honors that distinction: it may re-break the soft ones at a new width, and it must never merge across a hard one.

`grid_reflow(gd, sx)` (`src/ported/grid.rs`) does the work, and its shape is a rebuild rather than an edit. It walks the existing lines building a *new* line array at the target width `sx`: a run of lines joined by `GRID_LINE_WRAPPED` flags is treated as one logical line, concatenated, and then re-split into as many physical lines of width `sx` as it needs, each but the last carrying the wrapped flag again; a hard newline ends the logical line and starts a fresh one. The helper family `grid_reflow_add` (append a new empty line to the rebuild) and `grid_reflow_move` (move existing cells into it) keep the accounting straight, and the history size `hsize` is adjusted as lines are created or absorbed, because rewrapping a full scrollbar at a narrower width produces *more* lines than it started with. This is why making a pane narrower can lengthen its scrollbar: the same text, wrapped tighter, occupies more rows.

The cursor is the subtle part, because the row the cursor sits on may move during the rebuild — a line above it might split in two, pushing it down. `screen_resize_cursor` carries a `reflow` flag precisely so the resize path can track the cursor's *logical* position through the rewrapping and restore it to the corresponding physical row afterward, rather than leaving it stranded where the old geometry put it. Reflow is expensive — it reallocates the whole line array and touches every line of history — so it runs only on a width change, and only when the `window-size/reflow` policy calls for it; a pure height change resizes without rewrapping. The whole operation is a faithful port because reflow-on-resize is exactly the sort of behavior no specification pins down and every user notices: shrink your terminal, and the long lines in your scrollbar must re-wrap the way tmux re-wraps them, not truncate and not scramble. The parity discipline reaches it through `.sh` cases that resize a pane and read back the wrapped result.

A worked example: what one styled character costs in memory

Follow a single character all the way into the grid and the packed/extended split stops being abstract. Suppose a program prints a plain ASCII `A` in the default style. The screen-write layer builds a `grid_cell` whose `data` is the UTF-8 for `A`, whose `attr` is `0`, and whose `fg/bg/us` are all `8` (default). `grid_need_extended_cell` looks at that cell and answers *no*: the character is a single byte, the colors are palette indices that fit, no attribute above the low byte is set. So `grid_store_cell` writes a *packed* `grid_cell_entry` — five bytes: one flags byte plus the four-byte `grid_cell_entry_data` of `attr`, `fg`, `bg`, and the single data byte. That is the common case, and it is deliberately cheap, because a screenful of text is thousands of exactly this.

Now suppose the program prints a red `A` — SGR `38;2;220;50;47` then `A`. The cell's `fg` is now `(220<<16)|(50<<8)|47` or-ed with `COLOUR_FLAG_RGB` (`0x02000000`), which no longer fits the packed entry's one-byte `fg` slot. `grid_need_extended_cell` answers *yes*, and the cell is promoted: the packed entry

now stores an *offset* into the line's `extddata` array, and a `grid_extd_entry` at that offset carries the full `utf8_char`, a 16-bit `attr`, and the wide `fg/bg/us/link` fields — several times the packed size. The same promotion happens for a wide CJK glyph (which also claims a `GRID_PADDING_CELL` in the next column), a combining sequence, a curly underline (`GRID_ATTR_UNDERSCORE_2`, a flag above the low byte), or a hyperlink id. The lesson the example teaches is the one the whole grid is built on: you pay for richness only in the cells that use it, and `grid_need_extended_cell` is the exact predicate — ported from the C, not re-derived — that decides who pays.

Chapter 6 – The Input Parser

Bytes arrive from the program running in a pane; the input parser turns that VT stream into grid mutations. This chapter walks the state machine — escape sequences, CSI/OSC handling, and how the parser drives the screen.

The VT parser state machine

The input parser in `src/ported/input.rs` is a direct implementation of Paul Williams' well-known DEC ANSI-compatible parser — the state machine that virtually every serious terminal implements — extended, as tmux extends it, to handle UTF-8, to let OSC strings terminate on a bare `BEL` as well as the formal string terminator, and to add the APC, rename, and DCS-escape states tmux needs. Its own module documentation lays out the model, and its structure is faithful to it: a set of *states*, each with an *entry* action, an *exit* action, and a *transition table* keyed by input byte, plus a small set of *anywhere* transitions that apply in every state.

Concretely, there are twenty `input_state` values, each a struct carrying a name, an optional enter hook, an optional exit hook, and a slice of transitions: `GROUND` (the resting state, where printable bytes get written to the screen), `ESC_ENTER` and `ESC_INTERMEDIATE`, the CSI family (`CSI_ENTER`, `CSI_PARAMETER`, `CSI_INTERMEDIATE`, `CSI_IGNORE`), the DCS family (`DCS_ENTER`, `DCS_PARAMETER`, `DCS_INTERMEDIATE`, `DCS_HANDLER`, `DCS_ESCAPE`, `DCS_IGNORE`), the string states (`OSC_STRING`, `APC_STRING`, `RENAME_STRING`), and `CONSUME_ST` for swallowing a trailing string terminator. Each transition (`input_transition`) names a byte range (`first`, `last`), an optional handler function, and an optional next state; the `INPUT_STATE_ANYWHERE` transitions handle the bytes that mean the same thing in any state — `0x18/0x1a` (CAN/SUB) abort the current sequence back to ground, and `0x1b` (ESC) always begins a new escape.

The driver is `input_parse`, and it is a plain, fast loop: for each byte, read the current state's transition table, linear-scan it for the entry whose `first..=last` range contains the byte, run that entry's handler, and — unless the handler suppresses it — switch to the entry's next state, running the old state's exit hook and the new state's enter hook (`input_set_state`). Because printable characters are the overwhelmingly common case, they are special-cased: the ground state's printable range routes to `input_print`, and the loop avoids the collection-flush overhead for that path. The parser has entry points for the different sources it serves — `input_parse_pane` for a pane's program output, `input_parse_buffer` and `input_parse_screen` for other callers — but they all funnel into the same `input_parse` loop.

The parser is defensive in the ways a terminal must be, and the bounds are named constants rather than magic numbers. Intermediate bytes collect into a four-byte buffer, parameters into a sixty-four-byte buffer, the parameter list holds up to twenty-four parsed values, and the OSC/DCS string input grows on demand but is capped at `INPUT_BUF_LIMIT` (1048576 — one megabyte) so a runaway sequence cannot exhaust memory. A five-second timer (`input_start_timer`) guards against a sequence that begins and never completes, resetting the parser to ground so a pane cannot be wedged forever mid-escape by a malicious or broken program. These limits are tmux's, ported directly, and they are the difference between a parser that is merely correct and one that is also robust against hostile input.

CSI, OSC, and the escape-sequence surface

The three big escape families dispatch through three tables, and each table is a sorted, searchable map from the sequence's identifying bytes to a handler. For `ESC` sequences — the short two-byte codes like `ESC 7`

(save cursor), `ESC 8` (restore cursor), `ESC M` (reverse index), `ESC c` (full reset), `ESC #8` (the DEC alignment test) — `input_esc_dispatch` looks the final byte and intermediate up in `INPUT_ESC_TABLE` (keyed through `input_table_compare`) and calls the matching handler; the `input_esc_type` enum names each (DECALN, DECKPAM/PNM, DECRC/DECSC, HTS, IND, NEL, RI, RIS, the charset selectors). These are the terse controls that predate CSI, and they are handled in full.

CSI — the `ESC [` family — is the large one, and it is where a terminal earns its compatibility. `input_csi_dispatch` first parses the accumulated parameter buffer into the parameter list with `input_split` (splitting on `;`, and treating `:` as introducing sub-parameters, which is how the colon-form SGR color sequences are carried), then binary-searches `INPUT_CSI_TABLE` — a forty-entry table keyed by the final byte plus any intermediate — for the sequence's type from the `input_csi_type` enum. The catalogue is the full working set: cursor motion (CUU/CUD/CUF/CUB/CUP), erase (ED/EL), insert and delete of characters and lines (ICH/DCH/IL/DL), scroll (SU/SD), erase character (ECH), the scroll-region setter (DECSTBM), device status and attributes (DSR/DA), the SGR rendition setter, and the mode setters (SM/RM), each with private-mode variants dispatched through `_sm_private/_rm_private` for the `CSI ?` forms. Parameters are fetched with defaults through `input_get(ictx, idx, minval, defval)`, so a `CSI H` with no arguments correctly means “home.”

Replies are part of the CSI surface, because some sequences are *questions* the terminal must answer, and the parser answers them through an `input_reply!` macro that writes the response back toward the program. A Device Attributes request (`CSI c`) is answered `\x1b[?1;2c` — or `\x1b[?1;2;4c` when the sixel feature is built in, advertising graphics support — and a secondary DA is answered `\x1b[>84;0;0c`. A cursor-position report (`DSR` with parameter 6) is answered with the live cursor position, `\x1b[{row};{col}R`. These replies are how programs probe what the terminal can do, and getting them exactly right — including advertising sixel only when it is actually compiled in — is part of being a faithful terminal rather than one that merely renders.

OSC — the `ESC ]` family — carries the string-valued controls, and it is dispatched when the OSC string state exits (`input_exit_osc`): the leading numeric option is parsed and matched. `0` and `2` set the window title (via `screen_set_title`, gated by the `allow-set-title` option); `4` sets palette entries; `7` reports the working directory/path; `8` carries hyperlinks; `10/11/12` set the foreground, background, and cursor colors; `52` accesses the clipboard; `104/110/111/112` reset palette entries; and `133` carries the shell-integration prompt marks (FinalTerm/iTerm2-style semantic prompts). The neighboring string states handle the rest: APC strings, and the rename string (`ESC k`) that sets a window's name, gated by `allow-rename`. The gating options matter — they are the guardrails that stop a remote program from silently renaming your windows or repainting your palette, and the parser honors them.

From parsed sequence to grid mutation

The parser does not touch the grid directly; it drives the *screen-write* API, and that indirection is deliberate. Every handler in `input.rs` ultimately calls a `screen_write_*` function (`src/ported/screen_write.rs`) — `screen_write_cursormove`, `screen_write_linefeed`, `screen_write_scrollup`, `screen_write_clearline`, `screen_write_insertcharacter`, and so on — and those functions perform the actual grid mutation, update the cursor, respect the scroll region, and mark the affected cells dirty for redraw. The parser's job ends at “decide what should happen”; the screen-write layer's job is “make it happen to the grid correctly.” This is the same policy/mechanism split the client/server relationship showed, one level down: the parser is policy, screen-write is mechanism.

The most common mutation — printing a character — goes through a batched fast path, because it is where throughput lives. `input_print` does not write each character to the grid individually; it accumulates printable characters and hands them to `screen_write_collect_add`, which batches them into a run, and the batch is flushed (`screen_write_collect_end`) when a non-print sequence arrives or the run must be committed. This collection path is why *cat*-ing a large file through a tmux pane is fast: the parser turns a stream of thousands of printable bytes into a handful of batched grid writes rather than thousands of individual cell stores. The SGR state — the current color and attributes — rides along in the parser's `input_cell`, so each batched character is stamped with the active rendition without re-deriving it.

Character-set state is carried in the parser too, and it is a small but load-bearing example of state the parser

must track across sequences. The `input_cell` holds the G0 and G1 charset selections and which is currently active; the ESC charset selectors (`5C5G0/5C5G1` on and off) set them, and the shift-in/shift-out control bytes switch between them. When the ACS (alternate character set) charset is active, the `GRID_ATTR_CHARSET` flag is set on written cells so the drawing layer knows to map them through the line-drawing glyphs. This is how a TUI's box borders render as lines rather than as the letters `q` and `x`: the charset state, tracked across the byte stream by the parser, tags the cells, and the terminal back end (Chapter 10) maps them on the way out.

The color sequences are the richest mutation and show the whole pipeline in one place. `input_csi_dispatch_sgr` walks the parsed SGR parameters and updates the `input_cell`'s attributes and colors: `0` resets, `1` sets bright, `3` italics, `4` underline, `7` reverse, the `30–37/90–97` set the foreground from the ANSI palette, `38/48/58` introduce extended (256-color or RGB) foreground, background, and underscore colors through `_sgr_256/_sgr_rgb`, and `39/49/59` reset each to default. Both the semicolon form and the colon sub-parameter form (`input_csi_dispatch_sgr_colon`) are handled, because real programs emit both. The result of an SGR sequence is not a grid write at all — it is a change to the *cell template* — and the next printable characters inherit it. That is the model in miniature: the parser maintains style state, and printing applies it.

Semantic prompts and the clipboard: OSC 133 and OSC 52

Two OSC sequences the parser handles are worth singling out because they are how a modern shell and a modern terminal talk to the multiplexer, and because one of them is the raw material for a superset feature (Chapter 13). OSC 133 is the FinalTerm/iTerm2-style *semantic prompt* protocol: a shell emits `OSC 133 ; A` before its prompt, `; B` after it, `; C` before command output, and `; D` after — marking, in the byte stream, where prompts and command outputs begin and end. When the parser sees these, it sets grid line flags (`START_PROMPT`, `START_OUTPUT`) on the affected lines, so the grid itself records the structure of the session's history. Because that structure is stored, features that navigate by command — jump to the previous prompt, select the last command's output — have their data already in the grid; they are extensions the port is *positioned* for rather than ones needing new plumbing. `zshrs` running inside a pane emits exactly these marks, which is why the integration is more than aspirational: the tracking is live.

OSC 52 is the clipboard access sequence: a program inside a pane can set the system clipboard by emitting `OSC 52 ; c ; <base64>`, and tmux gates this behind the `set-clipboard` option, because letting arbitrary pane programs write your clipboard is a security decision, not a default. When enabled and the outside terminal supports it, tmux forwards the clipboard set outward; when disabled, it is ignored. The gating is the recurring theme of the OSC handlers (Chapter 6's `allow-set-title`, `allow-rename`): the parser recognizes the sequence but honors a policy option before acting, so a remote program cannot silently repaint your palette, rename your windows, or hijack your clipboard. Porting the gates, not just the sequences, is what keeps ztmux's security posture identical to tmux's.

Fidelity to xterm and the tmux input model

Fidelity here has a precise meaning: the parser must interpret the byte stream the same way tmux does, which is (very nearly) the way xterm does, so that any program written for a real terminal behaves correctly in a ztmux pane. The way ztmux guarantees this is not by aspiring to xterm compatibility in the abstract but by porting tmux's parser, table for table and handler for handler, and then checking the result against the real tmux with the parity suite. The state machine, the transition tables, the reply strings, the buffer limits, the gating options — all of them are tmux's, and when they diverged during porting, the parity suite caught it.

The tmux input model has one important twist over a bare emulator: tmux is *itself* a program running in someone else's terminal, so it has two input surfaces, not one. The parser in this chapter handles the bytes *from the programs running inside panes* — turning their output into grid mutations. A separate path (`src/ported/input_keys.rs`, `src/ported/tty_keys.rs`) handles the bytes *from the outside terminal's*

keyboard — turning the user’s keystrokes into tmux key codes that the key tables (Chapter 9) dispatch. The `input.rs` parser is the inner one; it consumes what pane programs produce, not what the user types. Keeping the two straight matters, because they are different state machines solving different halves of the multiplexer’s job.

The parser’s faithfulness is what makes everything above it trustworthy. Copy mode (Chapter 5) is only correct if the grid it navigates was mutated correctly; the layout engine (Chapter 7) only tiles panes whose contents are right; the format system (Chapter 9) only reports pane state that was tracked right. All of that rests on the parser interpreting the VT stream exactly as tmux does, which is why this subsystem is one of the most heavily line-for-line ported in the tree and one of the first the parity suite’s format and pane cases exercise indirectly. A multiplexer whose parser drifts is a multiplexer that corrupts editors and TUIs subtly; ztmux avoids that by not letting the parser drift from the C.

There is genuine subtlety the port has to preserve that a casual reading would miss — the exact handling of wrap-at-margin (does the cursor sit past the last column, or wrap immediately?), the interaction of the scroll region with insert/delete-line, the precise conditions under which a cell becomes extended, the timing of the auto-margin flag. These are the corners where terminals disagree and where TUIs have quietly grown to depend on one specific behavior. Because ztmux ports tmux’s exact resolutions of these corners — with the C citation on each function and the parity suite over the whole — it inherits tmux’s exact compatibility, including the parts no document fully describes. That inheritance is the point of porting rather than rewriting, and the parser is where it pays off most visibly.

The pending-wrap corner, in detail

One of those corners deserves to be spelled out, because it is the single most common source of subtle emulator bugs and the parser handles it exactly as tmux does. The question is: when a program prints a character in the last column of a line, does the cursor immediately wrap to the next line, or does it stay put? The naive answer — wrap immediately — is wrong, and TUIs depend on it being wrong. A terminal with the auto-wrap mode on does *not* move the cursor when it writes the last column; instead it sets a *pending-wrap* flag and leaves the cursor visually on the last column. The wrap happens only when the *next* printable character arrives, at which point the terminal first moves to column zero of the next line and then prints. If instead a cursor-motion or an erase sequence arrives, the pending-wrap flag is cleared and no wrap ever happens.

This deferred wrap is what lets a program fill the last column of the last row without scrolling the screen — draw a full-width box border, print a character in the bottom-right corner — which every full-screen TUI does constantly. Get it wrong by wrapping eagerly and the screen scrolls by one line every time a program touches the corner, corrupting the display. The parser and screen-write layer track the pending-wrap state on the screen and apply it at exactly the moment tmux does, and because the behavior is observable, it is the kind of thing the parser’s line-for-line port and the parity suite together pin. It is a one-flag detail that no casual reading of “the parser writes characters to the grid” would surface, and it is precisely the sort of undocumented-but-depended-on behavior that porting (rather than rewriting from a spec) inherits for free.

DCS, passthrough, and the sixel path

The escape-sequence surface described so far — CSI, OSC, plain escapes — covers what ordinary programs emit. There is a fourth family the parser must handle, and it is the one where a multiplexer’s position in the middle becomes genuinely awkward: DCS, the device control string. A DCS sequence has an introducer, a payload of arbitrary length, and a terminator, and unlike CSI it is not a compact command with numeric parameters — it is a container for a sub-protocol.

The parser’s structural obligation is the easy half. Whatever a DCS carries, the state machine must recognise the introducer, consume the payload without interpreting stray bytes inside it as commands, and return to ground state on the terminator. Getting that wrong is catastrophic in a specific way: a parser that mis-detects the end of a DCS payload will start interpreting the remainder of the payload as escape sequences, and a long

payload can then issue arbitrary cursor motions and colour changes derived from what was meant to be data. The state machine's separate DCS states exist precisely so payload bytes cannot leak into the command path, and the port keeps the state structure rather than short-cutting to "scan for the terminator," because the difference shows up on malformed input.

The interesting half is what to *do* with the payload, and there are two answers in tmux and both are ported.

The first is **passthrough**, governed by `allow-passthrough`. A program inside a pane wraps a sequence in the passthrough DCS form, and the multiplexer — instead of interpreting it — forwards the inner bytes to the outer terminal. This exists because the multiplexer cannot possibly implement every escape sequence every terminal supports, and a program that wants to use a capability ztmux does not model needs a way to reach the real terminal. It is deliberately opt-in and off by default, because passthrough is a hole in the abstraction: bytes sent this way bypass the grid entirely, so ztmux does not know what they did to the screen and cannot redraw over them correctly. A pane that passes through a cursor move has desynchronised the multiplexer's model from the terminal's reality, and the next redraw will paint from a model that no longer matches. Being off by default is the right trade — the feature is for users who know precisely what they are sending and accept the consequence.

The second is the **sixel** path, and it is different in kind because it is *interpreted* rather than forwarded. Sixel is an image format carried in a DCS payload, and `image_sixel.rs` parses it into an image which `image.rs` associates with a region of the pane's screen. This is genuinely modelled state rather than a hole: the multiplexer knows an image occupies a rectangle, which means it can redraw it, scroll it with the rest of the grid, and discard it when the region is cleared. It is also, as Appendix F notes, the one fully-implemented capability in the tree that sits outside the pure-text model everything else assumes.

That last point is worth dwelling on, because it exposes an assumption the rest of the book has been able to leave implicit. Every other subsystem treats a pane as a matrix of cells: the grid stores cells, reflow rewraps cells, copy mode extracts cells, the tty back end paints cells. An image is not cells. It occupies screen area without being addressable as characters, it cannot be copied by `grid_string_cells`, and it does not participate in reflow in any meaningful way — which is why the image layer keeps its own association between images and screen regions rather than trying to encode them into the grid. The design lesson generalises past sixel: the cell matrix is the right model for text and the wrong model for everything else, and the honest way to support non-text content is a parallel structure with explicit rules about how the two interact, not a heroic attempt to make cells carry pixels.

Chapter 7 – Layouts

Panes are tiled by a layout engine: the classic tmux layouts, manual splits, and the layout string that serializes an arrangement. This chapter covers the geometry.

The layout tree and cell geometry

A window's panes are arranged by a tree, not a list, and the tree is the layout engine's central data structure. Each node is a `layout_cell` (defined in `src/lib.rs`, operated on in `src/ported/layout.rs`): it carries a `type_`, a `parent` pointer, a size (`sx`, `sy`) and an offset (`xoff`, `yoff`) in the window's coordinate space, a `wp` pointer to a window pane, and a `cells` tail queue of children. The `type_` is one of just three values (`layout_type`): `LAYOUT_LEFTRIGHT` for a horizontal split (children side by side), `LAYOUT_TOPBOTTOM` for a vertical split (children stacked), and `LAYOUT_WINDOWPANE` for a leaf that holds an actual pane. Interior nodes split space; leaves point at panes. Every arrangement of panes you can make in tmux is some tree of these three node types.

The geometry is exact and it composes. A leaf's `sx/sy/xoff/yoff` are the pane's rectangle within the window; an interior `LAYOUT_LEFTRIGHT` node's width is the sum of its children's widths plus one column per internal border, and its height is each child's height; an interior `LAYOUT_TOPBOTTOM` node is the transpose. Because the offsets are absolute window coordinates rather than relative, drawing a pane is a matter of reading its leaf cell's rectangle, and finding which pane a mouse click landed in is a walk down the tree comparing the click position to each cell's rectangle. `layout_create_cell` builds a node (starting its sizes and offsets at `u32::MAX` as an "unset" sentinel until the layout is computed), and the tree is torn down by `layout_destroy_cell` and `layout_free`.

The one-column-per-border accounting is not a detail you can hand-wave, because it is where a layout's arithmetic either matches tmux or does not. The borders between panes occupy real columns and rows on screen, and the layout engine has to reserve them: an 80-column window split into two side-by-side panes is not two 40-column panes, it is (roughly) a 40 and a 39 with a border column between them. Getting that accounting exactly right — including which pane absorbs the odd column — is what the even-horizontal rounding fix (below, and Chapter 11) is about, and it is why the layout code carries dedicated tests locking the exact split sizes.

The tree also answers the navigation questions that pane-movement commands ask. "What pane is to the right of this one?" is a geometric query over the tree: find panes whose rectangles are adjacent on the relevant edge. "Resize this pane" is a mutation that adjusts a cell's size and propagates the change to its siblings and up to its ancestors. Because the layout is a tree with explicit geometry rather than an implicit tiling rule, all of these operations are concrete pointer-and-arithmetic walks, ported directly from tmux's `layout.c`. The tree *is* the arrangement, and everything you do to your panes is an operation on it.

Preset layouts and manual splits

There are two ways panes get arranged: you split them by hand, or you apply a preset layout, and the engine supports both against the same tree. Manual splitting is `layout_split_pane` (`src/ported/layout.rs`): given a pane and a direction, it takes the pane's leaf cell, turns it into an interior split node, and gives it two children — the original pane and a new one — dividing the space between them. This is what `split-window -h` and `-v` do: grow the tree by one leaf, in place, at the chosen pane. Closing a pane (`layout_close_pane`)

does the reverse, collapsing the split and handing the freed space back to the sibling, and `layout_spread_out` redistributes space when a collapse leaves things lopsided.

The preset layouts — the ones `select-layout` and the `Space` prefix key cycle through — live in `src/ported/layout_set.rs` as a `layout_sets` table mapping names to builder functions. There are five, matching tmux exactly: `even-horizontal` (all panes in one row, equal widths), `even-vertical` (one column, equal heights), `main-horizontal` (one large pane on top, the rest in a row below — plus a `-mirrored` variant that flips top and bottom), `main-vertical` (one large pane on the left, the rest stacked at the right — plus its mirror), and `tiled` (a grid as square as the pane count allows). `layout_set_lookup`, `layout_set_select`, `layout_set_next`, and `layout_set_previous` are the dispatch and cycling functions; applying a preset rebuilds the window's layout tree from scratch to the preset's shape.

The even layouts are where the arithmetic bites, and the code is explicit about it. Distributing N panes across a dimension of size S means each pane gets S/N and there is a remainder of $S \bmod N$ to hand out. The question is *to whom*, and tmux's answer — which ztmux now matches — is that the leftover columns go to the *leading* panes, one at a time, not dumped on the last pane. `layout_spread_cell` computes this, and its comment spells out the case that had to be fixed: for a size-80, two-pane split, each gets 39 and the remainder is 1, so the first pane becomes 40 and the second 39 — `40|39`, matching tmux, not `39|40`. Chapter 11 tells the debugging story; here the point is that a preset layout is not “roughly equal panes,” it is a *specific* distribution that has to match tmux column for column.

Manual splits and presets coexist because applying a preset is just replacing the tree, and manual splitting is just growing it. After you apply `tiled`, you can split a pane by hand, and the layout becomes a custom tree again (its name reported as the serialized string rather than a preset name); apply a preset again and the hand-work is replaced. This is exactly tmux's model, and it falls out naturally from representing every arrangement — preset or hand-built — as the same kind of `layout_cell` tree. There is no separate “preset mode”; there is one tree, built either way.

Resizing, zoom, and the layout string

Resizing a pane is the layout engine's most intricate operation because space is conserved: growing one pane must shrink another, and the change must respect every pane's minimum size and propagate correctly through the tree. `layout_resize_pane` (and `layout_resize_pane_to` for an absolute target) handle a single pane's resize; they walk to find the space to give or take, checking `layout_resize_check` for how much a subtree can yield before hitting minimums, and applying the change through `layout_resize_pane_grow/layout_resize_pane_shrink`. Whole-window resize — triggered when a client attaches at a new size or a terminal is dragged — is `layout_resize`, which scales the tree to the new dimensions, using `layout_resize_adjust` to hand out or reclaim space one column or row at a time in a round-robin so the panes stay proportional and no pane is starved.

That round-robin adjustment is the same leading-cell distribution as the even layouts, and it is tested the same way. `layout_resize_adjust` is the primitive that actually drives an 80-column two-pane window to `40|39` when the window grows into that size, and the layout code carries a test (`spread_via_resize_adjust_yields_40_39`) that drives the real primitive and asserts the first pane ends at 40 and the second at 39. Locking the resize arithmetic with a unit test, in addition to the parity suite's end-to-end check, is belt-and-suspenders on exactly the kind of off-by-one that is easy to reintroduce and invisible until a status-line format prints the wrong width.

Zoom is a lighter-weight operation that does not touch the layout tree at all, and that is by design. Zooming a pane (`resize-pane -Z`) temporarily makes one pane fill the whole window, hiding the others, and un-zooming restores the arrangement. Because this must be perfectly reversible, zoom does not rebuild the tree — it saves the window's layout and swaps in a temporary full-window one, restoring the saved tree on un-zoom (the `window_zoom/window_unzoom` functions in `window.rs`, with the zoom flag on the window). The layout tree is preserved untouched under the zoom, which is why un-zooming lands you back in exactly the arrangement you left, down to the column.

The *layout string* is how an arrangement is serialized — printed by `#{window_layout}`, accepted by `select-layout`, and saved by session-restore tools — and `src/ported/layout_custom.rs` implements it faithfully, checksum and all. `layout_dump` emits a string of the form `"{checksum:04x},{tree}"`, where the tree recursively encodes each cell as `"{sx}x{sy},{xoff},{yoff}"` followed by the pane id for a leaf or a bracketed list of children for a split (`[...]` for top-bottom, `{...}` for left-right). The four-hex-digit checksum is tmux's exact rotate-and-add over the string bytes:

```
csum = (csum >> 1) + ((csum & 1) << 15);
csum += *layout as u16;
```

`layout_parse` verifies that checksum before it will parse a layout string, so a corrupted or hand-mangled layout is rejected rather than misinterpreted. This is why a layout string copied from tmux pastes into ztmux and vice versa: the format, the recursion, and the checksum algorithm are byte-identical.

display-panes and pane navigation

`display-panes` is the small command that overlays each pane with a big number so you can jump to one by pressing its digit, and it is a good lens on how navigation and geometry meet. The overlay draws each pane's index (from the layout geometry — the number is centered in the pane's rectangle) and waits for a key; the digit you press selects the corresponding pane. The command lives in `src/ported/cmd_display_panes.rs`, and it leans on the same tree-to-rectangle mapping the renderer uses: knowing where to paint the “3” is knowing pane 3's cell offset and size. It is a tiny feature, but it shows that “which pane is where” is always answerable from the layout tree.

Directional navigation — `select-pane -L/-R/-U/-D` to move to the pane on a given side — is a geometric query the layout engine answers, not a stored adjacency list. Moving right from the current pane means finding the pane whose rectangle is nearest on the right edge, and the engine computes it from the cells' offsets and sizes. This is why pane navigation feels spatial: it *is* spatial, computed from the on-screen geometry each time, so it does the intuitive thing even in irregular custom layouts that no preset would produce.

The pane-movement commands that restructure the tree — `break-pane` (promote a pane into its own window), `join-pane` (pull a pane from another window into this one), `swap-pane` (exchange two panes' positions), `move-pane` — are all tree surgery, and they route through the layout functions this chapter has named. `break-pane` removes a leaf and rebuilds the source window's layout to close the gap while starting a new window around the removed pane; `join-pane` splits a destination pane and grafts the moved pane's leaf into the new split. Because these operate on the shared tree representation, a pane keeps its screen, its scrollbar, and its running program across the move — only its position in the geometry changes.

The whole layout subsystem, then, is a small pure-geometry engine sitting between the panes (which own content) and the terminal back end (which owns drawing). It knows nothing about what a pane contains or how it is drawn; it knows only rectangles, splits, and the arithmetic of distributing space among them faithfully. That separation is why it can be tested with plain arithmetic assertions and pinned with parity cases independent of any actual terminal — and why the one place it *had* to match tmux exactly, the rounding of leftover space, gets both a unit test and a parity case. With geometry settled, the book turns to the layer that lets a user drive all of this: the command language.

Reading a layout string by hand

Because the layout string is a plain, deterministic serialization, you can read one without any tooling, and doing so once demystifies the whole subsystem. Take an 80×24 window split once vertically into two stacked panes. `#{window_layout}` reports something like:

```
bf3c,80x24,0,0[80x12,0,0,0,80x11,0,13,1]
```

Read it left to right. `bf3c` is the four-hex-digit checksum — the rotate-and-add over the rest of the string that `layout_parse` recomputes and verifies before it will trust the layout. `80x24,0,0` is the root cell: 80 wide, 24

tall, at offset `(0,0)`. The `[...]` brackets mean a top-bottom split (a `LAYOUT_TOPBOTTOM` interior node); `{...}` would mean left-right. Inside are two children: `80x12,0,0,0` is the top pane — 80×12 at `(0,0)`, pane id `0` — and `80x11,0,13,1` is the bottom pane — 80×11 at `(0,13)`, pane id `1`. Notice the arithmetic that Chapter 7 insisted on: the two heights are 12 and 11, not 12 and 12, because the border row between them consumes row 12, and the bottom pane starts at `yoff` 13. Twelve plus one border plus eleven is twenty-four. The layout string is the geometry made legible, and the fact that a checksum guards it is why one copied out of `tmux` pastes into `ztmux` and lays out identically — same recursion, same offsets, same checksum algorithm, byte for byte.

Chapter 8 – The Command Language

tmux is driven by a command language — `new-window`, `bind-key`, `split-window`, and a hundred more — parsed by a real grammar. This chapter covers the command layer: the lalrpop grammar, argument parsing, and the one-command-per-file structure (`cmd_*.rs`) that mirrors tmux's own `cmd-*.c`.

The command grammar (`cmd_parse.lalrpop`) and the parser

Everything a user or a config file tells tmux to do is a command, and commands have real syntax — not just a name and arguments, but sequences separated by semicolons or newlines, brace-delimited blocks that group commands, `if/else` conditionals, the `%if/%elif/%else/%endif` config-file directives, and variable assignments. That syntax deserves a real parser, and ztmux uses one: `src/cmd_parse.lalrpop` is an LALRPOP grammar (LALRPOP is a Rust parser generator) that compiles to a generated `cmd_parse.rs` at build time. The grammar's productions map directly onto the language's structure — `Lines`, `Statements`, `Statement`, `Commands`, `Command`, `Arguments`, `Argument`, the `Condition/Elif` productions for `if/else`, the `Assignment` forms, and the `IfOpen/IfElif/IfElse/IfClose` productions for the `%if` family.

The grammar does not tokenize the raw bytes itself; it sits on top of a hand-written lexer (`crate::cmd_parse::lexer`, producing a `Tok` enum), declared to LALRPOP through its `extern` token block. This split — hand-written lexer, generated parser — is deliberate and it mirrors tmux, whose own command parser (`cmd-parse.y`) is a yacc grammar over a hand-written lexer. The tokens are what you would expect from the language: newline, semicolon, left and right brace, the `%hidden/%if/%elif/%endif/%else` keywords, and the value-carrying tokens `Format` (a `#{...}`-bearing string), `Token` (a bare word or quoted string), and `Equals` (an assignment). A tmux command line is genuinely a small program, and it is parsed like one.

The reason a full grammar is warranted rather than a `split(' ')` becomes obvious the moment you look at a real `.tmux.conf`. Brace blocks let a single `bind-key` map to a sequence of commands; the `if-shell` and `%if` conditionals branch on runtime or build-time conditions; assignments set the environment the following commands run in; and all of this nests. A naive splitter cannot handle a brace block that contains semicolons, or a conditional that spans lines, or a quoted argument with a space in it. The grammar handles all of them because it models the language's actual structure, and it does so the same way tmux does, which is what lets a config file written for tmux parse identically under ztmux.

The conditionals interact with parsing in a way worth calling out, because it is subtle and faithful. A `%if false` block is not merely evaluated to nothing at runtime — the parser, consulting the parse state's scope flag, *discards* the commands inside a false branch as it parses, so they never become command objects at all. This matters for config files that use `%if` to guard version-specific or platform-specific syntax: the guarded commands are only constructed when their condition holds, exactly as tmux does it. The grammar and the scope tracking together make the conditional a parse-time construct, not just a runtime one.

Arguments, flags, and target resolution (`arguments.rs`, `cmd_find.rs`)

Once a command's name and raw argument words are parsed, they have to be interpreted against that command's declared argument specification, and that is `arguments.rs`'s job. Each command declares

what flags it takes, which flags carry values, and how many positional arguments it expects (its `args_parse` spec, part of the command's entry). The argument parser (a faithful port of tmux's `args.c`) walks the words, separates flags from positionals, associates values with the flags that take them, and produces an `args` structure the command's implementation queries with helpers like `args_has`, `args_get`, and `args_count`. This is why `split-window -h -l 30` correctly means “the `-h` flag, and the `-l` flag with value 30, and no positional” — the argument spec drives the interpretation.

Target resolution is the other half of interpreting a command, and it is where tmux commands get their characteristic `-t` targets. Most commands act on a session, window, or pane, named by a target string like `mysession:2.1` (session `mysession`, window 2, pane 1), or `-` for the last, or `{marked}` for the marked pane, or a relative form. `src/ported/cmd_find.rs` resolves these: given a target string and the context (the current client, session, window, and pane), `cmd_find` walks the server's state tree to find the session, window, and pane the target names, applying all of tmux's defaulting and search rules. The result is a `cmd_find_state` — resolved pointers to the actual session, window, and pane — that the command then operates on.

The defaulting rules are more subtle than they look, and porting them faithfully is much of the work. When a command takes a `-t` target but none is given, the target defaults to the current pane of the current window of the current session *of the client that issued the command* — which is why `split-window` with no arguments splits the pane you are in. When a target names a window but not a pane, the pane defaults to the window's active pane. When it names a session but not a window, the window defaults to the session's current window. `cmd_find` encodes all of this, plus the special targets and the search-by-name-or-index logic, so that every command inherits the same target semantics — which is exactly why tmux's targeting feels consistent across a hundred commands: they all resolve targets through one function.

Flags, positionals, and targets together are the command's *interface*, and the faithful port of all three is what makes ztmux's commands drop-in compatible. A script that relies on `-t {last}` or on the defaulting of an omitted target or on a flag taking an optional value behaves identically, because the argument parser and `cmd_find` are tmux's, ported with their C citations and checked by the parity suite's `.sh` cases (Chapter 11), several of which do exactly this — run a command with a target, then query the result — precisely to pin the resolution behavior.

One command per file: the `cmd_*.rs` mirror of tmux's `cmd-*.c`

tmux organizes its commands one per file — `cmd-new-window.c`, `cmd-split-window.c`, `cmd-bind-key.c` — and ztmux mirrors that structure exactly: `src/ported/cmd_new_window.rs`, `src/ported/cmd_split_window.rs`, `src/ported/cmd_bind_key.rs`, and so on, sixty-odd files in all (the tree carries 65 `cmd_*.rs` files). The mapping is mechanical and the C-stem convention is even encoded in the tooling: the annotation script maps `cmd_kill_pane.rs` back to `cmd-kill-pane.c` by replacing underscores with hyphens. This one-to-one structure is not incidental tidiness; it is what makes the port auditable command by command — you can hold `cmd_split_window.rs` next to `cmd-split-window.c` and check the two directly, and the C citation on each function tells you exactly which C lines to compare.

Each command file defines two things: a `cmd_entry` — the command's metadata — and an `exec` function that runs it. The `cmd_entry` struct (`src/lib.rs`) carries the command's `name`, an optional `alias` (so `neww` works as `new-window`), the `args_parse` spec that drives argument parsing, a `usage` string, the source and target specifications, a set of flags, and the `exec` function pointer. The flags (`cmd_flag`) are the command's contract with the server: `CMD_STARTSERVER` means running this command should autostart a server if none exists, `CMD_READONLY` means it is safe in a read-only client, `CMD_AFTERHOOK` governs hook timing, and the `CMD_CLIENT_*` flags govern client-flag handling. These flags are how the command queue knows, generically, how to treat a command without special-casing each one.

All the entries are collected into one table, `CMD_TABLE`, an array of ninety command entries (`&CMD_ATTACH_SESSION_ENTRY` through `&CMD_WAIT_FOR_ENTRY`). Command lookup is `cmd_find(name)` — distinct from the *target-finding* `cmd_find` of the previous section — which scans the table, resolving aliases through `cmd_`

`get_alias`, and returns the matching entry or an error. When you type a command, this is the table consulted to find the code that implements it. Some files hold more than one related entry — `cmd_set_option.rs` carries `set-option`, `set-window-option`, and `set-hook`; `cmd_select_layout.rs` carries `next-layout` and `previous-layout` alongside `select-layout` — mirroring `tmux`, where related commands share a source file.

The `exec` function is where a command finally does its work, and its signature is uniform: it takes the parsed command and the command-queue item it is running under, and returns a `cmd_retval`. Inside, it queries its arguments, resolves its targets through `cmd_find`, and calls into the server's state — creating a window, splitting a pane, setting an option, binding a key. Because every command has the same shape — entry plus `exec`, resolved through the same table, run through the same queue — the command layer is uniform and extensible: adding a command is adding a `cmd_*.rs` file with an entry and an `exec`, and (for the port) checking it against the corresponding `cmd-*.c`. That uniformity is the whole reason `tmux` has a hundred commands that all behave consistently, and `ztmux` inherits it by inheriting the structure.

The target syntax, in detail

The `-t` target string deserves its own reference, because it is the notation every command uses to say *what* to act on, and `cmd_find.rs` resolves all of its forms. The full form is `session:window.pane` — `work:2.1` means session `work`, window index 2, pane 1 — but nearly every part is optional and defaults intelligently. `work:2` names a window (pane defaults to the window's active pane); `work:` names a session (window defaults to its current window); `:2.1` names a window and pane in the current session; a bare `1` in a pane context is a pane index. Beyond the numeric forms there is a vocabulary of special targets:

- `-` — the last (previously current) session, window, or pane, depending on context.
- `{last}`, `{next}`, `{previous}` — relative selectors.
- `{marked}` — the marked pane (`select-pane -m`), the implicit source for `join-pane` and `swap-pane`.
- `+/ -` as window indices — the next/previous window (`:+`, `:-`).
- `:=N` — an *exact* index match (used by the default number-key bindings, `select-window -t:=0`), distinct from a name-or-index search.
- `{top}`, `{bottom}`, `{left}`, `{right}`, `{up-of}`, `{down-of}`, and the directional pane selectors resolved geometrically from the layout tree (Chapter 7).

A target that names a session or window by *name* rather than index is searched (an exact match preferred, then a prefix, then an `fnmatch` pattern), which is why `attach -t wo` can find session `work`. All of this defaulting and searching lives in `cmd_find`, ported with its C citations, so that every one of the ninety commands inherits identical target semantics — which is exactly why `tmux`'s targeting feels consistent across the whole command surface: there is one resolver, and every command routes through it.

The command queue and how commands execute against server state

Commands do not run the instant they are parsed; they are *queued*, and the queue is what serializes every mutation of server state into a single, ordered stream. `src/ported/cmd_queue.rs` implements it: a parsed command list becomes a sequence of `cmdq_items` appended to a queue (`cmdq_append`, `cmdq_insert_after`), and the server's main loop pumps the queue by calling `cmdq_next`, which fires each item's command (`cmdq_fire_command`) or callback (`cmdq_fire_callback`) in turn. This is why order is deterministic and why a `bind-key` that runs three commands runs them in sequence against a consistent state — the queue guarantees one item completes before the next begins, and the server loop drains it (Chapter 3) every iteration.

The queue is not just a list; it carries *state* per item, and that state is what makes commands composable. `cmdq_new_state` builds a `cmdq_state` from a `cmd_find_state` (the resolved target context) and, for key-triggered commands, the key event that triggered it. That state flows to the command's `exec`, so a command

knows the client, session, window, and pane it was invoked against, and knows the key (if any) that fired it. This is how `send-keys` knows which pane to send to, how a bound command knows which client pressed the key, and how hooks know their context. The state travels with the item through the queue rather than being looked up afresh, which keeps a queued sequence coherent even as the world changes around it.

Commands can also *wait*, and the queue models that too, which is essential for the commands that are asynchronous. `if-shell` runs a shell command and branches on its exit status; `run-shell` runs one and optionally captures output; `confirm-before` and `command-prompt` wait for user input; `wait-for` blocks on a named channel. Each of these suspends its queue item until the asynchronous event completes, then resumes the queue. The `cmdq_running` state and the item's continuation machinery are how a queued command can pause without blocking the server's single event loop — the item yields, the loop keeps running, and the item is re-fired when its condition is met. This is the queue's most important property: it serializes state mutation *without* serializing the server, so a slow `if-shell` does not freeze every other client.

The command queue is, in a sense, the server's transaction log: every change to the session tree, every option set, every binding, every window created or pane split passes through it as an ordered item with a resolved context. That centralization is why the server can reason about “is any command still pending?” when deciding whether to exit (Chapter 3), why hooks can fire reliably after the commands that trigger them, and why the whole system stays consistent under concurrent clients. Parsing (the grammar), interpreting (arguments and targets), dispatching (the command table), and executing (the queue) are four clean stages, each a faithful port of its tmux counterpart, and together they are the command language that drives everything the previous chapters built. The next chapter covers the three systems that make that language *configurable*: formats, options, and keys.

A worked example: parsing one `bind-key` with a brace block

Take a single line from a real `.tmux.conf` and watch the four stages act on it:

```
bind-key -T prefix S { split-window -v ; select-layout even-vertical }
```

The lexer (`crate::cmd_parse::lexer`) turns the bytes into tokens: the word `bind-key`, the flag word `-T`, the word `prefix`, the word `S`, a left brace, then the inner tokens, and a right brace. The generated LALRPOP parser reduces those tokens through its productions: the outer `Command` is `bind-key` with arguments `-T prefix S` and a final argument that is itself a `Commands` group — the brace block containing two `Commands` separated by the `;` token. Crucially, the brace block is parsed into a *command list object*, not a string; the semicolon inside it is a command separator, not a character in an argument, which is precisely the case a naive `split(' ')` cannot handle.

Stage three, dispatch: `bind-key`'s entry is found in `CMD_TABLE` (via `cmd_find` on the name, resolving the alias `bind` to the same entry). Stage two, arguments: `arguments.rs` interprets `-T prefix` as the table flag with value `prefix`, `S` as the first positional (the key, parsed by `key_string.rs` into a `key_code`), and the brace block as the command list to bind. Stage four, execution: the queued item fires `bind-key`'s `exec`, which calls `key_bindings_add` to insert a `key_binding` into the `prefix` key table mapping `S` to that two-command list. Press the prefix then `S` later and the *same* command-list object is dropped onto the command queue and run in order — split, then re-layout. One line, four faithful stages, and the binding's value is a parsed program rather than a string, which is why tmux's most elaborate default bindings are possible at all.

Five entry points, one queue

The queue's value comes from being the *only* road into server state, and that is easier to appreciate by enumerating the roads that lead onto it. There are five, they look completely different from the outside, and they converge before anything is executed.

A key binding. A client presses a key, the server looks it up in the active key table (Chapter 9), and the binding's command list is dropped onto the queue with a state built from the pressing client's resolved context. This is the path most commands take in practice and the one Chapter 22 traces keystroke by keystroke.

A command from a client's command line. Running `zterm new-window` from a shell starts a client that connects, sends `MSG_COMMAND` (Chapter 17) rather than attaching, and exits when the command completes. Server-side, the command line is parsed and queued exactly as a binding's would be, with the state built from the requesting client. The only difference is what happens at the end: the item's completion triggers the reply that lets the one-shot client exit with the right status.

The config file. `cfg.rs` reads `.termux.conf` line by line and queues each command (Chapter 9), which is why a config file is not a special declarative format at all — it is a command script, and anything you can type interactively you can put in it, in the same syntax with the same semantics.

A hook. A notification fires (Chapter 21), a hook is bound to it, and the hook's command list goes onto the queue with a state derived from the event's context — the window that was renamed, the pane that died. Because hooks queue rather than execute inline, a hook cannot re-enter the operation that triggered it, which is what keeps a hook on a pane-changed event from recursing when it changes a pane.

Control mode. A control-mode client (Chapter 17) sends command lines over its socket and they are queued identically. A program driving `zterm` programmatically and a user pressing keys are, past this point, indistinguishable.

The convergence is the design. Five very different-looking sources produce the same kind of item carrying the same kind of resolved state, executed by the same loop in a single defined order, so there is exactly one place where server state changes and exactly one ordering to reason about. This is what makes the reactive parts of the system safe to compose: a hook that runs a command that fires a notification that triggers another hook is a sequence of queued items, not a recursive descent through the command implementations. It is also why concurrency between clients is a non-issue despite there being no locks anywhere (Chapter 35) — two clients pressing bindings at the same moment produce two items, and the queue orders them.

The practical consequence for anyone writing configuration is that command ordering is predictable in the strongest sense. Commands within one list run in order. A list queued later runs entirely after a list queued earlier. An asynchronous command (`if-shell`, `run-shell`, `confirm-before`) suspends its own item and everything queued behind it waits, which is the behaviour you want — a conditional that gates two subsequent commands actually gates them — and which a system that ran commands inline could not provide without blocking the whole server.

Chapter 9 – Formats, Config, and Keys

The glue that makes tmux configurable: the `#{...}` format language, the config file, key tables and bindings, and the options system. This chapter covers how a `.tmux.conf` becomes running behavior.

The `#{...}` format language and its expansions

The most distinctive thing about tmux’s configurability is that it has a small *expression language* embedded in strings — the `#{...}` format syntax that appears in status-line definitions, `display-message` arguments, `if-shell` conditions, and window and pane titles. `src/ported/format.rs` implements it, and it is the single largest module in the tree (over two hundred kilobytes, with more than two hundred C citations), because the format language is genuinely large: it does variable substitution, arithmetic, string operations, conditionals, comparisons, loops over sessions/windows/panes/clients, time formatting, and shell-command interpolation, all inside `#{...}`. A format string is expanded by `format_expand` (with a loop guard, `FORMAT_LOOP_LIMIT = 100`, so a self-referential format cannot recurse forever), and each `#{...}` is handled by `format_replace`.

The power comes from *modifiers* — single characters or operators after a `#{` that transform the expansion — and ztmux ports the full set. The simplest is a bare variable: `#{pane_current_command}` expands to the pane’s foreground command. The modifiers layer transforms on top: `#{b:pane_current_path}` is the basename, `#{d:pane_current_path}` the dirname, `#{t>window_activity}` a formatted time, `#{=10:pane_title}` a width-limited version, `#{s/foo/bar/:...}` a substitution, `#{n:...}` a length, `#{q:...}` a shell-quote. There are comparison and boolean operators (`#{==:a,b}`, `#{&&:...}`, `#{||:...}`), an arithmetic operator (`#{e|+|:2,3}` adds), a conditional (`#{?condition,then,else}`), and loop modifiers (`S`, `W`, `P`, `L`) that expand a sub-format once per session, window, pane, or client. Each is a flag in the `format_modifiers` bitset, parsed in the big modifier `match` and applied during replacement.

The literal modifier `l` is small but instructive, because it is where a real bug lived. `#{l:some text}` means “expand to this literal string, unescaped” — it is how you put characters into a format that would otherwise be interpreted. Its handler calls `format_unescape`, which walks the string turning `#`-escapes back into literal characters. The C original is a `for (; s != end; s++)` loop whose `s++` runs on every iteration, including the ones that `continue`; an earlier version of the Rust port translated it to a `while` loop that forgot to advance `s` on the normal branch, so the loop never progressed — it spun forever, writing past the `xmalloc(strlen + 1)` buffer it had allocated, and crashed the server. Chapter 11 tells how the parity suite caught it; the fix advances `s` explicitly on every path, matching the C’s per-iteration increment. The `format.rs` source carries the whole explanation as a comment above `format_unescape`, citing `vendor/tmux/format.c:4281`, so the trap is documented where the next person will see it.

The format language is also where the multiplexer’s whole state becomes readable as data, and that is worth appreciating. Nearly every field of a session, window, pane, or client is exposed as a format variable — `session_name`, `window_index`, `pane_width`, `client_tty`, `pane_current_command`, and hundreds more — and the format engine resolves them from the live server state at expansion time. This is why the status line can show anything, why `list-panes -F '#{pane_pid}'` can extract a specific field, and why the parity suite (Chapter 11) is *mostly* format cases: a `.fmt` case is a single format string, expanded by `display-message -p` against a fixed session and diffed byte-for-byte against tmux, and because formats read so much of the state, a format that matches is strong evidence the underlying state matches too. The format language is simultaneously tmux’s configuration expression language and its introspection interface, and ztmux ports both roles.

Options: server, session, window, pane scopes

Configuration values live in the *options* system, and its defining feature is that options are *scoped* and *inherited*. `src/ported/options.rs` (with the big default table in `src/ported/options_table.rs`) implements four scopes, defined as bit flags in `src/lib.rs`: `OPTIONS_TABLE_SERVER` (server-wide, like `escape-time`), `OPTIONS_TABLE_SESSION` (per-session, like `status` and `history-limit`), `OPTIONS_TABLE_WINDOW` (per-window, like `window-status-format`), and `OPTIONS_TABLE_PANE` (per-pane, like `allow-rename`). A given option belongs to one or more scopes, and a lookup for, say, a window option checks the window's own options, then falls back to the parent scope's value, then to the built-in default — the inheritance chain that lets you set an option globally and override it per window or per pane.

Each option has a *type*, and the type system (`options_table_type`) is what makes options more than untyped strings. An option is a string, a number, a key, a color, a flag (boolean), a choice (one of an enumerated set), or a command, and some are arrays. The type drives parsing and validation: a number option carries a minimum and maximum and rejects out-of-range values; a choice option carries its list of valid strings; a color option parses `#RRGGBB` and named colors through the same `colour.rs` machinery the grid uses. The default table entry for `history-limit`, for instance, declares it an `OPTIONS_TABLE_NUMBER` in the session scope with a minimum of 0, a maximum of `i32::MAX`, and a default of `2000`, with the help text noting the change applies only to new panes. The whole options table is this kind of declarative entry, ported from tmux's `options-table.c`.

The functions that read options are the ones the rest of the code calls constantly, and they encode the scope-and-inheritance logic. `options_get_only` fetches an option set directly on a given scope; `options_get` walks the inheritance chain to the parent; `options_get_string`, `options_get_number`, and their typed cousins return the value in the form the caller wants; `options_default` produces the built-in default from the table. Setting goes through `options_set_string`, `options_set_number`, and `options_parse` (which parses a value against the option's type). `set-option`, `set-window-option`, and `show-options` are the user-facing commands, and they route through exactly these functions — which is why `set -g` (global) versus `set` (current scope) versus `setw` (window) all behave as tmux documents: they are choosing which scope's options tree to write.

Scope resolution — deciding *which* scope a bare `set-option foo bar` means — is itself a ported function, `options_scope_from_name`, because tmux's rules are specific: an option's name determines its scope (the table says whether `foo` is a server, session, window, or pane option), and the command's flags (`-g`, `-w`, `-p`, `-s`) can further constrain it. Some options live in a combined window-and-pane scope. Getting this right matters because a config file that sets an option in the wrong scope silently does nothing useful, and tmux users have precise expectations here built over years. `options.rs` ports the resolution with its C citation (`vendor/tmux/options.c:863`), so the scope a given `set` lands in is exactly tmux's.

A worked inheritance example

Walk one option lookup to see the inheritance chain in action. Suppose `history-limit` is a session option (it is), and consider a pane in window 2 of session `work`. Nothing has set `history-limit` on that window or session, but the global session tree has it at the default `2000`. When a new pane is spawned and asks “how much scrollback should I keep?”, the lookup is `options_get`, which walks the chain: check the pane's own options — not set; fall back to the window's options — not set; fall back to the session's options — not set; fall back to the *global* session options — found, `2000`; and if even that were absent, `options_default` would produce the built-in table default. The pane gets `2000`.

Now `set -g history-limit 50000` writes the *global* session tree, so every future pane inherits 50,000. But `setw -t work:2 history-limit 100000...` would fail, because `history-limit` is a session option, not a window option — `options_scope_from_name` rejects the wrong-scope set, exactly as tmux does, rather than silently storing a value nothing reads. And `set -t work history-limit 100000` sets it on the `work` session specifically, so panes in `work` inherit 100,000 while panes in other sessions still inherit the global 50,000. The same walk governs every option, which is why “set it globally, override it per session, override that per window or pane” is a single consistent mental model across the whole configuration surface — and

why getting the *order* of the fallback chain byte-identical to tmux matters: a config that relies on a per-session override winning over a global default depends on the chain resolving in exactly this sequence.

Option arrays, user options, and the subscript syntax

Not every option is a single scalar. A handful are *arrays* — indexed collections of values — and they carry some of tmux's most-used configuration: `terminal-overrides` and `terminal-features` (Chapter 10), `command-alias` (the table that lets `set-option` map custom short names to command strings), `update-environment` (the variable list of the next section's environment machinery), and the several `status-format` slots. The table marks these `OPTIONS_TABLE_IS_ARRAY`, and `src/ported/options.rs` stores an array option not as a string but as an `options_array` — a red-black tree of `options_array_items` keyed by integer index, so an array can be sparse and can be addressed one slot at a time. `options_array_get`, `options_array_set`, `options_array_assign`, and `options_array_clear` are the ported operators over it.

The user-facing syntax mirrors that structure exactly, and porting it faithfully is what lets a real `.tmux.conf` full of array manipulation load unchanged. A bare `set-option` on an array replaces the whole thing; `set -a` (append) adds an element after the last, which is why the canonical truecolor incantation `set -ag terminal-overrides " ,xterm-256color:Tc"` adds an override rather than clobbering the existing list. The subscript form `foo[3]` addresses one index — `set -g command-alias[100] 'split-pane=split-window'` writes slot 100 specifically — and `show-options -A` expands an array back into its indexed `foo[0]` ... `foo[n]` lines so the output round-trips into commands that would recreate it. Each of these is a specific behavior a config depends on, and each routes through the same `options_array_*` functions the port carries from `vendor/tmux/options.c`.

User options are the other departure from the scalar table, and they are what makes tmux extensible without a plugin API. Any option whose name begins with `@` — `@my-plugin-state`, `@theme`, `@continuum-restore` — is a user option: it has no entry in `options_table.rs`, it can hold any string, and it is stored and read like any other option through the same scoped tree. `options.rs` special-cases the `@` prefix so that setting or getting an unknown name is not the error it would be for a misspelled built-in but a legitimate write to a user slot. This is the mechanism the entire tmux plugin ecosystem — `tpm`, `tmux-resurrect`, status-line themes — stores its state in, and because ztmux ports the `@`-prefix rule and the scoped storage exactly, those plugins' `set -g @foo ...` lines behave identically: the value is stored on the chosen scope, inherited down the same chain (the worked example above), and read back by whatever `#{@foo}` format or `if-shell` consults it. A superset that broke user options would break the plugin ecosystem it most wants to inherit; keeping them byte-faithful is part of the parity promise.

Key tables and bind-key (cmd_bind_key.rs)

Every keystroke you send tmux is looked up in a *key table*, and the key-table system (`src/ported/key_bindings.rs`) is how `bind-key` turns a key into a command. A `key_table` is a named, red-black tree of `key_bindings`, each mapping a `key_code` (a key plus its modifiers, and possibly a repeat flag) to a command list. The default tables are the familiar tmux ones: `prefix` (the table entered after the prefix key, where most bindings live), `root` (bindings that fire without a prefix), and the copy-mode tables (`copy-mode` and `copy-mode-vi`). When you press a key, the server consults the active table — normally `root` first, and `prefix` after the prefix key has been seen — and if the key is bound, runs its command list through the command queue; if not, the key is forwarded to the active pane.

`bind-key` (`src/ported/cmd_bind_key.rs`) is the command that populates these tables, and its implementation is a thin wrapper over `key_bindings_add`: it parses the target table (from `-T`, defaulting to `prefix`), the key (parsed from its string form by `src/ported/key_string.rs`, which turns `C-b` or `M-Left` into a `key_code`), an optional `-r` repeat flag and `-N` note, and the command list to bind, then adds the binding to the named table. `unbind-key` (`cmd_unbind_key.rs`) removes one via `key_bindings_remove`, and `list-keys` (`cmd_list_keys.rs`) walks the tables to print them. Because a binding's value is a full command list — parsed

by the Chapter 8 grammar — a single key can be bound to a brace block of many commands, which is how tmux's more elaborate default bindings work.

The `key_bindings` module ports the whole table lifecycle, not just add and remove. `key_bindings_init` builds the default tables at server start (Chapter 3); `key_bindings_get_table` finds or creates a table by name; `key_bindings_get/key_bindings_get_default` look up a key's binding (falling back to a default binding if the user has not overridden it); `key_bindings_first/_next` iterate a table's bindings for `list-keys`; and `key_bindings_reset` restores a key to its default. The reference counting on tables (`key_bindings_unref_table`) exists because a table can be referenced while it is being modified, and freeing it early would be a use-after-free — the sort of memory-lifetime concern the port handles with the same care the C does, because it is porting the C's exact ownership model.

The result is that key handling is table-driven and fully faithful: the same `C-b` prefix, the same default bindings, the same `bind-key -T copy-mode-vi` extensions, the same repeat behavior for pane resizing. A user's `.tmux.conf` full of `bind` and `unbind` commands produces the same key tables under ztmux as under tmux, because the tables, the key parsing, and the lookup are ported line for line. And because a binding's value is a command list, the key system composes with everything in Chapter 8 — a key is just a queued command with a trigger, which is exactly how the command queue's per-item state (which carries the triggering key event) is designed to treat it.

Loading config (`cfg.rs`) and `source-file`

A config file is just a sequence of commands, and loading one is running them — which is why the config loader (`src/ported/cfg.rs`) is thin: it does not have its own mini-language, it hands the file to the command parser (Chapter 8) and queues the result. `start_cfg` kicks off configuration at server start, and `load_cfg/load_cfg_from_buffer` do the work: read the file (or buffer), parse it with the command grammar, and append the resulting commands to the command queue to run in order against the freshly started server. The `source-file` command (`cmd_source_file.rs`) is the same machinery exposed to the user — it loads and runs another config file at runtime — which is why a `.tmux.conf` can `source-file` a modular set of sub-configs and they compose exactly as if inlined.

Because config loading is command execution, everything in the command language is available in a config file: brace blocks, `%if/%endif` conditionals, `if-shell`, assignments, and of course `set-option` and `bind-key`. This is the unifying insight of tmux's design that ztmux inherits wholesale — there is no separate “configuration format,” there is only the command language, and a config file is a batch of it. It is why the same line works typed at the command prompt, passed to `tmux` on the command line, sourced from a file, or bound to a key: all four paths parse the same grammar and run through the same queue.

Errors during config loading are collected rather than fatal, and the collection mechanism is itself a nice example of the port's fidelity. As commands in a config file run, any that produce an error add a *cause* — a message describing what went wrong and where — through `cfg_add_cause`, and after loading, `cfg_print-causes/cfg_show-causes` surface them (a config error shows up as a message you can read, not a silent failure or a crash). This is why a typo in your `.tmux.conf` leaves tmux running with an error message rather than failing to start, and ztmux reproduces the behavior because it ports the cause-collection machinery, not just the happy path.

Config, options, formats, and keys are four faces of one idea: tmux is configured by running commands that set typed, scoped options and populate key tables, with a format language threaded through it all for computed values, and a config file is just a script in that same command language. ztmux ports all four faithfully — the format engine with its modifiers and its hard-won `format_unescape` fix, the scoped options table with its typed defaults, the red-black key tables with their reference counting, and the config loader with its error collection — so that the entire configurability surface of tmux carries over unchanged. The last piece of the faithful core is the layer that turns all of this into actual output on a real terminal: the terminal back end.

Chapter 10 – The Terminal Back End

The other edge of the program: how ztmux drives the outside terminal — terminfo capabilities, the output optimizer, truecolor, and the features it negotiates with the host terminal.

Terminfo and capability negotiation

At the far edge of the server, past the grid and past the layout, is the problem of actually making a real terminal show what tmux computed — and real terminals differ, so tmux (and ztmux) drive them through *terminfo*, the database of terminal capabilities. `src/ported/tty_term.rs` is the terminfo layer, and it reads a terminal's capabilities through the `terminfo-lean` crate (the one non-libc, non-parser dependency doing real work here), building a `tty_term` that answers questions like “how does this terminal move the cursor?” or “does it support truecolor?” The client reads the capability list on its side (`tty_term_read_list`, Chapter 4) and ships it to the server in the identify handshake; the server builds the `tty_term` from it with `tty_term_create`. From then on, every drawing operation asks the `tty_term` for the right escape sequence rather than hard-coding one.

Capabilities are named — `TTYC_CUP` for cursor-position, `TTYC_SMCUP`/`TTYC_RMCUP` for entering and leaving the alternate screen, `TTYC_CLEAR`, `TTYC_SGR0`, and so on — and the back end emits them by name through `tty_term`, which knows each terminal's actual byte sequence. This indirection is why tmux runs correctly across the enormous variety of terminals that have existed: it never assumes a sequence, it looks one up. ztmux ports the capability names, the lookup, and the emission faithfully, so a terminal that works under tmux works under ztmux for the same reason — the capability database says how to talk to it.

Terminfo alone is not enough, because terminal databases are frequently wrong or incomplete, so tmux layers *overrides* on top, and ztmux ports the override machinery. `tty_term_apply` and `tty_term_apply_overrides` apply the `terminal-overrides` and `terminal-features` options — user- or default-supplied corrections that add, remove, or fix capabilities the terminfo entry gets wrong. This is how tmux knows that a terminal advertising a certain `TERM` actually supports truecolor even when its terminfo entry omits it, or how a user forces a capability their terminal has but its database lacks. The override system is a faithful port because it is load-bearing: without it, a large fraction of real terminals would render subtly wrong.

Feature negotiation goes a step further than static terminfo, and it is where the back end talks *back and forth* with the terminal. Some capabilities cannot be known from the database and must be probed at runtime — by sending a query escape sequence and reading the reply. tmux probes for features like truecolor, synchronized output, and the terminal's actual identity during startup, using the Device Attributes and other reports the input parser (Chapter 6) knows how to send and receive. The `tty_features.rs` module tracks the resulting feature set. This runtime negotiation is why tmux increasingly “just knows” what a modern terminal can do without configuration, and porting it keeps ztmux current with that behavior.

The output path and its optimizations

Driving a terminal is bottlenecked by bytes on the wire — an escape sequence to move the cursor and paint a cell is far larger than the cell itself — so the output path is where tmux spends real effort optimizing, and `src/ported/tty.rs` ports that effort. The core idea is *differential redraw*: the server does not repaint the whole screen every frame, it computes what changed since the last draw and emits only the sequences to update those cells. The grid's dirty-tracking (Chapter 5) feeds this: cells the screen-write layer marked changed

are the only cells the tty layer redraws, and the `tty_cmd_*` family of functions turns high-level changes (this line scrolled, these cells cleared, this region changed) into minimal terminal sequences.

The output is buffered through the event loop, and the buffering is where flow control lives. The tty's output goes into an `evbuffer` (`tty.out`), drained to the client's descriptor by the `tty_write_callback` when the descriptor is writable. When a program floods a pane faster than the terminal can consume — a `yes` loop, a giant `cat` — the output buffer grows, and tmux applies flow control rather than growing it without bound: when the buffered bytes exceed `TTY_BLOCK_START` (computed as $1 + (sx*sy)*8$, a multiple of a screenful), the tty enters a blocked state, stops producing more output, and arms a timer (`TTY_BLOCK_INTERVAL`, 100 milliseconds) to resume. This is what keeps a runaway program in one pane from making the whole terminal unresponsive — the back end throttles it. ztmux ports the exact thresholds because flow-control behavior is observable and the parity discipline extends to it.

The individual output helpers show the optimization mindset at the small scale. `tty_putc`, `tty_puts`, `tty_putn`, and `tty_add` append to the output buffer, and they mirror everything they emit to an optional output log (`tty_create_log`, controlled by the logging machinery) so a developer can capture the exact byte stream tmux sent — a faithful port of tmux's own debugging aid. Cursor movement is optimized: rather than always emitting an absolute cursor-position sequence, `tty_cursor` emits the shortest move from the current position (a single “cursor right” when the target is one cell over, a carriage return when it is column zero), because the current position is tracked and the cheapest sequence is chosen. These micro-optimizations sound minor and they are not — across a full-screen redraw of a busy editor they are the difference between a responsive terminal and a laggy one.

The whole output path is also where the `charset` and `attribute` state from the grid gets turned back into escape sequences, closing the loop the input parser opened. When the back end draws a cell, `tty_attributes` emits the SGR sequences to set the cell's color and attributes (only the changes from the previous cell, again minimizing bytes), and `tty_colours` handles the color-specific part, choosing between the 16-color, 256-color, and truecolor forms based on what the cell holds and what the terminal supports. The ACS charset flag on a cell (Chapter 6) causes the back end to switch the terminal into line-drawing mode via the `TTYC_SMACS`/`TTYC_RMACS` capabilities, mapping the box-drawing cells to the terminal's line glyphs. The grid stores tagged, abstract style; the back end renders it to concrete, minimal, terminal-specific bytes.

Truecolor and terminal features

Color is the feature that most visibly separates a modern terminal from a legacy one, and the back end handles all three color depths a cell can carry (Chapter 5) against whatever the terminal actually supports. If a cell holds a 24-bit RGB color and the terminal supports truecolor (known from terminfo, an override, or runtime probing), the back end emits the `38;2;r;g;b` / `48;2;r;g;b` direct-color SGR sequences. If the terminal supports only 256 colors, tmux can be told (or can decide) to approximate, and if only 16, it falls back further. The point is that the *grid* always stores the true color the program requested, and the *back end* renders it as faithfully as the target terminal allows — the fidelity is preserved as far up the pipeline as possible and only degraded at the last step, in the terminal that cannot do better.

Beyond color, the back end negotiates a set of modern terminal features that tmux has grown to support, and `tty_features.rs` is the registry. *Synchronized output* — the private mode that lets an application say “buffer this whole update and show it atomically” — is one; when the host terminal supports it, tmux can wrap a complex redraw in it so the user never sees a half-drawn frame. Others include extended underline styles (the curly, dotted, dashed underlines the grid can store), the overline attribute, bracketed paste, focus reporting, and the various mouse-reporting modes. Each feature is a capability the back end checks before using, so tmux uses a feature exactly when the terminal can handle it and not otherwise — the same conservative discipline as the escape-sequence support surface.

Sixel graphics are the one feature here that is fully implemented in ztmux and worth singling out, because the Cargo manifest calls it out specifically as an exception to its “options exist but aren't wired up yet” caveat. `src/ported/image.rs` and `src/ported/image_sixel.rs` implement sixel image support — the protocol for drawing bitmap graphics in a terminal — and its presence changes an observable behavior noted back

in Chapter 6: the Device Attributes reply advertises sixel (`\x1b[?1;2;4c` instead of `\x1b[?1;2c`) when the feature is compiled in. This is the sort of end-to-end consistency the port aims for: a feature is not “supported” until every place that should reflect it does, including the capability it advertises to the programs running inside panes.

The general shape of feature handling is negotiation, not assumption, and it runs both directions. Outward, tmux advertises to pane programs (through DA replies and terminfo) what *it* supports, so a program inside a pane knows it can send truecolor or sixel. Inward, tmux probes and reads the host terminal's terminfo to know what the *outside* terminal supports, so it knows what it can safely emit. tmux sits in the middle, translating between what pane programs want and what the real terminal can do — which is a large part of why running a program inside tmux can work even when the outside terminal is more limited than the program expects: tmux negotiates the gap. ztmux ports this middle-man negotiation faithfully, feature by feature.

A worked example: one RGB color, three terminals

Follow a single 24-bit color out to three different terminals and the “store true, degrade late” principle becomes concrete. A program prints text in `{e}`-computed RGB (220, 50, 47) — a specific red. The grid stores it exactly: the cell is promoted to extended (Chapter 5), its `fg` holding (220<<16)|(50<<8)|47 or-ed with `COLOUR_FLAG_RGB`. The true color is now in the grid regardless of who is looking.

Client 1 is a modern truecolor terminal. `tty_colours` checks the cell's tag and the terminal's capabilities (terminfo, an override, or runtime probing say truecolor is supported) and emits the direct-color SGR `\x1b[38;2;220;50;47m` — the exact red. Client 2 is a 256-color terminal: no truecolor capability, so tmux maps the RGB to the nearest entry in the 256-color cube and emits `\x1b[38;5;NNNm` — a close approximation, the best that terminal can show. Client 3 is a legacy 16-color terminal: tmux falls back further, to the nearest of the 30–37/90–97 ANSI codes. All three clients can be attached to the *same session* at once (Chapter 4), each rendered to its own capabilities from the one shared grid, because the tty back end is instantiated per client while the grid is singular. The fidelity is preserved as far up the pipeline as possible — the program's exact color lives in the grid untouched — and degraded only at the very last step, in the terminal that cannot do better. A port that stored the *approximated* color in the grid would have thrown away information the truecolor client could have used; storing the true value and degrading per-client at output is the faithful design, and ztmux ports it.

Where the back end meets the client render surface

The terminal back end and the client (Chapter 4) meet at a specific seam, and naming it precisely ties the whole faithful core together. The server owns the grid, computes the differential redraw, and produces the terminal byte stream through the tty layer — but the descriptor those bytes are written to is the *client's* terminal, passed to the server across the socket during the identify handshake as duplicated file descriptors. So the back end in `tty.rs` is driving hardware it does not own: it holds the client's `stdout` fd (dup'd over the socket) and the client's terminfo capabilities, and it draws the server's grid onto the client's screen. The client, meanwhile, does no drawing at all — it lent its terminal and it relays keys. The tty layer is the server-side end of that loan.

This seam is why the same session can render correctly to two different clients at once (Chapter 4's multi-client story). The server maintains one grid, but it can run the tty back end once per attached client, each with that client's own terminfo and output buffer, drawing the same grid to each client's terminal in whatever sequences that specific terminal needs. A truecolor terminal and a 256-color terminal attached to the same session both show the session correctly, each rendered to its own capabilities, because the color and the capability negotiation are per-client on the output side while the grid is shared on the state side. The back end is instantiated per client; the grid is singular.

The seam also explains the resize dance one more time, from the output side. When a client sends `MSG_RESIZE`, the server updates the session's size, the layout re-tiles (Chapter 7), the grids resize and reflow (Chapter 5), and then the back end redraws the new arrangement to that client's terminal. Every step is

server-side except the trigger (the client's `SIGWINCH`) and the destination (the client's terminal). The client participates only at the two ends; everything between — the layout arithmetic, the grid reflow, the differential redraw, the capability-correct output — is the server driving the client's borrowed terminal through the tty layer. This is the architecture in one sentence: the client lends a terminal and relays keys, and the server does everything else, meeting the terminal again only at the tty back end.

With the back end, the faithful core is complete: a socket and an event loop (Chapter 3), a thin client that lends its terminal (Chapter 4), a grid and screen model (Chapter 5), a VT parser feeding it (Chapter 6), a layout tiling the panes (Chapter 7), a command language driving it (Chapter 8), and formats, options, and keys configuring it (Chapter 9), all rendered out through terminfo (Chapter 10). Every one of those is a faithful port of a specific tmux C module, cited line by line and checked against the C. The next two chapters are about *how* that fidelity is proven and defended — the parity suite that measures it and the anti-drift gate that protects it — because in a port, the proof of correctness is as much a part of the work as the code.

A differential redraw, cell by cell

To make the differential-redraw claim concrete, follow what happens when a single cell changes. Suppose the clock in a status line ticks from `:07` to `:08`. The status code re-expands the format and writes the new characters through the screen-write layer, which compares each new cell against what the grid already holds and marks *only the differing cells* dirty — here, the one cell holding the `7/8` digit. When the redraw runs, it does not repaint the status line, let alone the screen; it asks the tty back end to update that one cell. The back end then does its own minimization: `tty_cursor` computes the shortest way to move the terminal's cursor from wherever it currently is to that cell — a bare carriage return if the target is column zero, a single “cursor right” if it is one column over, an absolute `CUP` only if nothing cheaper reaches it — and `tty_attributes` emits SGR only if the cell's color or attributes differ from the last cell drawn. The net wire traffic for a clock tick is a handful of bytes: a short cursor move, maybe an SGR, one digit.

Multiply that discipline across a full-screen editor scrolling one line and the payoff is obvious. A naive terminal would repaint the whole screen — thousands of cells, each with its own positioning and color sequences. tmux emits a scroll region command and repaints only the newly exposed line, and where a real scroll is not possible it repaints only the cells that actually changed. The `tty_cmd_*` family (`tty_cmd_scrollup`, `tty_cmd_clearline`, `tty_cmd_cell`, and the rest) is the vocabulary of these minimal operations, each turning a high-level change into the fewest terminal bytes. This is why a tmux session over a slow SSH link stays responsive: the back end is relentlessly stingy with the wire, and ztmux ports that stinginess exactly because the byte stream it produces is itself observable and, in principle, parity-checkable.

Chapter 11 – The Parity Suite: Correctness as Method

A port’s central claim is fidelity, and a claim of fidelity is worthless unless it is measured. This chapter is about how ztmux measures it: a parity suite that runs the real tmux and ztmux on identical inputs and diffs their output byte-for-byte, treated not as a test folder but as the project’s definition of correct.

Correct means the vendored tmux, byte-for-byte

The first decision the parity suite makes is the most important one, and it is a decision about *what correct means*. For ztmux, correct is not “matches the tmux man page” or “behaves reasonably” or “matches whatever tmux the developer has installed” — correct is “produces the exact same bytes as the specific tmux under `vendor/tmux`,” which is the `next-3.7` development version pinned in the repository. The parity runner (`parity/run_parity.sh`) builds and uses `vendor/tmux/tmux` as its reference, not the system tmux, and its header comment says why in plain terms: the truth being measured against is *that exact tmux*, because version drift causes false diffs — Ubuntu’s tmux 3.4 would report divergences that are really just version differences, not port bugs. Correctness is defined against a committed, versioned reference, so it can never be ambiguous.

Byte-for-byte is the other half of the definition, and it is deliberately unforgiving. For each case, the runner captures the reference tmux’s combined stdout and stderr and ztmux’s combined stdout and stderr, and compares them with `cmp` — a raw byte comparison, not a normalized or fuzzy one. A single differing byte fails the case. This is much stricter than a typical test suite, which asserts on parsed or normalized output, and the strictness is the point: a multiplexer that is “almost” byte-compatible with tmux is one whose status-line format prints a subtly wrong width, or whose layout string has an off-by-one in a checksum, and those are exactly the divergences that break real configs and scripts. Only byte-for-byte identity guarantees the compatibility the superset ambition (Chapter 1) rests on.

The measurement has a three-act history, and telling it straight is part of the honesty. Act one: the suite reached **122 of 122 cases passing**, byte-for-byte identical to the vendored tmux on both Linux and macOS — 100.00 percent green across two platforms, and the strength on which the README first called ztmux a daily driver. Act two, once that first set was exhausted, was to *grow the suite* aggressively — from 122 cases to **1115** — pushing into arithmetic, comparisons and regex, string operations, conditionals, padding, loop modifiers, and multi-command `.sh` scenarios that the first 122 never touched. Against that much larger and more adversarial target the suite did what a good suite is supposed to do: it *broke*, surfacing about twenty deterministic, individually pinned port gaps. Act three was to close them — each of those twenty was traced to its cause in the port and fixed — so the suite now stands at **1115 of 1115 passing (100%)**, byte-for-byte identical to the vendored tmux on both platforms. The parity summary JSON records the count (`total 1115, passed 1115, failed 0`) and the reference (`tmux next-3.7`) with a generation timestamp, so the claim is an artifact, not a boast. The important lesson across the three acts is philosophical: a green 122/122 could mean “the suite is too small to fail,” so the project deliberately expanded the suite until it *did* fail, fixed every failure it found, and then treated each failure as a work item. A parity number that never drops is a parity number that has stopped measuring anything.

Defining correctness this way has a profound effect on how the port is developed, because it makes disagreements *decidable*. When a behavior is in question — does an even split round up or down? does a format modifier trim trailing spaces? — there is no debate and no appeal to intuition: run both binaries on the input

and look at the bytes. The reference tmux is the oracle, and the parity suite is the machine that consults it. This is what “correctness as method” means: correctness is not a property the developer judges, it is a measurement the suite takes, and the developer’s job is to make the measurement pass.

The two case flavors: `.fmt` and `.sh`

The parity cases come in two forms, and the split reflects the two things a multiplexer’s behavior divides into: computing values, and manipulating state. The first and more numerous form is the `.fmt` case — a file containing a single tmux format string. The runner expands it with `display-message -p` against a fresh detached session and compares the two binaries’ expansions. A case named `010_arith_add.fmt`, for instance, contains `{e|+|:2,3}`, and the suite checks that both binaries expand it to `5`. Because the format language reads so much of the server’s state (Chapter 9), a format case is a probe into that state: `{window_panes}` after a split checks the pane count, `{pane_current_command}` checks process tracking, and a matching expansion is evidence the underlying state matches. The format cases are the bulk of the suite for exactly this reason — they are cheap to write and they cover an enormous surface of state and computation through one uniform mechanism.

The second form is the `.sh` case — a shell scenario for behavior that takes more than one command to exercise. The runner exports `$TM` as the binary bound to a private socket, and the script just runs `$TM <command>` lines and prints deterministic output. A case might run `$TM new-window; $TM list-windows -F '#{window_index}'` and check that the window index sequence matches, or split a pane and then query the pane count, or set an option and then read it back. These cases test the *state-manipulating* commands — creating windows, splitting panes, renaming sessions, setting options — by performing an action and then reading the result back through a format, so the assertion is again a byte comparison of output. The two flavors together cover both halves: `.fmt` for the computational and introspective surface, `.sh` for the state-mutating command surface.

The mechanical uniformity of the two flavors is what makes the suite easy to grow, and it is exactly that ease that took the suite from 122 cases to 1115: the `parity/cases/` directory now holds those 1115 `.fmt` and `.sh` files spanning arithmetic, comparisons, string operations, conditionals, padding, loops, and the window/pane/session/option command families. All 1115 report green today; the roughly twenty that the expansion first broke on are documented in Appendix D as *what the growth caught and the port then fixed*, each still guarding its corner as a regression test. Every new behavior the port tackles becomes one or more cases in one of these two forms, and because the forms are so simple — a format string, or a few shell lines — adding coverage is low-friction, which is precisely what you want a correctness suite to be.

The two flavors also map cleanly onto the debugging workflow, because a failing case is a minimal reproduction by construction. A `.fmt` case that fails is a single format string that expands differently — you can paste it into both binaries and see the divergence immediately. A `.sh` case that fails is a short command sequence you can run by hand. There is no large test harness to understand, no fixture to untangle; the case *is* the reproduction. This is why the three bugs the next section describes were each caught and fixed cleanly — the parity case that caught each one was also the minimal input needed to debug it.

Writing a new case

Because the case formats are so simple, adding coverage is a low-friction habit, and the workflow is worth stating so a contributor can extend the suite. To pin a *computed* behavior, write a `.fmt` file — a single format string — named with the next number in its family and a descriptive suffix, e.g. `230_str_upper.fmt` containing the format under test. The runner will expand it under both binaries and diff; if they match, the case is green and the behavior is now pinned against regression. To pin a *state-manipulating* behavior, write a `.sh` file that runs `$TM <cmd>` lines and prints deterministic output — perform an action, then read the result back through a format, so the assertion is a byte comparison of the printed output.

The discipline on a new case is the determinism rule below: no `{host}`, `{version}`, `{pid}`, wall-clock time, or socket path, because those differ between the reference and the port or between runs. A good new

case is one that *fails first* — if a case passes the moment you write it, it is either covering already-correct behavior (fine, as a regression guard) or it is not exercising the corner you meant to. The suite grew from 122 to 1115 precisely by authors writing cases aimed at suspected-weak corners and keeping the ones that failed as pinned gaps (Appendix D). The case is cheap, the reproduction is the case itself, and the byte comparison is the oracle — which is what makes growing the suite the natural way to make a claim about ztmux checkable.

Determinism: fixed geometry, `LC_ALL=C`, no host/time/pid

Byte-for-byte comparison only works if the output is *deterministic*, and a multiplexer’s output is full of things that are not — the hostname, the time, process ids, socket paths, the terminal size. The parity runner controls every one of these, and the controls are as much a part of the methodology as the comparison itself. Geometry is pinned: every server is started at a fixed 80x24 size, so any format that reports a width or height (`{window_width}`, `{pane_height}`) is stable across runs and across the two binaries. Without fixed geometry, a window-width format would depend on the terminal the suite happened to run in, and the comparison would be meaningless.

The locale is pinned to `LC_ALL=C`, because locale affects string collation, number formatting, and character handling, and a suite run under a different locale on a different machine would diverge for reasons that have nothing to do with the port. Each case runs against an isolated server per binary — a unique socket label (`-L <uniq>`), no config file (`-f /dev/null`), and a `timeout` wrapping the run so a hang fails the case rather than wedging the suite — and the server is killed after the case. The isolation means cases cannot interfere with each other or with any real tmux or ztmux the developer is running, and the empty config means a case tests default behavior, not the developer’s `.tmux.conf`.

The non-deterministic *content* is handled by a rule rather than a mechanism: cases must avoid host-, time-, version-, and pid-dependent output. A case cannot use `{host}`, `{version}`, `{pid}`, wall-clock times, or the socket path, because those differ between the reference and the port (the port’s version string and socket prefix are deliberately its own) or between runs. This is a discipline on case authors, documented in the runner and the roadmap, and it is the right place to put the constraint: rather than trying to normalize non-deterministic output after the fact (which would weaken the byte-for-byte comparison), the suite simply excludes non-deterministic *inputs*, keeping the comparison exact.

Determinism is what lets the suite run in CI across platforms and mean the same thing every time, and it is why the parity claim can be qualified with “on both Linux and macOS.” The same fixed geometry, the same locale, the same isolation produce the same reference bytes on both platforms, so a case that passes on Linux and fails on macOS is a real platform-specific port bug — which is exactly how the third bug in the next section was found. The determinism controls turn the parity suite from “a comparison that usually works” into “a comparison that is reproducible and portable,” and that reproducibility is what makes it usable as the definition of correct rather than just a smoke test.

Three bugs the suite caught

The parity suite’s value is best shown by what it caught, and ztmux’s history has three clean examples, each a real divergence the byte comparison exposed and each fixed against the C. The first was a *crash*: the `{l:...}` literal format operator brought the ztmux server down. The parity case was a simple `l:` format string that tmux expanded fine and ztmux died on. Root-causing it led to `format_unescape` (Chapter 9): the C loop’s `s++` runs on every iteration including `continue`, and the Rust port had translated it to a `while` that failed to advance on the normal branch, so it spun forever writing past its `xmalloc(strlen + 1)` buffer until it crashed. The fix advances the pointer on every path, and the function now carries a comment explaining the trap so it cannot be reintroduced silently.

The second was an *arithmetic* divergence: the `405_select_layout` case exposed the even-horizontal layout rounding an 80-column two-pane split to `39|40` where tmux produces `40|39`. The port had carried an older algorithm that dumped the leftover column on the *last* pane; current tmux distributes leftover columns to the

leading panes, one at a time. Re-ported `layout_spread_cell` against the current C fixed it (Chapter 7), and the fix was locked with two unit tests asserting `spread_sizes(80, 2) == [40, 39]` in addition to the parity case — because an off-by-one in space distribution is invisible until a format prints the wrong width, and worth pinning twice. This one is a perfect illustration of why byte-for-byte matters: `39|40` versus `40|39` is a one-column difference no reasonable-behavior test would flag, and it would silently break any config that depends on exact pane widths.

The third was a *platform-specific* bug, and it is the one that shows the cross-platform determinism paying off. The `294_pane_cmd_fmt` case checks `{pane_current_command}`, and on macOS it reported `ztmux` instead of the actual child command (`sleep`), while passing on Linux. The root cause was in the pane fork path (Chapter 3's `spawn`): the child, between `fork` and `exec`, called `closefrom` to close inherited descriptors, and the port's `closefrom` looped `close()` from the low fd up to `RLIMIT_NOFILE` — which the server raises to a huge value — so the child spun through millions of `close()` calls and hung before it could `exec`, leaving `pane_current_command` reporting the server binary that had not yet been replaced. The fix ports tmux's macOS-specific `closefrom` variant, which enumerates the process's actually-open descriptors with `proc_pidinfo(PROC_PIDLISTFDS)` and closes only those, so the child reaches `exec` immediately. The bug was only observable on macOS, and only the cross-platform parity run surfaced it.

Three bugs, three categories — a crash from a dropped pointer increment, an arithmetic off-by-one in space distribution, and a platform-specific hang between fork and exec — and all three were caught by the same mechanism: run both binaries on the same input, diff the bytes, investigate the difference against the C. None of them would have been caught by a test suite that asserted “reasonable” behavior, because each produced output that looked plausible in isolation; they were only visible as *divergences from tmux*. That is the argument for correctness-as-parity in one paragraph: the bugs that matter most in a port are the ones where the port does something reasonable that is nonetheless not what the original does, and only a byte-for-byte comparison against the original finds them. The parity suite is not one test folder among many — it is the operational definition of what it means for ztmux to be tmux.

The expansion to 1115 and closing the last gaps

Reaching 122/122 posed a question the project answered the honest way. A suite that is 100 percent green is either a perfect port or too small to catch the imperfections, and there is no way to tell which from the green bar alone. So rather than bank the number, the project *expanded the suite* until it started failing — from 122 cases to 1115 — deliberately pushing into corners the first set never touched: bitwise arithmetic (`{e|&|:...}`, `{e|\|:...}`), float formatting and precision (`{e|/|f2:...}`), regex substitution with backreferences (`{s/.../\1/:...}`), the logical-not operator (`{!:...}`), left and right numeric padding (`{p-N:...}`), the loop modifiers `{S:}/{W:}/{P:}` with their injected `loop_index/loop_last_flag`, named paste buffers, and layout pinning by `main-pane-width`. The result was **twenty failing cases** — and those twenty were the point: they are what the expansion was *for*.

Every one of the twenty was deterministic — identical across repeated runs, with no host/time/pid dependence — and every one was pinned to the exact case number that exposed it, so each failure was a work item with a reproduction attached rather than a rumor. They clustered into a handful of small, nameable gaps (Appendix D lists each with its case numbers and its fix): `{!:...}` printed empty where tmux prints `1`; `{s/.../.../}` did not substitute `\1\2` backreferences; `{p-N:...}` left-padding padded to `N` *plus* the content length instead of a field width of `N`; `{pane_at_top}/{pane_at_bottom}` formatted as Rust `true/false` (`format!("{}", flag)`) instead of the C's `%d` → `1/0`; the `S/W/P` loop modifiers didn't inject `loop_index/loop_last_flag` (real tmux vars — see `vendor/tmux/format.c:4776`); named buffers (`set-buffer -b <name>`) reported “empty buffer name”; and `main-vertical` ignored `main-pane-width`. Each was a bounded, mechanical fix against the C, not an architectural hole — exactly the profile a good expansion produces: it converted “we don't know what's wrong” into a short, specific list. And then the project worked the list: every one of the twenty was traced to its C counterpart and fixed, and the suite now stands at **1115 of 1115 passing (100%)**.

The policy on those cases is worth stating because it is unusual and it survives their being fixed: **cases stay in the suite forever, never deleted**. A parity case is the most valuable artifact the project can hold — a

byte-exact statement of tmux's behavior — and the twenty that once failed are now the twenty best regression guards in the suite, standing red-turned-green over exactly the corners that once diverged. The roadmap file ([parity/Parity_Roadmap.md](#)) records the arc; the runner reports the count; and the number [1115/1115](#) in [parity/parity_summary.json](#) ([total 1115, passed 1115, failed 0](#)) is reported as-is. This is the difference between a suite that measures a port and a suite that flatters one: the 100% is credible precisely because the suite was allowed — was *made* — to fail first.

Chapter 12 – The Anti-Drift Gate

The parity suite proves behavioral fidelity. A second gate proves *structural* fidelity — that the port has not quietly grown functions tmux does not have, inflating its apparent completeness. This chapter covers the anti-drift machinery: the fake-fn test, the C-function index it checks against, the allowlist that documents the exceptions, and the annotation script that keeps two thousand C citations honest.

The fake-fn problem: inflating completeness

There is a specific way a port can lie about its progress, and it is subtle enough to be worth naming precisely. A port's completeness is naturally measured by how many of the original's functions it has implemented — and that metric is gameable. If you invent helper functions in the port that have no counterpart in the original — a `parse_the_thing` here, a `handle_special_case` there, convenience wrappers and Rust-idiomatic splits — the port *looks* larger and more complete than it is, because it has more functions, even though many of them correspond to nothing in the source of truth. The project calls these “fake fns,” and its documentation is blunt about the risk: a Rust function whose name has no C counterpart is an invented helper that inflates the port's apparent completeness.

This is not a hypothetical concern; it is the natural drift of any porting effort. Writing idiomatic Rust *wants* to introduce helpers — extract a closure, factor a common pattern, split a big C function into smaller typed pieces — and every one of those, however reasonable, moves the port away from a one-to-one correspondence with the C and toward a codebase that is “inspired by” tmux rather than a faithful transliteration of it. Once that correspondence is lost, the C citations stop meaning anything (there is no C function to cite), the parity comparison can't localize a divergence to a function, and the port report's completeness number becomes fiction. The whole auditability apparatus of Chapter 2 depends on the port's functions mapping to the C's functions, and fake fns break the mapping.

ztmux's answer is to make the fake-fn problem a *build failure* rather than a matter of discipline. The rule is simple and absolute: every module-level free function in the Rust source must correspond to a real function in the vendored tmux C source. If a Rust function's name has no C counterpart, the build fails — unless the function is explicitly listed in an allowlist that documents why it is a legitimate exception. This turns “don't invent functions” from a guideline a reviewer might miss into a gate that CI enforces on every push, and it makes the anti-drift discipline mechanical, the same way the parity suite makes the fidelity discipline mechanical.

The reason this deserves its own gate, separate from parity, is that it catches a different failure mode. Parity catches *behavioral* drift — code that does the wrong thing. The anti-drift gate catches *structural* drift — code that is organized wrong, that has wandered from the C's decomposition even while (perhaps) behaving correctly. A port can pass parity while failing anti-drift (it produces the right bytes but through an invented structure), and such a port is a maintenance and auditability trap even though it “works.” Both gates are necessary because faithfulness is both behavioral and structural, and a port that is faithful in only one sense is not the thing ztmux is trying to be.

ported_fn_names_match_c.rs and the C index

The gate is a single test, `tests/ported_fn_names_match_c.rs`, and it runs in CI like any other test — but what it tests is the whole source tree's structural honesty. It works in two halves: collect every free function name in the Rust source, build an index of every function name in the vendored C source, and check that

each Rust name is in the C index or the allowlist. The Rust half (`collect_free_fns`) parses the source character by character, tracking brace depth so it counts only functions at depth zero — free functions, not methods inside `impl` or `trait` blocks, since those correspond to C’s struct-of-function- pointers idiom rather than to free C functions. It correctly skips braces inside comments, strings (including raw strings), and char literals, strips the `pub/unsafe/async/ extern/const` qualifiers, normalizes raw identifiers (`r#loop` to `loop`), and skips `mod tests` blocks wholesale. This is a small hand-written parser, and it is careful because a sloppy one would produce false violations.

The C half (`c_fn_names`) builds the index heuristically but robustly, by scanning `vendor/tmux/*.c`, `vendor/tmux/compat/*.c`, and the `cmd-parse.y` grammar for line-initial identifiers immediately followed by `(` with an opening brace on that line or within a few lines — the shape of a C function *definition*, as opposed to a `;`-terminated *prototype*, which it excludes. It excludes C keywords, and it asserts the resulting index has more than 500 names as a sanity check that the vendored tmux is actually present (a missing `vendor/tmux/` would otherwise make every Rust function look fake). The heuristic does not need to be a full C parser because it only needs the set of function *names*, and names are cheap to extract reliably.

The comparison itself is strict in a way worth noting: there is *no* trailing-underscore leniency. A Rust function named `foo_` is not automatically accepted as an implementation of C’s `foo` — it is treated as a deviation and must be explicitly allowlisted. This matters because the trailing underscore is exactly the convention the port uses when it has to *split* a C function or wrap a macro (below), and those splits are precisely the deviations the project wants to track rather than wave through. By refusing to auto-accept `foo_` as `foo`, the gate forces every such split to be a documented, listed exception, so the count of deviations is always visible and always shrinking-toward-zero as a goal.

When the gate fails, it fails helpfully. It prints one greppable line per violation — `FAKE-FN <name> (<file>:<line>) - no tmux C counterpart` — and panics, so a developer who introduced an unlisted function sees exactly which functions are the problem and where. And the fix is either to rename the function to match its C counterpart (if one exists and the name simply drifted) or, if the function is a legitimate exception, to add it to the allowlist with the understanding that the allowlist is an audit trail, not a dumping ground. The gate does not just say “no”; it points at the exact functions and leaves the developer a clear, honest path forward.

A worked example: what the gate catches

To see the gate working, imagine a well-meaning refactor. A contributor porting `input_csi_dispatch` finds it long and, in good Rust taste, extracts a helper: `fn parse_csi_params(...)`. The code compiles, the parity cases still pass (the behavior is unchanged), and everything looks fine — except that `parse_csi_params` is a function that exists in `ztmux` and *nowhere in tmux*. On the next push, `cargo test --test ported_fn_names_match_c` collects `parse_csi_params`, fails to find it in the C index or the allowlist, and prints:

```
FAKE-FN parse_csi_params (src/ported/input.rs:1423) - no tmux C counterpart
```

and panics, failing the build. The contributor now has a clear, honest choice. If the logic belongs inside `input_csi_dispatch` (it does — the C keeps it there), inline it back and the gate goes green. If a genuine, unavoidable helper is truly needed, add it to the allowlist *with the understanding that the allowlist is an audit trail to shrink, not a dumping ground*. Either way the drift is surfaced at the moment it is introduced, not discovered months later when the C citations have quietly stopped meaning anything. This is the whole value of making anti-drift a build failure rather than a review guideline: the reviewer who would have caught `parse_csi_params` by eye might miss it, but CI never does, and the port’s one-to-one correspondence with the C is protected on every push. The example is invented, but the mechanism is exactly what runs, and it is why the port’s function set has not silently drifted into a codebase merely “inspired by” tmux.

The allowlist as an audit trail, not a free pass

No real port can achieve a *perfect* one-to-one function correspondence, and pretending otherwise would make the gate useless — so there is an escape hatch, `tests/data/ fake_fn_allowlist.txt`, but its framing is

what makes it work. The file's own header calls it “a frozen snapshot of pre-existing, accepted exceptions” and “an AUDIT TRAIL, not a free pass,” with the explicit goal to *shrink* it, never grow it. It holds roughly three hundred entries, and — crucially — they are not arbitrary; they fall into a small number of documented, legitimate categories, so that a reader can see not just *that* a function is excepted but *why* its exception is reasonable.

The categories are specific. First, *macro-paired helpers*: C's variadic functions (a `format_add(fmt, ...)`) are ported as a Rust `macro_rules!` carrying the real C name plus a `_`-suffixed helper the macro forwards to (`format_add_`), because Rust macros and functions occupy different namespaces — the macro can keep the C name, so the helper takes the underscore. Second, *C-function splits*: one C function (like `xstrdup`) ported as several typed Rust overloads (`xstrdup_`, `xstrdup__`, `xstrdup___`) to be merged back later. Third, *libc shims*: wrappers around library functions that have no `vendor/tmux` definition at all (`fnmatch`, `basename`, `memcpy`), which are real functions, just not tmux's. The event loop used to swell this category with its whole API — `event_*`, `evbuffer_*`, `bufferevent_*` — and no longer appears in it at all: living under `src/extensions/`, it is outside what the gate measures, which is where code that is deliberately not a port belongs. Fourth, *genuine Rust glue*: the handful of truly Rust-only helpers (`cstr_to_str`, `assume_init_ref`) that exist because Rust needs them and C did not.

A large share of the entries are the BSD intrusive-data-structure macros that tmux uses throughout — the red-black tree family (`rb_insert`, `rb_remove`, `rb_foreach`, and the rest), the tail-queue family (`tailq_foreach`, `tailq_insert_tail`), the list family — which are C *macros* in tmux (so they have no function definition to match) but are ported as Rust functions. These are the data structures Chapters 3, 5, 7, and 9 all rely on (WINDOWS is an RB tree, CLIENTS a TAILQ, key tables are RB trees), and porting the macros as functions is unavoidable — Rust has no equivalent of C's token-pasting intrusive-structure macros. Listing them documents exactly which “functions” are really ported macros, keeping the fake-fn count honest without pretending they are tmux functions.

The regeneration story reinforces that the allowlist is meant to be curated, not auto-grown. The file can be regenerated from the gate's own FAKE-FN output — run the test, pipe the FAKE-FN lines through `sort -u` into the allowlist — but doing so blindly would also require manually re-adding the header, and the header is the whole point: it is the statement that these are accepted exceptions to be shrunk over time. The intended workflow is not “regenerate to make the gate pass,” it is “understand each new exception, decide if it is legitimate, and if so add it deliberately.” The allowlist is version-controlled, so its growth or shrinkage over time is itself a visible signal of whether the port is drifting toward or away from clean C correspondence.

annotate_c_links.py and the C-signature citations

The anti-drift gate checks function *names*; a companion script checks function *signatures*, and together they close the structural-fidelity loop. `scripts/annotate_c_links.py` walks the Rust source and, for every function whose name matches a tmux C function, writes (or refreshes) the doc-comment citation this book has quoted throughout — the `/// C `vendor/tmux/<file>.c:<line>`: `<full C signature>`` line. The citation includes the C function's *full signature*, return type and parameter types and parameter *names*, not just its name — and that is deliberate. A reviewer diffing the Rust parameters against the cited C parameters can catch a renamed, reordered, dropped, or invented argument: a “fake param,” the argument-level analog of a fake fn. There are more than two thousand of these citations across the tree, one per ported function.

The script is careful about which C definition it cites, because a name can appear in more than one C file. It prefers the C file whose stem matches the Rust file's stem — `grid.rs` cites `grid.c`, `cmd_kill_pane.rs` cites `cmd-kill-pane.c` (mapping underscores to hyphens) — so a function is cited from its natural home. Failing a stem match, if the name is defined in exactly one C file, it cites that; if the name is ambiguous across several files with no stem match, it *skips* rather than guessing, on the principle that no citation is better than a wrong one. This conservative citation policy is why the citations can be trusted: a citation that exists is one the script was confident about.

The script is idempotent and self-updating, which is what keeps two thousand citations from rotting. It recognizes an existing citation directly above a function and *replaces* it in place rather than duplicating it, so re-running the

script after a vendored-tmux refresh updates every citation's line number and signature to match the new C — the citations track the C automatically rather than being hand-maintained. A `--check` mode writes nothing and exits non-zero if any citation is missing or outdated, printing how many functions need attention, so the citation freshness can itself be a CI gate: the build can require that every ported function's citation is present and current.

Between them, the three tools form a structural-fidelity system that parallels the behavioral-fidelity system of Chapter 11. The name gate (`ported_fn_names_match_c.rs`) ensures every function corresponds to something real in the C. The signature citations (`annotate_c_links.py`) ensure every function's interface matches the C's, and stay current as the C moves. The allowlist documents and bounds the exceptions, with a stated goal of shrinking. And all three are derived from the vendored C rather than from hand-maintained lists, so none can drift out of sync with the source of truth. This is what lets ztmux claim to be a *faithful* port and not merely a *working* one: fidelity is measured behaviorally by parity and enforced structurally by the anti-drift gate, on every push, against a committed reference. With correctness both proven and defended, the book can finally turn to the forward-looking question — what a port this faithful is *for*, beyond reproducing tmux.

Chapter 13 – Beyond tmux: The Superset

The chapter that earns the word *superset*. ztmux’s goal was never to stop at parity: a memory-safe, own-able core is a platform for features the C tmux does not have. With the core provably identical to tmux — 1115/1115 byte-for-byte — that platform is open, and it has been built on. The first three extensions were structured (JSON/CSV/table) output for the `list-*` commands, a live `ztmux dashboard`, and an interactive `ztmux switch` picker, all living in a dedicated `src/extensions/` tree that the anti-drift gate leaves free.

Those three were the beachhead. That tree now holds **119 modules and roughly 29,400 lines** — the named `ztmux <name>` subcommands and the interactive surfaces catalogued through the rest of this chapter. It is bigger than several of the ported subsystems it sits next to, and it is the reason the word on the cover is *superset* rather than *port*: a port is what Chapters 3 through 12 describe, and this is the part that is deliberately, checkably not tmux.

The count is not a figure of speech, and the reader can reproduce it:

```
ls src/extensions/*.rs | wc -l
```

This chapter stays honest about the line between shipped and planned — it marks what exists today and what is still roadmap — and lays out where a Rust-owned multiplexer can go that a frozen C program cannot.

Parity first: what “faithful” commits the superset to pre-serve

The superset ambition has an order of operations, and it is not negotiable: parity comes first, extensions come second, and the second must never break the first. A superset is a *strict* extension — it does everything the original does, exactly, and then more — so every feature ztmux adds is constrained by a promise it made in Chapter 1: the config files, the keybindings, the format strings, and the muscle memory of every tmux user must keep working unchanged. That promise is what the parity suite (Chapter 11) and the anti-drift gate (Chapter 12) exist to protect, and it is why they had to come before this chapter. You cannot credibly offer “tmux plus extras” until the “tmux” part is provably identical.

This ordering is also a design constraint on the extensions themselves, not just a sequencing of work. A feature that changed the meaning of an existing format variable, or altered a default, or reinterpreted a command’s arguments, would break parity even if it added capability — and so it is disallowed by construction. The extensions ztmux can add are the ones that are *additive*: new format variables that don’t collide with existing ones, new commands that don’t shadow existing ones, new options that default to the existing behavior, integrations that are invisible unless invoked. The superset grows the surface without moving any of the existing surface, which is a stricter discipline than “add features” and the only one compatible with the parity promise.

The commitment cuts deeper than the user-visible surface, too. Because the port keeps its C correspondence (Chapter 12), a superset feature ideally slots in *alongside* the ported functions rather than tangling into them — an added capability should be legible as an addition, not a modification of a cited-from-C function, so that the port’s faithfulness stays auditable even as it grows. This is the same principle the `ported/` and `extensions/` split (Chapter 2) enforces: keep the ported core legible and keep your own additions distinguishable from it. A

superset built this way can always answer “what have you changed about tmux?” with “nothing — here is what I’ve added.”

The reward for accepting this discipline is that ztmux’s extensions inherit tmux’s entire ecosystem for free. Every plugin, every dotfile, every tutorial, every piece of tmux muscle memory continues to work, and the extensions are pure upside on top. This is the opposite of a fork, which asks users to give up the ecosystem in exchange for new features; a faithful superset asks them to give up nothing. Parity-first is not a constraint that limits the superset — it is the strategy that makes the superset worth using, because it means adopting ztmux costs a tmux user exactly nothing and the new capabilities are all gain.

The extensions tree: `ported/` and `extensions/`

The discipline above is enforced by the source layout, not just by intent. As the port matured, `src/` split into two trees. `src/ported/` holds the C-faithful port — every file mirrors a `vendor/tmux/*.c` module, every free function is held to its C name by the anti-drift gate (Chapter 12). `src/extensions/` holds original ztmux features that have *no* tmux counterpart, and it is deliberately **exempt** from the gate: the check skips any path containing `src/extensions/`, and `extensions/mod.rs` says so in as many words — “features with no upstream tmux counterpart... deliberately NOT a port of tmux.” That exemption is what lets extensions exist at all without tripping the fake-fn rule, and the split is what keeps the exemption honest: a function is either in `ported/` (and must cite a C origin) or in `extensions/` (and must be a genuine addition, off to the side). There is no third place to hide an invented helper. The extension modules are also written in a self-denying style — their logic lives in inherent methods and associated functions rather than free `fns` — so even inside the exempt tree they add no module-level names a reviewer must vet.

Shipped: structured output for the `list-*` commands

tmux’s `list-sessions`, `list-windows`, `list-panes`, `list-clients`, and `list-buffers` print one formatted line per item, shaped by a `-F` format string. That is perfect for a status bar and awkward for a script, which must then parse the text back apart. ztmux’s first shipped extension (`src/extensions/structured.rs`) adds a `-o <format>` flag to those commands that emits the rows as *data* instead: `json`, `jsonl` (a.k.a. `ndjson`), `csv`, `tsv`, and the human-facing `table`. The formats are parsed by `OutputFormat::parse`; an unrecognized name is a reported error, and no `-o` at all leaves the classic text output untouched — additive, by construction.

The mechanism reuses the ported core rather than duplicating it. Each list command owns a static schema — a slice of `Field { key, fmt, kind }`, where `key` is the column/JSON name, `fmt` is an ordinary `#{...}` format string, and `kind` is `Str`, `Int`, or `Bool`. So the *same* format engine (Chapter 8) that produces the text line produces the structured value; every format variable is available, and any `-f` filter still applies. A `Structured` accumulator expands each field against the item’s `format_tree`, coercing per `kind` (an unparseable-but-empty integer becomes JSON `null` rather than silently dropping data), then `render()` dispatches on the format: `render_json` (array or line-delimited), `render_sv` (with `,` or `\t`, quoting per RFC 4180 when a field contains the separator, a quote, or a newline), and `render_table` (measure each column’s width, left-align, pad with spaces, trim the trailing pad, and underline the header). The same rows, five renderings:

```

$ ztmux list-sessions -o table
NAME      WINDOWS  ATTACHED
work       3         1
scratch    1         0

$ ztmux list-sessions -o json
[{"name":"work","windows":3,"attached":true},
 {"name":"scratch","windows":1,"attached":false}]

$ ztmux list-sessions -o csv
name,windows,attached
work,3,true
scratch,1,false

```

This is the completion of the “json / csv / ...” family with the aligned, human-readable `table` view — and, not incidentally, the machine-readable side is what the two TUI extensions below consume to read the server without linking against its internals.

Shipped: ztmux dashboard

`ztmux dashboard` (`src/extensions/dashboard.rs`) is a live, full-screen overview of the running server — sessions, windows, and panes at a glance, refreshing in place — built on `ratatui` 0.29. It is a *client* subcommand, not a server command: it does not reach into server memory but polls the server exactly the way any other client would, by re-invoking its own binary (`ztmux -S <socket> list-* -o json`) through the shared query layer and deserializing the JSON with `serde`. `ratatui::init()` puts the terminal into the alternate screen and raw mode, the draw loop renders the session/window/pane tree with key handling for navigation and quit, and `ratatui::restore()` guarantees the terminal is put back on the way out. It is the sort of thing tmux has never had — a birds-eye console for a server you are not attached to — and it is cheap precisely because the structured-output extension already exposes the state as clean JSON.

Shipped: ztmux switch — the interactive picker

`ztmux switch` (`src/extensions/switch.rs`) is an interactive fuzzy picker to jump to any session, window, or pane. It polls the same JSON snapshot, flattens it into a flat list of *targets* — every session, every window, every pane, each with a display label — and then you type to filter (a case-insensitive subsequence match, so `wrk` finds `work`), move with the arrow keys, and press Enter. On Enter it runs the ordered commands that select the target against the chosen socket: a session runs `switch-client -t <session>`; a window runs `switch-client` then `select-window -t <session>:<index>`; a pane runs `switch-client`, `select-window`, then `select-pane -t <pane-id>`. Esc (or Ctrl-C) cancels. It is the picker `fzf`-driven tmux users hand-roll with shell glue and a dozen `tmux` calls — here it is one subcommand, and the target-building and fuzzy-matching logic ships with unit tests that assert a pane target selects session → window → pane in order and that `nvim` matches only the pane running it.

The two TUIs share a small read-only module, `src/extensions/tmux_query.rs`, which is the whole reason they stay clean: it defines the `Session/Window/Pane/Client` structs, the `ztmux_cmd` helper that builds a `ztmux -S <socket> ...` invocation of the current binary, and `poll()` which fetches the full tree via the `-o json` list commands. The extensions read the server the way a script would — through the public, structured interface the first extension added — never through a private back door. That is the additive discipline made literal: a new capability built entirely on top of the ported surface, touching none of it.

Shipped: the extension catalog and the interactive surfaces

The three extensions above are the ones this chapter dissects, but they are no longer the whole of what has shipped. `src/extensions/` now carries 110 named `ztmux <name>` subcommands, listed once in the `EXTENSION_COMMANDS` table at the top of `src/extensions/mod.rs`, which is the single source of truth for the CLI, the zsh completion, and the man page. Most are read-only *inspections* — pipeable, `-o json`-capable views over the live server built on the same structured `list-* -o json` output the dashboard consumes: process tables (`ps`, `pstree`, `mem`), geometry (`size`, `density`, `layouts`), repository state (`git`, `cwd`, `changes`, `worktree`), network (`ssh`, `net`, `ports`), and client and configuration queries. A smaller set are *actions* (`prune`, `equalize`, `retitle`, `bcast`, `pick`). Every one is additive and gate-exempt by the same rule the three flagship extensions follow — there is no third kind.

Four larger systems reach past one-shot subcommands, and each is opt-in and invisible until invoked. `ztmux triggers` revives tmux's removed `monitor-content` as a general sense→act loop: a rule in `~/.ztmux/triggers.json` runs any ztmux command when a regex matches a pane's output, armed with `ztmux triggers arm` and authorable through an inline wizard. `ztmux resurrect save / restore` persist the whole server — every session, window, and pane with its layout, cwd, and command — to `~/.ztmux/resurrect/` and recreate it across restarts, never clobbering a live session; `set -g @ztmux-resurrect-auto on` adds continuum-style automatic re-saving every fifteen minutes from a pidfile-guarded daemon. `ztmux open` scans the focused pane for URLs and file paths and offers a picker that opens the selection (a URL through `open/xdg-open`, a file in `$EDITOR` at its `file:line`) or copies it to the tmux buffer and OS clipboard. And an optional **ratatui UI layer** — on by default, disableable with `@ztmux-ratatui off` to fall straight back to the byte-for-byte parity draw path — renders interactive surfaces the server draw never had: a which-key hint bar on the prefix, a completing command palette, a ratatui clock and display-panes, and a family of zellij-style behaviors gated behind `@ztmux-zellij-mode on` (off by default) — inset pane frames a program can never overwrite, a collapsing pane stack (`prefix +`), a window tab bar (`ztmux tabs`), a filterable session manager (`ztmux sessions`), a persistent floating pane (`prefix [-f]`), and prefix-free modal keybindings (`ztmux modal`).

None of this moves the shipped/roadmap line the rest of the chapter draws. Every one of these surfaces is *additive*, sitting beside the ported core rather than modifying it, and every one leaves the classic tmux surface — and the parity suite — untouched when its setting is off. What remains roadmap is the deeper stack integration the next sections describe.

What an additive feature looks like

The rule that every extension must be *additive* (never moving existing surface) is easier to believe with a concrete shape. Suppose ztmux wanted to add prompt-aware navigation — a command to jump the copy-mode cursor to the previous shell prompt. Every piece it needs already exists and none of it disturbs parity: the OSC 133 marks are already parsed into grid line flags (`START_PROMPT`, Chapter 6); the grid reader already walks stored lines (Chapter 18); the copy mode already has a cursor and key tables. The feature is a *new cmd_*.rs* command with a *new* default `copy-mode` binding on a key tmux leaves unbound, whose `exec` walks the grid reader backward to the nearest `START_PROMPT` line. It adds a command, adds a binding on a free key, and reads state that is already tracked — it shadows no existing command, redefines no existing format variable, and changes no default. A tmux user who never presses the new key sees exactly tmux; a user who does gets pure upside. That is what “additive” means operationally: the new surface sits *alongside* the ported core (Chapter 12's principle), legible as an addition rather than a modification of a cited-from-C function, so the port's faithfulness stays auditable even as it grows. The feature is roadmap, not shipped (the next section is emphatic about that) — but its *shape* is the template every superset extension follows.

The MenkeTechnologies-stack integrations ztmux is positioned for

The natural extensions for ztmux are not speculative features dreamed up in the abstract; they are the integrations the surrounding MenkeTechnologies stack already wants, and being a Rust program that the same author owns is what makes them reachable. The clearest is `zterminal`, the stack's GPU terminal emulator, which already speaks a tmux wire protocol and already live-edits a running tmux server's options, buffers, and keybindings from its control panel. A tmux server it can talk to *natively*, built in the same language by the same hand, is a tighter and more capable target than the real tmux's protocol allows — the two can share types, share a socket convention, and evolve the protocol together rather than one reverse-engineering the other.

The `ztmux-core` sibling is the other half of that picture. Where this repository is the whole server and client, `ztmux-core` is a native tmux *client engine* — a library for speaking the tmux protocol — and the two are deliberately separated so the client- of-tmux story and the whole-multiplexer story stay distinct. But they are built to meet: a client engine and a server that share a codebase can offer an integration neither could alone, letting a stack application embed multiplexer functionality without shelling out to a separate `tmux` process. This is the kind of integration that is trivial when you own both sides and nearly impossible when one side is a frozen external binary.

Then there is the shell and the panes' contents. `zshrs` — the stack's Rust rewrite of zsh — runs inside the panes, and `zpw` paints inside them; a multiplexer that is aware of the shell integration protocols (the OSC 133 prompt marks the input parser already recognizes, Chapter 6) can offer prompt-aware features that a shell-agnostic multiplexer cannot. Because ztmux already parses the semantic-prompt sequences into grid line flags (`START_PROMPT`, `START_OUTPUT`), the raw material for prompt-jumping, command-output selection, and per-command navigation is already in the grid — these are extensions the port is *positioned* for because the underlying state is already tracked, not features that would require new plumbing.

The honest framing of all this is that these are *positions*, not shipped products, and this section is careful to say so. What is real today is that ztmux is a faithful, own-able tmux in the same language and stack as the tools that would integrate with it — which is the precondition for the integrations, not the integrations themselves. The value of the position is that each of these becomes a normal engineering task rather than a cross-project protocol negotiation with a frozen dependency. The superset's promise is that when the stack wants tighter multiplexer integration, ztmux is the piece that can grow to provide it, because it is code the stack controls.

Shipped vs. roadmap — kept honest

The single most important thing this chapter can do is draw the shipped/roadmap line sharply, because a superset chapter is exactly where a project is tempted to describe ambitions as if they were features. So, plainly: what is *shipped* today is the faithful core, plus the additive extension surface catalogued above. The server, client, grid, screen, input parser, layout engine, command language, format system, options, key bindings, config loading, and terminfo back end are ported and, to the extent the parity suite measures, verified — the suite reached 122 of 122 byte-for-byte, was then expanded to 1115 cases (surfacing twenty gaps the port then closed), and stands at **1115 of 1115 passing** on Linux and macOS. Sixel graphics are implemented. The socket is namespaced so ztmux coexists with tmux. That is the real, usable multiplexer this book has spent ten chapters describing.

What is *partial* is also stated in the code itself, and the Cargo manifest is candid about it: many of the feature flags and options exist “to make matching upstream's configuration options easier” — they are accepted and stored so a real `.tmux.conf` parses without error, but not every one is wired to behavior yet. This is a specific, honest kind of incompleteness: the *configuration surface* is broad (because parsing a tmux config faithfully requires accepting every option tmux accepts) while the *behavior behind some options* is still being connected. A user's config will load; some of its more exotic options may not yet do anything. Chapter 15 keeps the running ledger of which subsystems are complete versus partial.

What is *roadmap* is the superset proper — the stack integrations of the previous section, and the deeper

“done right” work of the next chapter — and none of it should be read as present today. There is no shipped `zterminal` native-integration feature, no shipped prompt-navigation UI, no shipped cross-stack multiplexer embedding. Those are where a Rust-owned core *can* go, and the argument of this chapter is that the port’s faithfulness and ownership make them reachable — but “reachable” is not “shipped,” and this book marks the difference rather than rounding up, because the whole series’ claim to honesty depends on drawing exactly this line.

The reason to be this careful is that the superset’s credibility is cumulative with the parity claim. A project that overstated its extensions would invite doubt about its parity numbers too; a project that is scrupulous about saying “this is roadmap” earns trust that its “this is measured” claims are equally exact. The 1115/1115 is real *because* the roadmap is labeled roadmap and the twenty gaps were printed rather than hidden — named, pinned, and fixed in the open, not quietly deleted — the same discipline produces all three. So the honest ledger is not a hedge that weakens the superset story; it is what makes the superset story believable when its features do ship, because the project will have a track record of describing its own state accurately.

Why owning the code in Rust is what makes the superset possible

Step back and ask why a superset of tmux is even a sensible goal, when tmux itself is mature, excellent, and actively maintained. The answer is ownership, and it has two components. The first is *language*: tmux is C, and C’s memory model means every new feature is written against the same manual-memory discipline, with the same class of bugs (use-after-free, buffer overrun — the very bug the `format_unescape` crash was) one mistake away. ztmux is Rust, and while the port is currently full of `unsafe` (the subject of Chapter 14), its trajectory is toward safe Rust, where whole categories of those bugs become impossible. A superset built on a memory-safe core can add features without the per-feature risk that C imposes.

The second component is *control*. tmux is governed by its maintainers, and rightly so — a widely-dependended-on tool should be conservative about what it absorbs. That conservatism is a virtue for tmux and a wall for anyone who wants to extend it deeply: a stack-specific integration, a new architecture for embedding, a feature that only makes sense inside the MenkeTechnologies ecosystem has no path into upstream tmux, nor should it. ztmux removes the wall by being owned by the same project that wants the extensions. It can move at the stack’s speed, adopt the stack’s conventions, and integrate with the stack’s other tools, because it is not asking anyone’s permission — it is the stack’s own multiplexer.

Ownership without faithfulness would be worthless, which is why the whole first two-thirds of this book had to come first. A fork that diverged from tmux would be owned but would have thrown away the ecosystem and the muscle memory — it would be a *different* multiplexer, competing with tmux on features while lacking tmux’s maturity, which is exactly the losing proposition the project’s own bar rejects. The value is in the combination: faithful *and* owned. Faithful means the ecosystem transfers for free; owned means the extensions are reachable. Neither alone is a superset; together they are.

This is the deepest reason ztmux is a port and not a rewrite-from-spec, and it closes the loop back to Chapter 1. A rewrite could have been owned, but it would have diverged in a thousand undocumented behaviors and lost faithfulness. A wrapper could have been faithful (it *is* tmux), but it would have owned nothing and could extend nothing. Only a *faithful port* — tmux’s exact behavior, in code the stack controls — is simultaneously compatible enough to inherit the ecosystem and own-able enough to grow past it. The superset is not a feature ztmux has; it is the strategic position the faithful port creates, and everything in the roadmap is a move from that position. The next chapter is about the work that turns the position from a transliteration into something worthy of the word in the README: *done right*.

Chapter 14 – The Road to “Done Right”: Shrinking `unsafe`

The README’s tagline is `[TMUX, REWRITTEN IN RUST – DONE RIGHT]`, and “done right” is a claim the current code has only partly earned. This chapter is honest about the gap: what “done right” means past parity, why the port is full of `unsafe` today, and why the order of operations is parity first, safety second, features third.

What “done right” means past parity

A faithful port can be *correct* without being *done right*, and the distinction is the whole subject of this chapter. Correct means the parity suite is green — ztmux produces tmux’s exact bytes. Done right means something more: that the code is idiomatic, memory-safe Rust that takes advantage of the language’s guarantees rather than merely transliterating C’s patterns into Rust syntax. Today’s ztmux is emphatically the first and only partway to the second. It is a near-mechanical transliteration of tmux’s C — same function names, same control flow, same intrusive data structures, the same event-loop API called the same way, same raw pointers — which is exactly what made it possible to verify against the C, and exactly what “done right” has not yet finished.

The evidence is visible in the code’s texture. Nearly every ported function is `pub unsafe fn`; the data structures are `#[repr(C)]` structs threaded on raw pointers through ported BSD tree and tailq macros; the globals are `static mut`; strings are often `*mut u8` and null-terminated rather than `String` or `&str`; callbacks are `extern "C-unwind" fn` pointers. This is not accidental and it is not sloppy — it is the *honest starting point* of a port that must first match C exactly. A grid cell that is a `#[repr(C)]` struct with the same field layout as tmux’s `grid_cell` can be reasoned about against the C directly; a grid cell rewritten as an idiomatic Rust enum could not. The transliteration is a phase, and “done right” is the phase after it.

So “done right” is best understood as a *direction*, not a current state, and the tagline is a statement of intent that the rest of this chapter qualifies. The direction is: take a subsystem that is faithful (green on parity, clean on the anti-drift gate) and rewrite it from C-shaped Rust into Rust-shaped Rust — replacing raw pointers with references and owned types, `static mut` with safe interior mutability, null-terminated `*mut u8` with `str`, unsafe FFI patterns with safe wrappers — *without breaking parity*. Each such rewrite is a step from “correct” toward “done right,” and each is gated by the same parity suite that proved the subsystem correct in the first place. The tagline will be fully earned when that process has swept the tree; today it names the goal.

It is worth being clear that this is a legitimate and common shape for a serious port, not a confession of a half-finished project. Rewriting a large C program in Rust in one pass, idiomatically, with no reference to check against, is how ports fail — they diverge in a thousand places and no one can tell where. Transliterate-then-idiomatize is the disciplined alternative: get to faithful first, with the C as a checkable reference, then improve the code’s *form* while the parity suite guards its *behavior*. ztmux is a project that has finished the first phase for its core and is early in the second, and saying so plainly is exactly the honesty the series demands.

The `unsafe` surface today and why it exists

The `unsafe` in ztmux is not incidental; it is structural, and understanding *why* it is there is the key to understanding what shrinking it entails. It exists because the port mirrors tmux’s memory model, and tmux’s memory model is manual: it allocates with `malloc`, threads objects on intrusive linked structures via raw pointers,

shares mutable global state, and hands raw buffers to libevent and libc. To match tmux’s behavior exactly — including allocation patterns the parity suite can observe, and pointer identities the code relies on — the port initially reproduces that model rather than replacing it, and reproducing a manual-memory model in Rust means `unsafe`. The custom global allocator in `main.rs` (a shim over libc `malloc/free`) is the clearest tell: the port even routes Rust’s allocations through libc so that memory behaves as C’s does.

There are a few distinct sources of `unsafe`, and separating them clarifies which are temporary and which are more fundamental. The largest source is the *data model*: raw- pointer-threaded `#[repr(C)]` structs and the intrusive `rb_*/tailq_*` structures that every subsystem uses. This is the most reducible — Rust has safe equivalents (owned trees, `Vec`, slotmaps), and converting a subsystem’s data model to them is the bulk of the “done right” work. A second source is *global state*: the `static mut` server globals (`CLIENTS`, `SESSIONS`, `SOCKET_PATH`), which safe Rust would express with interior mutability or by threading a context. A third is *FFI*: the libc calls, which are inherently `unsafe` at the boundary but can be wrapped in safe interfaces so the `unsafe` is confined to thin shims rather than smeared across every caller.

The FFI source deserves a note because it will never go fully to zero, and that is fine. It is also smaller than it was: the event loop used to be C, and every dispatch was a call across the boundary, until it was replaced by the Rust loop of Chapter 3, leaving libc as the only C ztmux calls. What remains is called directly and deliberately, because the port’s allocation and syscall behaviour has to match the C’s. Calls across that boundary are `unsafe` by definition; C has no Rust guarantees to offer. But the *right* amount of FFI `unsafe` is a small, audited set of wrapper functions, each encapsulating one C call behind a safe Rust signature, so that the rest of the code calls safe wrappers. “Done right” for the FFI is not eliminating it but *confining* it — turning a tree-wide scattering of raw C calls into a thin, reviewable boundary layer.

Seen this way, the current `unsafe` surface is a map of the remaining work, and it is a map the anti-drift allowlist already partly draws — the `rb_*`, `tailq_*`, `evbuffer_*`, and `evtimer_*` entries in `fake_fn_allowlist.txt` (Chapter 12) are exactly the ported macros and FFI shims that carry much of the `unsafe`, and shrinking that allowlist and shrinking the `unsafe` surface are nearly the same project. That is a satisfying convergence: the structural-fidelity gate’s “shrink the allowlist” goal and the “done right” chapter’s “shrink the `unsafe`” goal point at the same code, so progress on one is progress on the other. The port has, in effect, already instrumented the work that remains.

Idiomatic Rust as the endpoint, not the starting point

The endpoint the tagline names is a ztmux whose code a Rust programmer would recognize as Rust — safe references and owned types instead of raw pointers, `enum` and pattern matching instead of tagged-integer unions, `Result` and `?` instead of C-style error returns and out-parameters, `RAll` instead of manual free — but reaching it is a *migration*, subsystem by subsystem, each verified by parity. This is why idiomatic Rust is the endpoint and not the starting point: had ztmux started idiomatic, it could never have been checked against the C, because idiomatic Rust and C have no line-for-line correspondence. The faithful transliteration is the bridge that makes the idiomatic destination *reachable with a safety net*.

The migration order is not arbitrary; it follows the same logic as everything else in the project. A subsystem is a candidate for idiomatization once it is faithful (green on parity, clean on the gate) and once its correspondence to the C is no longer the thing being actively verified — that is, once the C has been fully mined for its behavior and the parity cases capture it. At that point the C’s *structure* has done its job (guiding the transliteration) and can be left behind in favor of Rust’s structure, with the parity cases now serving as the specification that the idiomatic rewrite must satisfy. The parity suite, built to *prove* faithfulness, becomes the tool that *preserves* faithfulness across the idiomatic rewrite — the same green cases, a very different implementation underneath.

There is a real tension the migration has to manage, and honesty requires naming it: making a subsystem idiomatic can *reduce* its auditability against the C, because the C citations that mapped each function to its origin become less meaningful once the function no longer mirrors the C. The project’s answer is sequencing — audit fully *first*, against the transliterated version, then idiomatize — so that the auditable phase and the idiomatic phase are distinct in time. The C citations and the anti-drift gate are most valuable while a subsystem is still being verified; once it is verified and pinned by parity, the value shifts to code quality, and the citations

can relax. Trying to have both maximal C correspondence and maximal Rust idiom *simultaneously* is the mistake; having them in sequence is the plan.

The payoff of reaching the endpoint is what makes the whole migration worth the care. An idiomatic, safe-Rust ztmux is a multiplexer where the class of bugs that produced the `format_unescape` crash — pointer arithmetic overrunning a manual buffer — is impossible by construction, where the superset's new features can be written without the per-feature memory risk C imposes, and where a contributor who knows Rust but not tmux's C idioms can work on the code. That is the multiplexer the tagline promises: not just tmux's behavior in Rust syntax, but tmux's behavior in *Rust*, with everything the language's guarantees buy. The transliteration got the behavior right; the idiomatic migration is how it becomes done right.

What one idiomatization step actually looks like

To make stage two concrete rather than abstract, picture the first safe-Rust conversion of a small subsystem — say the tab-stop storage. Today it is a `bitstr` (a ported C bit-string, `src/ported/bitstr.rs`) threaded through the screen as a raw pointer, with `bit_set/bit_clear/bit_test` operations and manual allocation. The idiomatic version would be a `Vec<bool>` or a `bitvec`-style owned bitset with methods, no raw pointer, no manual free, no `unsafe` at the call sites. The conversion is not a rewrite from scratch: the tab-stop parity behavior (default every eighth column, `HT5` sets a stop, `TBC` clears, cursor tab motion honors them) is already pinned, so the parity suite becomes the specification the new implementation must satisfy. You change the storage type, update the handful of call sites, keep the behavior, and run the parity cases; green means the idiomatic version is behaviorally identical to the C-shaped one it replaced.

Two things make this safe to do. First, the anti-drift allowlist already names the `bit_*` shims (Chapter 12), so retiring them and shrinking the allowlist are the same action — progress is measurable. Second, the C citations on the old code can *relax* once the subsystem is no longer being verified against the C, because the parity cases now carry the fidelity that the citations carried during the porting phase. That is the whole sequencing argument (below) in one subsystem: audit against the C first, with the transliteration and its citations; then idiomatize, with the parity cases as the guard. The tab stops are a tiny example; the format engine, the grid, and the layout tree are the same story at larger scale, and each is a discrete, parity-guarded step from correct toward done right.

The order of operations: parity, then safety, then features

The whole project resolves into a single ordering, and stating it plainly is the best summary of ztmux's method. First, *parity*: match tmux byte-for-byte, verified by the parity suite and defended by the anti-drift gate, using a faithful transliteration whose C correspondence makes the matching checkable. Second, *safety*: with parity green and pinned as a specification, migrate the transliterated core toward idiomatic, memory-safe Rust, subsystem by subsystem, with the parity suite guaranteeing no behavior changes. Third, *features*: from the faithful, safe, owned core, grow the superset extensions the stack wants, each additive so parity is preserved. Parity, then safety, then features — in that order, never skipping.

The order is not a preference; it is a set of dependencies, and reversing any two links breaks the chain. Safety before parity would mean idiomatizing code before its behavior was verified against the C — the divergent-rewrite trap, where you can no longer tell whether a difference from tmux is a bug or an intended change. Features before parity would mean building extensions on an unverified base, so a parity failure later could be in the core or the extension and you could not tell which. Features before safety would mean writing new `unsafe` code for every extension, inheriting C's per-feature bug risk into the new work. Each stage is the precondition for the next: parity gives safety a specification to preserve, and safety gives features a foundation that does not multiply risk.

This ordering also explains why the book is shaped the way it is, and why the port looks the way it does. Ten

chapters on the faithful core, two on the machinery that proves and defends its fidelity, and only then the forward-looking chapters — because that is the actual order of the work. At this stage, stage one (parity) is done for the core and measured at 1115/1115 (a suite grown from a green 122/122 until it was hard enough to fail, then each failure pinned and fixed); stage two (safety) is early, its remaining work mapped by the `unsafe` surface and the anti-drift allowlist; stage three (features) is roadmap. A reader who wants to know “where is ztmux?” can read it off the ordering: at the boundary between stage one and stage two, with stage three clearly labeled ahead.

There is a discipline in refusing to do the stages out of order that is easy to underestimate. It is tempting, at every point, to jump ahead — to idiomatize a subsystem that is *almost* verified, to add a compelling feature before the safety migration reaches its part of the tree. The project’s answer is that the ordering is what makes each claim trustworthy: the parity number means something because features did not come first; the eventual safety will mean something because it came after verification; the eventual features will mean something because they sat on a verified, safe base. Rushing a stage would forfeit exactly the trust the careful ordering builds. “Done right” is, in the end, as much about the *order* the work is done in as about the work itself — and the last chapter is the honest accounting of exactly how far down that order the project has come.

Chapter 15 – Status and the Honest Gaps

Every book in this series ends by walking its own boundaries. This chapter states where the port stands, which subsystems are complete and which are partial, and how the port report and doc-count gates keep that picture accurate rather than aspirational.

The port report: what's ported, what's pending

The first honest answer to “how complete is ztmux?” is a generated one, not a claimed one. The port report ([scripts/gen_port_report.py](#), published as [port_report.html](#)) walks the Rust source and the vendored C, matches functions across the two using the same C index the annotation script builds, and reports the coverage — how much of tmux's function surface has a ported counterpart and how much remains. Because it is derived from the vendored C rather than from a hand-maintained checklist, it cannot flatter the port: refresh the vendored tmux, re-run the script, and the report reflects reality. This is the same anti-inflation discipline as the C citations and the anti-drift gate, applied to the aggregate progress number.

What the report shows, today, is a core that is substantially ported — every subsystem this book walked has its functions present and cited — with the pending surface being the long tail of tmux's less-central commands, the deeper corners of features whose common paths are done, and the platform and build variations behind the feature flags. The report is the right place to look for the *specific* pending items, because it is per-function and current; this book does not reproduce its list, because a list transcribed into prose would be stale the moment the next function is ported. The report is live; the book points at it.

The report pairs with the anti-drift gate (Chapter 12) to answer two different completeness questions. The report answers “how much of tmux is covered?” — coverage against the C. The gate answers “is everything in the ported tree actually from tmux?” — integrity of what is already there. A reader who wants the full honest picture of the port's state consults both: the report for breadth, the gate for the absence of invented surface. Neither is a number the author asserts; both are artifacts the tooling generates from the committed reference.

The value of having the completeness question answered mechanically is that it removes the temptation to round up, which is the single most common dishonesty in port projects. A project that describes its completeness in prose will, however well-intentioned, drift toward optimism. A project whose completeness is a generated report against a committed reference cannot — the number is what it is, and it moves only when real code moves it. ztmux's status, in the aggregate, is whatever the port report says today, and the book's job is to point you at the live artifact rather than to freeze a number that will be wrong tomorrow.

Complete vs. partial subsystems today

Zooming from the per-function report to the subsystem level, the picture is legible and worth stating directly. The subsystems that are *complete* in the sense that matters — ported and verified byte-for-byte by the parity suite on both Linux and macOS — are the ones this book spent its core chapters on and the ones the 122 parity cases exercise: the format language and its modifiers (the bulk of the cases), arithmetic and comparison and string operations within formats, the core session/window/pane commands ([new-window](#), [split-window](#), [list-windows](#), [select-layout](#), and their kin), the options get/set surface, and the layout arithmetic. Where a `.fmt` or `.sh` case pins a behavior, that behavior is complete in the strong sense.

The subsystems that are *ported but partial* are the ones present in the tree and transliterated but not yet exhaustively pinned by parity or not yet fully wired to behavior. The configuration surface is the clearest example: the options table accepts every option tmux accepts — it must, to parse real config files — but, as the Cargo manifest admits, some options are stored without yet driving behavior, existing “to make matching upstream’s configuration options easier.” Sixel is the notable counter-example the manifest calls out as genuinely implemented. The right way to read a partial subsystem is: the common paths are ported and the structure is faithful, but the coverage of edge cases and the wiring of every option is still being completed, and only a parity case makes a specific behavior a guarantee.

The `unsafe` dimension (Chapter 14) is orthogonal to the ported/verified dimension and worth holding separately in mind. Every subsystem, complete or partial, is currently `unsafe`-heavy — being verified against the C does not make a subsystem idiomatic Rust, it makes it faithful C-shaped Rust. So “complete” here means “faithful and verified,” not “done right” in the Chapter 14 sense. A subsystem can be fully parity-verified and still be entirely raw pointers and `static mut`; the format engine is exactly this — thoroughly exercised by the parity suite, and thoroughly `unsafe`. Keeping the two axes distinct is part of the honesty: verified is one claim, idiomatic is another, and at this stage the core is far along the first axis and early on the second.

The one number that summarizes the verified axis is the parity count, and it deserves its caveats stated once more so it is never overread. 1115/1115 means: of 1115 specific cases, each a format string or a short command scenario, all 1115 produce byte-identical output between the vendored `next-3.7` tmux and ztmux, on Linux and macOS, under fixed geometry and `LC_ALL=C` (twenty of them guarding gaps the expansion once caught and the port has since fixed; Appendix D). It does *not* mean every tmux behavior is covered — 1115 cases is a large sample, not the whole of tmux’s behavior. It is a strong, growing, reproducible measurement of a real and central slice of fidelity, and it is exactly as strong as its case count, no more. Read that way, it is the most trustworthy status indicator the project has, precisely because it claims neither more nor less than it measures.

The doc-count and man-page gates

A port that is legible today can become illegible tomorrow if its documentation is not gated, and ztmux inherits the stack-wide documentation gates for exactly that reason. The doc-count gate watches the ratio of documented to undocumented public surface and fails the build if it regresses; the man-page gate ensures the reference material stays present and buildable. These do not measure correctness — parity does that — but they measure whether the project stays *understandable*, which for a port whose entire pitch is auditability is its own kind of correctness. A faithful port that no one can read is a faithful port that no one can verify, and the doc gates keep that from happening quietly.

These gates compose with the C-signature citations (Chapter 12) to give the project an unusually high documentation floor for its age. Every ported function already carries a citation to its C origin — over two thousand of them, kept fresh by `annotate_c_links.py`’s `--check` mode — so the codebase is self-documenting in the specific dimension that matters most for a port: provenance. The doc-count gate ensures that beyond the citations, the public API carries real documentation, and the man-page gate ensures the user-facing reference exists. Together they mean a reader arriving at any level — a function, a module, a command — finds either a C citation, a doc comment, or a man page pointing them at what they need.

The gates also serve the same anti-drift purpose in documentation that the fake-fn gate serves in code. Just as code can drift from the C by growing unmatched functions, documentation can drift from the code by falling behind it — new functions with no docs, changed behavior with stale man pages. Gating the documentation ratio and the man-page presence turns “keep the docs current” from a chore that slips into a build requirement that cannot. This is the same philosophy as the rest of the project: do not rely on discipline where a gate will do, because gates run on every push and discipline does not.

That the documentation is gated at all, this early in the port, is itself a status signal. A project that gates its docs this early is a project that intends the code to be read — by reviewers checking the port against the C, by contributors learning the subsystem, by the author six months from now. The gates are a bet that legibility is worth enforcing before the code is even feature- complete, which is the right bet for a port: the whole value

proposition is that the fidelity is *checkable*, and checkable requires readable. The doc gates are how the project keeps the promise that its faithfulness can be audited rather than merely believed.

A subsystem-by-subsystem ledger

To make the complete-versus-partial distinction concrete, here is the honest state of the subsystems this book walked, read against the two axes that matter — *verified* (pinned by parity) and *idiomatic* (the Chapter 14 axis). Every subsystem below is *unsafe*-heavy, so the idiomatic axis is uniformly “early”; the interesting variation is on the verified axis:

- **Format engine** — heavily verified (the bulk of the 1115 cases), fully green, including the four corners the expansion once caught and the port then fixed (`#{!:...}`, `#{s/.../\1/}`, `#{p-N:...}`, loop-var injection; Appendix D). The most exercised subsystem in the tree.
- **Layout** — verified on the arithmetic that matters (even-split rounding, resize distribution), pinned by both unit tests and parity cases, with `main-vertical` + `main-pane-width` a named gap.
- **Options / commands** — the core get/set surface and the common session/window/pane commands are pinned by `.sh` cases; named buffers are a current gap.
- **Server / client / protocol** — exercised indirectly by every `.sh` case (each spins up a real server and client), and directly by the coexistence and detach behaviors.
- **Grid / screen / input parser** — exercised indirectly (every format reads grid-derived state; every `.sh` scenario drives the parser), and structurally faithful with full C citations, but with fewer *direct* byte-comparison cases than the format engine.
- **Terminfo back end** — ported with the capability lookup, overrides, and output minimization; its output byte stream is observable but less directly parity-cased than formats.
- **Copy mode, status line, spawn, hooks/alerts** — ported and functional, with the common paths covered and the long tail of edge cases still being pinned.

The honest one-line reading: the *computational and command surface* is thoroughly verified, the *rendering and I/O surface* is faithfully ported and indirectly exercised, and *everything* is still C-shaped Rust awaiting the idiomatic pass. Where a `.fmt` or `.sh` case pins a behavior, it is complete in the strong sense; elsewhere the structure is faithful and the coverage is growing. That is the ledger, and it is exactly as current as the parity count it is read against.

How a gap gets closed: the shape of one work item

A ledger of what is complete and what is partial is only useful if the transition from the second column to the first is a defined procedure rather than an act of inspiration. It is, and describing it makes the status claims in this chapter concrete: every remaining gap is an instance of the same work item, and the steps are the same regardless of which subsystem it lives in.

Start from a failing observation, not from the code. A gap is worth closing when something *observable* differs from the reference — a parity case fails, a command produces different output, an option is set and nothing happens. Starting from the code instead (“this function looks incomplete”) produces work with no acceptance criterion, and work with no acceptance criterion is how a port acquires plausible-looking bodies that do not quite do the right thing. The anti-drift gate of Chapter 12 exists because that failure mode is real.

Reduce it to a minimal case. Whatever the original observation, boil it down to the smallest input that still shows the divergence, under the determinism rules of Appendix D: fixed geometry, fixed locale, nothing host- or time- or pid-dependent. This is a prerequisite rather than a courtesy — the case *is* the specification for the fix, and a case that carries incidental context will pin incidental behaviour along with the behaviour you meant.

Add the case before the fix. It goes into the suite failing. This is what makes the subsequent work verifiable in the direction that matters: a case that has never been seen to fail proves nothing when it passes, because it might be passing for reasons unrelated to what it claims to test. Watching it go red first, then green, is the difference between a regression guard and a decoration.

Read the C. Find the corresponding function in `vendor/tmux/` — via the port’s own C citations where the Rust already claims a counterpart, or by name where it does not — and read what it does, including the branches your case does not exercise. The temptation at this point is to implement just enough to make the case pass, and it is the wrong move: the case is one probe into a behaviour, not the whole of it, and a fix shaped to the probe will leave the other branches wrong in ways the next case discovers.

Port the function, do not reinvent it. With the C in front of you, transliterate, following the idioms of Chapter 35. Keep the name, keep the structure, keep the error paths and their message strings. If the port needs substrate that does not exist yet — a helper module, a struct field, a whole subsystem — that is the finding, and it gets surfaced as such rather than papered over with an approximation that happens to satisfy the case.

Run the whole suite, not the case. The case going green is necessary and not sufficient. The relevant risk in a port is that a faithful implementation of one function corrects a behaviour another part of the tree had been compensating for, and the compensation now over-corrects. Only the full suite catches that, and it is cheap to run relative to the cost of shipping it.

Then check the gates. The anti-drift gate (Chapter 12) confirms the new function is real and correctly named against the C index; the C-citation tooling confirms the provenance annotation is present and accurate; the port report picks up the changed state mechanically. None of these are asked to be persuaded — they are asked to be run.

The reason to spell this out in the status chapter specifically is that it converts the remaining gaps from a worry into an estimate. Each item on the pending list is one pass through those seven steps. Some are longer than others because the C function is longer or the missing substrate is larger, but none of them requires a new idea, and none of them requires re-litigating the method. That is the actual claim this chapter is making about the project’s state: not that it is finished, but that finishing it is a known, repeatable procedure applied a countable number of times.

From port to superset: the road ahead

The honest summary of where ztmux stands is a single sentence: it is a faithful, parity-verified, `unsafe`-heavy transliteration of tmux’s core, at the boundary between the “achieve parity” stage and the “become idiomatic and safe” stage, with the superset features labeled clearly as the stage after that. Every claim in this book resolves to that sentence. The core chapters describe the transliteration; the parity and anti-drift chapters describe how its fidelity is proven and defended; the superset and “done right” chapters describe the two stages ahead. Real, usable, and still early in its own roadmap — even as the crate version tracks tmux’s 3.7 line for compatibility — is precisely the state this book describes.

The road ahead is the ordering of Chapter 14, walked forward. Stage two, safety, is the near-term work: sweep the tree subsystem by subsystem, converting faithful C-shaped Rust into idiomatic safe Rust, each conversion guarded by the parity suite so behavior is preserved and each shrinking both the `unsafe` surface and the anti-drift allowlist (the two being nearly the same set). Stage three, features, is the longer horizon: from the faithful, safe, owned core, grow the additive superset extensions the stack is positioned for — the `zterminal` and `ztmux-core` integrations, the prompt-aware features the OSC 133 tracking already lays groundwork for — each preserving parity by construction. Neither stage is a rewrite; both are migrations from a base that is already correct.

What will *not* change on that road is the discipline that got the project here, and that is the most important thing to carry out of this book. The parity suite will keep defining correct; the anti-drift gate will keep defending structure; the C citations will keep provenance legible; the port report will keep completeness mechanical; and the doc gates will keep the whole thing readable. These are not scaffolding to be removed once the port matures — they are the permanent instrumentation that lets ztmux claim faithfulness as a measured property rather than an asserted one, at every stage, forever. A superset that abandoned them would be a fork wearing tmux’s name; a superset that keeps them is what the README means by done right.

And so the book ends where it began, with the honesty that is the point. ztmux is tmux, reimplemented in Rust, ported against the C and verified against the C, byte for byte — a faithful core today, an idiomatic and safe

core tomorrow, a superset the day after. It is early software, and this book has marked the early parts as early rather than rounding up, because the project itself does: it measures its parity, gates its drift, cites its sources, and labels its roadmap. Read against that standard, ztmux is exactly what it claims to be and nothing it does not — the multiplexer itself, rebuilt in Rust, done carefully enough that every claim about it is one you can check. That checkability is the real inheritance from tmux, and the real gift to whoever reads the code next.

Chapter 16 — The Format Mini-Language in Depth

Chapter 9 introduced the `#{...}` format language as one of the three systems that make `tmux` configurable. It deserves a chapter of its own, because it is the single largest module in the tree (`src/ported/format.rs`, over two hundred kilobytes, more than two hundred C citations), it is where the bulk of the parity suite lives, and it is a genuine little programming language embedded in every status string, `display-message`, pane title, and `if-shell` condition. This chapter walks the operators one family at a time — with real inputs and the exact bytes they expand to — and marks, honestly, the handful of operators that are the current parity gaps.

The expansion pipeline: `format_expand` and `format_replace`

A format string is expanded by `format_expand` (`src/ported/format.rs`), which scans for `#{`, `#[`, and `#` escapes and hands each `#{...}` to `format_replace`. Two design details govern everything else. First, expansion is *recursive with a guard*: a variable's value can itself contain `#{...}`, and the engine re-expands it, but a counter capped at `FORMAT_LOOP_LIMIT = 100` stops a self-referential format (`set -g x '#{x}'; display -p '#{x}'`) from recursing forever — it expands a hundred deep and stops, exactly as `tmux` does, rather than blowing the stack. Second, the engine keeps a `format_expand_state` carrying flags like `FORMAT_EXPAND_TIME` (evaluate `#{t:...}` time formats) and `FORMAT_EXPAND_NOJOBS` (do not spawn `#(shell)` jobs — used where a job would be unsafe or non-deterministic), threaded through the recursion so a nested expansion inherits the right evaluation mode. The parity suite runs with jobs effectively inert and time excluded precisely because `#(date)` and `#{t:...}` are non-deterministic; the state flags are how `tmux` itself expresses that distinction.

The `#{` handler parses, in order: an optional block of *modifiers* (the operators this chapter catalogues), then a *variable name or sub-expression*, then, for the operators that take them, comma-separated *arguments*. The modifiers are accumulated into a `format_modifiers` bitset (`FORMAT_BASENAME`, `FORMAT_DIRNAME`, `FORMAT_LITERAL`, `FORMAT_QUOTE_SHELL`, `FORMAT_TIMESTRING`, `FORMAT_SESSIONS`, `FORMAT_WINDOWS`, `FORMAT_PANES`, `FORMAT_CLIENTS`, `FORMAT_LENGTH`, `FORMAT_WIDTH`, `FORMAT_NOT`, and the rest), each corresponding to one letter or symbol after the `#{`. The big `match` that parses them is the heart of the module, and every case below is one arm of it.

Variable substitution and the `FORMAT_TABLE`

The simplest format is a bare variable: `{pane_current_command}` expands to the pane's foreground process name, `{session_name}` to the session's name, `{window_index}` to the window's index. These names resolve two ways. Dynamic per-object fields (`session_name`, `window_width`, `pane_pid`, and hundreds more) are computed from the live server state at expansion time by `format_create_add`-populated entries and the big resolver. A second set — the *callback* variables — live in `FORMAT_TABLE`, a sorted array binary-searched by `format_table_get`; these are the ones whose value is computed by a small function rather than stored, like `host_short`, `pid`, `version`, `cursor_x`, `client_terminame`, `mouse_x`, `pane_mode`, and the alternate-screen and scroll-region reporters. The table is kept sorted so lookup is a `binary_search_by`, and Appendix B lists the full variable set.

Worked example, against a fresh 80×24 detached session (which is exactly the parity harness's environment):

```
$ ztmux display-message -p '#{window_width}x#{window_height}'
80x24
$ ztmux display-message -p 'panes=#{window_panes} idx=#{window_index}'
panes=1 idx=0
```

Because so much of the server's state is reachable this way, a format string is also an *introspection query* — `list-panes -F '#{pane_pid} #{pane_current_command}'` is a structured dump of the pane table — and that dual role is why the parity suite is mostly format cases: a matching expansion of `#{window_panes}` after a split is evidence the pane tree itself matches.

Arithmetic: `#{e|op|:a,b}`

The `e` modifier introduces arithmetic, and its syntax is `#{e|OP|FLAGS:A,B}` where `OP` is the operator, the optional `FLAGS` control float precision, and `A,B` are the operands (themselves formats, so they can be nested). The operators map to an `Operator` enum evaluated in `f64`: `+` add, `-` subtract, `*` multiply, `/` divide, `%` modulus, `&` bitwise-and, `|` bitwise-or, and the comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) which yield `1` or `0`. Worked examples straight from the parity cases:

```
#{e|+:2,3}      → 5      (010_arith_add.fmt)
#{e|-:10,4}     → 6
#{e|*:6,7}      → 42
#{e|/:7,2}      → 3      (integer division by default)
#{e|=:17,5}     → 2
#{e|/f2:22,7}   → 3.14   (f2 = two decimal places)
#{e|&:6,3}      → 2      (bitwise and)
```

Two faithful subtleties the parity suite pins. Division and modulus by zero must match tmux's exact output rather than crash or print `inf` (`202_arith_divzero.fmt`, `203_arith_modzero.fmt`) — tmux has a defined result and ztmux reproduces the bytes. And equality on floats is not `==` on IEEE bits but an epsilon compare — `(mleft - mright).abs() < 1e-9` — because tmux's `#{e}` treats near-equal floats as equal, and a naive port that used exact float equality would diverge on `#{e|=:0.1,0.1}` computed two different ways. These are the corners a spec would miss and a byte comparison catches.

Comparisons and booleans: `#{==:a,b}`, `#{&&:...}`, `#{||:...}`, `#{m:...}`

Outside arithmetic, formats have direct comparison operators that work on strings and numbers: `#{==:a,b}`, `#{!=:a,b}`, `#{<:a,b}`, `#{>:a,b}`, `#{<=:...}`, `#{>=:...}`, plus the boolean combinators `#{&&:a,b}` and `#{||:a,b}`, and the pattern matcher `#{m:pattern, string}` (fnmatch-style glob) with a `#{m/r:...}` regex variant. Each yields `1` (true) or empty (false) — note *empty*, not `0`, for the boolean/conditional forms, which matters because conditionals test for emptiness. Examples:

```
#{==:abc,abc}   → 1
#{==:abc,xyz}   → (empty)
#{&&:1,1}        → 1
#{m:*.rs,main.rs} → 1      (fnmatch glob)
#{m:*.rs,main.txt} → (empty)
```

The `!` logical-not operator, `#{!:...}`, inverts a truth value, and this is one of the current parity gaps: tmux prints `1` for `#{!:}` on an empty (false) argument, and ztmux prints empty because the `!` operator is not yet wired into the modifier parse (cases 548–550, Appendix D). It is a small, bounded fix — one arm of the modifier `match` — and it is kept red in the suite until it lands.

Conditionals: `#{?condition,then,else}`

The conditional is `#{?COND,THEN,ELSE}`: if `COND` expands to a non-empty string it yields `THEN`, otherwise `ELSE`. The condition is commonly a variable (truthy when non-empty) or a comparison. This is the workhorse of status-line configuration — showing one thing when a pane is zoomed and another when it is not, one color when a window has activity and another when it does not:

```
#{?pane_active,[active],[inactive]}    → [active]    (in the active pane)
#{?window_zoomed_flag,Z,}              → Z or empty
#{?#{==:#{window_panes},1},single,multi} → single      (one pane)
```

The last example shows nesting: the condition is itself a `#{==:...}` over `#{window_panes}`, and the whole thing composes because every part is a format re-expanded by the same engine. The comma is the argument separator, and a literal comma inside a branch is escaped as `#,` — a detail the parity string-op cases exercise.

String operations: `#{s/.../.../:}`, `#{=N:}`, `#{b:}`, `#{d:}`, `#{q:}`, `#{n:}`

The string family is large. `#{b:path}` is basename, `#{d:path}` is dirname — both implemented against the ported `basename/dirname` shims:

```
#{b:/usr/local/bin/ztmux} → ztmux
#{d:/usr/local/bin/ztmux} → /usr/local/bin
```

`#{=N:string}` limits width to `N` columns from the left (`#{=-N:string}` from the right), respecting wide characters; `#{=/N/...:string}` adds a trailing marker when truncated. `#{n:string}` is the length. `#{q:string}` shell-quotes (escaping so the result is safe to hand to a shell), and `#{q:...}` in `FORMAT_QUOTE_STYLE` form escapes for a style string instead. `#{t:variable}` formats a time-valued variable through `strftime` (with `#{t/f/FORMAT:...}` choosing the format and a `p` flag for a “pretty”/relative form).

The substitution operator `#{s/PATTERN/REPLACEMENT/:string}` runs a regex substitute (`src/ported/regsub.rs`), with a trailing `g` flag for global replace:

```
#{s/foo/bar/:foobar}      → barbar    (first match)
#{s/o/0/g:foobar}         → f00bar     (global)
```

Regex *backreferences* in the replacement — `\1`, `\2` referring to capture groups — are the second current parity gap: on input `abcd` with a pattern capturing pairs, `#{s/(.)(.)/\2\1/g:abcd}` should yield `badc`, but `ztmux` emits the literal digit next to the capture (`b2a1d2c1`) because the backreference substitution is not wired in the replacement builder (cases 566–568, 572, Appendix D). Like `#{!:...}`, it is a localized fix in one function, held red until done.

Padding: `#{pN:}` and the left-pad gap

The `p` modifier pads a field to a fixed width: `#{pN:string}` right-aligns to width `N` (padding on the left), and the fill character defaults to a space. This is how status lines keep columns aligned regardless of content width. The right-pad and left-pad forms are `#{pN:}` and `#{p-N:}`. The left-pad form `#{p-N:...}` is the third current parity gap: `tmux` treats `N` as the *total field width*, but `ztmux` currently pads to `N` *plus* the content length, so a `#{p-5:ab}` that should produce a five-wide field produces a seven-wide one (cases 589, 592, Appendix D). Again bounded, again kept red until the arithmetic is corrected to a total-width computation.

Loops: `#{S:...}`, `#{W:...}`, `#{P:...}`, `#{L:...}`

The loop modifiers expand a sub-format once per object: `#{S:fmt}` once per session, `#{W:fmt}` once per window in the current session, `#{P:fmt}` once per pane in the current window, and `#{L:fmt}` once per client. They are implemented by `format_loop_sessions`, `format_loop_windows`, `format_loop_panes`, and the client loop, each of which iterates the relevant tree and re-expands `fmt` with that object pushed as the current context:

```
#{W:#{window_index}:#{window_name} } → 0:zsh 1:vim 2:logs
```

Inside a loop, tmux injects two synthetic variables per iteration — `loop_index` (the zero-based position) and `loop_last_flag` (1 on the final iteration) — which `format_loop_sessions` sets via `format_add!(nft, "loop_index", ...)` and `format_add!(nft, "loop_last_flag", ...)`. The `#{S:}/#{W:}/#{P:}` capital-letter loop modifiers are the fourth current parity gap: they iterate correctly but do not inject `loop_index/loop_last_flag`, so an inner `#{loop_index}` expands empty where tmux prints the counter (cases 676–678, Appendix D; the C reference is `vendor/tmux/format.c:4776`). The lowercase per-object loops used above work; the gap is specifically the injected iteration variables in the capital-letter forms.

Literals and the `format_unescape` trap

The `l` modifier, `#{l:some text}`, means “this is a literal — unescape it and stop,” which is how you put otherwise-interpreted characters into a format. Its handler calls `format_unescape(src/ported/format.rs`, citing `vendor/tmux/format.c:4281`), and this small function is the site of the most instructive bug in the port’s history. The C is a `for (; s != end; s++)` loop whose `s++` runs on every iteration, including the ones that hit a `continue`; an early Rust port translated it to a `while` loop that advanced `s` on most branches but forgot to on the normal path, so on a plain character it looped forever, writing past the `xmalloc(strlen + 1)` buffer and crashing the server. The parity case was a single `#{l:...}` string that tmux expanded and ztmux died on. The fix advances the pointer on every path, matching the C’s per-iteration increment, and the function now carries a comment documenting the trap so it cannot be reintroduced silently. It is the clean archetype of the whole methodology: a byte-for-byte parity case turned a mysterious crash into a one-line, C-anchored fix.

Quoting, escaping, and the `#[...]` style directive

Two mechanisms sit next to the `#{...}` operators and are easy to confuse with them. The first is *escaping*: because `#` introduces a format and `,` separates arguments, a literal `#` is written `##` and a literal comma inside a conditional branch is written `#,`. The `#{q:string}` operator (`FORMAT_QUOTE_SHELL`) does the inverse for shell contexts — it backslash-escapes a string so it is safe to hand to a shell, which is how `run-shell "grep #{q:pane_title}"` avoids being broken by a title containing spaces or quotes. A parallel `#{q:...}` style-quote form (`FORMAT_QUOTE_STYLE`) escapes for a style string instead. Getting the quoting exactly right matters because a format that under-quotes is a shell-injection bug and a format that over-quotes prints stray backslashes; the parity string cases (`037_str_quote` and `kin`) pin it.

The second mechanism is the `#[...]` style directive, which is not a `#{...}` expansion at all but an inline styling instruction the drawing layer (`format_draw.rs`, Chapter 20) interprets. `#[fg=red,bold]` sets the foreground and bold attribute for the text that follows in a status string; `#[default]` reverts; `#[reverse]` inverts. Styles and formats compose in the same string — a status line is typically a mix of `#{...}` values and `#[...]` directives — but they are processed by different machinery: `format_expand` resolves the `#{...}` into values, and `format_draw` walks the resulting string turning `#[...]` runs into actual cell attributes and recording the style *ranges* that make status-line regions clickable (Chapter 20). Keeping the two straight — expansion versus styling — is the last piece of reading a real status-line format fluently, and both are ported so a `.tmux.conf`’s status string renders identically, colors, clickable ranges, and all.

More operators: character, color, search, and re-expansion

Beyond the big families there is a tail of single-letter operators the modifier `match` handles, each small but real. `#{a:N}` (`FORMAT_CHARACTER`) turns a numeric code point into its character — `#{a:65}` is `A` — which is how a format produces a byte it could not write literally. `#{c:colour}` (`FORMAT_COLOUR`) resolves a color name or spec to its numeric value through the same `colour.rs` machinery the grid uses, useful for computing styles. `#{w:string}` (`FORMAT_WIDTH`) reports the display *width* of a string in columns, counting wide characters as two — distinct from `#{n:string}` (`FORMAT_LENGTH`), which counts characters — so a status line can budget columns correctly for CJK text.

Two operators control *re-expansion*, which is subtle enough to trip up format authors. `#{E:variable}` (`FORMAT_EXPAND`) expands the variable's value a second time, so a variable whose value itself contains `#{...}` gets those inner formats evaluated; `#{T:variable}` (`FORMAT_EXPANDTIME`) does the same but also enables time expansion for the inner pass. These exist because, by default, a variable's value is used verbatim, and sometimes you have stored a format *in* an option (a user status template, say) that you want evaluated when read. The `#{N:variable}` operator (`FORMAT_WINDOW_NAME` / `FORMAT_SESSION_NAME`, selected by a `/w` or `/s` argument) checks whether a named window or session exists, yielding truthy or empty — the building block for a status line that shows something only when a particular session is alive. And `#{C:pattern}` (the search modifier) searches the pane's content for a pattern and reports the matching line, which is how a format can surface “the line where the error is.” Each of these is one arm of the same big modifier `match`, ported from `tmux`, and each is a reminder that the format language is genuinely a language: it has character construction, type coercion, existence tests, and controlled re-evaluation, not just variable interpolation.

Why the format language is the parity suite's center of gravity

Step back and the reason formats dominate the parity suite is structural. A format case is a single deterministic string, expanded by `display-message -p` against a fixed session, diffed byte-for-byte — cheap to write, trivially reproducible, and, because formats read so much of the server's state, an enormous surface covered through one uniform mechanism. Arithmetic, comparisons, string ops, conditionals, padding, loops, and every per-object variable are all reachable as `.fmt` cases, which is exactly why the suite grew from 122 to 1115 mostly by adding formats, and exactly why the four format corners the expansion caught (`#{!:...}`, `#{s/.../\1/}`, `#{p-N:...}`, loop-var injection) were each a named case with a reproduction rather than an unknown — and are now green, each one still guarding its corner. The format engine is where the port is most thoroughly exercised, most thoroughly `unsafe`, and most cleanly measured — a microcosm of the whole project.

Chapter 17 — The Wire Protocol and `imsg` Framing

Chapters 3 and 4 walked the server and the client as two halves of one binary. This chapter is about the seam between them: the socket messages that carry a client's attach, its keystrokes, its resizes, and its detach to the server, and the server's replies back. `tmux` does not invent a bespoke protocol framer — it uses OpenBSD's `imsg` library, the same small message-passing layer OpenBSD daemons use to talk between privilege-separated processes — and `ztmux` ports the message set and the framing faithfully, because the wire format is an internal contract both halves must agree on exactly.

`imsg`: length-prefixed messages with passed file descriptors

`imsg` is a deliberately small idea: a message is a fixed header followed by a payload, buffered and framed over a Unix domain socket, with the ability to pass an open file descriptor alongside the bytes. The header carries a message *type* (a `u32` — the `MSG_*` values below), a *length*, a *peer id*, and a *pid*, and the library handles the buffering so that a partial read never delivers half a message: a reader pulls complete messages out of an internal buffer, and a writer queues messages that drain as the socket becomes writable. `ztmux` drives this through `src/ported/proc.rs` (the `tmuxproc/tmuxpeer` abstraction) and the buffered event layer, with the `evbuffer_*` and `imsg` calls appearing in the anti-drift allowlist (Chapter 12) precisely because they are library functions with no `vendor/tmux` definition to match — real functions, just not `tmux`'s own.

The file-descriptor-passing capability is not a nicety here; it is load-bearing to the whole architecture. When a client attaches, it does not *describe* its terminal for the server to open — it passes the actual open descriptors for its stdin and stdout across the socket, using `imsg`'s `SCM_RIGHTS`-based fd transfer. That is what `MSG_IDENTIFY_STDIN` and `MSG_IDENTIFY_STDOUT` carry: duplicated file descriptors, not paths. The consequence, revisited from Chapter 10, is that the server's tty back end writes directly to the client's real terminal fd — it is driving hardware it received over a socket. No other part of the design would work without fd passing; it is why the server, a daemon with no controlling terminal of its own, can nonetheless paint your screen.

`PROTOCOL_VERSION = 8` and the version handshake

The very first thing the two halves do is agree they speak the same protocol. `src/ported/tmux_protocol_h.rs` declares `pub const PROTOCOL_VERSION: i32 = 8`, and it is the first field checked: on connect the server sends `MSG_VERSION`, and the client compares the server's version against its own `PROTOCOL_VERSION`. On mismatch it prints `protocol version mismatch (client {}, server {})` and refuses to proceed. This guard exists because the `imsg` protocol is an *internal build contract*, not a stable ABI — the message set and payload layouts can change between builds, and a client from one build talking to a server from another could silently corrupt state. The version bump is how “these two processes must come from the same code” is enforced mechanically rather than hoped for. It is also why, in normal use, you never see the check fire: the client that autostarts a server is the same binary, so the versions always match unless you have mixed builds behind one socket.

The message catalogue

The `MSG_*` set is small and every message has a clear job. They group naturally:

- **Version and readiness.** `MSG_VERSION` (protocol check), `MSG_READY` (server signals the attach is complete), `MSG_WAKEUP` (nudge a waiting client).
- **The identify handshake** — the client's self-description, one fact per message: `MSG_IDENTIFY_LONGFLAGS` (64-bit client flags), `MSG_IDENTIFY_FLAGS`, `MSG_IDENTIFY_TERM` (the `TERM` string), `MSG_IDENTIFY_FEATURES` (negotiated terminal features), `MSG_IDENTIFY_TTYNAME`, `MSG_IDENTIFY_CWD` and `MSG_IDENTIFY_OLDPWD`, `MSG_IDENTIFY_TERMINFO` (one per terminfo capability), `MSG_IDENTIFY_STDIN` and `MSG_IDENTIFY_STDOUT` (the passed fds), `MSG_IDENTIFY_CLIENTPID`, `MSG_IDENTIFY_ENVIRON` (one per environment variable), and `MSG_IDENTIFY_DONE` to close the stream. The server assembles a client object as these arrive.
- **Command and control.** `MSG_COMMAND` (run a command line), `MSG_SHELL` (the shell-command path), `MSG_EXEC` (have the client `exec`l a program in its own process), `MSG_LOCK/MSG_UNLOCK` (run the lock command), `MSG_FLAGS` (client flag changes), `MSG_RESIZE` (new terminal geometry — sent on attach and on every `SIGWINCH`).
- **Exit and detach.** `MSG_DETACH` and `MSG_DETACHKILL` (detach, optionally killing the session), `MSG_SUSPEND` (stop the client with `SIGTSTP`), `MSG_EXIT`, `MSG_EXITED`, `MSG_EXITING`, `MSG_SHUTDOWN`.
- **File I/O over the socket.** `MSG_READ_OPEN/MSG_READ/MSG_READ_DONE/MSG_READ_CANCEL` and `MSG_WRITE_OPEN/MSG_WRITE/MSG_WRITE_READY/MSG_WRITE_CLOSE` — the messages behind `load-buffer/save-buffer` and control-mode file transfer, which let the server read and write files *in the client's context* (with the client's permissions and working directory) rather than its own.
- **Legacy fd carriers.** `MSG_OLDSTDIN/MSG_OLDSTDOUT/MSG_OLDSTDERR` — the older descriptor-passing messages, kept for the compatibility they name.

An attach, message by message

Putting the catalogue in motion, here is the sequence when you run `ztmux attach`:

```
client → connect() to <dir>/ztmux-<uid>/default
server → MSG_VERSION (8)
client → (checks version, aborts on mismatch)
client → MSG_IDENTIFY_LONGFLAGS, _TERM, _FEATURES, _TTYNAME, _CWD,
        _TERMINFO (*N), _STDIN(fd), _STDOUT(fd), _CLIENTPID,
        _ENVIRON (*N), _IDENTIFY_DONE
server → (builds the client, binds it to a session, sizes it)
server → MSG_READY
client → sets CLIENT_ATTACHED, sends initial MSG_RESIZE
... steady state: keys in, drawing out, MSG_RESIZE on SIGWINCH ...
client → MSG_DETACH (or server → MSG_DETACH / MSG_SHUTDOWN)
server → tears down the client; session keeps running
```

Two properties of this exchange are worth naming. It is *asymmetric*: the client front-loads everything about its terminal in the identify burst, then goes quiet and becomes a relay, because all intelligence lives server-side (Chapter 4). And it is *idempotent across re-attach*: a second `attach` from a different terminal runs the identical handshake with that terminal's facts, and the server rebinds the same session — the protocol has no notion of “resume,” because there is nothing to resume; the session never stopped. The wire protocol is thin for the same reason the client is thin: it carries a terminal's identity and a stream of keys and resizes, and lets the server do the rest.

Control mode: the protocol as a text stream

There is a second face of the protocol worth naming, because it is the seam the superset's integrations lean on hardest. A client can attach in *control mode* (`ztermux -CC`), and instead of driving a terminal it exchanges a line-based text protocol with the server: it sends commands and receives structured %-prefixed notifications — `%output` when a pane produces bytes, `%window-add`, `%window-close`, `%layout-change`, `%session-changed`, and the rest — each corresponding to the internal notifications of Chapter 21. `src/ported/control.rs` and `src/ported/control_notify.rs` implement this, translating the server's internal events into the control-mode text lines a program can parse. Control mode is how a GUI front end embeds a tmux server: it drives the server programmatically and mirrors the server's state from the notification stream rather than scraping a rendered terminal.

This is precisely the seam `zterminal` uses today against the real tmux, and the reason the superset story (Chapter 13) is more than aspiration: a control-mode client and a server that share a codebase can evolve the protocol together — share types, share the socket convention — rather than one reverse-engineering the other. Porting control mode faithfully keeps ztmux a drop-in target for anything that already speaks tmux control mode, and porting the underlying notification vocabulary (Chapter 21) is what makes the control-mode stream identical. The text protocol and the `imsg` protocol are two views of the same event flow: `imsg` for the terminal-driving client, control mode for the programmatic one, both fed by the same server-side state changes.

Why porting the protocol byte-exactly matters

It would be tempting, in a Rust rewrite, to “improve” the protocol — use a nicer serialization, drop the legacy messages, restructure the identify burst. The project refuses, and the refusal is the same discipline as everywhere else. The protocol is observable at the boundary with `ztermux-core` (the sibling client engine that speaks the tmux wire protocol) and it is the seam the superset's `zterminal` integration (Chapter 13) will build on. Keeping `PROTOCOL_VERSION` and the `MSG_*` set faithful to tmux's — and evolving them deliberately rather than incidentally — is what keeps that seam legible. A protocol that drifted casually would be a protocol no sibling tool could trust, which is exactly the kind of un-owned dependency the whole project is built to avoid.

Chapter 18 — Copy Mode and the Grid Reader

Chapter 5 introduced copy mode as a cursor and a selection over the grid, and promised a closer look. This chapter is it: how `src/ported/window_copy.rs` turns a pane into a navigable, searchable, extractable view of its own scrollbar, driven by the motion primitives in `src/ported/grid_reader.rs`, and how the whole thing is a *mode* layered on the grid rather than a separate program. Copy mode is where the grid stops being a rendering buffer and becomes a data model you can walk.

The mode layer and the window mode stack

A pane is normally showing the live output of its program, but tmux can push a *mode* onto it — copy mode, the clock, the tree/menu choosers — that takes over the pane’s display and key handling until it is popped. The machinery is `src/ported/mode_tree.rs` and the per-mode `window_mode` structs; copy mode’s is `window_copy_mode`. Entering copy mode (`copy-mode`, or the default prefix-`[]`) calls `window_copy_init`, which snapshots the pane’s current screen into a copy-mode screen and starts a virtual cursor at the live cursor position. From that instant the keys you press are looked up in the `copy-mode` or `copy-mode-vi` key table (Chapter 9) rather than sent to the program — which is why `j` and `k` move the copy cursor under vi keys, and `C-Space` starts a selection under emacs keys. The program underneath keeps running and its grid keeps updating; you are navigating a view, not pausing the world.

The vi-versus-emacs split is not a special case in the copy-mode code — it is just which key table is active, set by the `mode-keys` option. `bind-key -T copy-mode-vi v` adds a vi-mode binding; the same motions dispatch to the same `window_copy_*` functions regardless. This is the payoff of the table-driven key system: copy mode inherits two full keymaps for free, and a user’s custom copy-mode bindings compose with the defaults exactly as under tmux.

The grid reader: motions over stored lines

The virtual cursor’s movements are the `grid_reader` — a `gd` grid pointer plus a `cx/cy` position — and `src/ported/grid_reader.rs` holds the motion vocabulary, each a faithful port of tmux’s `grid_reader.c`. `grid_reader_start` initializes it; then the motions: `grid_reader_cursor_left/right/up/down` for single steps, `grid_reader_cursor_start_of_line/end_of_line` for line ends, `grid_reader_cursor_back_to_indentation` for the first non-blank, `grid_reader_cursor_next_word/previous_word/next_word_end` for word motion, `grid_reader_cursor_jump/jump_back` for in-line character search (the `f/F/t/T` family), and `grid_reader_line_length` and `grid_reader_in_set` as the primitives the others build on. Two details make these faithful rather than approximate. `grid_reader_handle_wrap` decides whether moving off the end of a line advances to the next line or stops, honoring the wrapped-line flag so a soft-wrapped long line is navigated as one logical line. And word motion consults the `word-separators` option through `grid_reader_in_set`, so what counts as a “word” is exactly tmux’s configurable set, not a hard-coded `\w`.

The reader also seamlessly crosses the boundary between the visible screen and the scrollbar above it, because it walks the grid’s full coordinate space including history — which is why scrolling up in copy mode flows smoothly from the current screen into stored lines with no seam. `window_copy_scroll_up/scroll_`

`down/_scroll_to` move the viewport over that space, and the copy cursor's `cy` can range across the whole `hsize + sy` of stored lines.

Selection and extraction

A selection is a `screen_sel` on the copy-mode screen — `sx/sy` start, `ex/ey` end, a `rectangle` flag for block versus stream selection, and a cell template for how selected cells are drawn (Chapter 5). As the cursor moves with a selection active, `window_copy_update_selection` grows it, and the renderer calls `screen_check_selection(px, py)` for every cell to decide whether to draw it highlighted. Block selection (the rectangle flag, toggled by `v` under vi keys) selects a column range across multiple lines rather than a stream from start to end — the same distinction tmux makes, ported exactly.

Extraction — the “copy” in copy mode — is defined against the grid's string functions. `window_copy_copy_selection` walks the selected region and builds a string through `grid_string_cells`, whose `grid_string_flags` decide whether the copy preserves styling (embedding the SGR escape sequences via `grid_string_cells_fg/_bg/_us/_code`) or strips to plain text, and whether trailing spaces are trimmed. The result goes into a paste buffer (`src/ported/paste.rs`), and the sibling `window_copy_copy_pipe` and `window_copy_copy_line` route it instead to a pipe command or a single line. This is why a copy-mode selection can round-trip color and attributes when you want it to and come out clean when you do not: the extractor can regenerate the exact escape sequences that produced each cell, or omit them, on a flag.

Search over the grid

Incremental search in copy mode (`/` and `?` under vi keys) is a scan over the grid, and `window_copy_search` with its `window_copy_search_compare` and `window_copy_search_back_overlap` helpers implement it. A search walks lines comparing against the query — plain or, with the regex option, as a pattern — and moves the copy cursor to the match, tracking the current-match and all-matches highlight styles (`copy-mode-current-match-style`, `copy-mode-match-style`, `copy-mode-mark-style` — real options in the table). Backward search and the overlap handling exist so a match spanning a wrap boundary, or overlapping the current position, is found correctly. Because search is just a grid scan, it searches the scrollbar too, which is the whole point: copy mode's search is how you find a line that scrolled off the screen a thousand lines ago.

Copy mode as proof that the grid is a data model

The recurring theme of Chapter 5 returns here fully realized: the grid is not merely a buffer that gets painted, it is a queryable, navigable, extractable data structure, and copy mode is the feature that exercises every one of those verbs. Navigation is the grid reader; query is search; extraction is `grid_string_cells`; selection is `screen_sel`. None of these required new storage — they are all operations on the same grid the input parser fills and the tty back end draws. That is why copy mode can scroll into history without a seam, preserve styling on copy, and search a thousand lines of scrollbar: it is not a separate subsystem bolted on, it is the grid used as what it always was — a model, not just a picture.

Chapter 19 – Spawning Panes and Windows

Every pane runs a real program in a real pseudo-terminal, and getting from a `split-window` command to a running shell in a new `pty` is the job of `src/ported/spawn.rs`. It is a small module — `spawn_window`, `spawn_pane`, and the logging helper `spawn_log` — but it is where the server crosses from managing abstract state into managing operating-system processes, and it is the site of one of the three headline parity bugs (Chapter 11). This chapter walks the spawn path and the fork/exec dance that a faithful port has to get exactly right.

From command to child: `spawn_window` and `spawn_pane`

`spawn_window` creates a whole new window with its first pane; `spawn_pane` adds a pane to an existing window (the split case). Both take a `spawn_context` — the resolved target, the command to run (or `None` to run the default shell), the working directory, the environment, and flags — and both funnel into the same core work: allocate the window and/or pane structures, create the pane's screen and grid (Chapter 5), open a pseudo-terminal, fork a child, and in the child set up the environment and `exec` the program. `spawn_pane` is what `split-window` and `respawn-pane` call; `spawn_window` is what `new-window`, `new-session`, and `respawn-window` call. The `respawn_*` variants reuse an existing pane whose program has exited, which is why `remain-on-exit` can leave a dead pane in place and `respawn-pane` can bring it back to life in the same slot.

The environment handed to the child is assembled, not inherited raw. The `update-environment` option names the variables copied from the attaching client into the session environment (`src/ported/envIRON.rs`), so that `DISPLAY`, `SSH_AUTH_SOCK`, and the rest reflect the client you attached from rather than the stale values the daemon was started with. `spawn_pane` merges the session environment, the window's overrides, and the per-spawn additions into the child's environment, and sets `TMUX` (well, the `ztermux` equivalent) so a program inside a pane can tell it is running under the multiplexer. This is faithful `tmux` behavior and it is why re-attaching from a new SSH session and opening a new window gets the *new* session's `SSH_AUTH_SOCK`.

Where a new pane starts

One `spawn_context` field deserves separate treatment because it is the source of more user confusion than the rest of the spawn path combined: the working directory. The question “where does a new pane start” has a layered answer, and the layers exist because three different parties have an opinion.

The command can say so explicitly, and an explicit path wins. Absent that, the session carries a default path, which is what makes “new windows open where I told this session to open them” work across a long-lived session. Absent that, the resolution falls back to the context the command was issued from. The layering is the same shape as option scoping (Chapter 33): the most specific statement of intent wins, and each fallback is a broader default.

The subtlety is that the *pane's current* directory and the *pane's start* directory are different facts, and only the second is the multiplexer's. Once a shell is running, it changes its own directory freely, and the multiplexer knows this only because a format variable exposes what the pane's process is currently using — a fact read from the operating system, not a fact the server set. This is why a split “starting in the same directory” is not

automatic in the way users expect: it requires the spawn to be told to use the source pane's *current* directory rather than the session default, which is precisely why configurations that want that behaviour set it explicitly on the split and new-window bindings. The port reproduces the layering rather than smoothing it, because smoothing it would change where every existing configuration's new panes open.

The pseudo-terminal and the fork/exec window

A pane is backed by a pty pair: the server holds the master fd (watched by the event loop — when the child writes, an event fires and the bytes flow to the input parser, Chapter 6), and the child runs with the slave fd as its controlling terminal. Between `fork` and `exec` the child does the delicate work that must happen in the child but before the new program takes over: it puts itself in a new session (`setsid`), makes the pty slave its controlling terminal, sets the terminal size to the pane's size, redirects stdin/stdout/stderr to the slave, restores default signal handling, and closes the inherited descriptors it must not leak to the child program.

That last step — closing inherited descriptors — is where the macOS parity bug lived, and it is worth retelling from the spawn side because it shows how load-bearing “faithful” is even in a corner this deep. The child called `closefrom` to close every descriptor at or above a low number, and the port's `closefrom` looped `close()` from that number up to `RLIMIT_NOFILE`. On the server, `RLIMIT_NOFILE` is raised to a huge value, so on macOS the child spun through *millions* of `close()` calls and hung before it could `exec` — leaving `#{pane_current_command}` reporting `ztmux` (the not-yet-replaced server binary) instead of the real command. The parity case `294_pane_cmd_fmt` caught it, passing on Linux and failing on macOS, and the fix ports tmux's macOS-specific `closefrom` that enumerates the actually-open descriptors with `proc_pidinfo(PROC_PIDLISTFDS)` and closes only those. The child reaches `exec` immediately, and the format reports the true command. A whole-second hang between fork and exec, invisible on one platform, surfaced only because the parity suite runs cross-platform and compares bytes.

Reaping children and pane death

The other end of a pane's life is handled by the server's `SIGCHLD` path (Chapter 3): `server_child_signal` dispatches to `server_child_exited` or `server_child_stopped`, which find the pane by pid and mark it dead (recording the exit status and signal, exposed as `#{pane_dead}`, `#{pane_dead_status}`, `#{pane_dead_signal}`) or continued. What happens to a dead pane is policy: by default the pane and, if it was the last, the window are destroyed; with `remain-on-exit` set, the dead pane stays, its `remain-on-exit-format` shown, ready for `respawn-pane`. The whole lifecycle — spawn a child into a pty, watch its master fd, reap it on exit, apply the death policy — is the server acting as a process manager, and `spawn.rs` is the front door where new processes enter that management.

remain-on-exit as an instrument

`remain-on-exit` was introduced above as the policy that keeps a dead pane in place, and it is usually described as a convenience. It is more useful than that: it is the only way to see the *output of a program that died*, which is otherwise destroyed along with the pane in the same instant the interesting thing appeared on it.

The failure it addresses is specific and common. A pane's command fails at startup — a missing binary, a bad argument, a shell that exits because its rc file is broken — and the pane, having no more program, is destroyed. The error message was printed, rendered into the grid, and freed within milliseconds. Turning `remain-on-exit` on converts that into a pane that stays, showing both the final screen and, via `remain-on-exit-format`, a rendering of the exit status and signal the reaping path recorded. From there `respawn-pane` retries in place, which makes the edit-retry loop fast: fix the config, respawn, observe, without recreating the layout.

The relationship to the dead-pane format variables is what makes the diagnosis complete rather than approximate. The reaping path records the exit status and the signal separately, and both are exposed, so “exited

non-zero” and “was killed” are distinguishable — a distinction that matters exactly when you are debugging why a pane will not stay up, because the two have entirely different causes. A pane that died from a signal was killed by something; a pane that exited with a status decided to leave.

Chapter 20 — The Status Line and Screen Redraw

The visible frame a tmux user stares at all day is drawn by two subsystems working together: `src/ported/screen_redraw.rs`, which composites the panes and their borders into the client's terminal, and `src/ported/status.rs`, which builds the status line at the top or bottom. Both are downstream of everything the earlier chapters built — the grid supplies the pane contents, the layout supplies the rectangles, the format engine supplies the status text — and both are heavily driven by options and formats, which makes them a good closing look at how the faithful core turns state into pixels.

Screen redraw: compositing panes and borders

`screen_redraw_screen` is the top-level draw for a client: it walks the window's visible panes and, for each, calls `screen_redraw_draw_pane` to blit that pane's grid into its rectangle, then `screen_redraw_draw_borders` to draw the lines between panes. The border drawing is more subtle than it looks, because a border cell's glyph depends on its neighbors — a `+` where four panes meet, a `T` where three do, straight lines elsewhere — and `screen_redraw_cell_border` and `screen_redraw_check_cell` compute, per cell, which border character and style belong there. The active pane's border can be styled differently (`pane-active-border-style`), and the border can carry a per-pane status line (`pane-border-status`, drawn by `screen_redraw_draw_pane_status`) formatted from `pane-border-format`. This is why a tmux screen's borders “know” the shape of the layout: each border cell is computed from the geometry the layout tree (Chapter 7) defines.

Redraw is differential wherever it can be, tying back to Chapter 10's output path. The screen-write layer marks cells dirty as the grid changes; the redraw walks only what changed and hands minimal `tty_cmd_*` operations to the tty back end, which turns them into the shortest terminal sequences. A full redraw (`screen_redraw_check_is`) decides when one is needed — a resize, a mode change, a client refresh) repaints everything; the common case repaints a few cells. The status line, pane contents, and borders are all composited into the same client terminal through the same back end, per client, so two clients of different sizes each get a correct frame (Chapter 4's multi-client story, seen from the draw side).

The status line: `status.rs` and the format-driven bar

The status line is built by `status_redraw`, and it is almost entirely a format-expansion exercise. The bar has a left section (`status-left`, width-capped by `status-left-length`), a right section (`status-right`, `status-right-length`), and a middle list of window-status entries — one per window, formatted by `window-status-format`, with the current window using `window-status-current-format` and separators from `window-status-separator`. Every one of those is a `#{...}` format string expanded through Chapter 16's engine against the relevant session/window context, then laid out by `status_redraw` honoring `status-justify` (left/centre/right/absolute-centre) and `status-position` (top or bottom). `status_line_size` and `status_at_line` compute how many rows the status occupies (it can be multi-line) and where it sits, so the pane area and the status area never overlap.

Because the whole bar is formats, everything Chapter 16 described is available in it: a default `status-right` like `#{?client_prefix,^B ,}"#{=21:pane_title}" %H:%M` uses a conditional (highlight when the prefix is pending), a width limiter, and — where time is allowed — a strftime format. `format_draw.rs` is the piece

that renders an *already-expanded* status string with its embedded `#[...]` style directives into actual cells, handling the style ranges (`STYLE_RANGE_LEFT`, `STYLE_RANGE_WINDOW`, and the rest) that let a click on a window-status entry know which window it hit. The style-range machinery is what makes the status line *interactive* under the mouse, not just decorative.

The width budget: what happens when the bar does not fit

The status line is one row of a fixed number of columns, and the three regions competing for it — left, window list, right — are independently configured by users who did not coordinate. Overflow is therefore the normal case on a narrow terminal, not an edge case, and how the program resolves it is a behaviour worth understanding because it is the difference between a bar that degrades legibly and one that turns into noise.

The first mechanism is the explicit cap. `status-left-length` and `status-right-length` are not cosmetic; they are the budget each end section is allowed to claim before the window list gets what remains. Setting a generous `status-right` without raising its length option produces a common complaint about a status line — content that is silently cut — and the fix is a configuration change rather than a bug. The design is deliberate: the window list is the part of the bar that is *about the sessions*, and the fixed ends are decoration around it, so the ends are the parts with hard caps.

The second mechanism is the format language itself. Chapter 16's width limiter is the tool for content whose length you do not control, and the default `status-right` uses it on the pane title for exactly that reason: a title is whatever the running program decided to set, which can be a full path or a hundred characters of a query. Truncating at the format level rather than at the layout level is better because the format author chooses *what* gets truncated; layout-level truncation cuts whatever happens to be last.

The third is `status-justify`, which decides where the window list sits within the space the ends left it. This interacts with overflow in a way that is easy to miss: with many windows, the list itself exceeds its space, and the justification determines which end of the list falls off. A session with thirty windows on an eighty-column terminal is a case where the current window can scroll out of view entirely, which is why users with many windows either shorten `window-status-format` or move to a multi-row status.

That last option is the escape hatch worth naming, because it changes the arithmetic rather than the priorities. `status` takes a count, and `status-format` is an array with one entry per row (Appendix C), so a bar that will not fit in one row can be given two, with the ends on one and the window list on the other. `status_line_size` and `status_at_line` — named above as the functions computing how many rows the status occupies and where it sits — are what make the pane area shrink correspondingly, which is the reason a multi-row status is a layout change and not just a drawing change.

Prompts, messages, and the status line's other lives

The status line row is time-shared with two other uses, and `status.rs` handles both. `status_message_redraw` shows the transient messages that `display-message` and command errors produce — the yellow bar that appears for `display-time` milliseconds and then reverts. `status_prompt_redraw`, with `status_prompt_key`, `status_prompt_update`, and `status_prompt_set`, implements the interactive command prompt (`command-prompt`, the prefix-: line) and the `confirm-before` and `command-prompt -p` prompts — reading a line of input in the status row with its own editing keys (`status-keys`, emacs or vi), history (`status_prompt_save_history`, persisted so your prompt history survives a server restart — one of the cleanup steps Chapter 3 named), and completion. So the single status row is, at different moments, an information display, a menu of windows, a transient message, and a line editor — three subsystems multiplexed onto one strip of terminal, each a faithful port of its tmux counterpart, and all of them ultimately drawing through the same grid-and-tty path as everything else in the program.

Chapter 21 — Hooks, Alerts, and Notifications

tmux is event-driven not only at the socket level but at the *semantic* level: when a window is renamed, a session is created, a pane changes mode, or a bell rings in a background window, tmux fires a named event that user configuration can hook and that attached clients can be told about. This chapter covers that layer — `src/ported/notify.rs`, `src/ported/alerts.rs`, and the hook machinery threaded through the command queue — because it is where a user’s `.tmux.conf` reaches past static options into *reactive* behavior, and because it ties together the command queue (Chapter 8), options (Chapter 9), and the status line (Chapter 20).

The notification system: named events

`src/ported/notify.rs` is the dispatcher for tmux’s internal events. When something noteworthy happens, the code calls one of the typed `notify_*` entry points — `notify_session`, `notify_window`, `notify_pane`, `notify_client`, `notify_winlink`, `notify_paste_buffer`, `notify_session_window` — with the name of the event and the objects it concerns. Each queues a notification through `notify_add`, whose callback (`notify_callback`) does two things: it runs any hook bound to that event name, and it tells interested clients (control-mode clients especially) that the event occurred. The event names are a fixed vocabulary, and reading the list is reading tmux’s model of “things worth reacting to”: `session-created`, `session-closed`, `session-renamed`, `session-window-changed`, `client-detached`, `client-session-changed`, `window-linked`, `window-unlinked`, `window-renamed`, `window-pane-changed`, `window-layout-changed`, `pane-mode-changed`, `paste-buffer-changed`, and `paste-buffer-deleted`.

The reason this indirection exists rather than each site directly doing the work is uniformity: every state change that matters flows through one dispatcher, so hooks and control-mode notifications compose the same way regardless of what triggered them. A control-mode client (Chapter 17’s `-CC`) subscribes to these events to keep its own model of the server in sync — which is exactly how the `zterminal` integration of Chapter 13 would learn that a window was renamed without polling. The notification layer is the server’s “something changed” broadcast, and `ztmux` ports the whole vocabulary and the dispatch.

Hooks: commands bound to events

A *hook* is a command list bound to run when an event fires — the reactive counterpart to a key binding (which runs on a keystroke). Hooks are set with `set-hook` (a sibling entry in `src/ported/cmd_set_option.rs`, alongside `set-option`) and shown with `show-hooks`, and they live in the options tree as a special option kind, which is why they inherit and scope like options do. When `notify_hook` (or `notify_insert_hook/notify_insert_one_hook`) fires an event, it looks up the hook of that name in the relevant scope and, if present, queues its command list on the command queue with the event’s context as the queue-item state (Chapter 8) — so a hook command knows the session, window, or pane the event concerned, and resolves `-t` targets against it.

Two families of hooks are worth naming. The event hooks share the notification names above: bind a command to `window-renamed` and it runs whenever a window is renamed, with the renamed window as its target. The *command* hooks — `after-<command>` and `before-<command>` — fire around the execution of

specific commands (the `CMD_AFTERHOOK` flag on a `cmd_entry`, Chapter 8, governs the after-hook timing), so `set-hook -g after-split-window '...'` runs your command every time any `split-window` completes. Because hooks are command lists on the command queue with resolved context, they compose with everything else: a hook can run a brace block, branch with `if-shell`, set an option, or send keys, exactly as a key binding can. This is how a `.tmux.conf` turns from a static configuration into a small reactive program.

Alerts: activity, bell, and silence

`src/ported/alerts.rs` handles the three background-window alerts a user actually sees: *activity* (a background window produced output), *bell* (a program rang the terminal bell), and *silence* (a background window produced *no* output for a configured interval — the inverse alert, useful for watching a build that has stalled). The monitoring is gated per window by `monitor-activity`, `monitor-bell`, and `monitor-silence` (Appendix C), and when a monitored condition is met, `alerts_queue` schedules a check that `alerts_callback` services on the event loop. The check functions — `alerts_check_activity`, `alerts_check_bell`, `alerts_check_silence`, gathered by `alerts_check_all` and `alerts_check_session` — decide whether the condition has actually fired for a window and, if so, apply the configured *action*.

The action is where alerts meet the rest of the configuration surface. `activity-action`, `bell-action`, and `silence-action` (session options) choose *which* clients are alerted — any, current, or none, gated by `alerts_action_applies` — and `visual-activity`, `visual-bell`, and `visual-silence` choose *how*: a visible message in the status line (`alerts_set_message`, drawn by `status_message_redraw` of Chapter 20) versus an actual terminal bell versus both. When an alert fires it also sets the window's alert flags (`window_activity_flag`, `window_bell_flag`, `window_silence_flag`, Appendix B), which the `window-status-*style` options then use to restyle that window's entry in the status line — so an active background window can glow in the status bar. `alerts_reset` and `alerts_reset_all` clear the state when the window is next viewed. The whole path — monitor option, condition check, action policy, visual/bell delivery, status-flag restyle — is a faithful port, because “which windows light up when and how” is exactly the kind of long-established behavior tmux users depend on.

Reentrancy: hooks that cause the events they hook

The composability described above has an obvious hazard, and a configuration language that lets an event run arbitrary commands has to have an answer for it. A hook on `window-renamed` that renames a window fires an event that runs the hook that renames a window. A hook on `after-select-window` that selects a window is the same loop with a command hook. Neither is a contrived example; both are shapes a user reaches for naturally when trying to enforce an invariant — “whenever a window is renamed, normalise the name” is a reasonable thing to want.

The structural reason this is survivable rather than instantly fatal is the command queue. A hook does not execute inline at the notification site; `notify_callback` queues its command list, and the queue is drained by the server loop (Chapter 3). So a self-triggering hook does not recurse on the stack — it enqueues, returns, and the queued item runs on a later pass, enqueueing again. The failure mode is therefore a server that makes no progress rather than a server that overflows its stack, which is a meaningful difference: the loop is still turning, clients are still being serviced, and the situation is diagnosable through the mechanisms of Chapter 36 rather than being a crash.

It is still a real hazard, and the practical defences belong to the configuration rather than to the program. The first is to make the hook's action conditional on the state it is about to establish — check the name before renaming, so the second firing is a no-op and the cycle terminates. The second is to notice that many “enforce an invariant” hooks are better expressed as the option that already enforces it: `automatic-rename` and `automatic-rename-format` (Appendix C) exist precisely so that naming policy is a setting rather than a reactive script, and a hook that reimplements a setting has taken on the setting's job plus a reentrancy problem.

The general lesson generalises past hooks. Every mechanism in this layer turns a state change into an action,

and any action that produces the same class of state change closes a loop. Alerts are safe here by construction — they deliver a message or a bell rather than mutating session state — which is a good illustration of why the reactive surface is split into a general mechanism with a hazard and specific mechanisms without one, rather than everything being hooks.

Why this layer completes the picture

Hooks, alerts, and notifications are the reactive skin over the state machine the rest of the book described. The static layers — options, key tables, formats — configure what tmux *is*; this layer configures what tmux *does when things happen*. And it reuses every mechanism already built: hooks are command lists on the command queue with resolved context; alerts are options-gated checks that deliver through the status line; notifications are the broadcast that control-mode clients and the superset's integrations subscribe to. Nothing here is a new subsystem so much as the composition of the existing ones into event-driven behavior — which is precisely why porting it faithfully mattered: a hook that fired at the wrong time, or an alert that applied to the wrong clients, would break configs that have quietly relied on the exact semantics for years.

Chapter 22 — Two Lifecycles: A Keystroke and a Pane

The book has walked the subsystems one at a time; this capstone walks *through* all of them twice, following two end-to-end journeys — the life of a single keystroke, and the life of a pane from birth to death. Both cross nearly every module named so far, and tracing them is the best way to see how the faithful pieces compose into the whole machine. Nothing here is new mechanism; it is the existing mechanism in motion.

The life of a keystroke

You press `⌘-b c` to make a new window. Follow the bytes. Your terminal sends the raw byte for `⌘-b` to the ztmux *client*, which is in raw mode (Chapter 4) and forwards every byte to the server over the socket — the client decides nothing. On the server, the bytes arrive on the client's input and go to `tty_keys.rs`, which decodes the terminal's byte sequence into a tmux *key code* (`⌘-b` becomes the internal code for control-B). The server looks that key up in the client's active key table — normally `root`, but because `⌘-b` is the prefix key, the lookup instead switches the client's key table to `prefix` and consumes the key: the client's `client_key_table` is now `prefix`, which is exactly what `#{client_prefix}` reports and what the status-line recipe in Appendix J highlights.

Now you press `c`. Its byte is forwarded, decoded by `tty_keys.rs` into the key code for `c`, and looked up — this time in the `prefix` table, where `key_bindings_get` finds the default binding added by `key_bindings_init` (Chapter 9, Appendix I): `c` maps to the command list `{ new-window }`. That command list is dropped onto the command queue (Chapter 8) as a `cmdq_item` carrying a `cmdq_state` built from the resolved context — the client that pressed the key, and through it the current session, window, and pane (Chapter 8's target resolution). The client's key table resets to `root`.

The server's event loop (Chapter 3) pumps the queue on its next iteration: `cmdq_next` fires the item, which runs `new-window's exec` (`src/ported/cmd_new_window.rs`). That resolves its target through `cmd_find`, then calls into `spawn.rs` (Chapter 19): `spawn_window` allocates a window and its first pane, creates the pane's screen and grid (Chapter 5), opens a pty, forks a child, and in the child sets up the environment (Chapter 19's `update-environment` merge) and `execs` your shell. The new window is inserted into the session's window list; the layout engine (Chapter 7) gives its single pane the full window rectangle. A `window-linked / session-window-changed` notification fires (Chapter 21), any hook bound to it runs, and the status line is marked for redraw.

Finally the frame comes back. The server recomputes the visible state, the screen-redraw layer (Chapter 20) composites the new window's pane and the updated status line, and the tty back end (Chapter 10) emits the minimal escape sequences — through the differential redraw and `tty_cursor/tty_attributes` minimization — to the client's borrowed terminal fd. The client writes those bytes to your actual screen. One keystroke pair crossed the client, the socket, the key decoder, two key tables, the command queue, target resolution, the spawn path, the pty and fork/exec, the layout engine, the notification system, and the redraw and tty layers — and every one of those was a faithful port of the corresponding tmux module. That traversal *is* the architecture.

The life of a pane

Now widen the lens from an instant to a whole lifetime. A pane is born when a command that spawns one — `new-window`, `split-window`, `new-session` — reaches `spawn_pane/spawn_window` (Chapter 19). Its screen and grid are created (`screen_init` → `grid_create`), sized to the rectangle the layout tree assigns it, and a `pty` is opened with the server holding the master fd. The child forks, does its between-fork-and-exec setup (new session, controlling terminal, size, descriptor closing — the macOS `closefrom` corner of Chapter 19), and `execs` a program. The server arms an event-loop watch on the master fd. The pane is alive.

Through its life the pane is a pipe between a program and a grid. When the program writes, the master-fd event fires (Chapter 3), the bytes are read and fed to `input_parse` (Chapter 6), which drives the screen-write layer (Chapter 5) to mutate the grid — printing characters through the batched collect path, moving the cursor, scrolling the region, pushing lines into scrollback when they fall off the top (`grid_scroll_history`, trimmed by `grid_collect_history`). The changed cells are marked dirty; on the next redraw the `ty` layer paints only those cells to every client viewing the pane. If you enter copy mode (Chapter 18), a virtual cursor and a `grid_reader` navigate the pane's stored lines and scrollback while the program keeps running underneath; a selection extracts styled or plain text into a paste buffer. If you resize the window, the layout engine re-tiles, the pane's screen resizes and its grid reflows long lines to the new width (Chapter 5), and the pane redraws — all without the program noticing more than a `SIGWINCH`.

A pane can move without dying. `break-pane` promotes it into its own window, `join-pane` grafts it into another window's split, `swap-pane` exchanges it with another — all tree surgery on the layout (Chapter 7) that preserves the pane's screen, scrollback, and running program; only its rectangle changes. It can be zoomed to fill the window and un-zoomed back to the exact column it left, because zoom saves and restores the layout tree rather than rebuilding it (`window_zoom/window_unzoom`).

The pane dies when its program exits. The server's `SIGCHLD` path (Chapter 3) reaps the child and marks the pane dead, recording the exit status and signal (`#{pane_dead_status}`, `#{pane_dead_signal}`). Policy decides what happens next: by default the pane is destroyed and, if it was the last in its window, the window is closed and the layout collapses the freed space back to its siblings; with `remain-on-exit` set, the dead pane stays in place showing its `remain-on-exit-format`, ready for `respawn-pane` to bring it back to life in the same slot. When the last window of the last session is gone and the `server_loop` exit conjunction is finally satisfied (Chapter 3), the server itself exits — and the pane's whole lineage, from `grid_create` to the reaped child, has been a single coherent walk through the ported machine. The two lifecycles together are the book's argument in motion: a faithful port is faithful not module by module in isolation, but in the way the modules compose into the behavior a `tmux` user has always known.

The life of a config file

The two lifecycles above follow data through the machine. There is a third traversal worth making because it crosses the same modules in a different order and answers the question a user asks most often when something is not working: what exactly happened between “I edited my config” and “the program behaves that way.”

It begins before any of it, at server start. The very first client invocation with no server running autostarts one (Chapter 4), and among the server's startup work is `start_cfg` in `cfg.rs`, which arranges for the configuration to be loaded. The order here has a consequence people trip over: the server's *defaults* are established first — the option table's defaults (Chapter 33) and the default key bindings (`key_bindings_init`, Appendix I) — and the config file then runs *against* an already-populated world. A config file does not describe the world; it edits it. That is why a `.tmux.conf` that binds nothing still leaves you with every default binding, and why removing a default requires an explicit `unbind-key` rather than the absence of a `bind-key`.

Loading itself is command execution, not parsing into a data structure. Each line is parsed by the command grammar (Chapter 8) and queued, so the file is a script in the same language you type interactively, with the same lexer, the same brace blocks, and the same target resolution. This is why a multi-line `bind-key` with a

brace block works in a config file exactly as it does at the command prompt, and why there is no separate configuration syntax to learn or, from the port's side, to implement.

The failure handling is where the design shows judgement. A line that fails does not abort the load — it produces a cause (Chapter 35's out-parameter idiom) which is accumulated, and loading continues to the next line. So one unrecognised option costs you that option and nothing else. But the accumulated causes are not discarded either: there may be no client attached at load time, since the server may have autostarted in the background, so the causes are held and reported to the first client that attaches. That is the mechanism behind the familiar experience of attaching and immediately seeing your config errors — they happened before you were there, and they waited for you.

`source-file` re-enters the same path at any time, which makes the whole thing re-runnable. This is the reason the standard “reload my config” binding works at all, and also the reason it does not do quite what people expect: re-sourcing runs the commands again against the *current* world, so settings are overwritten but nothing is reset. Options you set in a previous version of the file and then deleted from it stay set, because nothing instructed them to change. Bindings you removed from the file stay bound. The config is a diff applied to live state, and applying a diff twice is not the same as reverting and re-applying.

From there the effects fan out into the subsystems the book has already covered, and each takes a different route to becoming visible. An option set into the options tree (Chapter 9) is consulted the next time the relevant code reads it — which for a status-line option means the next status redraw, and for `history-limit` means the next pane created, since existing grids were sized when they were created. A `bind-key` inserts into a key table and is live on the next keypress. A `set-hook` arms a hook that fires on the next matching notification (Chapter 21). A command with an immediate effect — creating a session, splitting a pane — simply runs, which is how a config file that builds an entire workspace on startup does its job.

The traversal is worth having explicitly because the “why didn't my change take effect” question almost always resolves to one of three answers visible in it: the option is read at object-creation time and your objects already exist; the config line failed and the cause is sitting in the message ring (Chapter 36) where you have not looked; or the setting landed in a different scope than the one being read (Chapter 33). Three answers, all mechanical, all consequences of the path a config file actually takes through the program.

Chapter 23 – Performance and the Hot Paths

A multiplexer sits in the path of every byte a program prints and every key you type, so it has to be fast, and tmux is — through a handful of specific optimizations in its hottest paths. Because ztmux ports those paths line for line rather than reinventing them, it inherits the performance characteristics along with the behavior. This chapter names the hot paths and the optimizations that make them fast, and explains why a faithful port has to keep them exactly rather than substitute a “more idiomatic” approach that might dispatch differently.

The character-printing fast path

The single most-traveled path in the program is a program printing text to a pane, and it is optimized end to end. When bytes arrive, `input_parse` (Chapter 6) special-cases the printable range so a run of ordinary characters never pays the collection-flush overhead of a control sequence. Printable characters route to `input_print`, which does *not* store each character in the grid individually; it accumulates them and hands the run to `screen_write_collect_add`, which batches them, and the batch is committed by `screen_write_collect_end` only when a non-print sequence arrives or the run must flush. The active SGR style rides along in the parser’s `input_cell`, so each batched character is stamped with the current color and attributes without re-deriving them. The result is that *cat*-ing a megabyte file through a pane turns a stream of thousands of printable bytes into a handful of batched grid writes, not thousands of individual cell stores. This batching is tmux’s, ported exactly, because it is not just a speed trick — the *order* in which the collected run is flushed relative to control sequences is observable, so a different batching strategy could change output.

Bounded reallocation and the packed cell

Two memory optimizations from Chapter 5 are performance choices as much as space choices. Grid lines grow through `grid_expand_line`, which snaps the allocation up through a bounded sequence of sizes — a quarter of the screen width, then half, then full — so a line that fills up reallocates a small, bounded number of times rather than once per character, turning what could be $O(\text{width})$ reallocations into $O(\log \text{width})$. And the packed/extended cell split (the five-byte packed `grid_cell_entry` for the common case, promotion to a `grid_extd_entry` only when needed) keeps a screenful of ordinary text compact, which means better cache behavior and less memory traffic on the redraw walk. Both are exact ports, because the allocation pattern is part of the behavior the parity discipline pins — but they are also why a grid of ten thousand scrollbar lines does not cost ten thousand full-width allocations.

Differential redraw and output minimization

The output path (Chapter 10, and the cell-by-cell walk earlier in this book) is where tmux spends the most optimization effort, because bytes on the wire are the bottleneck: an escape sequence to position the cursor and paint a cell is far larger than the cell. The server repaints only cells the screen-write layer marked dirty; `tty_cursor` emits the shortest cursor move from the tracked current position (a carriage return, a single “cursor right,” or an absolute `CUP` only when nothing cheaper reaches the target); and `tty_attributes` emits SGR only for the differences from the previously drawn cell. The `tty_cmd_*` family turns high-level changes

(a scrolled line, a cleared region) into the fewest terminal operations. Across a full-screen redraw of a busy editor these micro-optimizations are the difference between a responsive terminal and a laggy one, especially over a slow link — and ztmux ports them because the emitted byte stream is itself observable and, in principle, comparable.

Flow control: surviving a flood

A program can produce output faster than a terminal can consume it — a `yes` loop, a giant `cat` — and an unbounded output buffer would let one runaway pane exhaust memory and freeze the whole terminal. tmux applies flow control, and ztmux ports the exact thresholds. The tty's output goes into an `evbuffer` drained to the client when writable; when the buffered bytes exceed `TTY_BLOCK_START` (computed as $1 + (sx*sy)*8$, a multiple of a screenful), the tty enters a blocked state, stops producing more output, and arms a `TTY_BLOCK_INTERVAL` (100 ms) timer to resume. The runaway pane is throttled, the rest of the terminal stays responsive, and memory stays bounded. The specific constants matter — they are observable timing behavior — so the port keeps them rather than picking its own.

Housekeeping: the tidy timer and returning memory

Finally, the server does periodic housekeeping so long-lived sessions do not accumulate waste. The hourly tidy timer (`SERVER_EV_TIDY`, an `evtimer` set to 3600 seconds, whose callback `server_tidy_event` fires from the event loop) expires stale format jobs and, off macOS, calls `malloc_trim(0)` to return freed heap back to the operating system — so a server that once held a huge scrollbar and released it actually gives the memory back rather than holding the high-water mark forever. This is a small, faithful touch (the macOS build skips `malloc_trim`, which glibc provides and the macOS allocator does not), and it is the kind of long-run hygiene a daemon that may live for weeks needs. Performance in a multiplexer is not one clever algorithm; it is a dozen small, exact decisions in the hottest paths, and porting them faithfully is how ztmux is fast for the same reasons tmux is.

What a port inherits, and what it must pay for separately

The claim that ztmux inherits tmux's performance characteristics by porting its hot paths line for line is true and needs a boundary drawn around it, because inheritance covers the *algorithms* and not the *substrate*. Three things sit outside the ported code and are worth naming honestly rather than leaving as an implied “and everything else is the same.”

The first is allocation. tmux allocates constantly — grid lines, cell arrays, format strings, command arguments — and its performance depends on those allocations being cheap. Rust's default global allocator is not the C library's, so a naive port would run the same allocation *pattern* through a different allocator with different characteristics, and the memory behaviour of the two programs could diverge without a single line of ported logic differing. This is why the entry point installs a global allocator backed by the C library's (Appendix F, `main.rs`): with it in place, the same sequence of allocation and free calls reaches the same implementation underneath, which keeps both the performance profile and the interaction with the tidy timer's `malloc_trim` coherent with the C. It is a small piece of setup that makes a large claim honest.

The second is bounds checking. Rust checks slice indexing where C does not, and in a program whose hottest loop walks cell arrays, unconditional bounds checks in the wrong place are a real cost. The port's answer is a consequence of its style rather than a separate optimisation effort: the ported code manipulates the same raw pointers the C does, inside `unsafe`, so the hot paths are not paying for checks the C never paid for. This is one of the less comfortable implications of Chapter 14's ordering — the `unsafe` that is scheduled for removal is also, in the hot paths, doing performance work — and the honest framing is that the idiomatisation sweep has to keep an eye on it. Converting a cold module to safe indexing costs nothing measurable; converting the

cell walk in the redraw path is a change that needs its performance verified, not assumed. The parity suite proves behaviour is preserved and says nothing about cost.

The third is the FFI boundary itself, and this is the one place where the accounting has improved rather than held. It used to read: every libevent callback is a C-to-Rust transition, every libc call a Rust-to-C one, at least one per readable descriptor per loop iteration, unavoidable without reimplementing the event loop. The event loop was then reimplemented (Chapter 3), and the dispatch half of that traffic is now Rust calling Rust, which the compiler can see through and inline. The libc calls remain, they are cheap — this is not a marshalling boundary, just a call — and they are not going anywhere, because matching the C's syscall and allocation behaviour is the point.

Set against those costs, there is one structural advantage the port has that is worth stating with the same precision: it is not faster, it is *safer at equal speed* in the parts that have been converted. A subsystem that has been through the idiomatisation sweep and is now safe Rust with no `unsafe` in its hot loop has the same asymptotics as the C — the algorithms were ported, not replaced — with a class of bug removed. That is the actual performance story the project can defend today: parity of characteristics with the C, an allocator chosen to keep that parity real, a known set of boundary costs, and a safety improvement that accrues subsystem by subsystem as the sweep proceeds. Anything stronger would require measurement this book does not have, and the discipline that applies to behavioural claims applies to performance claims identically.

Chapter 24 – Colour: Names, Spaces, and the Nearest Match

Chapter 5 showed how a cell *stores* colour — a tagged `i32` that encodes an ANSI index, a 256-palette entry, or a 24-bit RGB value in one field. This chapter is about the machinery that sits underneath that storage: `src/ported/colour.rs` (a port of tmux's `colour.c`), which turns colour *names* into those tagged integers, converts between the three colour spaces, finds the nearest palette entry when a terminal cannot show a true colour, and applies a window's palette overrides. It is small, unglamorous, and load-bearing — every `#[fg=...]` style directive, every `fg/bg` option, and every degraded-colour redraw passes through it.

Three colour spaces in one integer

The colour field's encoding is the fact everything else in the module is organized around, so it is worth restating precisely. A colour is an `i32`. The low values `0–7` and the bright `90–97` (aixterm) forms are direct ANSI palette indices; `8` is `COLOUR_DEFAULT` (the terminal's own default, and `9` is treated as default too); a value or-ed with `COLOUR_FLAG_256` (`0x01000000`) carries a 256-colour palette index in its low byte; and a value or-ed with `COLOUR_FLAG_RGB` (`0x02000000`) packs a 24-bit colour as `(r << 16) | (g << 8) | b`. The two flag bits are mutually exclusive and sit above any real index, so a single test on the tag bits tells `colour.rs` which space a value lives in before it unpacks anything.

The pack and unpack helpers are the primitives the rest of the module builds on. `colour_join_rgb(r, g, b)` folds three bytes into the RGB-tagged form, and `colour_split_rgb(c, &r, &g, &b)` reverses it, masking off the flag and shifting the components back out. These are trivial bit arithmetic, but they are *exactly* tmux's bit arithmetic — the same shift amounts, the same flag constants — because the packed value is not private: it is what gets stored in a grid cell (Chapter 5), passed through a format's `{c:...}` operator (Chapter 16), and written into an option, so any divergence in the packing would be observable downstream. The module keeps the C's representation to the bit.

Naming: `colour_fromstring` and `colour_tostring`

The bridge between the human world of config files and the integer world of the grid is a pair of functions. `colour_fromstring` parses a colour *name* or *spec* into the tagged integer, and it accepts the full tmux vocabulary: the bare ANSI names (`black`, `red`, ..., `white` and their `bright*` cousins), the numbered forms `colour0` through `colour255` (with the British spelling tmux uses, and `color/grey` variants accepted), the hex spec `#rrggbb`, the keyword `default`, and the large table of X11/web colour names (`orange`, `slateblue`, `darkolivegreen`, and the rest) that newer tmux added so a style string can name a colour the way CSS does. A `#rrggbb` parses through `colour_join_rgb`; a `colourN` becomes `N | COLOUR_FLAG_256`; a name is looked up in `colour_byname`, which returns the RGB-tagged value for the web names and the small index for the ANSI ones.

`colour_tostring` is the inverse, and it is what `show-options` and `{c:...}` print. Given a tagged integer it produces the most faithful string form: an RGB value comes back as `#rrggbb`, a 256-palette entry as `colourN`, an ANSI index as its name (`red`) or `colourN`, and `8` as `default`. The round-trip matters — a colour set as `#d75f5f` should read back as `#d75f5f`, and one set as `colour167` should read back as `colour167` — because configs and scripts compare these strings, and tmux's exact spelling (British `colour`, lowercase

hex, the specific name chosen for each ANSI index) is what a parity `.fmt` case pins. `colour.rs` ports the tables and the formatting so the strings are identical to the byte.

The nearest match: `colour_find_rgb` and `colour_256to16`

The most interesting function in the module is the one that runs when a terminal *cannot* show what the grid holds. When a cell carries a true 24-bit RGB colour but the client's terminal is only 256-colour (Chapter 10's degrade-late principle), the back end must approximate, and `colour_find_rgb(r, g, b)` computes the approximation: the index of the nearest colour in the xterm 256-colour palette. The algorithm is tmux's exact one, and it is cleverer than a linear scan. The 256-palette's colour cube is a 6×6×6 grid whose axis values are not evenly spaced but the specific ramp `{0x00, 0x5f, 0x87, 0xaf, 0xd7, 0xff}`; `colour_to_6cube` maps each input component to its nearest ramp step, giving a candidate cube colour, and separately the input is matched against the palette's 24-step greyscale ramp. The function then compares the squared distance from the input to the cube candidate and to the grey candidate and returns whichever is closer, or-ed with `COLOUR_FLAG_256`. Picking between the cube and the grey ramp is why a near-grey RGB value degrades to a clean grey rather than a muddy cube colour.

`colour_256to16` is the next step down, a straight table lookup from a 256-palette index to the nearest of the sixteen ANSI colours, used when even 256-colour is unavailable. The reason both of these are ported byte-for-byte rather than reimplemented with “a reasonable nearest-colour metric” is the recurring one: the *approximation itself is observable*. Two multiplexers that both “approximate reasonably” but choose different palette entries for the same RGB value produce visibly different output on a 256-colour terminal, and a user who tuned their theme against tmux's approximation would see it shift. The specific ramp values, the cube-versus-grey tie-break, the 256 → 16 table — all of it is tmux's, so ztmux degrades a colour to the *same* index tmux would, and the parity discipline can in principle diff the resulting SGR bytes.

Palettes: per-window overrides and OSC 4

The last piece is the *palette* — the per-window remapping of the 256 base colours that OSC 4 (Chapter 6) and the `pane-colours`/palette options install. A program inside a pane can redefine what `colour4` means by emitting an OSC 4 sequence, and a user can set palette entries through options; both feed a `colour_palette` structure that the back end consults before emitting a colour. `colour_palette_init` and `colour_palette_clear` manage its lifetime, `colour_palette_set(palette, n, c)` installs an override for index `n`, and `colour_palette_get(palette, c)` resolves a colour through the overrides — returning the remapped value when the palette has an entry for it and the original otherwise. The default foreground and background slots are tracked specially, because `fg/bg` “default” must resolve to the window's configured default rather than the terminal's when a palette sets them.

The palette is where the whole colour story closes its loop back to the input parser and the back end. OSC 4 comes *in* through `input.rs` (Chapter 6), lands in the window's `colour_palette`, and then every subsequent draw of a cell holding that palette index resolves through `colour_palette_get` on the way *out* through `tty_colours` (Chapter 10) — so redefining a palette entry retroactively recolours every cell already on screen that used it, exactly as on a real terminal. That end-to-end path, from an escape sequence a pane program emits to the bytes the outside terminal receives, is a colour subsystem in miniature, and `colour.rs` is the piece in the middle that every other stage leans on. It is a good example of a module that is tiny in lines and total in reach: get its nearest-match wrong and every degraded redraw is subtly off; get its name table wrong and every config's colours shift; port it to the bit and colour “just works” for the same reasons it works in tmux.

Chapter 25 – UTF-8, Wide Characters, and Combining Sequences

A grid cell holds a *character*, and in a Unicode world a character is not a byte. This chapter covers `src/ported/utf8.rs` (a port of tmux's `utf8.c`), the module that decodes UTF-8 from the byte stream, measures how many terminal columns a character occupies, packs multi-byte sequences into the compact handle a grid cell stores, and handles the two hard cases every terminal must get right: *wide* characters that occupy two columns and *combining* characters that occupy none. Getting this wrong does not crash anything — it misaligns everything, which is worse, because a status line whose width is off by one and a TUI whose box borders drift are exactly the quiet breakage the parity discipline exists to prevent.

The `utf8_char` handle and the `utf8_data` it stands for

A grid cell cannot afford to store an arbitrary-length UTF-8 sequence inline — most cells hold a single ASCII byte, and Chapter 5's packed/extended split depends on the common cell staying small. So tmux represents a character in a cell as a `utf8_char`: a fixed-size handle (a `u32`) that stands for a UTF-8 sequence rather than containing it in general. The expanded form is `utf8_data` — a small struct carrying the raw bytes (`data`, up to `UTF8_SIZE`), the byte length `size`, and the display `width` (0, 1, or 2). `utf8_from_data` converts a decoded `utf8_data` into the packed `utf8_char` handle, and `utf8_to_data` expands a handle back into its bytes when the back end needs to emit them. Short sequences pack directly into the handle's bits; longer or unusual ones are interned through a lookup table so the handle stays a fixed `u32` no matter how many bytes the character actually took.

This indirection is the reason a cell can carry an arbitrary grapheme without bloating the common case, and it is why the `data` field of a packed `grid_cell_entry` (Chapter 5) can be a single byte for ASCII while a wide CJK glyph or an emoji-with-modifier lives as an extended cell whose `utf8_char` handle resolves through this table. `utf8.rs` ports the packing scheme exactly, because the handle values themselves are internal, but the *behavior* they produce — which sequences round-trip, how a combining mark attaches — is observable through copy-mode extraction and `capture-pane`, and must match tmux.

Decoding: `utf8_open`, `utf8_append`, and the incremental path

The input parser (Chapter 6) does not receive whole characters; it receives bytes, one at a time, and a multi-byte UTF-8 character arrives across several parser iterations. So decoding is *incremental*, driven by three functions. `utf8_open(ud, byte)` starts a new character on a lead byte, reading the high bits to learn how many continuation bytes to expect and stashing the first byte in a fresh `utf8_data`; it returns `UTF8_MORE` if more bytes are needed. `utf8_append(ud, byte)` adds each continuation byte, returning `UTF8_MORE` while the sequence is incomplete, `UTF8_DONE` when it completes (at which point the character's width is computed and it is ready to write to the grid), or `UTF8_ERROR` on a malformed sequence — an unexpected byte, an overlong encoding, a continuation without a lead. The parser holds a partial `utf8_data` across bytes and feeds it through this small state machine, which is why a UTF-8 character split across two reads from the pane's pty decodes correctly rather than tearing.

Error handling here is a fidelity detail that matters more than it looks. A broken UTF-8 stream — and real

programs emit them, through bugs or binary output accidentally sent to a terminal — must degrade the *same* way tmux degrades it, not crash and not silently drop bytes, because what a terminal shows for invalid UTF-8 is observable. `utf8.rs` ports tmux’s exact treatment: an invalid sequence is handled deterministically (the offending bytes rendered as tmux renders them) so a pane fed garbage looks under ztmux the way it looks under tmux. The incremental decoder’s three return values and their exact triggers are the ported contract.

Width: the one-flag decisions that keep columns aligned

Once a character is decoded, the single most consequential question is: how many columns does it occupy? `utf8_width` answers it, returning 0 for a combining or zero-width character, 1 for an ordinary one, and 2 for a wide (East Asian fullwidth, or wide emoji) character. The answer drives everything visual: a width-2 character claims its cell *and* a following `GRID_PADDING_CELL` (Chapter 5) so the next real character starts two columns over; a width-0 combining mark does not advance the cursor at all but attaches to the base character already written; a width-1 character is the ordinary case. Get a width wrong and every character after it on the line is one column out of place — the box border of a TUI drifts, a status line’s right-aligned clock lands in the wrong spot, a CJK document renders as a diagonal smear.

The width data is a table — the classification of code points into zero/one/two columns — and tmux ships its own rather than trusting the host’s `wcwidth`, because `wcwidth` varies by platform and locale and would make width a non-portable, non-deterministic quantity. A ztmux built on the C library’s `wcwidth` would report a different width for the same emoji on macOS than on Linux, and the parity suite would light up with false diffs that are really libc differences. So `utf8.rs` ports tmux’s width determination, which is *why* the parity suite can pin `#{w:...}` (the display-width format operator, Chapter 16) across platforms: both binaries consult the same ported table, so the same string is the same number of columns everywhere.

Combining characters and the string helpers

Combining characters are the case that makes “a cell holds a character” subtle. A base letter followed by one or more combining marks — an `e` plus a combining acute accent, a family of emoji joined by zero-width joiners — is *one* grapheme occupying *one* cell, not several. When the parser decodes a width-0 character, it does not write a new cell; it appends the combining sequence to the `utf8_char` of the cell already under the cursor, extending that cell’s stored sequence (which is exactly why such a cell becomes *extended* in Chapter 5’s sense — its byte sequence no longer fits the packed form). `utf8.rs` handles the append, so a decomposed accented character and a precomposed one both end up as a single cell that copies and renders correctly.

Around the decoder sits a family of string helpers the rest of the code leans on, each UTF-8-aware where a naive byte routine would miscount. `utf8_strlen` and `utf8_cstrwidth` measure a string’s length in characters and its width in columns respectively — the distinction Chapter 16’s `#{n:...}` (length) versus `#{w:...}` (width) operators expose. `utf8_padcstr` and `utf8_rpadcstr` pad a string to a column width (not a byte count), which is how status-line fields align regardless of whether they contain CJK. `utf8_rtrimcstr` trims trailing columns. And `utf8_stravis/utf8_strvis` produce a *visible* rendering of a string with non-printable bytes escaped, used where tmux must show a control character rather than emit it. Every one of these is a place a byte-oriented shortcut would silently break on multi-byte input, and every one is ported so that width and length mean, to the column and to the character, what they mean in tmux. The alignment you never think about — because it just works — is this module doing its job.

Chapter 26 — Mouse Events and the Window Modes

Two subsystems that turn tmux from a keyboard program into an interactive one share this chapter because they meet: the *mouse*, which delivers clicks and drags and wheel scrolls to the right pane, and the *window modes*, the takeover screens (`choose-tree`, `choose-buffer`, `customize-mode`, and copy mode) that a pane can enter to become an interactive picker or editor. The mouse is how you point at them; the modes are much of what you point at. Both are faithful ports, and both are where the multiplexer’s UI, such as it is, actually lives.

A mouse click is a key: parsing and routing

The first thing to understand about mouse handling is that, to tmux, a mouse event is just another *key*. The outside terminal, when mouse reporting is on, encodes a click or a scroll as an escape sequence (the modern SGR form, `\x1b[<b;x;yM`, or the legacy X10/1000 forms), and that sequence arrives in the same input stream as the user’s keystrokes. `src/ported/tty_keys.rs` recognizes it — `tty_keys_mouse` parses the button, the column and row, and the press/release/drag/wheel disposition out of the sequence — and packages it as a `key_code` carrying the `KEYC_MOUSE` marker with a `mouse_event` struct attached. From there it flows through the same key-dispatch path as any keystroke (Chapter 9’s key tables), which is the unifying trick: because a mouse event is a key, everything the key system already does — table lookup, binding to a command list, the command queue — applies to the mouse for free.

What makes a mouse event *usable* is that its coordinates must be resolved to an object. A click at column 40, row 12 means nothing until it is turned into “pane 2 of window 3.” The `mouse_event` carries the raw `x/y` (and the previous `lx/ly` for drags), and helpers `mouse_get_pane`, `mouse_get_window`, and the status-line hit-testing translate those coordinates against the current layout (Chapter 7) and status line (Chapter 20) into the pane, window, or status element under the cursor. This is how `MouseDown1Pane` knows which pane you clicked and `MouseDown1Status` knows which window in the status line you selected. The resolution is geometric — it walks the same layout tree the renderer used to place the panes — so the pane the click lands on is exactly the pane drawn there.

The default mouse bindings and forwarding to programs

Because a mouse event is a key, the *default* mouse behavior is expressed as bindings in the `root` key table (Chapter 9), on synthetic key names tmux defines for each event: `MouseDown1Pane` selects the pane clicked, `MouseDown1Pane` begins a copy-mode selection, `WheelUpPane` enters copy mode and scrolls up, `MouseDown1Status` selects a window, `MouseDown1Border` resizes panes, and so on. These are ordinary bindings — you can see them in `list-keys`, rebind them, or unbind them — which is why the mouse is configurable through the same mechanism as everything else and why `bind -n MouseDown3Pane ...` (a right-click binding) is a thing a user can write. The synthetic key names, the default set, and their command lists are all ported so that mouse behavior out of the box is tmux’s.

There is a crucial second path: a program *inside* a pane can itself request mouse reporting (an editor, a pager, a full-screen app that wants clicks). When it has, and the click lands in that pane, tmux does not act on the event — it *forwards* it, re-encoding the mouse event as the escape sequence the inner program expects and

writing it into the pane (through the `input_key` path, the outbound cousin of Chapter 6’s parser). So a click in `vim-with-mouse` goes to `vim`, while a click in a plain shell pane is handled by tmux’s own bindings. The `mouse` option gates the whole system (off by default in stock tmux, though most configs turn it on), and the decision of whether an event is tmux’s or the program’s — based on the pane’s mode and whether the inner program enabled reporting — is ported exactly, because “does my click go to tmux or to the app?” is behavior a user builds precise expectations around.

Window modes: a pane that becomes a UI

A *window mode* is what a pane enters when it stops showing its program and starts showing an interactive tmux surface instead. Copy mode (Chapter 18) is the most familiar, but it is one of several, and they share a structure. Each mode is a `window_mode` — a struct of function pointers (`init`, `free`, `resize`, `key`, and a `key_table` naming the key bindings that apply while the mode is active) — and a pane enters one through `window_pane_set_mode` and leaves it through `window_pane_reset_mode`. While a mode is active, the pane’s screen is drawn by the mode, not the program: the program keeps running underneath (its output still mutates its real screen), but what you see is the mode’s own screen, and keystrokes are dispatched through the mode’s key table rather than sent to the program. This is the same policy/mechanism separation the rest of the book keeps running into: the mode is a temporary alternate view and input handler bolted onto a pane.

The modes beyond copy mode are the interactive pickers, and tmux builds them on a shared toolkit — `mode-tree.c`, ported as the mode-tree helper — that provides a scrollable, filterable, taggable tree with a live preview. `choose-tree` (the `window_tree` mode) presents the whole session/window/pane hierarchy as a navigable tree and switches to whatever you select; `choose-client` (`window_client`) lists attached clients; `choose-buffer` (`window_buffer`) lists the paste buffers (Chapter 28) with a preview of each; and `customize-mode` (`window_customize`) is an interactive options-and-keys editor, letting you browse and change the scoped options (Chapter 9) and key bindings live. Because all four are built on the one mode-tree toolkit, they share their navigation keys, their `/`-search, their tagging for bulk actions, and their preview pane — which is why they feel like one interface, and why porting the toolkit once ports the consistency across all of them.

Why the modes and the mouse belong together

The mouse and the modes are the two halves of the same interactive loop, and a worked example shows them meeting. Scroll the wheel over a pane: the terminal sends a wheel escape, `tty_keys.rs` parses it into a `WheelUpPane` key with resolved coordinates, the `root` binding for `WheelUpPane` fires, and its command puts the pane into copy mode and scrolls up one step — a mouse event, routed as a key, entering a window mode. Now drag to select: each `MouseDownPane` event, coordinates resolved to grid cells, extends the copy-mode selection (Chapter 5’s `screen_sel`). Release, and the selection is copied into a paste buffer (Chapter 28). One gesture traversed the mouse parser, the key tables, a window mode, the grid’s selection model, and the buffer store — every stage a faithful port of the corresponding tmux piece.

That traversal is the argument for why these subsystems are ported rather than reinvented as “reasonable mouse support.” A reimplementation would almost certainly resolve a click to a slightly different pane at a border, or scroll a different number of lines per wheel notch, or bind the middle button somewhere else — small divergences that a mouse-driven tmux user would feel immediately as *wrong*, even if they could not say why. By porting the coordinate resolution, the synthetic key names, the default bindings, the forwarding decision, and the mode toolkit, ztmux inherits tmux’s exact feel under the mouse, the same way it inherits tmux’s exact behavior under the keyboard. The interactive surface is the part of a multiplexer users touch most directly, and faithfulness there is what makes the switch from tmux feel like nothing changed.

Chapter 27 — Jobs, run-shell, and pipe-pane

tmux runs other programs — not the ones in panes, which are the whole point, but *helper* processes: the shell command behind `run-shell`, the condition behind `if-shell`, the `#(...)` job in a status line, the sink of `pipe-pane`. All of them go through one small subsystem, `src/ported/job.rs` (a port of tmux's `job.c`), which forks a child, wires its I/O to the event loop, and calls back when output arrives or the child exits — without ever blocking the server's single thread. This chapter is about that subsystem and the four features built on it, because “multiplexer that can shell out asynchronously” is a surprising amount of what makes tmux scriptable.

job_run: a child wired into the event loop

The core is one function, `job_run`, and its signature tells the story: it takes a command to run (as a shell string or an argv), an environment and working directory and originating session, and a set of *callbacks* — an update callback fired as the child produces output, a complete callback fired when it exits, and a free callback for teardown — plus flags and an initial size. It forks a child, sets up a socketpair (or a pty, if the `JOB_PTY` flag is set) connecting the child's stdout back to the server, `execs` the command in the child, and in the parent returns a `struct job` whose output end is registered with the event loop. From then on the job is *asynchronous*: the child runs concurrently, its output arrives as read events that fire the update callback, and its death arrives as a `SIGCHLD` that the server's child-reaping (Chapter 3's `server_child_signal`) routes to the job's complete callback. The server never waits on the child; it goes back to its event loop and services everything else.

The flags encode the variations the callers need. `JOB_NOWAIT` runs a fire-and-forget job whose output is discarded (a `run-shell -b` background command). `JOB_KEEPPWRITE` keeps the write end open so the job can be *fed* input, not just read from — the mechanism `pipe-pane` needs. `JOB_PTY` gives the child a pseudo-terminal instead of a pipe, for commands that behave differently when they think they are on a terminal. `job_free` tears a job down, and two functions matter to the rest of the server: `job_kill_all`, which the server calls on exit to terminate every outstanding job (Chapter 3), and `job_check_died`/the still-running check, which the server-loop exit decision consults so the server does not quit out from under a job that is still working (Chapter 3's exit conjunction). The jobs subsystem is thus wired into the server's lifecycle at both ends: started asynchronously, and accounted for when deciding whether the server is idle.

run-shell and if-shell: commands that wait

The two user-facing commands built directly on jobs are `run-shell` and `if-shell`, and they show how a *command* can be asynchronous without stalling the command queue (Chapter 8). `run-shell` runs a shell command and, unless backgrounded, waits for it — capturing its output and displaying it, or feeding it to a `-t` pane. `if-shell` runs a command purely for its *exit status* and then branches: on success it runs one command list, on failure another. Both are the reason the command queue models *waiting* (Chapter 8): when `if-shell`'s `exec` runs, it launches a job and then *suspends its queue item* rather than returning a result, and the item stays suspended — the server loop continuing to run everything else — until the job's complete callback fires with the exit status, at which point the item resumes and queues the chosen branch.

This is the property that makes tmux scriptable without freezing. A `.tmux.conf` that does `if-shell "test -n \"$SSH_CONNECTION\" " "source ~/.tmux-remote.conf"` runs a real subprocess during config load,

and a status line that shells out must not hang the server while it does. Because the job is asynchronous and the queue item merely suspends, a slow `if-shell` — a command that takes a second to decide — leaves every other client responsive; the server is not blocked, the item is just parked. `ztermux` ports the suspend/resume continuation exactly, because “does a slow `if-shell` freeze my other panes?” (it must not) is behavior, and getting it wrong would be a regression no format diff would catch but every user would feel.

Format jobs: `#(command)` in a status line

The format language's `#(command)` operator (Chapters 9 and 16) runs a shell command and interpolates its output into a format string — the classic status-line `#(uptime)` or `#(whoami)`. Naively that would fork a process every time the status line redraws, several times a second, which would be absurd; so `format.rs` keeps a *jobs cache* — an RB tree of format jobs keyed by the command string — so that `#(uptime)` runs *once*, its output is cached and interpolated on every redraw, and the job is only *re-run* on an interval (tied to `status-interval`). The first expansion of a `#(...)` starts the job and yields its last-known output (empty the first time); the job's update callback stores the fresh output; the next expansion picks it up. The stale-job expiry that the hourly tidy timer performs (Chapter 3, Chapter 23) is the reaping of format jobs whose commands have not been referenced recently.

This caching is also where the parity discipline meets the jobs subsystem, through a deliberate suppression. The parity suite (Chapter 11) cannot run `#(date)` and compare bytes — the output is non-deterministic — so format expansion carries a `FORMAT_EXPAND_NOJOBS` flag (Chapter 16) that tells the engine *not* to spawn jobs, leaving `#(...)` inert, and the suite runs with it set. The flag is not a parity hack; it is `tmux`'s own mechanism for the contexts where spawning a job would be unsafe or meaningless, ported along with everything else. So the same machinery that makes a status line's `#(...)` efficient in normal use makes it deterministic under the parity harness, which is the sort of dual-purpose fidelity the port aims for: the feature and its testability come from the same ported code.

`pipe-pane`: tee'ing a pane's output to a process

The fourth job-backed feature is `pipe-pane`, and it is the one that runs a job *fed* by a pane rather than read from. `pipe-pane "cat >> ~/pane.log"` opens a job whose stdin is connected to a *copy* of the target pane's output byte stream: every byte the pane's program writes is both processed normally (parsed into the grid, Chapter 6) and tee'd into the job's input, so the log file accumulates exactly what scrolled past. This is the `JOB_KEEPPWRITE` case from the first section — the write end is kept open so the pane can keep feeding the command — and it is wired at the pane's output path (`window_pane_start_output` / the pane's pipe fd) so the tee happens as output is read, not as a separate pass. Running `pipe-pane` again with no command toggles it off, closing the job.

`pipe-pane` completes the picture of what the jobs subsystem is *for*: it is the multiplexer's whole relationship with helper processes, in every direction. `run-shell` reads a job's output; `if-shell` reads a job's exit status; a format job caches a job's output for a status line; `pipe-pane` feeds a job a pane's output. Four different couplings, one `job_run`, all asynchronous, all reaped through the same `SIGCHLD` path, all accounted for in the same exit decision — and all ported from `job.c` with their callbacks and flags intact, so that a config or a script that leans on any of them behaves the way it does under `tmux`. The next chapter covers two more pieces of shared server state that these features touch — the paste buffers a copy fills and `run-shell` can read, and the environment a job inherits.

The command string as cache key, and what follows from it

The format-job cache is keyed by the command string, and that one implementation choice has consequences worth drawing out, because they explain several behaviours that look arbitrary from the outside.

The first is deduplication, and it is the intended effect. A status line that mentions the same `#(...)` command in two places — the left section and a window-status format, say — runs one job, not two, because the second lookup finds the first's entry. Two commands that differ only in whitespace are, by the same rule, two different jobs: the key is the string, not a normalised form of it. This is the right trade for a cache that must be cheap to consult on every expansion, but it means a config that writes the same command two ways pays twice for no benefit.

The second follows from the order of expansion. A `#(...)` whose command text itself contains a format — a command constructed from the current pane's identity, for instance — is expanded before it becomes a key, so the key is the *expanded* command. Switching panes therefore produces a different key, hence a different job, hence a fresh command per distinct value. That is the behaviour you want semantically and it is a cache-population pattern worth being aware of: a status line whose shell command varies with something that changes often does not have a cache, it has a job factory. The tidy timer's expiry of stale entries (Chapter 3) is what keeps that from being unbounded, which is the same expiry mentioned above seen from the direction of what fills the table.

The third is the first-expansion behaviour. Because the job is asynchronous, the expansion that *starts* a job cannot have its output, so it yields what it has — nothing, the first time. A status line containing a brand-new `#(...)` therefore renders with a gap for one refresh cycle and fills in afterwards. This is not a defect to be fixed by blocking on the job; blocking is precisely what the whole subsystem exists to avoid, and a status line that stalled the server until a slow command returned would be worse in exactly the way Chapter 3's single-threaded event loop is designed to prevent. The gap is the visible cost of not blocking, and it lasts one interval.

A practical consequence for configuration: a command whose output is expensive to compute should be made cheap by the command, not by the multiplexer. The cache refreshes on `status-interval`, so the command runs on that period regardless of how long it takes; a command taking longer than the interval is simply always in flight. The standard resolution is the same one used for any polled metric — have the expensive thing write a file on its own schedule and have the `#(...)` command read the file.

Chapter 28 — Paste Buffers and the Environment

Two small server-global stores round out the faithful core, and both are the kind of shared single-copy state Chapter 3 argued belongs in the server. The *paste buffers* are tmux's internal clipboard — the stack of text that copy mode fills and `paste-buffer` empties. The *environment* is the set of variables a session carries and hands to the programs it spawns. Neither is glamorous; both are ported carefully, because both have exact behaviors — buffer auto-naming and trimming, the `update-environment` merge — that scripts and configs depend on.

The paste buffer store: two orderings and automatic names

The paste buffers live in `src/ported/paste.rs` (a port of `paste.c`), and the defining detail is that they are held in *two* red-black trees at once: one keyed by name (`paste_by_name`) and one keyed by an insertion order counter (`paste_by_time`), so a buffer can be found by its name *or* walked newest-first. Each `paste_buffer` carries its data and size, its name, an order number, and an `automatic` flag. That flag is the crux of the naming behavior: a buffer created *automatically* — by a copy-mode copy, which does not name its result — is given an auto-generated name (`buffer0000`, `buffer0001`, ...) from a counter, while a buffer created *explicitly* through `set-buffer -b <name>` carries the name the user gave it. The two kinds coexist in the same store, distinguished by the flag.

The automatic buffers are bounded, and the bound is a ported policy. The `buffer-limit` option (default 50) caps how many automatic buffers the store keeps; when a new automatic buffer pushes the count over the limit, `paste_add` frees the *oldest* automatic buffer (found through the by-time tree), so the clipboard is a bounded stack of recent copies rather than an unbounded accumulation. Explicit named buffers are *not* subject to the limit — you asked for them by name, so they persist until you delete them. This distinction (automatic buffers roll off, named buffers stay) is exactly tmux's, and it is the sort of thing a user relies on without articulating: your last fifty mouse-copies are there to paste, but the buffer you deliberately named for a script does not silently vanish. The operators around it — `paste_get_top` (the most recent), `paste_get_name` (by name), `paste_set`, `paste_replace`, `paste_rename`, `paste_walk` (iterate for `list-buffers`) — are the ported API the buffer commands sit on.

The buffer commands and a fixed parity gap

The user-facing buffer commands are a family, and they all route through the `paste.rs` store: `set-buffer` (create or replace a buffer from a string), `load-buffer/save-buffer` (from and to a file), `show-buffer` (print the top buffer), `paste-buffer` (insert a buffer's contents into a pane, optionally with `-p` bracketed-paste and `-r` no-newline-translation), `delete-buffer`, and `list-buffers` (walk them with a `-F` format). `choose-buffer` (Chapter 26's `window_buffer` mode) is the interactive picker over the same store. Copy mode's copy commands (Chapter 18) create buffers through `paste_add`, which is the loop that connects “select and yank in copy mode” to “paste on the command line.”

This store is also where one of the parity-suite gaps (Chapter 11, Appendix D) lived and was fixed, which makes it a concrete example of the method. Creating a *named* buffer with `set-buffer -b <name>` reported

“empty buffer name” in an early state of the port — the named-buffer path was not correctly wired — where tmux accepts it and stores the buffer under that name. The suite’s expansion caught it as a deterministic, pinned failure; the fix wired the `-b <name>` path through `paste_set/paste_rename` to match the C, and the case turned green and now guards the corner. It is the same shape as every other gap the expansion surfaced: a bounded, mechanical divergence from `paste.c`, caught by a byte comparison, fixed against the source of truth, and kept as a regression test forever.

The environment: per-session variables and `update-environment`

The other store is the environment, in `src/ported/envIRON.rs` (a port of `envIRON.c`), and it exists because a session must carry a set of variables to hand to the programs it spawns — and *which* variables, and from *when*, is a genuinely subtle question a multiplexer has to answer well. The environment is a red-black tree of `envIRON_entry` (a name, a value, and flags), and there is a *global* environment (built from the server process’s own environment at startup, via `envIRON_create` and `envIRON_copy`) plus a per-session environment layered on top. The operators are the expected set: `envIRON_set/envIRON_put` to add, `envIRON_unset` to remove, `envIRON_get` to read, and `envIRON_update` to merge one environment into another. When a pane is spawned (Chapter 19), the effective environment is assembled from these layers and applied to the child with `envIRON_push` just before `exec`, so the program starts with exactly the variables tmux intends.

The behavior that makes this subsystem worth a section is `update-environment`, and it solves a real problem detach-survival creates. A long-lived session outlives the client that started it, but some environment variables — `DISPLAY`, `SSH_AUTH_SOCK`, `SSH_CONNECTION`, `SSH_AGENT_PID` — are tied to the *client*’s connection, not the session, and go stale the moment you detach and re-attach from a different terminal or a new SSH connection. The `update-environment` option is a *list of variable names* that tmux, on each attach or client switch, copies from the *attaching client*’s environment into the session’s environment (through `envIRON_update`), so the session picks up the new client’s `DISPLAY` and SSH agent rather than clinging to the ones from whenever it was first created. This is why you can detach an X or SSH session, re-attach from a fresh connection, open a new pane, and have `ssh` find the *current* agent and a GUI program find the *current* display — the session’s environment was refreshed from the client that just attached. `envIRON.rs` ports the option, the merge, and the timing exactly.

Why these small stores are worth porting exactly

It would be easy to treat the buffers and the environment as trivia — a clipboard and some variables — and reimplement them “reasonably.” The reason ztmux does not is that both have precise, depended-on behaviors hiding in their simplicity. The paste store’s automatic-versus-named distinction and its `buffer-limit` roll-off govern whether a script’s named buffer survives and whether your recent copies are where you expect; get the trimming policy or the auto-naming scheme wrong and a paste-buffer-driven workflow breaks in ways that are maddening to diagnose. The environment’s `update-environment` timing governs whether a re-attached session can reach the X server and the SSH agent; get the merge or its trigger wrong and re-attaching over SSH silently loses your agent forwarding. These are exactly the “reasonable but not identical” divergences (Chapter 29’s argument) that a byte-for-byte port avoids and a from-spec rewrite invites.

Both stores also close loops the rest of the book opened. The paste buffers are the sink of copy mode (Chapter 18) and the source of `paste-buffer`, the thing a mouse drag (Chapter 26) ultimately fills, and a store `run-shell` (Chapter 27) can read; the environment is what a job (Chapter 27) inherits and what a spawned pane (Chapter 19) starts with, assembled from the global and session layers and refreshed by the client that attaches. They are small, but they are *connected* — shared server-owned state that several subsystems read and write — which is precisely why they live in the server (Chapter 3) and precisely why porting them faithfully matters: a divergence here would surface not in one feature but in every feature that touches them. With the buffers and the environment ported, the account of the faithful core is complete down to its smallest

shared stores, and the remaining chapter steps back from mechanism to answer the sharp questions the whole approach invites.

Chapter 29 – Design Decisions and Objections

A port this opinionated invites questions, and the honest thing is to answer the sharp ones directly rather than let them sit as unstated doubts. This chapter takes the objections a skeptical reader would raise — about the `unsafe`, the FFI, the vendored trees, the byte-for-byte bar — and answers each from the project's actual reasoning. None of the answers is “trust us”; each is a consequence of the method the earlier chapters laid out.

“Why is it full of `unsafe`? Isn't the point to be safe Rust?”

The point is to be safe Rust *eventually*, and the `unsafe` is the honest cost of getting there faithfully (Chapter 14). `ztmux` reproduces `tmux`'s manual-memory model — raw-pointer intrusive trees, `#[repr(C)]` structs, `static mut` globals, `libc` FFI — precisely because that is what lets the behavior be checked against the C line for line. A grid cell that is a `#[repr(C)]` struct with `tmux`'s exact field layout can be reasoned about against the C directly; the same cell rewritten as an idiomatic Rust enum could not. The `unsafe` is a phase, not a destination: with parity green and pinned as a specification, the tree gets swept subsystem by subsystem toward safe Rust, each conversion guarded by the parity suite. Starting idiomatic would have thrown away the one thing that makes the port checkable. So the `unsafe` is not a failure of the safety goal — it is the disciplined route to it.

“Why write your own event loop instead of linking `libevent` or using an async runtime?”

Take the two halves of that question separately, because they have different answers.

The async runtime is refused for the reason it always was. A faithful port of an event-loop program must dispatch in the *same order*: which descriptor is serviced before which, when a timer fires relative to the command queue, whether a redraw lands this iteration or the next — all of it is observable behavior. A general-purpose async runtime, however good, brings its own scheduler, its own fairness policy, and its own notion of when a task is ready, and it would make subtly different decisions in exactly the places the parity suite measures. That objection is about *semantics*, and it has not weakened.

Linking the C library is a different matter, and the distinction is the whole point: `libevent`'s semantics are what the port needs, and the C library is merely one implementation of them. For a long time `ztmux` linked it, and the cost was paid entirely at the edges of the project rather than in its behavior — every build needed `libevent-dev` or a Homebrew prefix or a `pkg-config` probe; the release pipeline built a static `x86_64` `libevent` from source just to cross-compile for Intel macs; the Homebrew formula carried a `depends_on`; a Rust program shipped a C dependency for the one subsystem where its behavior was most precisely specified and most thoroughly tested. So the semantics were kept and the dependency dropped: the loop was reimplemented in Rust (`src/extensions/event_loop/`, Chapter 3), against the same API the ported code already called, with the same backends `libevent` would have been forced onto, and verified two ways — its own unit tests for the parts a suite cannot see (timer firing, persistent events, deletion from inside a callback, signal delivery, buffer semantics), and the full parity suite for the only claim that ultimately matters, which is that `ztmux` still produces `tmux`'s bytes.

That reimplementaion is emphatically *not* the async runtime the first paragraph refuses. It is a hundred-odd lines of dispatch, a readiness call, a timer list, and a self-pipe for signals — libevent's model, not a new one. And it found two real bugs on the way in, both worth naming because they are the kind a port only learns by doing: tmux takes a pointer into an `evbuffer`, appends a terminating NUL, and then reads the pointer it already holds, which works because libevent never moves the bytes already in a buffer on append; and the first fix for that reserved the slack *after* the append instead of before, so at one capacity boundary the data moved anyway and every message of exactly fifteen bytes vanished. Both were caught by the parity suite — 155 failures, then 14 — and both are now pinned by unit tests. A dependency you have replaced is a dependency you finally understand.

“Why vendor a whole C source tree? That bloats the repo.”

Self-containment is worth the bytes. `vendor/tmux/` is committed as a SHA-pinned plain copy so that a clone of ztmux carries everything needed to build the port, build the reference, run the parity suite, and audit every function against its origin — with no network access and no version ambiguity (Chapter 2). “Faithful to tmux” then means faithful to a *specific, committed* tmux, not to whatever tmux a contributor happens to have installed, which would produce false parity diffs from version drift. Every alternative — a submodule, a download step, a documented “install tmux 3.7 first” — reintroduces exactly the ambiguity the pin exists to remove, and does so silently, in whatever environment happens to be running the suite. The repo size buys auditability that cannot be had any cheaper.

“Byte-for-byte is fanatical. Isn't ‘behaves reasonably’ enough?”

No, and the three headline bugs (Chapter 11) are the proof. The `39|40-versus-40|39` layout rounding, the `pane_current_command` reporting `ztmux` on macOS, the `#{1:}` crash — each produced output that looked perfectly *reasonable* in isolation. They were only visible as *divergences from tmux*, and only a byte-for-byte comparison against the reference found them. The whole value of a superset is muscle-memory compatibility: the configs, keybindings, and status-line format strings people have accumulated over years must keep working unchanged, and those depend on exact widths, exact format output, exact defaults. “Almost compatible” is the state in which a status line prints a subtly wrong width and a script silently breaks. Byte-for-byte is not fanaticism; it is the only bar that guarantees the compatibility the superset rests on.

“A green suite is easy to fake — why trust ztmux's 1115/1115?”

Because the number is *earned*, and the method is designed to make faking it obvious. The suite reached 122/122, and rather than bank a green bar that might only mean “the suite is too small to fail,” the project grew it to 1115 — deliberately into corners it suspected were weak — until it *did* fail, pinned each of the twenty failures to a specific gap (Appendix D), and then fixed every one against the C. A parity case is an asset: a byte-exact statement of tmux's behavior with a minimal reproduction attached, and it is never deleted — the twenty that once failed now stand as regression guards. So 1115/1115 is a stronger claim than the original 122/122 was, not merely a bigger one — it is measured against a harder target that was deliberately made to fail first, and every corner it caught is named, reproducible, and now green.

“Why keep the C function names and structure? Rust has better idioms.”

It does, and they are the endpoint, not the starting point (Chapter 14). Keeping the C names is what makes the anti-drift gate, the C-signature citations, and a reviewer’s side-by-side diff all possible — the shared name is the join key between the Rust and the C. Rewrite `grid_create` as `Grid::new` and the citation has nothing to point at, the parity comparison can no longer localize a divergence to a function, and the port becomes “inspired by” `tmux` rather than a checkable transliteration of it. The idiomatic cleanup comes *after* a subsystem is verified, with the parity cases as the specification the rewrite must satisfy — so the auditable phase and the idiomatic phase are distinct in time. Having maximal C correspondence and maximal Rust idiom *simultaneously* is the mistake; having them in sequence is the plan.

“Why not run an automated C-to-Rust translator over the sources?”

Because those tools translate the syntax and keep the memory model, and the memory model is the entire thing you were trying to leave behind. Point one at a C program and what comes back is every pointer still a raw pointer, every `malloc` still a `malloc`, every `free` still a `free`, every lifetime still tracked in the programmer’s head — the same manual allocation discipline, the same use-after-free and double-free surface, the same aliasing rules that no compiler is checking, now spelled in Rust. Rust exists to make that class of bug unrepresentable. A tool whose output reproduces it wholesale has produced a program that pays Rust’s costs and collects none of its guarantees: you have taken on a borrow checker you are routing around, a build system, and a second toolchain, in exchange for nothing at all. That is not a slow start toward safety. It is a program that is exactly as unsafe as the C and harder to read than the C.

Machine translation is worse than useless here, and the reason is specific rather than aesthetic. This book has argued at length (Chapters 2, 14, and the section above) that a faithful, `unsafe`, C-shaped transliteration is the right *first* phase — so the objection lands close to home, and the difference has to be stated precisely. `ztmux`’s transliteration is written by hand, keeps `tmux`’s function names and structure, and is pinned by a parity suite and an anti-drift gate; every one of those properties exists to make the phase *escapable*. The names give the citations something to point at, the structure lets a reviewer diff a function against its C, and the parity cases pin the behavior precisely enough that a subsystem can be rewritten into safe Rust and *proved* to still behave identically. Machine output has none of that. Its names and control flow are artifacts of the translator rather than of the C, so nothing can be audited against anything; no suite pins its behavior, so nothing can be safely rewritten; and no human ever formed a model of it, so no human can idiomatize it. It is a transliteration with the exits removed — the phase, permanently.

Which is why the honest thing to do with such output is delete it. This port started by evaluating a machine-shaped first-pass Rust tree and concluded it was not a foundation but a liability: every module had to be rewritten from the C anyway, and the rewriting was slower for having the generated code in front of it than it would have been from a blank file. The C is the specification. A tool that stands between you and the specification, emitting something that looks like progress while preserving precisely the semantics you meant to eliminate, has cost you the work twice.

Chapter 30 – Sessions, Groups, and the Window Index Space

Chapter 3 introduced the state tree as three levels — sessions hold windows, windows hold panes — and that picture is correct as a first approximation and wrong in one important detail. A window is not *owned* by a session. It is *referenced* by one, through a link record, and the same window can be referenced by several sessions at once, at different indexes, simultaneously visible in each. Almost every session-level command in Appendix A makes more sense once that indirection is visible, and several tmux behaviours that look arbitrary — why `kill-session` sometimes leaves a window alive, why window numbering has holes, why `link-window` exists at all — fall directly out of it. This chapter is about the middle layer of the tree: the link between a session and a window, the index space that link lives in, and the session-level policies built on top.

The window link and why windows are shared, not owned

The record that binds a session to a window is the *window link* — tmux's `winlink`, ported under the same name. It carries three things: an index (the number you see in the status line and type in `select-window -t 3`), a pointer to the window itself, and its membership in the session's ordered set of links. The window on the other end of that pointer knows nothing about which session is looking at it; it knows its panes, its layout tree, its name, and its size, and that is all. Ownership runs the other way from what the tree diagram suggests: sessions point at windows, windows point at nothing above themselves.

The consequence is that a window's lifetime is governed by *how many links reference it*, not by any single session. `link-window` adds a second link to an existing window from another session; the window now appears in two sessions' window lists, and a program running in one of its panes is visible from both. It is genuinely the same window, not a mirrored copy: type in a pane from session A's client and session B's client sees the characters appear, because there is one grid and both clients are drawing it. `unlink-window` removes one link. Only when the last link is gone does the window itself get destroyed and its panes killed. This is ordinary reference counting, and it is why `kill-session` on a session full of linked windows can leave every one of those windows alive — the other session still holds links, so nothing is orphaned.

`move-window` is `link-window` followed by `unlink-window` on the source, which is why it can move a window *between* sessions and not merely renumber it within one. `swap-window` does not move windows at all: it exchanges the window pointers inside two link records, leaving the indexes where they are. That difference is observable — after a `swap-window` the indexes are unchanged and the contents swapped, and after two `move-window` calls the contents are unchanged and the indexes swapped — and it is exactly the sort of distinction the parity suite pins, because a status line rendering `#{window_index}` alongside `#{window_name}` prints different text under each.

The index space: `base-index`, holes, and `renumber-windows`

Window indexes are not array positions. They are keys in a sparse, ordered map, and the session's link set is kept sorted by them. The first index a new session uses comes from `base-index` (default 0, and the option exists because a great many users set it to 1 to line the indexes up with the number-key bindings). When a command asks for the next free index, the search starts at `base-index` and walks upward until it finds a gap — so the index space is allocated by first-fit, not by an incrementing counter.

That first-fit rule is what produces the familiar hole behaviour. Open windows 0, 1, 2, close window 1, and the session's links are 0 and 2 with a hole at 1. Open a new window and it lands *in the hole*, at index 1, not at 3. Users who dislike that set `renumber-windows`, which after a window closes renumbers the remaining links to a contiguous run starting at `base-index`, so closing window 1 renumbers the old window 2 down to 1 and the next new window goes to 2. Both behaviours are correct; they are different policies over the same sparse map, and the option selects between them.

The sparseness also explains the target forms in the window commands. `new-window -t 3` asks for index 3 specifically and fails if it is taken unless you also ask for the kill-and-replace behaviour; `new-window -a` asks for “the index after the current window,” which means inserting into the space just above the current index — and if that index is occupied, the renumbering push that makes room is itself a defined operation rather than an error. `move-window -r` is the standalone renumber. None of this is complicated once the index space is understood as a sorted map with an explicit allocation policy; all of it is confusing if you assume windows live in a list at positions 0 through *n*.

Panes have the same structure one level down, with `pane-base-index` as the analogous first-index option, and the same first-fit reuse of freed pane indexes. The symmetry is not accidental — both are the same “sparse ordered keys with a configurable base” pattern, and both are ported with it intact.

The current window and the window stack

A session has a *current* window — the one its clients display — and, separately, a stack of recently-current windows. `select-window` sets the current window and pushes the previous one onto the stack; `last-window` pops back to the top of that stack. The stack is not merely “the previous window,” which is why repeatedly pressing the `last-window` binding toggles between two windows rather than walking backwards through your whole history: selecting B from A pushes A, and selecting A from B pushes B, so the top of the stack alternates. `next-window` and `previous-window` do something different again — they walk the *index order*, wrapping at the ends, ignoring the stack entirely. Three different notions of “the other window” coexist because users want all three, and a port that collapsed them into one would break muscle memory immediately.

The same pattern repeats at the pane level (`last-pane` over a per-window pane stack) and at the session level (`switch-client` over the client's session history). Each stack is maintained where the object it orders lives — the window stack in the session, the pane stack in the window — and each is updated as a side effect of the corresponding select operation rather than by a central history service. That is tmux's structure and the port keeps it, which also means the stacks are correctly scoped for free: destroying a window removes it from exactly one session's stack, and nothing has to scan a global list.

Session groups: many sessions, one window set

Session groups are the feature that makes the link indirection pay off visibly. `new-session -t existing` does not create an independent session; it creates a session in the same *group* as `existing`, sharing that group's window set. Both sessions list the same windows at the same indexes, but each has its own current window, its own window stack, and — critically — its own client attachments and therefore its own size.

The use case is two terminals of different sizes wanting the same work. Attach a laptop client to one session of the group and a large external display to another; each client gets the session's own current window and each session sizes independently, but they are looking at one shared set of windows and one set of running programs. Switch windows in one and the other does not follow, because the current window is per-session. Create a window in one and it appears in both, because window membership is per-group. This is exactly the behaviour the reference-counted link model produces without special-casing: a group's sessions hold parallel link sets over the same windows, and creating a window in the group adds a link in each member session.

Grouping also interacts with the destroy policies. `destroy-unattached` destroys a session when its last client detaches, which for an ungrouped session takes its windows with it (last link gone) and for a grouped session usually does not (the other members still hold links). `detach-on-destroy` decides what a *client* does when the session it is attached to is destroyed underneath it — detach entirely, or move to another session, with the choice values covering the most-recently-used, the next, and the previous. Under grouping, that choice is what makes a workflow where sessions come and go beneath a long-lived client feel continuous rather than like a series of ejections.

Why this layer is worth porting exactly

None of this is glamorous machinery. It is a sorted map, a reference count, three stacks, and a handful of policy flags. But it is the layer users' fingers are trained on most deeply, because the window index is the thing they type dozens of times an hour, and the index allocation policy is the thing that decides where a new window lands. A port that got `base-index` right but first-fit reuse wrong would place new windows one index off from tmux in exactly the situation — after closing a middle window — where a user is most likely to type the index from memory without looking. That is the failure mode the byte-for-byte bar exists to prevent, and it is why `#{window_index}`-bearing format cases are worth as much in the parity suite as any escape-sequence test.

Chapter 31 – Resize: From SIGWINCH to Reflow

Resizing is the operation that touches the most subsystems at once. A single terminal window drag has to cross the client, the socket, the session's sizing policy, the layout tree, every pane's screen, every pane's grid, every pane's child process, and the redraw path — and it has to do all of that fast enough to feel instantaneous while a user is still dragging. Chapters 4, 5, and 7 each described their own slice of this; this chapter follows one resize end to end and then examines the policy that decides what “the size” even means when several clients of different dimensions are attached to one session.

The client's job: report, do not decide

The client's involvement is deliberately trivial, and for the reason Chapter 4 gave: all intelligence lives server-side. The client installs a `SIGWINCH` handler; when the outer terminal resizes, the handler sets a flag, the client's event loop notices it, queries the new size of its terminal, and sends a `MSG_RESIZE` (Chapter 17) carrying the new geometry. That is the entire client-side story. The client does not resize anything, does not recompute a layout, does not redraw from its own knowledge of the new dimensions — it reports a fact and waits for the server to send it a new frame.

This is worth dwelling on because it is what makes multi-client resizing coherent at all. If clients decided their own layouts, two clients on one session would compute two layouts and the session would have no single answer to `#{window_layout}`. Because clients only report, the server holds exactly one layout tree per window and decides, under an explicit policy, which client's dimensions it should be sized to.

The sizing policy: window-size and the four answers

The policy option is `window-size`, and its choices are the four defensible answers to “which client's size wins.”

`smallest` sizes the window to the smallest attached client's dimensions. Every client can then display the whole window, and the larger clients letterbox — unused rows and columns filled with `fill-character`. This is the safe choice: nothing is ever cut off for anybody.

`largest` sizes to the largest client. The window uses the biggest screen fully, and smaller clients see only a portion of it rather than the whole. This trades universal visibility for not wasting a big display.

`latest` sizes to whichever client most recently did something — the client with the most recent activity gets the window sized to it, and the size follows you between terminals as you use them. This is the choice that makes the two-terminal workflow feel right: the terminal you are typing in is always the one the layout fits.

`manual` freezes the size. The window stays at whatever it was set to and no client attachment changes it, so the resize commands are then the only way to change the dimensions. This is for sessions whose layouts encode a fixed geometry the user does not want disturbed.

Sitting alongside these is `aggressive-resize`, a per-window option that narrows the question from “all attached clients” to “clients actually *viewing this window* right now.” Without it, a window is sized against the session's clients regardless of what those clients are currently displaying — so a small client parked on window 1 constrains window 7 that nobody is looking at. With it, window 7 sizes to whoever is actually watching

window 7, and re-sizes when the audience changes. The cost is more resize churn, since switching windows can now trigger a resize; the benefit is that an idle small client stops taxing every window in the session. And `default-size` supplies the answer to the degenerate case: a session with no attached clients at all still needs dimensions for its panes' ptys, so it takes them from `default-size` (80x24 by default), which is why a detached `new-session -d` followed by `send-keys` behaves sensibly rather than against a zero-sized screen.

The cascade: window, layout, screen, grid, child

Once the server has decided the new window dimensions, `resize.rs` orchestrates a cascade, and each step is a subsystem the book has already covered doing its own job.

First the window's size is set. Then the layout tree (Chapter 7) is recomputed against the new outer rectangle: the tree's proportions are preserved where they can be, cells are scaled, and the integer rounding rules — the same rounding whose left-to-right order produced the 39|40-versus-40|39 parity bug of Chapter 11 — distribute the remainder. This is the step where a faithful port earns its keep, because the rounding decisions are invisible until you diff two layout strings and find the columns off by one.

Each pane then gets its new rectangle, and for each, the pane's screen is resized. Screen resize is two operations that are easy to conflate and behave differently. Changing the number of *rows* is comparatively simple: rows are added at the bottom or removed, with removed content pushed into scrollbar rather than discarded, and the cursor and scroll region clamped to the new bounds. Changing the number of *columns* triggers reflow (Chapter 5): lines previously soft-wrapped at the old width must be unwrapped and rewrapped at the new one, so a paragraph that occupied three lines at eighty columns occupies fewer at a wider size and the scrollbar above the visible screen changes line count. Reflow is the expensive part of a resize, it is proportional to the history size, and it is why a pane with a very deep scrollbar resizes measurably more slowly than an empty one.

Finally the child process is told. The pty's window size is updated with the pane's new dimensions, which causes the kernel to deliver `SIGWINCH` to the process group in the pane — and that is how the editor inside the pane learns to redraw itself. The chain is: outer terminal `SIGWINCH` → client → server → layout → screen → pty → inner `SIGWINCH`. The same signal at both ends of a path that crosses a socket, a tree, and a grid.

Then the redraw. Because the pane dimensions changed, this is one of the cases `screen_redraw_check_is` (Chapter 20) resolves to a *full* redraw rather than a differential one: the entire client frame, borders included, is recomposited and sent. That full repaint is the visible cost of a resize and the reason a drag-resize of a terminal window sends far more bytes than ordinary typing.

Zoom as a resize that remembers

`resize-pane -Z` is worth calling out here because it is a resize that deliberately does not follow the cascade in the usual direction. Zooming does not restructure the layout tree; it saves the tree, replaces it with a single cell occupying the whole window, and marks the window zoomed. Unzooming restores the saved tree verbatim. That is why a zoomed pane un-zooms back to the *exact* column widths it had before, down to the last cell, rather than to a freshly recomputed even split — the layout was never recomputed, only shelved. The panes that were hidden during zoom keep their screens and their running programs the whole time; only their visibility changed. It is the cheapest possible implementation of the feature and also the most faithful, and both properties come from the same decision to save and restore rather than rebuild.

Manual resize and where the columns go

`resize-pane -L/-R/-U/-D` with an amount does not resize one pane; it moves one *border*, which necessarily resizes at least two cells of the layout tree. The tree walk finds the appropriate parent — the nearest ancestor

whose split axis matches the direction — and moves the boundary between the two children, then propagates the change down each subtree. If the neighbouring pane cannot give up the space because it would fall below its minimum size, the resize is clamped rather than refused, so dragging a border into a wall stops rather than erroring. Minimum pane sizes are also why an over-split window eventually refuses further splits: the layout engine will not produce a cell smaller than the floor, and `split-window` fails rather than creating a degenerate cell.

The mouse-drag resize is the same operation driven differently: the mouse handling (Chapter 26) resolves a drag on a border into a target border and a delta and calls the same resize path. There is no separate mouse resize implementation, which is a small example of the general structural point — the port has one mechanism per behaviour, and the several user-facing routes into it are all thin.

Chapter 32 – Terminal Overrides and the Capability Correction Layer

Chapter 10 established the shape of the terminal back end: terminfo describes what the outside terminal can do, and the tty layer emits the shortest sequence that achieves an effect on that terminal. What that chapter left for later is the uncomfortable truth underneath it — terminfo is frequently *wrong*. Terminal emulators ship capabilities they do not implement, omit capabilities they do implement, and are identified by `TERM` values inherited from ancestors whose behaviour they no longer share. A multiplexer that trusted terminfo naively would emit sequences that garble output on real, widely-used terminals. tmux therefore maintains a correction layer over terminfo, and this chapter is about that layer: `terminal-overrides`, `terminal-features`, `default-terminal`, and the runtime feature registry that ties them together.

Two `TERM` values, pointing opposite directions

The first thing to be clear about is that a multiplexer sits between two terminals and has a `TERM` for each, and confusing them is the source of most terminal-configuration confusion.

The *outer* `TERM` is the one the client's real terminal advertises. The client sends it in the identify burst (`MSG_IDENTIFY_TERM`, Chapter 17), along with the terminfo entries themselves (`MSG_IDENTIFY_TERMINFO`, one message per capability) so the server does not have to look them up in its own environment — a critical detail, because the server may be a daemon started long ago in a different environment, and over a remote connection the terminfo database that knows about your terminal may only exist on the client side. This outer `TERM` governs what the server may emit *to your screen*.

The *inner* `TERM` is the one the server sets in each pane's environment, and it comes from `default-terminal`. It describes what ztmux itself implements — what programs running inside a pane may emit *to ztmux*. It is a different terminal, because ztmux's own capability set is fixed by ztmux's input parser (Chapter 6), not by whatever terminal happens to be on the other end. This is why the answer to “why is `TERM` inside the multiplexer not the same as outside” is not a workaround: they genuinely describe different machines. `screen-256color` and `tmux-256color` are the usual values, and the difference between them matters mostly for whether the italics capability is advertised, which is the classic reason a program shows reverse video where you expected italics.

The two directions are connected by translation, not by pass-through. A program emits an inner-`TERM` sequence, the input parser turns it into cell state in the grid, and the tty back end re-encodes that cell state into outer-`TERM` sequences. The multiplexer is a transcoder, and the two capability sets need not match at all.

`terminal-overrides`: patching the terminfo entry

`terminal-overrides` is an array option whose entries each match a `TERM` pattern and then add, replace, or remove capabilities for terminals matching it. The mechanics are simple — a pattern, then a list of capability assignments — and the power is entirely in what it lets you correct without touching the system terminfo database, which may be read-only, shared, or simply not the place you want per-user terminal fixes to live.

The canonical uses fall into three groups. *Adding* a capability the entry omits: your terminal supports something the database's entry for its `TERM` does not mention, and the override tells ztmux it may be used. *Removing* a

capability the entry falsely claims: setting a capability to empty makes the tty layer treat it as absent and fall back to whatever it does when the capability is missing, which is the fix when a terminal advertises something it renders incorrectly. *Replacing* a capability with a corrected sequence: the entry has the right idea and the wrong bytes.

The pattern matching against `TERM` is what makes a single config portable across the several terminals one user actually attaches from. A `.tmux.conf` accumulates a few override lines over the years, each one the residue of a specific terminal misbehaving on a specific day, and each one continues to apply only to the terminal it names. Because ztmux ports the option and its matching semantics rather than reinterpreting them, those accumulated lines keep working unchanged, which is precisely the compatibility promise Chapter 1 made.

terminal-features: capabilities as named bundles

`terminal-overrides` works at the granularity of individual terminfo capabilities, which is precise and tedious. Modern terminal behaviour tends to arrive in *bundles* — a terminal either supports the whole truecolor story or none of it; either it implements synchronized output or it does not — and describing those bundles one capability at a time is both verbose and error-prone. `terminal-features` is the higher-level option: it names a feature for a matching `TERM`, and the feature registry in `tty_features.rs` expands that name into the set of capabilities it implies.

The registry is the runtime model of “what can this specific attached terminal do,” assembled from three sources: the terminfo entry the client sent, the features implied by matching `terminal-features` entries, and features negotiated at runtime — because some capabilities can be *asked about*. A terminal that responds to a capability query tells ztmux directly what it supports, and the answer feeds the same registry. The three sources are merged into one per-client feature set, and every decision in the output path consults that set rather than re-reading terminfo.

The practical importance of the feature abstraction shows up in the truecolor case Chapter 10 sketched. Whether an RGB cell colour is emitted as a 24-bit SGR sequence, degraded to the nearest 256-palette entry, or degraded further to one of the sixteen ANSI colours (Chapter 24’s `colour_find_rgb` and `colour_256to16`) is a decision made per client, from the feature set, at draw time. Two clients attached to the same session, one on a truecolor terminal and one on a sixteen-colour one, get different bytes for the same grid cell — which is only possible because the capability model is per-client state and not a global.

tty_acs and the line-drawing fallback

One correction deserves its own mention because it is the one users notice as visual corruption rather than as a missing feature. Pane borders (Chapter 20) are drawn with line-drawing glyphs, and there are two ways to get them onto a terminal: the Unicode box characters, or the VT100 alternate character set, in which the terminal is switched into a graphics mode where ASCII letters render as line segments. `tty_acs.rs` is the mapping between the two, and which route is taken depends on the terminal’s advertised capabilities and UTF-8 support.

This matters because getting it wrong is spectacular. Emit alternate-character-set escapes to a terminal that does not honour them and the borders come out as literal letters strewn across the screen — a failure mode instantly recognisable to anyone who has run a curses program over a mis-configured connection. Emit Unicode box characters to a non-UTF-8 terminal and you get mojibake instead. The decision is made from the same feature registry, and the mapping table is ported entry for entry, because a table with one wrong row produces exactly one wrong glyph at exactly one kind of border junction — a bug that would survive casual testing indefinitely.

`pane-border-lines` sits on top of this as the user-facing choice of *which* border style to draw, and its options interact with the capability question: a heavy or double-line border style requires Unicode, so the selection and the capability model have to agree. The fallback is not an error; it is a quieter border.

Extended keys, and negotiation as a two-way conversation

The keyboard direction has the same problem in mirror image. Historically, terminals encode a limited set of modified keys, and combinations users want — control with a shifted character, several modifiers at once — are simply not representable in the legacy encoding. The extended-key encodings fix this, and `extended-keys` and `extended-keys-format` are the options that control whether ztmux asks the outer terminal to use them and how it reports modified keys to programs inside panes.

This is a negotiation with two independent halves, and both halves have to be tracked separately. On the outside, ztmux may request extended reporting from the client's terminal; whether it gets it depends on that terminal. On the inside, ztmux may emit extended sequences to a pane's program; whether that program understands them depends on the program. `input_keys.rs` (encoding key codes into the bytes a pane receives) and `tty_keys.rs` (decoding the outer terminal's bytes into key codes) are the two ends, and the extended-key options configure them independently. `user-keys` provides the escape hatch for the remaining cases: a raw byte sequence your terminal sends that nothing in the standard tables recognises can be declared as a user key and then bound like any other.

The general shape here is the shape of the whole chapter. A multiplexer cannot assume the terminals at either end are correct, complete, or honest about themselves. It maintains a model of each, patched by user configuration and by runtime negotiation, and it consults that model on every emission. Porting the model faithfully is not optional polish — a port that got the correction layer subtly wrong would work perfectly on the developer's terminal and corrupt output on somebody else's, which is the worst possible distribution of a bug.

Chapter 33 – The Options Table as a Data Structure

Appendix C lists the options and Chapter 9 explained their scopes. What neither covers is the thing that makes both possible: `options_table.rs`, a single static table of entries that is the sole authority on what options exist, what type each one has, what values are legal, what the default is, and which scope it belongs to. Nearly every behaviour users attribute to “tmux’s configuration system” is actually a property of that table’s shape, and walking the shape explains a set of otherwise unrelated-looking facts at once.

One entry, six facts

Each entry in the table carries, at minimum: a name; a type; a scope mask; a default value; where relevant, a set of legal values or a numeric range; and a flag set for the odd cases. The types are a small closed set — string, number, key, colour, flag (a boolean), choice (an enumerated string), command (a parsed command list), and array — and every option in Appendix C is one of them.

The type is not documentation. It is the parser and the validator. When you write `set -g status-position bottom`, the table entry for `status-position` says *choice* and lists its legal values, so the string is checked against that list and rejected with an error naming the valid options if it does not match. When you write `set -g history-limit 5000`, the entry says *number* and the value is parsed as one, with the range checked. When you write `set -g prefix C-a`, the entry says *key* and the string goes through `key_string.rs` to become a key code — which is why setting the prefix to an unparseable key name fails at `set-option` time rather than mysteriously producing a prefix key that can never be pressed. Validation at set time rather than at use time is a deliberate property: a typo in a config file surfaces as a startup error attributed to a line number (Chapter 9’s config causes) rather than as a feature that silently stopped working.

The scope mask and where a value lands

The scope mask on each entry is the mechanism behind `options_scope_from_name`, which Appendix C described from the outside. An entry declares which of the scopes — server, session, window, pane — it may live in, and the resolution rule is that the name determines the natural scope and the command’s flags constrain it further. This is why bare `set-option foo bar` works at all for most options: the table already knows where `foo` belongs, so the flag is redundant in the common case and only needed to disambiguate the options whose masks admit more than one scope.

That “more than one scope” case is the window-and-pane group in Appendix C — options like `remain-on-exit` and `synchronize-panes` whose masks include both window and pane. For those, `set -w` and `set -p` land in different trees, and the pane tree falls back to the window tree on lookup. The fallback chain is the second half of the story: reading an option walks from the most specific scope outward, so a pane option not set on the pane comes from the window, a session option not set on the session comes from the global session tree, and so on until the table’s default is used. Every read therefore terminates, and no option is ever unset in the sense of having no value — it has, at worst, the table’s default.

This is also the mechanism behind an under-appreciated behaviour: setting an option *globally* and setting it on a specific object are not the same operation, and unsetting the specific one reveals the global one again rather than reverting to the compiled default. Unsetting removes a value from a tree, and what you then observe is

whatever the next level out holds. A user who has set a global `status-style` and then a per-session override can remove the override and get the global back, not the shipped default. Getting that inheritance exactly right matters for the same reason everything else in this book does: config files depend on it, often without their authors realising they do.

Arrays and the subscript syntax

Array options — `terminal-overrides`, `terminal-features`, `command-alias`, `user-keys`, `update-environment`, `status-format`, `pane-colours` — are the type that behaves least like the others, and they behave that way because appending is the dominant operation. A user's config does not want to *replace* the whole `terminal-overrides` array, wiping out whatever defaults exist; it wants to add one entry. So the array type supports append as a first-class operation, and the subscript syntax addresses individual elements by index for the cases where you must replace or remove one specifically.

The subscript form also appears on the read side. `show-options` prints array options element by element with their indexes, which is what makes an array option's state legible — you can see exactly what accumulated and in what order, which for `terminal-overrides` is the difference between debugging a terminal problem in one command and guessing. And because the format language can read options, an array element is reachable from a `#{...}` expansion too, which is how a status line can reflect configuration rather than only state.

User options and the deliberate open extension point

The table is closed — an option that is not in it does not exist and `set-option` on it is an error — with exactly one hole: names beginning with `@` are *user options*. They are not in the table, they have no declared type (they are strings), no validation, and no default, and they may be set at any scope. They exist so that configuration and scripts have somewhere to keep their own state without colliding with the option namespace, forever.

This is a well-designed extension point and worth appreciating as one. The alternative designs are all worse: letting arbitrary names into the main namespace makes typos silent — you misspell an option, nothing complains, nothing works — while reserving a prefix keeps strict validation for real options and still gives scripts unlimited room. The `@` prefix is also what makes user options safe against future tmux versions adding new real options, since no real option can ever be named with a leading `@`. Plugins built on tmux lean on this heavily, and the port carries it because a superset that broke plugin state storage would break the ecosystem it wants to inherit.

Why “accepted but not yet wired” is the honest state

Appendix C carried a caveat worth restating in structural terms: every option in the table is accepted, validated, stored, and readable — the *configuration surface* is complete — while not every one is yet connected to behaviour. That sounds like a weakness and is in fact the correct order of operations for a port.

The reason is that the table is the thing every config file touches on line one. A real `.tmux.conf` sets dozens of options, and a single unknown-option error aborts config loading at that line, taking every subsequent line with it. If options were added to the table only as their behaviour landed, then every user's config would fail to load until the very last option they happened to use was implemented — and each failure would look, from the user's side, like a total failure rather than a single missing feature. Populating the table first means a real config parses cleanly today and the behaviours light up one at a time underneath it, with the ones still dark being individually observable rather than catastrophic.

It also means the gap is *measurable*, which is the recurring theme of Chapters 12 and 15. An option that is in the table but unread by any subsystem is a specific, greppable, countable thing — a name with no consumer

— rather than a vague sense that “options are partly done.” That is the same discipline as the anti-drift gate applied to configuration: make the incompleteness precise, and it becomes a work list instead of a worry.

Chapter 34 – Overlays: Menus, Pop-ups, and the Chooser Modes

Chapter 18 described copy mode as a *mode* pushed onto a pane, and named the other modes in passing — the clock, the choosers, the customizer. There is a second, structurally different way tmux draws interactive UI, and the difference is worth a chapter: an *overlay* is not attached to a pane at all. It belongs to the client, floats above whatever the client is displaying, and intercepts that client's keys until it is dismissed. Menus and popups are overlays; copy mode and the choosers are modes. The distinction determines what each can do, and both mechanisms are ported.

Modes versus overlays

A mode replaces a pane's display. It is per-pane state, it is sized to the pane's rectangle, every client viewing that pane sees it, and it composites through the ordinary pane redraw path (Chapter 20). Two clients looking at the same pane in copy mode see the same copy-mode screen — including the same cursor position and the same selection, because there is one mode and one screen. That shared-ness is occasionally surprising and entirely consistent with the rest of tmux's model, where panes are shared objects and clients are viewers.

An overlay belongs to one client. The client holds a reference to it along with the callbacks that draw it, handle its keys, and free it. When a client with an overlay redraws, the overlay is composited *after* the panes and status line, so it appears on top; when the client receives a key, the overlay's key callback gets first refusal and can consume it. Another client attached to the same session sees nothing — its own overlay reference is empty. This is why a `display-menu` triggered from one client does not appear in another, and why two clients can have two different menus open simultaneously over the same session.

The two mechanisms answer different questions. “Show a different view of this pane's content” is a mode. “Ask *this user* something” is an overlay. Copy mode is about the pane, so it is a mode; a confirmation menu is about the person who pressed the key, so it is an overlay.

Menus: a list, a position, and a command per item

`display-menu` builds a menu from a series of item triples — a title, a key, and a command — and draws it as a bordered box at a chosen position. The position arguments accept anchors as well as coordinates, and the mouse-driven case positions the menu at the click, which is what makes the default mouse bindings' context menus land where you clicked.

Selecting an item runs its command, which means a menu is not a special UI construct with its own vocabulary — it is a set of command-language invocations with labels. That has two consequences worth naming. A menu can do anything the command language can do, including opening another menu, because `display-menu` is itself a command. And each item's title is a *format string*, expanded at menu-build time against the target context, so menu labels can show live state — a pane's command, a window's name, the zoom flag — rather than static text. Both properties fall out of building the menu on top of existing machinery instead of beside it, and both are inherited by the port without extra work precisely because the port kept that structure.

Key handling in a menu is the overlay's key callback: navigation keys move the selection, the per-item keys jump directly to and activate an item, and the dismissal keys tear the overlay down. The item keys are why menus are fast for anyone who uses them twice — you stop navigating and start typing the letter.

Popups: a pane in a box, with a real program in it

`display-popup` is the more ambitious overlay because it does not display a list; it runs a *program*. A popup gets a pty and a child process much as a pane does (Chapter 19's spawn path), and a screen and grid to render into, and its output goes through the same input parser as any other program's output. It is drawn as a bordered floating rectangle over the client's display, and keys the client presses go into it.

The distinctive thing about a popup is where it sits in the object model: it is client-owned like a menu, but pane-like in its content. It is not in the layout tree, so it does not disturb the panes underneath — no resize cascade (Chapter 31), no layout string change, no `#{window_layout}` difference. It is not in the window's pane list, so `list-panes` does not show it and pane-navigation commands cannot select it. It exists for as long as its program runs or until it is dismissed, and then it is gone with no trace in the session's structure.

That combination is what makes popups useful for exactly the workflows they get used for: a scratch shell, a quick file browser, a repository interface — things you want *now*, over what you are doing, without rearranging your panes and without leaving a window behind to clean up. The costs are the flip side of the same properties: a popup is not part of the layout, so the operations that act on layout cells — splitting it, zooming it, tiling around it — do not apply, because none of the machinery that provides those things reaches an object outside the tree.

`mode_tree`: one widget behind four choosers

`choose-tree`, `choose-client`, `choose-buffer`, and `customize-mode` are four different commands presenting four different kinds of data, and they are all one implementation. `mode_tree.rs` is a general tree/list widget — a hierarchical item list with expand and collapse, a selection cursor, a filter, a sort order, a preview region, and a tagging facility for acting on several items at once — and each chooser supplies the data and the per-item behaviour.

So `window_tree.rs` supplies sessions containing windows containing panes, with a preview that shows the selected pane's content and commands that select or kill; `window_client.rs` supplies attached clients, with detach; `window_buffer.rs` supplies paste buffers, with paste and delete; `window_customize.rs` supplies the options and key tables, with edit. The navigation keys, the filtering, the tagging, the sorting, and the preview region are identical across all four because they are literally the same code.

This is the strongest example in the tree of the general principle Chapter 29 defended. Reimplementing four choosers as four idiomatic Rust UIs, each with its own list handling, would have produced code that was individually nicer and collectively divergent — four places for the filter key to behave slightly differently, four places for a sort order to be off. Keeping the shared widget keeps the four consistent with each other and with tmux, and keeps the parity comparison localised: a chooser bug is a bug in one of five files, and `mode_tree.rs` is the first one to check.

Why overlays complete the interaction model

With overlays described, the full set of ways ztmux can put something on your screen is visible, and it is exactly four. A pane's program writes to its grid and the grid is composited. A mode replaces a pane's display with a view of the pane's own data. The status line occupies its own rows and time-shares them between the bar, transient messages, and the prompt (Chapter 20). An overlay floats above everything for one client. Every visible element of a running ztmux is one of those four, they are drawn in that order, and each has a clearly-defined owner — pane, pane, session, client respectively.

That ownership column is the part worth remembering, because it predicts behaviour without consulting any documentation. Anything owned by a pane is seen by every client viewing the pane. Anything owned by a client is seen by that client alone. When a user asks why their menu did not appear on the other terminal,

or why a second person attached to the same session sees their copy-mode selection, the answer is in the ownership column, and the port preserves it exactly because it is the model users have internalised.

Chapter 35 – Porting Idioms: A Catalogue of C-to-Rust Translations

Chapters 2 and 14 established the method — transliterate the C, keep the names, keep the structure, accept the `unsafe`, idiomatise later — and Chapter 2 walked one function to demonstrate. What has not been collected in one place is the *vocabulary*: the recurring C constructs tmux uses, and the specific Rust rendering each one gets. That vocabulary is what makes the port tractable at scale, because once each construct has a settled translation, porting a new module is mostly mechanical, reviewing a port is mostly pattern-matching, and a deviation from the settled form is visible as a deviation rather than as a judgement call. This chapter is the catalogue.

Intrusive lists and trees

tmux, like most OpenBSD-lineage C, builds its data structures with the queue and tree macro families — doubly-linked lists and red-black trees defined as preprocessor macros. The defining property is that they are *intrusive*: the link fields live inside the element struct, so an element is a member of its container by virtue of its own fields, and there is no separate node allocation. A window carries the link fields that put it in the global window list; a session carries the tree fields that put it in the session tree.

This does not translate to a Rust `Vec` or `BTreeMap`, and the port does not try. Intrusiveness is observable: a pointer to an element is simultaneously a handle to its position in every container it belongs to, elements are removed in constant time from a bare pointer without knowing the container, and an element in several containers has several sets of link fields. Rewriting to owning collections would change the ownership model of the entire program, which is exactly the kind of wholesale restructuring that makes a port uncheckable. So the link fields stay as raw pointers in `#[repr(C)]` structs, and the macro operations become functions or macros in Rust with the same names and the same semantics, operating on the same fields. The `unsafe` this requires is real and is precisely the surface Chapter 14 plans to retire subsystem by subsystem — but retiring it means replacing a whole subsystem's container discipline at once, with parity cases as the specification, not sprinkling conversions through a port in progress.

`#[repr(C)]` and why layout is part of the contract

Every struct that corresponds to a tmux struct is `#[repr(C)]`, and Chapter 29 gave the reviewability argument. There are two mechanical reasons on top of it.

First, some of these structs cross the FFI boundary or are handed to libc, and those calls require C layout by definition. Second — and this is the subtler one — field layout affects memory cost, and memory cost is behaviour in the structure the book cares most about. The packed cell entry of Chapter 5 is the size it is because of exactly how its fields are arranged; a Rust-layout struct with the same fields could be reordered and padded by the compiler into something larger, changing the memory profile of a screenful of text and the cache behaviour of the redraw walk that Chapter 23 counted on. `#[repr(C)]` is what makes the size claim a statement about the port and not only about the C.

Unions get the same treatment, which is where the packed/extended cell split actually lives: a C union of the packed representation and the extended index, rendered as a Rust union with C layout and read through the

same discriminating flag the C uses. A Rust `enum` would be the idiomatic form and would carry its own tag, changing the size and the semantics of the flag the rest of the code tests.

`static mut` globals and the single-threaded assumption

tmux is single-threaded. Its event loop is the only thing running, and its global state — the session tree, the window list, the client list, the options trees, the command queue — lives in file-scope globals reached directly by every function that needs them. There is no lock anywhere in the program because there is nothing to lock against.

The port keeps that model, which means `static mut` and the `unsafe` that touching it requires. The alternative — wrapping globals in synchronisation primitives, or threading a context parameter through every function — would either add cost for a contention that cannot occur or change every function signature in the tree, destroying the C correspondence that the anti-drift gate (Chapter 12) checks. The honest framing is that the single-threaded invariant is a *precondition* the port inherits, and the `unsafe` blocks are where that precondition is being relied upon. It is not unchecked in the sense of unknown; it is unchecked in the sense of not machine-checked, with the event-loop architecture as the argument for why it holds.

Callbacks in the C shape

The event loop calls back into the program constantly — every readable fd, every timer, every signal. Those callbacks are `extern "C-unwind"` functions carrying libevent's exact signature, registered with the same event values tmux registers, and they kept that shape after the loop itself became Rust (Chapter 3): the ported code stores and passes them exactly as the C does, so changing their type would have meant editing every call site to no benefit. The `-unwind` part is deliberate rather than incidental: it declares that a Rust panic may propagate through the frame rather than being undefined behaviour, which is what makes the panic path (and the panic file of Chapter 36) meaningful in code entered through a C-ABI function pointer.

The same shape appears wherever a C API takes a function pointer, which in tmux is more places than the event loop: comparison functions for tree ordering, the per-item callbacks the command queue fires, the overlay draw/key/free callbacks of Chapter 34. In each case the Rust side is a plain `extern` function taking the same arguments and an opaque context pointer, cast back to its real type at the top of the body. This is not pretty, and it is the direct translation, and the direct translation is what lets a reviewer put the two functions side by side.

Strings, `xmalloc`, and the ownership of `char *`

C string handling is where a naive port produces the most subtle bugs, because Rust's `String` and `CString` carry guarantees a C `char *` does not — UTF-8 validity, and a length rather than a terminator. tmux's strings are byte strings that are *usually* UTF-8, sometimes deliberately not, and frequently sliced at byte boundaries by code that is counting terminal columns rather than characters.

So the port keeps C strings as C strings where the C does: nul-terminated byte buffers allocated by the `xmalloc.rs` wrappers, freed explicitly, passed as raw pointers. The `xmalloc` family — allocation that aborts rather than returning null, `xstrdup`, the formatted-string allocators — is ported as itself, because its failure semantics are part of the program's behaviour: tmux does not have out-of-memory error paths threaded through every call site precisely because `xmalloc` never returns null, and reintroducing fallible allocation would mean inventing error handling the C does not have and the parity suite cannot check.

The place this matters most is the format engine (Chapter 16), which is string manipulation end to end and where the difference between “byte index” and “character index” and “display column” is load-bearing three different ways in the same function. Chapter 25's UTF-8 machinery is the part that knows about characters

and widths; the surrounding string plumbing deliberately does not, and keeping the two separated the way the C separates them is what keeps the width calculations correct.

Out-parameters, `cause`, and error reporting

C reports errors through return codes plus out-parameters, and tmux's dominant idiom is the `cause` parameter: a function returns failure and writes an allocated, human-readable explanation into it, which the caller prints and frees. The command layer is built on this — it is how `set-option` reports an invalid choice value with the list of legal ones, and how config loading (Chapter 9) collects the per-line causes it reports at startup.

A Rust `Result` is the idiomatic rendering and the port does not use it here, because the message text and the exact set of conditions that produce it are observable output the parity suite compares. Preserving the `cause` idiom keeps the message construction in the same place in the same function producing the same string; converting to `Result` would invite restructuring the error paths, and restructured error paths produce differently worded messages, and differently worded messages are parity failures. The conversion is a good candidate for the idiomatisation phase — once the messages are pinned by cases, the plumbing can change while the strings stay — which is the general rule Chapter 14 stated, seen here in a specific instance.

Enumerations, flag sets, and the constant blocks

C enumerations and constant families become Rust constants of the underlying integer type, not Rust `enums`, and flag sets become integer constants combined with bitwise operators, not a wrapper type. The reason is the same in both cases: the values are compared, stored in `#[repr(C)]` struct fields, passed to C, and — for the flag sets — combined and tested with masks in ways that a closed Rust `enum` would make either impossible or noisy with conversions.

There is a real cost here, and it should be stated plainly: a Rust `enum` would make invalid states unrepresentable and an integer constant does not. This is precisely one of the conversions Chapter 14's idiomatisation phase exists to perform, and it is a good one to perform early in a subsystem's sweep, because it is local — the flag set for one module can become a proper type while the rest of the tree keeps integers, with conversions at the boundary. But it is a change to make *after* the module's behaviour is pinned, not during the transliteration that establishes what the behaviour is.

What the catalogue is for

Written out, these translations look like a list of compromises, and in a sense they are — each one trades a Rust guarantee for a correspondence with the C. The point of collecting them is that the compromises are *systematic*. Every intrusive list is handled the same way; every callback has the same shape; every `cause` out-parameter looks like every other. A reviewer reading an unfamiliar module recognises the idioms immediately and can spend attention on the logic instead of on the plumbing. A porter starting a new module does not re-decide any of this. And when the idiomatisation phase comes, the conversions are *mechanical at scale*, because there are a dozen recurring patterns rather than a thousand individual choices — which is the difference between a safety sweep that is a project and one that is a rewrite.

Chapter 36 — Failure, Diagnosis, and the Logs

Every chapter so far has described the program working. This one describes it failing, and how to find out why — which for a port is not a peripheral concern but a central one, because the most common thing a porter needs to do is take a divergence from the reference and localise it to a function. The tooling for that is the same tooling a user needs when something misbehaves, so it is worth one chapter to lay out what ztmux records, where, and how to read it.

Crash semantics: what a panic means in a ported program

The first question is what happens when Rust code panics inside a program that is a transliteration of C. In the C, the equivalent situations are undefined behaviour or a signal — a null dereference faults, an assertion aborts. In the port, a bounds violation or an explicit panic unwinds, and because the frames may have been entered from C through the `extern "C-unwind"` callbacks of Chapter 35, that unwinding is well-defined rather than undefined.

What ztmux does with it is write a panic file. The diagnostic artefacts Appendix L names — the panic file and the output log — are the two places a failure leaves evidence, and the panic file is the one that captures an unexpected termination with its message and location. This is a meaningful improvement over the C's behaviour and one of the few places the port is straightforwardly better rather than merely equivalent: a crashed tmux leaves a core file if the system is configured to keep one and nothing otherwise, while a crashed ztmux leaves a file naming what went wrong.

The subtlety is *whose* process crashed. The server holds every session, so a server panic is catastrophic — sessions and their child processes go with it. A client panic is not: the client is a relay (Chapter 4), the session keeps running, and re-attaching gets you back exactly where you were. This asymmetry is worth internalising when reading a bug report, because “ztmux crashed” describes two very different events, and the panic file's contents distinguish them.

The server log and verbosity

The server can be started with logging enabled, and what it records is the event-level history of the process: connections accepted, messages received and their types, commands parsed and fired, panes spawned and reaped, signals delivered. Because the server is single-threaded (Chapter 35), the log is a strictly ordered account of everything that happened, with no interleaving to untangle — which makes it far more useful than a multi-threaded program's log at the same level of detail.

The workflow this supports is the one that matters for a port. A parity case fails; the diff shows the output differing at a specific point; the log shows the sequence of internal operations leading to that point in both the failing run and a working one; the first divergence in the operation sequence localises the bug to a function far faster than reading the code that produced the output. The three headline bugs of Chapter 11 were all found this way in the general sense — a byte-level divergence narrowed to an operation-level divergence narrowed to a function.

The tty output log

`log.rs` also carries the tty output log, and this one is specialised: it records the bytes the tty back end actually emitted to a client's terminal. That is exactly the artefact needed to debug the output path, because the output path's whole job (Chapter 10, Chapter 23) is emitting the *shortest* correct sequence, and “shortest” and “correct” fail in different ways.

A too-long sequence is a performance bug and invisible to the eye. A wrong sequence is a visual corruption whose cause is unguessable from the corruption itself, because by the time you see a garbled screen the terminal has already interpreted the bytes and thrown them away. Capturing the byte stream means the failing sequence can be examined directly, replayed against a bare terminal, and compared against what the reference tmux emits for the same grid state — which turns “the borders look wrong on this terminal” from a report you cannot act on into a byte diff.

This log is also the mechanism that makes the output path *comparable* in principle rather than only in spirit. Chapter 23 noted that the emitted byte stream is observable behaviour; the tty log is where observing it happens.

show-messages and the in-server ring

There is a third record, and it is the one users interact with without thinking of it as logging. The server keeps a bounded ring of recent messages — the same messages that flash in the status line — and `show-messages` prints it. `message-limit` sets the depth.

The value of this record is that status-line messages are *transient by design*: a `display-message` shows for `display-time` milliseconds and vanishes, and command errors appear the same way. If a command in a key binding fails and its error flashes while you are looking at your editor, the error is gone before you know it happened. `show-messages` recovers it. For debugging a `.tmux.conf` — where a binding runs three commands and the second one silently fails — it is the first thing to check, and it is available without restarting anything or enabling any logging.

Config-time errors versus runtime errors

The two failure classes users encounter are worth distinguishing because they are surfaced differently and on purpose.

Config-time errors are collected. `cfg.rs` loads the config file, and a line that fails to parse or a command that fails to run produces a cause (Chapter 35's `cause` idiom) which is *accumulated* rather than printed immediately — because at config-load time there may be no client attached to print to. The collected causes are then reported when a client does attach, which is why a broken `.tmux.conf` greets you with its errors on your first attach rather than having failed silently in a daemon you never see. Loading continues past a failing line, so one bad option does not cost you your whole config — but every failure is reported, so it does not hide either.

Runtime errors go to the message ring and the status line, as above. The split is: errors that happen before there is anyone to tell are queued for the first person to appear; errors that happen while someone is watching are shown to them and kept in the ring.

Diagnosing a parity failure, concretely

Putting the pieces together, the standard workflow when a parity case (Chapter 11) fails runs in a fixed order, and the order matters because each step is cheaper than the next.

Start with the diff itself. A `.fmt` case's output is a single expanded format string, so the diff is usually short enough to read directly, and a surprising number of failures are diagnosable from the text alone — an off-by-one

width, an empty field where a value was expected, a boolean inverted. The failing expression names the format operator, and the format operator names the function in `format.rs`.

If the diff does not localise it, reduce the case. Parity cases are minimal by construction (Appendix D's determinism rules), but a failing case can usually be reduced further — strip format operators one at a time until the failure disappears, and the last one removed is the culprit. This is bisection over the input rather than over the code, and it is fast because each iteration is one command.

If the reduced case is still ambiguous, go to the C. The failing function's C counterpart is named by the port's own C citations (Chapter 12), which is the point of maintaining them: the citation takes you to the exact `vendor/tmux` source line the Rust claims to implement, and reading the two side by side is usually decisive. This is the step the whole name-preservation discipline of Chapter 29 exists to make possible.

And if the divergence is behavioural rather than textual — the output is right but something happened at the wrong time — the server log is the tool, because ordering bugs are invisible in output and obvious in an event trace.

Why the diagnostic surface is part of the product

It would be possible to build a port with no logging, no panic file, and no message ring, relying entirely on a debugger, and it would be a materially worse project. The parity suite tells you *that* something diverged; the diagnostic surface is what tells you *where*, and the ratio between those two costs determines how many gaps can be closed in a given amount of work. A suite of over a thousand cases (Chapter 11) is only tractable because a failure is cheap to localise. The logs are not developer conveniences bolted onto the side of the port — they are what makes the port's core method affordable.

Chapter 37 – Bug-Compatibility: What the Port Deliberately Inherits

The byte-for-byte bar has an uncomfortable implication that this book has so far stated only obliquely, and it deserves a direct treatment. If ztmux must produce the same bytes as tmux, then where tmux's behaviour is *wrong* — surprising, inconsistent, or the visible residue of an old implementation decision — ztmux must be wrong the same way. Faithfulness is not selectively applied to the parts one approves of. This chapter is about that commitment: why it is right, exactly what it does and does not cover, and where the superset's freedom actually lies.

The three categories, and only one of them is negotiable

Divergences between what tmux does and what a fresh implementation would do fall into three categories, and conflating them is what makes this topic confusing.

The first is **specified behaviour that happens to be surprising**. Chapter 6's pending-wrap corner is the archetype: writing to the last column of a line does not move the cursor to the next line immediately, it sets a pending-wrap flag that the *next* write consults. Users find this odd and it is not odd at all — it is what the VT specification requires, it is what every conforming terminal does, and programs depend on it. This is not a bug and there is nothing to consider deviating from.

The second is **implementation-derived behaviour that is observable and depended upon**. The layout rounding order of Chapter 11 belongs here: nothing specifies that a split's remainder column goes left rather than right, tmux's arithmetic puts it in a particular place, and that place is baked into every layout string every user has saved and every parity comparison anyone will ever run. It is arbitrary and it is load-bearing. Changing it would be defensible in a vacuum and catastrophic in practice, and the port reproduces it exactly. Most of the interesting cases live here.

The third is **behaviour nobody depends on and nobody can observe**. Internal ordering that produces no difference in output, allocation patterns with no timing consequence, dead code paths. These are genuinely free, and they are the only category where the port has discretion for reasons of code quality alone.

The operational rule follows directly: the port reproduces categories one and two without argument, and treats category three as license only after establishing that the behaviour really is unobservable — a bar that is higher than it sounds, since Chapter 23 established that allocation patterns and emitted byte counts are themselves observable if anyone chooses to look.

The temptation, and why it is a trap

The pressure to fix things during a port is constant and it comes from a good instinct. You are reading the C closely enough to see its rough edges; you know how you would do it better; the improvement is often small and obviously correct. Nothing feels more wasteful than transliterating something you can see is suboptimal.

The problem is that “obviously correct improvement” and “unnoticed behaviour change” are indistinguishable *from inside the change*. Every one of the three headline bugs of Chapter 11 would have looked, to the person who wrote it, like reasonable code — that was Chapter 29's argument for byte-for-byte, and it applies with

double force to deliberate deviations, because a deliberate deviation comes with a rationale that makes it *harder* to question later. A bug is found and fixed; an improvement is defended.

There is also a compounding effect. One improvement is a small divergence. Twenty improvements across twenty modules interact, and the port no longer has a single question to answer about any given behaviour — “does it match tmux?” — but twenty exceptions to remember. The value of the byte-for-byte bar is that it is *uniform*: there is exactly one rule, it applies everywhere, and any divergence is a bug by definition. A bar with exceptions is not a bar.

Where the freedom actually is: additive, not corrective

None of this makes the port a museum, and the superset story of Chapter 13 is not in tension with it — because the superset’s extensions are *additive*, and additive changes are exactly the ones that cannot break parity.

A new command that tmux does not have cannot change what an existing command outputs. A new format variable cannot change what an existing format string expands to. A new option defaults to off and, when off, changes nothing. Structured output for the `list-*` commands is reached through a flag that tmux does not define, so a `list-panes` invocation from an existing script takes the existing path and produces the existing bytes. The whole extension design is arranged so that every extension is invisible to a user who does not ask for it, which means every extension is invisible to the parity suite — which is the mechanical statement of “a faithful core that can grow.”

The distinction is between *adding a new answer* and *changing an existing answer*. The port may do the first freely and may not do the second at all. Chapter 13’s separation of `ported/` from `extensions/` is that rule expressed in the directory structure, so that a reviewer can see at a glance which side of the line a change is on, and Chapter 12’s anti-drift gate is what keeps the ported side from quietly acquiring changes that belong on the extension side.

Deprecated behaviour and the compatibility floor

A related question: what about behaviour tmux itself has deprecated — old command forms, superseded options, compatibility paths kept for configs written years ago?

The port keeps them, for the reason the whole compatibility argument rests on. The value proposition of Chapter 1 is that a user’s accumulated configuration keeps working, and accumulated configuration is precisely where deprecated forms survive. Someone’s `.tmux.conf` has a line they wrote long ago, that still works, that they have never revisited because there was no reason to. Dropping the deprecated path breaks that line, and the breakage is maximally annoying because the config worked yesterday under a program that claims compatibility.

`command-alias` deserves mention here as the mechanism that makes the compatibility surface *extensible by the user*: it is an array option mapping names to command lines, so a spelling the program does not implement can be supplied by configuration. That is the pressure valve — rather than the port carrying an unbounded set of historical spellings forever, the ones that are genuinely obsolete can be expressed as aliases, and the user who needs one has a one-line fix that does not require the program to change.

When tmux itself changes

The last question this raises is what happens when the reference moves. tmux is actively developed; a future release will change something, possibly something the port reproduces faithfully today.

The vendored-reference design of Chapter 2 is the answer, and it is worth seeing why it is the answer rather than merely a convenience. Because `vendor/tmux/` is a specific committed copy, “faithful to tmux” means faithful to *that* copy, and a change in upstream tmux is not an immediate parity failure — it is a decision to make

when the vendored copy is advanced. When it is advanced, the parity suite runs against the new reference and reports exactly which behaviours changed, since a case that passed against the old copy and fails against the new one has identified an upstream behaviour change with a minimal reproduction already attached. The version tracking of Chapter 1 — the crate version naming the tmux release targeted for compatibility — is the same idea expressed as a version string: the port states which tmux it is faithful to, rather than claiming faithfulness to a moving target.

That is the final shape of the commitment. The port is bug-compatible with a *named*, *committed* tmux; the suite measures that compatibility continuously; upstream changes arrive as an explicit, reviewed, one-time diff rather than as drift; and everything the project wants to build beyond tmux is built additively where it cannot disturb any of it.

Chapter 38 – The Trust Boundary

Every chapter so far has treated the server as a benign owner of state and the pane programs as things it renders. That framing is convenient and incomplete. A multiplexer sits at a junction of three parties who do not necessarily trust each other: the user at the keyboard, the programs running inside panes, and — if the socket is shared — other users on the machine. `tmux` has a considered position on each of those relationships, expressed not as a security subsystem but as a scattering of permission checks, gating options, and ownership rules across the tree. This chapter collects them into one picture, because a port that reproduces the mechanisms without understanding the boundary they draw will eventually “clean up” a check that was load-bearing.

The socket is the capability

There is no authentication in the `tmux` protocol. There is no password, no token, no handshake that establishes who you are; Chapter 17’s identify burst tells the server what your terminal is, not who you are. The entire access decision is made by the filesystem before a single byte of protocol is exchanged: if you can connect to the socket, you are in, and you can run any command the server implements — including `run-shell`, which executes arbitrary programs as the server’s user.

That is why `make_label` in `src/ported/tmux.rs` is a security-relevant function and not merely a path builder, and why Chapter 3 described its checks in that much detail. The socket directory is created `S_IRWXU` — owner-only — and then *verified*: it must be owned by the calling user and must carry no world-accessible bits. Both halves matter, and the second is the interesting one. Creating a directory with restrictive permissions proves nothing, because the directory may already exist; someone who can pre-create `<dir>/ztmux-<uid>` with permissive modes would otherwise get a socket they could connect to. Refusing to proceed when the directory is not owner-only, rather than fixing it up, is the correct response: the program cannot distinguish “leftover from an earlier misconfiguration” from “planted five seconds ago,” so it declines to use the path at all.

The consequence is worth stating plainly for operators. A `ztmux` socket is exactly as private as its containing directory, and putting a socket somewhere shared with `-S` is a deliberate act of delegation. The `ztmux-<uid>` prefix that Appendix L describes as the coexistence mechanism is also a separation mechanism: two users on the same machine get two directories, because the `uid` is in the path.

server-access: sharing a socket on purpose

Sharing a socket *is* a supported workflow — pair programming, a shared operations session, a teaching terminal — and it is what `src/ported/server_acl.rs` and the `server-access` command exist for. The model is an explicit list rather than filesystem permissions alone, because filesystem permissions are too coarse for the question being asked. Making the socket group-writable admits everyone in the group with full command authority; the ACL lets the owner name specific users and, separately, decide whether an admitted user may only observe or may also type.

The join happens at accept time. Chapter 3 noted that `server_accept` calls `server_acl_join` after creating the client, and that ordering is the mechanism: the client structure exists — so the server can report and log the attempt — but the ACL check runs before that client is allowed to become a full participant. A rejected peer is disconnected rather than silently ignored, which is the right behaviour for a collaboration tool: the person who could not join needs to know they could not join.

The read-only dimension deserves its own note, because “read-only client” is a claim that has to be enforced in more than one place to be true. A client that cannot send keys but *can* run commands is not read-only in any meaningful sense, since `send-keys` exists and does the same thing at one remove. This is a general property of the tmux command surface that the port inherits and that any future extension has to respect: the command language is the full authority surface, and any restriction that does not cover it is decoration. If the superset of Chapter 13 ever grows a finer-grained permission model, it will have to be expressed in terms of the command table, not in terms of key handling.

Pane output is untrusted input

The other direction of the boundary is subtler and affects far more users, because it applies even to a single-user, owner-only socket. The bytes arriving from a pane’s pty are *not* trusted content. They were produced by whatever program is running there, which includes `cat` of a file you downloaded, the output of a command on a remote host you reached over `ssh`, and the log line some service was persuaded to emit. Chapter 6’s parser consumes that stream, and every escape sequence it honours is an action a remote party can trigger.

tmux’s answer is a family of policy options that gate the sequences whose effects reach outside the grid, and Chapter 6 named them individually as they came up: `allow-rename` for the window-rename sequence, `allow-set-title` for OSC 0/2, `set-clipboard` for OSC 52, `allow-passthrough` for the DCS passthrough form. Read together they form a coherent policy, and the principle behind the grouping is legible: a sequence that only changes *what the pane looks like* needs no gate, because the pane is the program’s own canvas, while a sequence whose effect escapes the pane — renaming a window that appears in your status line, retitling your terminal, writing your system clipboard, reaching the outer terminal directly — is gated.

Passthrough is the extreme case and the reason `allow-passthrough` defaults off, for the reason Chapter 6 gave: the bytes bypass the grid entirely, so the multiplexer’s model and the terminal’s reality diverge. Note what that means from the trust angle rather than the correctness angle. Passthrough hands a pane program a direct channel to the outer terminal, which is to say it grants that program every capability the outer terminal implements, including ones the multiplexer would otherwise have gated. Enabling `allow-passthrough` on a pane where you run untrusted commands does not weaken one check; it makes the entire gating layer advisory.

The clipboard gate is one users change without thinking about it. `set-clipboard` on means any program that can write to your pane’s terminal can set your system clipboard, and the interesting risk is not reading the clipboard but writing it: whoever can put text on your clipboard is one paste away from your shell running it. tmux’s default is a security decision, exactly as Chapter 6 said, and the port keeps the default because changing a default is a behavioural change that would be invisible until it mattered.

Execution surfaces the server exposes

It is worth enumerating what a party with command access can cause the server to execute, because the list is longer than “run-shell.” `run-shell` and `if-shell` run shell commands through the job subsystem of Chapter 27. `display-popup` runs a command in an overlay. `new-window`, `split-window`, and the `respawn-*` family run programs through `spawn.rs`. `pipe-pane` runs a command fed by pane output. `source-file` reads and executes a configuration file, which can contain any of the above. `set-hook` arranges for commands to run later, on events the setter does not have to trigger. `command-alias` (Appendix C) redefines what a command name means. `lock-command` names a program run at lock time. `copy-command` and `editor` name programs those option groups invoke.

None of these are flaws — they are the point of a scriptable multiplexer — but the enumeration is what makes the earlier claim precise. There is no subset of the command language that is safe to expose to a party you would not give a shell to. This is also why the parity discipline extends to the gating options and not only to the visible behaviour: a port that implemented `allow-rename` as always-on would pass every format test in the suite and would have quietly widened the boundary.

Descriptor passing as privilege transfer

Chapter 17 described `imsg`'s ability to carry file descriptors alongside a message, and treated it as a mechanism for the terminal hand-off and for `load-buffer/save-buffer` running in the client's context. The trust reading of that mechanism is that a passed descriptor is a transferred capability: the server ends up holding an open file it did not open and could not necessarily have opened itself, because the client's process may have different privileges, a different working directory, or a different filesystem namespace.

This is deliberate and it is why file I/O works the way it does. When you run `load-buffer ./notes`, the path is resolved and opened by *your client*, in your context, and the descriptor travels to the server — so the file you can read is the file that gets loaded, and the server's own ambient authority is not the deciding factor. On a single-user socket this is merely convenient. On a shared socket it is the difference between “each participant can load their own files” and “every participant can read anything the server's user can read.” Porting the descriptor-passing path faithfully therefore preserves a property that is invisible in normal use and essential in the case the ACL exists to support.

Locking, and what it does not promise

`lock-server`, `lock-session`, and the `lock-after-time` idle timer run `lock-command` and mark clients locked. It is worth being precise about the guarantee, because the feature's name invites over-reading. Locking covers the *attached client's terminal*: the lock program takes over the display and the server refuses to service that client until it returns. It does not encrypt anything, it does not suspend the programs in your panes, and — crucially — it does not defend the socket. Anyone who can connect to the socket can attach a new client, and the sessions are right there.

That is not a defect to be fixed in the port; it is an accurate description of what a user-level lock over a Unix socket can be, and inheriting it unchanged is correct. The operational reading is the useful one: `lock-after-time` protects against the person walking past the desk, and the socket permissions protect against everyone else. Treating the first as if it were the second is the mistake, and it is a mistake a book about the program should say out loud rather than leave to the reader to discover.

Chapter 39 – Signals, and Why a Daemon Handles Them Twice

Signals are one of the more awkward parts of the Unix process interface, and a program that is simultaneously a daemon, a process supervisor, and an event-loop application has to deal with them in every one of those roles. Chapter 3 listed which signals the server honours and what each one does; Appendix L repeated the list as an operator’s reference. This chapter is about the *mechanism* — how signals get from the kernel into an event-loop program safely, why the fork sequence blocks them, what the child does with them between `fork` and `exec`, and why the whole arrangement is observable enough that the port had to reproduce it rather than modernise it.

The problem a signal handler cannot solve

A signal handler runs asynchronously, at some point between two instructions of whatever the process was doing. The set of things it is safe to do there is famously small: essentially, assign to a `volatile sig_atomic_t`, call a short list of async-signal-safe syscalls, and return. Everything the server would actually like to do in response to a signal — walk the client tree, queue a command, redraw, free memory — is forbidden, because the interrupted code may have been halfway through mutating the very structure the handler would touch, or halfway through an allocation.

The standard resolution is to make the handler do nothing but record the fact, and to do the real work later from the main loop, where the world is consistent. This is what the loop’s signal support provides and what the server uses: signals are registered as *events*, the same as a readable socket or an expiring timer, and the handler’s only job is to make the event loop notice.

The recording is worth seeing, because it is the whole of what runs in signal context. The handler sets one bit in an atomic word — one bit per signal number — and writes a single byte into a pipe the loop keeps open on itself. Both operations are async-signal-safe, which is the entire design constraint; nothing is allocated, no lock is taken, no data structure is walked. The read end of that pipe is watched like any other descriptor, so a signal arriving while the process sits in `select` or `poll` wakes it the same way a readable socket would. On the next turn the loop drains the pipe, swaps out the bit set, and activates whichever registered signal events those bits name. If the pipe were ever full, the wakeup byte is dropped and the bits are not: the loop still sees the signal on its next pass. `server_signal` is therefore not a classic signal handler doing the work inline; it is the deferred dispatch that runs from the loop, which is exactly why it can afford to reap children, tear down clients, and recreate a socket.

The consequence for a reader of the code is that “when does the server react to `SIGTERM`?” has a precise answer: at the next turn of the loop, in the same well-defined place that every other input is processed. There is no window in which a signal-triggered teardown races the command queue, because the two are serialised by the loop. Chapter 3’s five-stage walk through `server_loop` and this chapter’s signal deferral are the same design decision seen from two angles: everything that changes server state happens at a loop-defined point.

The fork window and the blocked signal set

`server_start` blocks all signals with `sigfillset` and `sigprocmask(SIG_BLOCK)` before forking, and unblocking happens on the far side in each process separately. It is worth spelling out what would go wrong

without that, because the sequence looks like ceremony until you enumerate the races.

Between `fork` and the point where the child has re-established its own handlers, the child holds the parent's handler *dispositions* but is a different process with a different job. A `SIGCHLD` arriving in that window would be delivered to a handler written for the client, in a process that has just become a server. A `SIGWINCH` from the terminal the parent was attached to would reach a process that no longer has that terminal's role. An impatient Ctrl-C could kill a half-initialised daemon that has already unlinked and re-created the socket. Blocking everything makes the window uninteresting: nothing is delivered until each side has decided what it wants delivered.

The child's side of that decision is the sequence Chapter 3 described — clear the inherited handlers, `event_reinit` the base, start its own `tmuxproc`, install `server_signal` — and the ordering inside it is not free either. `event_reinit` must happen before any event is armed, because a base carried across a fork holds state that belongs to the parent: the fork inherited both ends of the wakeup pipe and the parent's handler dispositions, so the child closes that pipe, opens its own, and re-arms a handler for every signal still registered. Using the inherited state instead is the kind of bug that manifests as one process's events firing in the other. That the port treats a failed `event_reinit` as fatal is the right call: there is no sensible degraded mode for an event-loop program whose event loop is in an unknown state.

SIGCHLD and the two things a child can be

The server forks children for two entirely different purposes, and `SIGCHLD` does not distinguish them. A pane's program (Chapter 19) is a long-lived child whose death means a pane died and a policy — `remain-on-exit` or destruction — has to be applied. A job (Chapter 27) is a short-lived helper whose death means a callback should fire, possibly resuming a suspended command-queue item. Both arrive as the same signal, so the dispatch has to identify which kind of child exited before it can decide what the exit *means*.

That is what `server_child_signal` does, and it is why the reaping path is written as a loop over exited pids that consults the pane and job state rather than as a handler per child. The subtlety a port must preserve is the *stopped* case: `server_child_stopped` exists because a child can stop rather than exit, and a stopped pane program is not a dead one. Collapsing the two — treating any `SIGCHLD` as a death — would produce a program that destroys a pane the moment its program is suspended, which is both wrong and the kind of wrong that only shows up when someone suspends a job inside a pane.

There is also an ordering property worth naming. Chapter 3's exit conjunction consults “no job still running,” and Chapter 27 noted the same. Reaping happens through the loop's deferred dispatch, so a job that died is not accounted as dead until its `SIGCHLD` has been processed. This is the correct direction to be wrong in: the server may briefly believe a finished job is still running and therefore decline to exit, and it will re-evaluate on the next pass. The opposite error — believing a running job is finished — would let the server exit out from under live work.

The signals the operator gets to use

Two of the wired signals are not internal plumbing but a deliberate operator interface, and they are worth understanding as design rather than trivia. `SIGUSR1` recreates the socket. That exists because the socket is a filesystem object with a lifetime independent of the server: a stale-socket cleanup script, a temp-directory reaper, or a mistaken deletion can remove the path out from under a running daemon, at which point the server is alive and holding all your sessions but is unreachable, since new clients have nothing to connect to. Without a recovery path the only option would be killing a server whose entire value is the state it holds. With it, the recovery is a signal:

```
# the socket file vanished; the server is still alive
kill -USR1 "$(pgrep -x ztmux | head -1)"
```

`SIGUSR2` toggles logging at runtime, which solves the complementary problem: the interesting failure is happening now, in a server that was started without verbosity, and restarting it to enable logging would destroy the state you are trying to diagnose. Chapter 36 covers what the logs then contain; the point here is the mechanism's shape. Both signals are "reconfigure a running daemon without restarting it," which is the classic reason Unix daemons claim the user signals at all, and both are cases where the faithful port also happens to be the right design.

What a pane's child does with signals

The last piece is the far side of the fork in `spawn.is`. A pane's child must not inherit the server's signal arrangements, and Chapter 19 listed restoring default signal handling among the between-fork-and-exec steps. The reason is that the program about to be `exec`'d will be a shell or an editor with its own expectations, and a signal disposition inherited from a multiplexer daemon is not one of them. Ignored signals in particular survive `exec` — dispositions set to ignore are inherited across it, unlike handlers, which are reset — so a child that did not restore defaults could `exec` a shell that silently ignores a signal it needs to act on.

The `setsid` call in the same sequence is the other half of the process-group story. It puts the child in a new session with the pty slave as controlling terminal, which is what makes job control work *inside* the pane: the shell running there can create process groups, put a job in the foreground, and have an interrupt reach it — because the pty, not the server's terminal, is the controlling terminal for that group. Without it, signals generated by the outer terminal's line discipline would go to the wrong place entirely, and the pane would be a display surface rather than a working terminal. That a single syscall in the fork sequence is the difference is a good illustration of why this part of the port is line-for-line rather than reimagined: each call in that window is there for a specific failure it prevents, and the failures it prevents are not visible in the code.

Chapter 40 – The Clock: Everything That Happens Without a Keystroke

An idle multiplexer is not doing nothing. It is waiting on a set of timers, each armed for its own reason, each with a period chosen for a specific human or machine constraint. This chapter collects the program's relationship with time in one place, because the timers are scattered across a dozen modules and are individually easy to dismiss as details — and because time is among the hardest things for the parity suite of Chapter 11 to test, which makes it the area where discipline has to substitute for measurement.

One “now” per loop iteration

Chapter 3 mentioned that `server_loop` stamps `CURRENT_TIME` with `gettimeofday` at the top of each pass, and gave the reason: every piece of work in the iteration should see one coherent notion of now. It is worth pressing on why that matters more than it appears to, because the alternative — each site reading the clock when it needs the time — sounds harmless.

Consider a single pass in which the status line is redrawn and an activity check runs. If each consults the clock independently, the timestamp rendered into `status-right` and the timestamp used to decide whether a window has been silent for its configured interval can straddle a second boundary. Nothing catastrophic follows, but the program has become one whose behaviour depends on where the second tick fell relative to its own internal ordering — which is precisely the class of nondeterminism that makes a bug irreproducible. Stamping once converts “when did this code run” into “which iteration was this,” and an iteration is a thing the program can reason about.

It also converts a stream of syscalls into one. That is a real if unglamorous saving in a loop that runs on every readable pane, and it is the sort of decision a port must not “improve” by inlining the clock read back into the call sites.

The timer catalogue

The periods the program uses fall into three bands, and the band a timer belongs to explains its value.

The *perceptual* band is where a human is waiting for something and the number encodes a judgement about human perception. `escape-time` is how long the input path waits after an `Escape` byte before concluding that it was a bare Escape key rather than the beginning of a function-key sequence — a number that must exceed the transit time of a multi-byte sequence and stay under the delay a vi user would notice. `repeat-time` is the window in which a `-r` binding repeats without the prefix, which encodes “how long after one press is another obviously part of the same gesture.” `prefix-timeout` bounds how long a pending prefix stays pending. `display-time` and `display-panes-time` govern how long a message and the pane-index overlay remain visible. Every one of these is a session or server option (Appendix C), because the right value is a property of the human, not of the program.

The *refresh* band is where the program decides how often to recompute something whose inputs it does not watch. `status-interval` is the archetype: the status line can contain `#{uptime}` and a clock, neither of which generates an event when it changes, so the only way to keep them current is to redraw periodically. Chapter 27 described how format jobs are tied to this interval rather than to redraw frequency, which is the

essential optimisation — the bar may repaint many times between refreshes without re-running anything. The interval behind `monitor-silence` is the same band read inversely: it is a period after which the *absence* of events is itself the event.

The *hygiene* band is where the program cleans up after itself on a schedule long enough that nobody notices the work. The hourly tidy timer of Chapters 3 and 23 is the member worth naming, at 3600 seconds, expiring stale format jobs and returning freed memory to the OS where the platform supports it. The period is chosen to be irrelevant: an hour is far longer than any interaction and far shorter than the weeks a server may live.

The flow-control resume timer of Chapter 23 sits outside all three bands, at 100 ms, because it is not about humans or freshness or hygiene — it is a machine-to-machine backoff, and its value is a throughput trade, not a perceptual one.

Why the server is idle when it is idle

A reasonable question about a program with this many timers is whether it wakes up constantly. It does not, and the reason is structural rather than clever. The loop's dispatch computes the earliest pending timeout and sleeps in the underlying multiplexing syscall until then or until a descriptor becomes ready. The program does not poll; it declares what it is waiting for and blocks. A server with no attached clients and no status line to refresh is genuinely asleep between its hourly tidy passes.

This is a central reason Chapter 29's objection about substituting an async runtime is answered the way it is. The idle behaviour of the program — how often a detached server wakes on a laptop running on battery — is a consequence of exactly which events are armed and what timeouts they carry. Reimplementing the dispatch would make that behaviour a new thing to get right rather than a thing inherited by construction, and it is behaviour nobody would write a test for.

There is an operational corollary. If a ztmux server is observed consuming CPU while apparently idle, the diagnosis is a timer or an event, not a busy loop: something is armed with a short period, or a descriptor is reporting readable and the read is not consuming it. Chapter 36's logging is the tool, and the question to ask it is "which event keeps firing," not "where is the loop spinning."

Time and the parity suite

Chapter 11's determinism rules exclude time for the obvious reason: a case that expands a clock format cannot be compared byte-for-byte between two runs, let alone between two programs. Chapter 27 described the corresponding mechanism for jobs — the expansion flag that leaves `#(...)` inert — and the same problem recurs everywhere the clock appears in output.

What that means honestly is that the time-dependent surface is the least mechanically verified part of the port. The suite can confirm that the *format operators* around a timestamp behave correctly by feeding them fixed values; it cannot confirm that `status-interval` actually elapses at the configured rate, that `escape-time` distinguishes the two Escape cases at the boundary, or that `repeat-time` closes its window when it should. Those are verified the way such things always are: by reading the ported code against the C, and by the fact that the values are options whose defaults come from the same table (Chapter 33) as everything else.

Naming that gap is more useful than papering over it. The general shape of a timing test that would help is one that measures an *interval* rather than an instant — arm a condition, wait, assert the state changed — and such cases are possible in the `.sh` flavour, at the cost of being slow and, if the margins are tight, flaky. The project's position, consistent with the rest of Chapter 15's honesty, is that the ported timer values and the ported arming logic are the evidence, and that a timing case is worth writing when a specific timing bug motivates it rather than as blanket coverage. A flaky case in a suite whose value is its trustworthiness costs more than it proves.

Chapter 41 – Nesting: A Multiplexer Inside a Multiplexer

Running a multiplexer inside a multiplexer is either a mistake or a workflow, depending on who is doing it. It is a mistake when it happens by accident — you connect to a host and reflexively attach to a session, forgetting you are already inside one — and a workflow when it is deliberate: an outer multiplexer on your laptop holding one session per remote host, each of those being an inner multiplexer's client on the far side. Because `ztmux` is the program and not a wrapper, both cases work exactly as they do under `tmux`, and both are worth walking through, because nesting is the situation in which every layered abstraction in this book gets exercised twice at once.

Why it works at all

The reason nesting is not a special case is that the inner server has no idea it is nested. Chapter 19 established that a pane runs an arbitrary program in a `pty`; Chapter 4 established that a client is a program that opens a socket and drives a terminal. An inner client attached inside an outer pane is simply the first thing being run as the second: a program in a `pty`, which happens to be a terminal-driving client of another server. The outer server sees a pane whose program writes escape sequences; the inner client sees a terminal whose size it can query and whose input it receives. Neither has a concept it needs to extend.

That is a strong statement about the architecture and it is worth noticing why it is true. The client/server split of Chapter 3 means the *terminal* the inner client drives is not required to be a hardware terminal or an emulator — it only has to be a `pty` with a `terminfo` entry that describes it. The outer server provides exactly that: a `pty` per pane, and a `TERM` value it advertises via `default-terminal`. Nesting works because the multiplexer's output side implements the same contract its input side requires, which is the compositional property that makes a program a real terminal rather than an approximation of one.

The immediate practical consequence is that the inner session's capabilities are bounded by what the *outer* multiplexer models, not by the outer terminal. If the outer server advertises a `TERM` without a capability, the inner client will not use it, even if the real terminal three layers out supports it. Chapter 32's two-`TERM` discussion is the general form of this; nesting simply makes it a three-`TERM` problem, and the middle value is the binding constraint.

The prefix problem and `send-prefix`

The visible difficulty of nesting is input routing. Both multiplexers watch for the same prefix key by default, and the outer one sees the keystroke first, so an unqualified prefix always addresses the outer session. The inner one would be unreachable if there were no way to say “this keystroke is not for you.”

Two answers are conventional, and they differ in kind. The first is `send-prefix`, which is a command the outer server runs that transmits the prefix key *into* the pane's program rather than acting on it. Bound in the outer configuration to the prefix key pressed twice, it produces the familiar gesture: an extra prefix press per level of nesting. The mechanism is worth appreciating because it is entirely built out of parts this book already described — a key binding (Chapter 9) running a command (Chapter 8) that writes bytes to a pane (Chapter 19's `pty`) via the same key-encoding path (`input_keys.rs`) that any `send-keys` uses. Nesting support required no nesting-specific code.

The second answer is to give the inner multiplexer a different prefix. This is the better choice for a deliberate nesting workflow, because it makes the two layers addressable without a modal gesture and without the count of key presses scaling with depth. It costs muscle memory, which is why it is the deliberate-workflow answer and `send-prefix` is the accidental-nesting answer.

A third technique deserves mention because it appears in many configurations: binding a key in the outer configuration that toggles the outer prefix off and on, so the outer layer becomes temporarily transparent. It is a legitimate use of the options system — a key binding that sets an option — but the failure mode is real. If the toggle's own key is itself prefix-qualified, turning the outer prefix off makes the toggle unreachable, and recovering means running a command against the outer server from elsewhere. The safe shape is a binding in the root table (Chapter 9's key tables), which is reachable without a prefix.

Knowing which server you are talking to

A pane's environment carries a variable identifying the server and session the pane belongs to — Chapter 19 noted that `spawn_pane` sets it so a program inside a pane can tell it is under the multiplexer. Nesting turns that variable into the mechanism by which the accident is prevented: a client that finds it set knows it is already inside a session, and the program refuses to attach a session to itself rather than creating an infinite regress. Overriding that refusal is possible and is what a deliberate nester does, at which point the inner attach proceeds and the variable is replaced in the inner pane's environment by the inner server's own value.

The corollary matters for scripts. A script that runs inside a pane and wants to talk to “the multiplexer” must decide *which* one, and the environment variable it inherits names the innermost. A script that wants the outer server needs the outer socket path explicitly, via `-5`. This is not a nesting quirk so much as the socket-is-the-address model of Chapter 38 asserting itself: there is no ambient “the server,” only the socket you name, with the environment providing a default.

What costs double

Nesting is not free, and it is useful to be precise about which costs multiply and which do not.

Parsing multiplies. Every byte a program prints inside an inner pane is parsed by the inner server's input parser into the inner grid, rendered by the inner server's tty layer into a new byte stream, parsed by the *outer* server's input parser into the outer grid, and rendered again by the outer tty layer to the real terminal. Chapter 23's fast paths therefore run twice per byte, and the differential-redraw optimisation applies twice — with the useful property that the second application operates on the output of the first, which is already minimised. Two multiplexers do not produce quadratic output; they produce minimised output of minimised output.

Scrollback multiplies in the worst way. The inner server holds the pane's history, and the outer server holds the *rendered view* of the inner session's screen, including whatever scrolled past in the outer pane. Two copies of history exist, with different contents, because the outer copy captured the composited frame — status line and all. Chapter 5's memory arithmetic applies to both, and the outer copy is the less useful one, since scrolling back in the outer pane shows fragments of previous inner frames rather than coherent output. This is the concrete reason experienced users scroll back in the *inner* session and treat the outer pane's history as noise, and the reason a nested setup is a place to consider a smaller `history-limit` on the outer session specifically.

Latency adds rather than multiplies, which is the good news: a keystroke traverses two socket hops and two command paths, and a redraw traverses two render paths, but neither is a loop. Flow control (Chapter 23) is the one place where the layering has a subtle effect: a flood in an inner pane is throttled by the inner server's block threshold, and the already-throttled stream then meets the outer server's threshold. The outer limit is computed from the outer pane's dimensions, so the effective throttling is set by whichever layer is tighter.

Two status lines, and the honest advice

A common complaint about nesting is cosmetic: two status lines, one of which is a row of the other's pane area. The usual configurations are to turn the inner status off, to distinguish the two by colour so a glance identifies which layer a keystroke will hit, or to make the outer one a single-purpose host indicator rather than a full session display. All three are ordinary option settings (Appendix C's `status`, the style options), applied in the inner or outer configuration as appropriate — which is the recurring theme of this chapter. Nesting requires no features. It requires understanding that every mechanism in this book is now present twice, and deciding, per mechanism, which copy should be doing the work.

Chapter 42 – Control Mode as an API Surface

Chapter 17 introduced control mode as the second face of the protocol and described what it is: a line-oriented text stream in which a client sends commands and receives %-prefixed notifications. That was the protocol chapter's view — control mode as an alternative encoding of the same event flow. This chapter takes the client's view instead, because control mode is the seam the superset of Chapter 13 is built on, and building against it correctly requires understanding a set of properties that are not obvious from the message list.

Why text, when the server already has a binary protocol

The first question a systems engineer asks is why a second protocol exists at all. The server speaks `imsg` (Chapter 17); a programmatic client could speak `imsg` too, and would get length-framed messages and descriptor passing for free.

The answer is the audience. `imsg` framing is a C library's convention, with structure layout, descriptor passing, and a version handshake in the mix; implementing it correctly from another language is a real project, and getting it *wrong* — off-by-one framing, a mismatched struct layout — fails in ways that are hard to diagnose from the far side. Control mode is deliberately the opposite: a stream of lines that any language can produce and consume with its standard library, debuggable by eye and reproducible by hand. The trade is that structure has to be re-imposed by the client, and that everything that cannot be expressed as text — notably descriptor passing — is simply unavailable.

That trade is right for the use case. A GUI front end embedding a multiplexer does not need to pass descriptors; it needs to know what the server's state is and to be told when it changes. Chapter 21's notification vocabulary is the substance of control mode, and the text encoding is just its transport.

The two streams a control client multiplexes

The property that catches most first implementations is that a control connection carries two logically independent streams over one channel: replies to commands the client sent, and notifications the server generated on its own. They interleave. A notification can arrive between a command being sent and its reply appearing, because the server has no reason to withhold an event just because a command is in flight.

A client that reads the stream as “send a command, read the response” will therefore eventually parse a notification as a command reply and desynchronise. The correct structure is a reader that classifies each line first — notification, or content belonging to a reply block — and routes accordingly, with commands tracked by their position in the reply sequence rather than by adjacency to the write that triggered them. This is the same discipline any asynchronous protocol demands, and it is the property most likely to be missed in a control client, because the failure mode is not a clean error: it is a client whose model drifts from the server's over hours.

The block structure of replies is what makes the classification possible. A command's output is delimited, so a client can accumulate the lines between the delimiters as that command's result and treat anything outside them as an event. Multi-line output — the whole point of `list-panes -F` against a large session — arrives as content inside one block rather than as a stream the client has to guess the end of.

%output and the volume problem

The notification that dominates traffic is `%output`, which carries bytes a pane produced. Every other notification is an occasional state change; this one is proportional to everything happening in every pane.

Three consequences follow for a client. First, the encoding matters: pane output is arbitrary bytes including newlines and control characters, and a line-oriented protocol must escape them, so a client that forgets to unescape will render literal escapes and a client that unescapes twice will corrupt binary output. Second, the client is now in the terminal-emulation business: it is receiving the bytes a pane produced, which means it has to parse them the way Chapter 6's parser does in order to know what the pane looks like. A control client is a terminal emulator that has outsourced session management, not a program that receives rendered text. Third, back-pressure is the client's problem. A pane running a runaway loop produces `%output` faster than a slow client can consume it, and Chapter 23's flow control governs the server's output to a *terminal* client. A control client that does not read fast enough is a control client whose socket buffer fills.

The design implication for the integrations of Chapter 13 is that a control client's consumption loop must be decoupled from its rendering loop. Read the stream continuously into a buffer; render from the buffer at whatever rate the UI can sustain; drop redundant intermediate states rather than falling behind on the read. This is standard practice for any UI over a firehose, and control mode is a firehose.

Mirroring state instead of polling it

The reason to accept all this complexity is the alternative. A programmatic client that does not use control mode has to poll — run a `list-*` command on a timer and diff the result — which is wrong in both directions: too slow to feel responsive, and too frequent to be cheap, since each poll is a command through the queue and a set of format expansions (Chapter 16) over the whole state tree.

The notification model inverts it. The client builds its model once, from an initial set of `list-*` commands, and thereafter *applies* events: a window was added, a window was closed, the layout changed, the session changed. The cost is proportional to change rather than to state size, which is the property that makes a front end over a large session feel instantaneous.

The discipline this requires is that the client must be able to *resynchronise*. Any event-sourced model can drift — a missed event, a state change the vocabulary does not cover, a bug in the client's own application logic — and a client with no way back to ground truth will show a stale window list forever. The correct design keeps the initial enumeration path available as a recovery operation, triggered on any inconsistency the client can detect. The enumeration commands are ordinary commands (Appendix A), so this costs nothing structurally; what it costs is the discipline to treat the model as a cache rather than as the truth.

Sizing, and who decides what the client is

A terminal-driving client tells the server its size, and the server sizes windows according to `window-size` (Chapter 4, Chapter 31). A control client has no terminal, and yet the sessions it drives must have a size, because a pane's program needs one to run.

`refresh-client` (Appendix A) is the command that lets a client state its dimensions explicitly, which is what makes a GUI front end able to own its own geometry: it renders at whatever size its window is, tells the server that size, and the server resizes the session accordingly. This is the same machinery Chapter 31 described, entered from a different door. A control client that never states a size gets the sizing policy's answer for a client whose size the server does not know, which is why a front end that has not wired this up sees panes at a geometry that does not match its window.

What porting this faithfully buys

Chapter 17 argued that porting the protocol byte-exactly keeps ztmux a drop-in target for anything that already speaks tmux control mode, and this chapter's detail is what that claim means concretely. An existing control client encodes assumptions about the block structure, the escaping of `%output`, the interleaving, the notification vocabulary, and the sizing command. Every one of those is a place where a rewrite that "improved" the protocol would break a client that has been working for years, and none of them would be caught by a test that only checked that the messages had the right names.

It is also the reason the notification vocabulary of Chapter 21 is worth porting in full rather than in the subset any one client happens to use. A control client's model is only as complete as the events it can receive; an event the server does not send is a state change the client can only find by polling, which is the thing control mode exists to avoid. Completeness of the vocabulary is the feature.

Chapter 43 – What the Server Forgets

The server holds everything. That has been the premise since Chapter 3 — sessions, windows, panes, grids, scrollback, layouts, options, key tables, paste buffers, environments — and it is what makes detach-survival work. The complement of that premise is less often stated: when the server exits, all of it is gone, and almost nothing is written down. This chapter is about that boundary — what persists, what does not, why the program does not try harder, and how the tooling that does try is built out of the command surface rather than into the server.

The persistence inventory

The honest inventory is short. The command-prompt history is saved, through `status_prompt_save_history` on the way out (Chapter 3) and reloaded on start, with `history-file` naming the path and `prompt-history-limit` bounding its depth (Appendix C). The socket itself persists as a filesystem object, which is a lifetime, not state. That is essentially the list.

Everything else lives only in memory. The grids and their scrollback, the layout geometry, which window is current and the window stack behind it (Chapter 30), the paste buffers (Chapter 28), the session environment, the options set at runtime rather than in a config, the key bindings added interactively, the hooks, the marked pane, the client list — all of it is a data structure in a process, and a process exit takes it.

The programs are the part that hurts. A pane is a child process with the `pty` slave as its controlling terminal; when the server exits, those children lose their terminal, and the usual consequence is that they are hung up and die. An editor's unsaved buffer, a half-finished build, a shell with hours of history not yet flushed — those are process state, and no multiplexer-level persistence scheme reaches them.

Why the program does not try harder

It would be technically possible to serialise more. The grid is a data structure; the layout already has a serialisation (below); options and environments are string maps. So the absence of a save/restore feature is a decision, and it is worth being explicit about why it is the right one rather than treating it as a gap.

The reason is that the valuable state is precisely the state that cannot be serialised. A session's worth is not the rectangle geometry — that is reconstructible in seconds — it is the *running programs*: the remote connection with an authenticated session, the REPL with a loaded dataset, the editor with unsaved work, the build at minute forty. None of those can be written to a file by the multiplexer, because none of them are the multiplexer's state. A feature that restored the geometry and the pane titles while every program had to be restarted by hand would deliver the cheap half of the value and imply the expensive half, which is worse than delivering nothing, because users would rely on it.

There is a second reason, which is the one that binds the port. Chapter 37's argument applies: a persistence format is a compatibility surface, and one the reference does not have. Inventing it inside the faithful core would create state the C `tmux` cannot read and a file whose format the project would owe compatibility to forever. If it belongs anywhere it belongs in the extensions tree of Chapter 13, built on the command surface, where it can evolve without touching parity.

The one thing that is serialisable

The exception is instructive. Chapter 7 described the layout string — `layout_custom.rs`'s dump, parse, and checksum — and it is the one piece of server state with a first-class textual representation the program can round-trip. You can read a layout out of a running session, store it, and apply it later:

```
ztmux display-message -p '#{window_layout}'
ztmux select-layout '<the string you saved>'
```

Why does this one exist when nothing else does? Because layout is *configuration-shaped* rather than *session-shaped*. A layout is a description of an arrangement, independent of which programs are running in it, and users want to name arrangements and reuse them across sessions. The checksum in the string exists precisely because the string is expected to travel through configs and scripts and be re-applied to a different window, where a corrupted or hand-mangled value must be rejected rather than half-applied.

That distinction is the useful design principle. State that describes an *arrangement* serialises well and belongs in configuration. State that describes a *moment* — cursor positions, scrollbar, which pane is active — does not, because the moment is only meaningful alongside the processes that produced it.

Reconstruction from the outside

The tooling that does restore sessions works from the command surface, and understanding how is worth doing even if you never write it, because it is a clear demonstration of why the format language of Chapter 16 is an interface and not a cosmetic feature.

Everything an external tool needs to describe a session is exposed as a format variable (Appendix B) and dumpable through the `list-*` commands with `-F`. A saver enumerates sessions, windows, and panes, capturing for each the identifying fields, the geometry, the window layout string, the pane's working directory, and the command it is running. Restoration is the inverse: `new-session`, `new-window`, and `split-window` with the recorded working directories, `select-layout` with the recorded layout string, and then whatever policy the tool has for re-running commands — which is the part where the honest tools stop and ask, because re-running an arbitrary recorded command line is a decision a program should not make silently.

Two properties of the port make this work better than it would against an ad-hoc interface. The first is that the format surface is *complete enough* to describe what it describes: the layout string round-trips exactly, so the geometry restoration is not an approximation. The second is Chapter 13's structured output for the `list-*` commands, which is exactly this consumer's problem solved at the source — a tool that parses `list-panes -F` output is writing a parser for a format string it also authored, and the structured form removes that step without changing any parity-relevant behaviour.

Restart, upgrade, and the running-server problem

The practical face of all this is upgrading. A running server is an old binary holding state that only it can interpret; a new binary on disk changes nothing until a server starts. So the upgrade path is the same as the restart path, and the same as the “everything is lost” path.

The mitigations are unglamorous and worth stating because they are what experienced users actually do. Keep the state that matters in files rather than in scrollbar — the `capture-pane` command exists partly for this, and `pipe-pane` (Chapter 27) turns a pane's output into a file continuously for the cases where you know in advance you will want it. Keep configuration in a config file rather than in interactively-set options, so a fresh server comes up identical. Expect long-lived work to be resumable at the process level: detachable jobs, checkpointed builds, editors that persist their state.

And note what the architecture does give you, which is that the restart is *voluntary*. A detached server survives the closing of every window and the ending of every remote session (Chapter 3), and the only things that end

it are a deliberate `kill-server`, the exit policy options, a crash, or a reboot. The failure mode this chapter describes is real and the program does not pretend otherwise — but it is a rare event by design, which is why mitigations aimed at the rare event are the proportionate response, and a persistence subsystem in the faithful core is not.

Chapter 44 – Measuring the Port Without Lying About It

This book contains no performance numbers. Chapter 23 described the hot paths and the optimisations in them and made an argument about *why* the port is fast — it inherits tmux’s algorithms line for line — without ever claiming a figure. That is a deliberate policy and it deserves a chapter, because “we did not measure it that way” is a more useful thing to tell a reader than a number they cannot reproduce, and because a reader who wants to measure it should be told how to do it without fooling themselves.

Why there are no numbers

A benchmark number is a claim about a specific binary, built a specific way, on a specific machine, driven by a specific workload, compared against a specific alternative. Published without all five, it is decoration. Published in a book, it goes stale the first time anyone touches either program, and it cannot be un-published.

The project’s position, consistent with Chapter 15’s ledger and Chapter 11’s insistence that correctness be operationally defined, is that a claim it cannot verify continuously is a claim it does not make. The parity suite runs in CI (Chapter 11) and reports 1115 of 1115; there is no equivalent standing measurement for performance, so there is no equivalent standing claim. If such a harness existed, its output would be the thing to cite, not a number in prose.

There is also a subtler reason. The performance argument this port actually makes is *structural*: it does the same work in the same order as the reference. That claim is verified by reading the ported code against the C — the same activity Chapter 2 describes for correctness — and it is a stronger claim than a benchmark, because it holds across workloads a benchmark would not have covered. A number would be weaker evidence for the thing being claimed.

What is worth measuring

If you do want to measure, the useful quantities are not the obvious one. “How fast is it” is not answerable; these are.

Byte latency is the time from a program in a pane writing a byte to that byte’s effect appearing on the real terminal. It traverses the pty read, the input parser, the screen write, the redraw decision, the tty encode, and the socket to the client. It is what makes a session feel immediate and it is the quantity nesting (Chapter 41) adds to.

Redraw amplification is the ratio of bytes the server emits to the client against bytes the pane produced. This is the quantity Chapter 23’s differential redraw exists to keep low, and it is the one where a multiplexer can be pathological: a naive implementation repainting a screen per input byte would have an enormous amplification. It is also a portable measurement, because it is a count rather than a duration — the tty output log Appendix L describes captures exactly the numerator.

Throughput under flood is how a large dump behaves: does the terminal stay responsive, does memory stay bounded, does the flow control of Chapter 23 engage and release. The question here is not speed but *bounded degradation*, and the pass criterion is behavioural rather than numeric.

Memory per unit of scrollbar is the quantity Chapter 5's worked example addresses analytically. Measuring it means filling a pane's history to a known line count and observing the process's resident size, which is meaningful only if you control the packed versus extended cell distribution — a screen of plain ASCII and a screen of styled double-width characters do not cost the same, and that is the interesting fact rather than a confounder to eliminate.

Startup splits into two very different numbers that are often conflated: the time for a client to attach to a running server, which is the one you feel dozens of times a day, and the time for the first client to autostart a server and load a config, which happens once per boot and is dominated by whatever the config does.

Constructing a comparison that means something

An A/B against C tmux is the comparison people want, and it is the easiest one to get wrong. Several things have to be equalised.

Build configuration first: a debug build of a Rust program is not a slower version of the release build, it is a different program, and comparing one against an optimised C build is meaningless. Appendix D notes that the parity harness can test a debug build; performance work cannot.

Environment second, and this is where most naive comparisons fail. Both programs must run with the same `TERM`, the same terminal emulator, the same locale, and the same options that affect work done per byte. Chapter 11's determinism rules — fixed geometry, `LC_ALL=C` — exist for the parity suite but are exactly the right controls here too. A comparison run in two different terminal emulators is measuring the emulators.

Configuration third: an empty config on both sides, or the identical config on both sides, and awareness that a status line containing `#(...)` jobs (Chapter 27) makes the measurement partly a measurement of those shell commands.

Then the confounders that are specific to this port. The readiness backend must be the same on both sides, since the dispatch mechanism is a real variable — and note that this is no longer an apples-to-apples comparison of one library against itself: tmux dispatches on libevent, ztmux on its own loop (Chapter 3), both landing on the same syscall. The tty output log must be off, since logging every emitted byte is not free. And the allocator: Appendix F notes the libc-shim global allocator that `main.rs` installs, which exists so the ported code's allocation behaviour matches the C's — it is a fidelity decision with performance consequences, and it is a legitimate thing to measure, but only if you know it is there.

Reporting honestly

The rules that follow are the ones this project applies to itself, and they are why the chapter exists rather than a table.

State the workload, not just the result — “measured with X” is the load-bearing half. Report the distribution rather than the best run, because a multiplexer's worst case is what a user notices. Separate the quantity you measured from the conclusion you drew; “emitted fewer bytes for the same input” is a measurement, “is faster” is an inference. Never compare a measurement of one thing against a memory of another. And when a measurement contradicts the structural argument — when the ported path is slower than the C it was ported from — that is a finding about a specific divergence, and the response is to find the divergence, not to publish the aggregate. A port that does the same work in the same order and is slower is doing something different somewhere, and locating it is more valuable than any number either side of it.

Chapter 45 – Exhaustion: Where the Program Draws Its Bounds

A daemon that lives for weeks, holds arbitrary amounts of program output, forks children on demand, and accepts connections from anything that can reach its socket has a great many ways to run out of something. Chapter 23 covered flow control as a performance mechanism and Chapter 3 covered the accept path's descriptor backoff as a lifecycle detail. This chapter takes the systematic view: every resource the program can exhaust, the bound that protects it, and whether that bound is a constant, an option, or an absence.

The bounds are almost all named

The first observation is that the bounds which matter have user-visible names. `history-limit` bounds scrollbar per pane, at a default of 2000 lines. `buffer-limit` bounds the number of paste buffers. `message-limit` bounds the server message ring `show-messages` reads. `prompt-history-limit` bounds the saved prompt history. The listen backlog is 128. The flow-control threshold is computed from the pane's dimensions.

That naming is a design property worth calling out. An operator asking “how much memory can this server use” has an answer expressible in the program's own vocabulary: panes times `history-limit` times per-line cost (Chapter 5's arithmetic), plus buffers times their sizes, plus the fixed structures. There is no hidden cache growing without a name. The practical consequence is that memory tuning is a configuration exercise rather than a source-reading exercise, which is what you want from a program that runs unattended.

Descriptors: three different exhaustions

File descriptors are a resource this program can run out of, and it does so in three distinguishable ways.

Accepting clients is the one Chapter 3 described: `server_accept` degrades on `ENFILE` and `EMFILE` by disabling accept for a second rather than spinning. The design point is that a descriptor shortage is usually transient, so the correct response is to stop making it worse and retry, not to fail.

Spawning panes is different, because a pane needs a pty pair and the failure is not transient in the same way — the user asked for a new pane and either gets one or gets an error. There is no backoff that helps; the command fails with a cause (Chapter 35's out-parameter idiom) and the user sees a message. That asymmetry is right: an accept failure has no one to report to, so retrying is the only useful behaviour, while a command failure has a user attached to it.

Jobs are the third, and they are the one that can be provoked by configuration rather than by the user's actions. Each format job (Chapter 27) holds a descriptor, and a status line with many distinct `#(...)` commands, refreshed on `status-interval`, is a steady demand. Chapter 27's caching keyed by command string is the mitigation — identical commands share one job — and the tidy timer's expiry of stale jobs is the reclamation.

Behind all three sits `RLIMIT_NOFILE`, which the server raises. Chapter 19 told the story of how that interacted disastrously with a naive `closefrom` in the fork path, and it is worth re-reading in this context because it is a genuine instance of the theme: a bound raised for one subsystem's benefit became a cost in another that iterated over it. Bounds are not free even when they are generous.

Memory: the honest position

The program's memory profile is dominated by grids, and the grid is bounded per pane by `history-limit`. What is *not* bounded is the number of panes, so the aggregate is bounded only by the user's own behaviour — a session with hundreds of panes each holding a full history is a large process, and that is a consequence of the user's configuration rather than a leak.

Two mechanisms make the aggregate better behaved than the arithmetic suggests. The packed cell representation of Chapter 5 means the common case — plain characters in default styling — costs the minimum, with the extended representation paid only by cells that need it. And the tidy timer's memory return (Chapters 3, 23) means a server that held a huge history and released it actually gives the memory back on platforms that support it, rather than holding the high-water mark for the rest of its life.

The failure mode when memory does run out is the one Chapter 35 named as an idiom: the `xmalloc` family treats allocation failure as fatal. This is inherited rather than chosen, and it is worth defending rather than apologising for. The alternative — propagating allocation failure through every call site in a program of this size — would mean an error path in every function that grows a grid, and error paths that are never exercised are error paths that do not work. Dying cleanly at the point of failure is more honest than unwinding through code that has never been tested unwinding. What a Rust port adds is that the death is diagnosable: Chapter 3's panic hook writes the backtrace to a file, so a daemon with no terminal leaves evidence rather than vanishing.

Output buffers and the slow client

A client that reads slowly is a form of exhaustion too, and it is the one with the most interesting dynamics because it couples two rates that have nothing to do with each other: how fast a pane's program writes, and how fast a terminal on the far side of possibly a network consumes.

Chapter 23's flow control is the answer on the pane side — the tty's buffered output crossing its threshold blocks further production and arms the resume timer. The property worth noting here is *who* gets throttled. The block is on the tty, which is per-client, so a slow client throttles the output destined for it. Two clients on the same session at different speeds (Chapter 4's multi-client story) do not force the fast one to the slow one's rate for arbitrarily long, because each has its own buffer and its own threshold.

The control-mode client of Chapter 42 is the case where this protection is thinnest, since `%output` volume is proportional to pane activity and the flow control that governs a terminal client's rendering does not translate directly. That is why that chapter's advice is to decouple reading from rendering: the client must consume, and if it cannot render at that rate it must discard intermediate states rather than stop reading.

What the exit conjunction protects

The last bound is not a resource at all but a policy, and it belongs here because it is what prevents the costliest failure in this catalogue: losing work. Chapter 3's exit conjunction — the server exits only if policy permits, and no command waits, and no job runs, and no config load is in progress — is a bound on the server's willingness to discard state.

The reason it is a conjunction of several conditions rather than one is that each conjunct guards a different way of being wrong. Dropping the policy check exits a server the user wanted kept. Dropping the command check exits while a queued command list is halfway through. Dropping the job check exits out from under an `if-shell` that has not decided yet. Dropping the config check exits during startup. All of them are rare; all of them are silent; all of them destroy state that cannot be recovered by Chapter 43's standards. This is why that conjunction is one of the specific things the port reproduces in order rather than rewriting as an equivalent-looking boolean expression, and why a `.sh` parity case pins it.

The catalogue as a design statement

Reading the bounds together, the pattern is that the program prefers *degrade* to *fail* wherever the failure has no addressee, prefers *fail with a cause* wherever a user is attached to the request, prefers *named option* to *hardcoded constant* wherever the right value is a matter of taste, and prefers *fatal* to *unexercised error path* where recovery would be a fiction. Those four preferences are not stated anywhere in the source as a policy; they are visible only by collecting the cases, which is what this chapter has done. They are also a good summary of what “faithful port” buys past correctness: the port inherits a set of decisions about failure that were made across decades of a program running unattended on other people's machines, and it inherits them intact because it did not re-derive them.

Chapter 46 – Decoding the Keyboard: the Other Parser

Chapter 6 described a parser: bytes arrive from a pane’s program, a state machine interprets them, and the grid changes. That parser has a twin running in the opposite direction, in the client rather than the server, and this book has named it repeatedly without ever opening it. `src/ported/tty_keys.rs` takes the bytes the user’s terminal sends — the raw output of a keyboard, a mouse, and whatever else the terminal emulator decides to report — and turns them into key codes. The glossary’s definition is the whole contract: a key code is “the internal representation of a key plus its modifiers and flags,” and `tty_keys.rs` is where terminal bytes become one.

The two parsers look symmetric and are not, and the asymmetry is the interesting part.

Why input is harder than output

The output parser has three structural advantages. Its input is machine-generated, so it arrives in well-formed sequences. Its grammar has explicit terminators — a CSI sequence ends at its final byte, an OSC string ends at its terminator — so the parser always knows when a sequence is complete. And a sequence it does not recognise can be discarded with no consequence beyond a missing effect.

Input has none of these. The bytes are generated by a human pressing keys, so a sequence can be *split arbitrarily in time*: the terminal sends an escape-introduced sequence for the up arrow, and those bytes may arrive in one read or in three, seconds apart, if the connection is slow enough. There is no terminator to wait for, because many key sequences are not self-delimiting — the decoder cannot look at the first two bytes and know how many more belong to them without knowing the whole table of possible continuations. And an unrecognised sequence cannot simply be discarded, because bytes the client cannot interpret still have to reach the program in the pane; a keystroke silently dropped is the most user-visible bug a terminal program can have.

So where Chapter 6’s parser answers one question — what does this byte do — the key decoder answers a three-valued one:

- **Match.** These leading bytes are exactly one known key sequence. Consume them, emit a key code, continue with the remainder.
- **No match.** These leading bytes cannot begin any known sequence. The first byte is therefore an ordinary key in its own right (a literal character, a control character); consume one byte and continue.
- **Partial.** These leading bytes are a *prefix* of one or more known sequences but do not yet complete any. Consume nothing; wait for more input.

The third answer is what makes this a different kind of program from the output parser. The output parser is a state machine over a stream that never needs to ask for more data, because it holds its state between bytes and can simply stop. The key decoder holds *unconsumed bytes* between reads and has to decide, each time it is called, whether the buffer it is holding is an incomplete sequence or a complete key that happens to look like the start of one.

The prefix collision, and why time is the only tiebreaker

That last case is not hypothetical; it is the normal case, and it centres on one byte.

`Escape` is a key. `Escape` is also the first byte of nearly every function key, arrow key, and modified key in the legacy encodings. When the decoder is holding a lone escape byte and nothing further has arrived, both readings are live, and — this is the part worth sitting with — *the bytes contain no information that distinguishes them*. It is not a hard parsing problem; it is an undecidable one at the level of content. The only signal available is arrival time: a machine-generated escape sequence's bytes arrive together, and a human's next keystroke after pressing Escape does not.

This is why `escape-time` exists, and it explains something that otherwise looks like a wart in the options table. Chapter 40 placed `escape-time` in the perceptual band of the timer catalogue and explained why its value is a judgement about human perception. What this chapter adds is *why the option has to exist at all* — not as a tuning knob for a tunable behaviour, but as the externalisation of an ambiguity the program cannot resolve internally. Every other parsing decision in the program is a function of the bytes. This one is a function of the bytes and the clock, and the clock's threshold is a property of the path between the user's keyboard and the client process, which the program cannot measure and the user can estimate. Handing it to the user is the only correct design; it is not an evasion.

The corollary is that a misconfigured `escape-time` produces two distinct symptoms with one cause, and they are worth being able to tell apart when reading a bug report. Too *short*, and a real escape sequence whose bytes were split by a slow link gets decoded as `Escape` followed by garbage — the user presses an arrow key and gets an escape, then stray characters typed into their editor. Too *long*, and a bare `Escape` is not delivered until the timeout expires — the user presses Escape in a modal editor and feels a lag that is exactly the option's value. Both get reported as “the escape key is broken”; they are opposite errors.

Decode, then re-encode: keystrokes are never forwarded

A design consequence of having a decoder at all is that ztmux does not forward keyboard bytes. It cannot: it has to decode every keystroke in order to find out whether that keystroke is the prefix, or a binding in the active key table, or something that should reach the pane. Chapter 22 traced that decision path. What follows from it is that a key destined for a pane's program must be *re-encoded* on the way in, and that is `src/ported/input_keys.rs` — the same pairing Chapter 32 named when discussing extended keys.

The round trip has three consequences that are easy to miss.

First, the bytes a program in a pane receives are not necessarily the bytes the outer terminal sent. They are whatever `input_keys.rs` produces for the decoded key code under the current encoding options. This is not a defect; it is the mechanism by which a multiplexer can present a *consistent* keyboard to programs regardless of which terminal the user happens to be attached from — the same normalising role the capability layer of Chapter 32 plays for output.

Second, the encoding chosen on the way in is a function of what the *pane's program* should see, and the decoding on the way out is a function of what the *outer terminal* sends. These are configured independently, which is exactly why `extended-keys` and `extended-keys-format` are two options and not one. Chapter 32 made this point as negotiation policy; here it is the same fact seen as a data path with two endpoints and two codecs.

Third, a decoding bug does not lose a keystroke, it *substitutes* one. If a sequence is matched too greedily, the extra bytes vanish into the wrong key; if too shallowly, the tail arrives as separate keys. Either way the user gets an action they did not ask for rather than no action, which is worse and much harder to describe accurately in a bug report.

user-keys, and admitting the table is incomplete

No table of key sequences is complete, because terminal emulators keep inventing them and because users remap their keyboards at layers below the terminal. A decoder built on a closed table has exactly two failure modes for an unknown sequence, and both are bad: treat it as an escape plus literal characters, which puts garbage into the pane, or drop it, which produces a dead key.

The `user-keys` option is the escape hatch, and its design is worth noting as a general pattern rather than a tmux trivium. It lets the user *declare* a byte sequence as a named key, after which it participates in the ordinary machinery — it decodes to a key code, it can be bound with `bind-key`, it prints in `list-keys`. The option does not add a special case to the decoder; it adds an entry to the data the decoder already consults. That is the same shape as the `@`-prefixed user options of Chapter 33: a table-driven subsystem extended by adding rows rather than by adding branches. A port that got this wrong — that implemented `user-keys` as a pre-pass or a post-pass around the real decoder — would produce a key that works in some contexts and not others, and the difference would surface only in whichever context the implementer did not test.

The same table-extension logic is why key *names* have to round-trip. `src/ported/key_string.rs` parses the names users write into key codes and prints key codes back into names; the glossary pairs it with `tty_keys.rs` as the producer of key codes for exactly this reason. Those are the two ends of a bijection that the configuration surface depends on: `bind-key` parses a name into the code the decoder will produce, and `list-keys` prints codes back into names the user can paste into a config file. If the parse and the print disagree about any key, `list-keys` output stops being valid configuration input — a failure mode with no error message, because both halves work fine in isolation.

Bursts: paste is the input path's flood

Chapter 23 discussed flood on the output side — a program producing more bytes than the terminal can consume — and the machinery that survives it. The input path has a mirrored problem with a different cause: a paste.

A paste is, at the byte level, indistinguishable from an extremely fast typist. Large volumes can arrive with no pause anywhere in them, which removes the timing signal the decoder relies on for its one ambiguous case, and which means the decoder's per-key work runs once per character rather than once per human action. Terminals that support bracketed paste address the *first* problem by wrapping the pasted region in explicit markers, which is why that capability appears in Chapter 10's feature list alongside truecolor and the mouse-reporting modes: it is not decoration, it is the terminal telling the client that the timing assumptions do not hold for the next stretch of bytes.

The second problem — throughput — is why the decoder's per-byte cost matters more than its apparent frequency suggests. A human generates a handful of keys a second; a paste generates as many as the pipe allows. This is the same reasoning Chapter 23 applied to the character-printing fast path on the output side, arrived at from the opposite direction.

What parity cannot see here

There is an honest limit worth stating plainly, and it is structural rather than a matter of effort.

The parity suite (Chapter 11, Appendix D) drives both binaries through commands. A `.fmt` case expands a format with `display-message -p` against a fresh detached session; a `.sh` case runs `$TM <command>` lines and prints output. Neither flavour attaches a terminal, and neither presses a key. Everything in this chapter — the three-valued match, the prefix collisions, the escape ambiguity, the re-encoding on the way into a pane — sits on the far side of that boundary. The suite can pin `bind-key` and `list-keys` behaviour, because those are commands and their output is text. It cannot pin which byte sequence `input_keys.rs`

emits for a modified function key under a given `extended-keys-format`, because getting that byte sequence out requires a pane, a program in it that echoes its input, and a client with a real terminal driving keys in.

Chapter 40 named the time-dependent surface as the least mechanically verified part of the port. This is a second such region, and it is broader: the *client-side input path* is outside the suite's reach, not because of timing but because the suite's two case flavours are both command-shaped. The evidence for this code is the evidence Chapter 2 described for everything else — the C citation on each function, the anti-drift gate of Chapter 12 confirming the ported function names match the C index, and reading the port against its C counterpart. That is real evidence and it is not the same kind of evidence as a byte comparison, and the difference should be visible to anyone deciding how much to trust which subsystem.

The shape of a case that *would* cover part of it is worth recording for whoever writes it: spawn a pane running a program that prints its input in a stable, escaped form; drive keys into it through `send-keys`; capture the pane with `capture-pane`; compare. That covers the encoding half under both binaries, deterministically, with no terminal involved, because `send-keys` enters the pipeline at the key-code level rather than at the byte level. It does *not* cover the decoding half, because nothing that is not a real attached terminal produces the bytes that half exists to consume. Half the path is reachable with the existing case flavours and half is not, and knowing which half is which is the difference between a documented gap and a blind spot.

Chapter 47 – The Command Line: One Binary, Two Programs

Every path through ztmux begins in the same place: a process starts, with a command line. Chapter 3 established that the binary is both client and server and that `tmux_main` decides which one this invocation is. Chapter 8 enumerated the five entry points that lead onto the command queue and described the second of them — a command from a client's command line — from the server's side. This chapter covers the part before all of that: what happens between process start and the point where either the queue or the event loop takes over. It is a small amount of code with an unusual amount of policy in it, and a large share of the operational surprises users have with this class of program originate here.

The decision that has to be made first

Before anything else can happen, the process must answer a question with no default: should this invocation talk to a server, and if so, which one?

Every part of that question is answered from the command line and the environment, and the order of resolution matters. `src/ported/tmux.rs` holds it — the appendix's module map records its job as argument handling, socket-label construction (`make_label`, the `ztmux-<uid>` path), and the client-versus-server decision. Three things, and they are strictly ordered, because the socket cannot be constructed until the arguments naming it have been parsed, and the client-versus-server decision cannot be made until the socket is known.

The socket is the address, and Appendix L gave its construction: the default is `<dir>/ztmux-<uid>/default`, with `-L <label>` naming a different socket in the same directory and `-S <path>` giving an explicit path. What deserves emphasis is that this is the *entire* addressing scheme. There is no server registry, no discovery protocol, no daemon that knows about other daemons. Chapter 41 made this point in the context of nesting — the socket-is-the-address model — and it is worth stating as a first-class property of the command line rather than a nesting quirk, because it is the reason two otherwise identical command lines can act on completely different worlds. `ztmux ls` and `ztmux -L build ls` are not variations on a query; they are queries to two different programs that share no state and cannot see each other.

Three properties follow, and each is something an operator eventually relies on:

- **Isolation is free and total.** Running work under a distinct `-L` label guarantees it cannot be affected by anything in the default server — not a stray `kill-server`, not a global option set from a config file, not a `kill-session` with a mistyped target. This is why the parity runner (Appendix D) starts each case's server with a unique `-L`; it is not a testing trick, it is the ordinary mechanism used deliberately.
- **There is no way to ask “what servers exist.”** Since the address space is the filesystem, enumerating servers means listing the socket directory, and the correspondence between files and live servers is not exact in either direction — Chapter 39 covers the inverse case, a live server whose socket file was removed out from under it, and the recovery signal that exists for it. Any tooling that wants a list of servers is building an inventory the program itself does not maintain.
- **The socket path is a capability.** Chapter 38 makes that argument in full and it is not repeated here; the command-line observation is narrower and worth stating on its own. `-S` is the one flag that moves the server's socket outside the permission-checked directory `make_label` builds, which is why it is the flag that requires thought.

`-f`, and the config file as an argument

The configuration file is a command-line concern rather than a startup concern, and the distinction has teeth.

Chapter 9 described config loading and Chapter 8 listed the config file as one of the five roads onto the command queue — a config file is a command script, not a declarative format. The command-line half is `-f`, which names the file to read, and its most important appearance in this book has been `-f /dev/null`: the parity runner (Appendix D) passes it to both binaries so that neither reads the developer's configuration and both start from compiled-in defaults.

That idiom deserves to be lifted out of the test harness, because it is one of the most useful diagnostic moves available to a user of either program. A report of the form “ztmux does X and tmux does Y” is worth little until both have been re-run with `-f /dev/null`, because a config file is a command script and a command script can change essentially anything the program does. Chapter 36 covers diagnosis of parity failures at the level of logs and byte diffs; this is the step before that, and it is the step that resolves a large fraction of reports without any of that machinery.

There is a subtlety that the config-is-a-command-script framing makes obvious and that a declarative-format framing would hide. `-f` does not merely select a file; it selects a *sequence of commands to run before anything else*, at a specific point in the startup order, with its errors handled by the config-time error path Chapter 36 distinguishes from the runtime one. A file that fails halfway leaves the server in the state the first half produced. That is not a bug to be fixed by making config loading transactional; it is the direct consequence of the decision that there is no separate configuration language, and the cost side of a trade whose benefit is that anything typable is configurable.

The command line as a command list

The other thing on a ztmux command line is, usually, a command — and this is where the parsing story gets more interesting than it looks.

`ztmux new-window` is a command name and no arguments. Add a flag and its value and the argument parser of Chapter 8 handles it. But the command grammar also allows *command lists* separated by semicolons, which means a single shell invocation can carry several commands, and those commands are parsed by the same grammar that parses the config file and the command prompt — `cmd_parse.lalrpop`, the one parser. This is the design's payoff, and it also means the shell command line is subject to the grammar's rules, including its quoting.

So a shell command line invoking ztmux crosses *three* independent parsers, and confusing them is a common source of “the command works when I type it and fails in my script”:

1. **The shell's parser**, which resolves the outer quoting, expands variables and globs, and decides where the argument boundaries are. This one runs before ztmux exists.
2. **The command grammar** (Chapter 8), which sees the resulting argument words and applies its own rules — the semicolon as a command separator, its own quoting, brace blocks.
3. **The format expander** (Chapter 16), which runs later still, on the *values* of arguments that take formats, and which has its own escape rules where `#` is special and `##` is a literal `#`.

Each layer consumes a level of quoting, and each has a different escape character. This is why Chapter 16's `{q:...}` operator exists in two flavours: `FORMAT_QUOTE_SHELL` for a value that will be handed to a shell, and `FORMAT_QUOTE_STYLE` for one that will be read as a style string. Those are two of the boundaries in the list above, and the operator's existence is an admission that crossing them by hand is error-prone enough to need library support.

The practical rule that falls out of the three-parser model is worth stating directly, because it is not obvious and it saves time: when a command line misbehaves, determine *which* parser is misreading it before changing any quoting. Adding a layer of shell quoting to fix a command-grammar problem produces a command that fails differently. The cheap experiment is to run the same command through a different entry point — from the

command prompt, where no shell parser is involved, or from a file loaded with `-f`, where neither a shell parser nor an interactive context is involved — and see whether the behaviour changes. If it changes, the shell was the problem; if it does not, the problem is in one of the two ztmux-side layers.

The autostart rule, and why it is a per-command property

`ztmux new-session` on a machine with no running server starts one. `ztmux list-sessions` on the same machine does not; it reports that there is no server. This distinction lives in the command table rather than in the command-line code, and Chapter 8 named the mechanism without drawing out the consequence: `CMD_STARTSERVER` is a flag on a command's `cmd_entry`, and the entry-point logic consults it when there is no server to connect to.

Making autostart a per-command property rather than a global policy is the right design and it is worth seeing why. A global always-autostart policy would mean any typo that happened to be a valid command name spawns a daemon, and would make `has-session` — whose entire purpose is to answer a question by exit status — capable of changing the answer by asking it. A global never-autostart policy would mean every user's shell profile needs a `start-server` call before anything else. Neither is acceptable, and the middle position cannot be derived from anything about the command line; it has to be declared per command, by whoever knows whether that command is a question or an instruction.

For scripting, the consequence is a rule Chapter 50 relies on: commands that *inspect* are safe to run against a possibly-absent server and will tell you it is absent, and commands that *create* will bring one into existence. Any script that wants to branch on the existence of a server must use the former kind, and `has-session` is the canonical instance because it communicates purely through exit status.

The one-shot client, in full

The command-line invocation that runs a command and exits is the most common ztmux process by count on any machine with automation, and its life is short enough to describe completely. Chapter 8 covered the server-side half of it and Chapter 4 covered the exit reasons; the whole shape, end to end, is this.

The process starts, parses arguments, builds the socket path, and finds a command on the line. It connects to the socket. If the connection fails and the command's entry carries `CMD_STARTSERVER`, it starts a server — the daemonisation of Chapter 3 — and connects to that. It sends `MSG_COMMAND` (Chapter 17) rather than attaching, which means it never performs the terminal hand-off of Chapter 4: no raw mode, no alternate screen, no render surface. Server-side, the command line is parsed by the same grammar and queued like any other item. When the item completes, the server replies, and the client exits with a status derived from that reply.

Two things about this are load-bearing for every script anyone will write against the program.

The client does no interpretation. Chapter 4 established this for the attached case — the client is a relay, all intelligence is server-side — and it holds just as strictly for the one-shot case. The command is parsed by the server, resolved against server state by the server, and executed by the server. The local process is a courier. This is why a command run from a shell inside a pane and the same command run from an entirely different session behave identically apart from their default target, and it is why there is no such thing as a client-side ztmux command.

The exit status is a protocol, not a convenience. It comes back over the socket from the command's `cmd_retval`, through the client-exit machinery Chapter 4 describes. This means the status a script sees is the *command's* verdict, not the local process's — a distinction that matters the moment the failure is a connection failure rather than a command failure, because those are two different exit reasons in the enumeration Chapter 4 describes, and a script that treats all non-zero statuses identically will report “the command failed” when the truth was “there was no server.”

Why this is not a `main()` chapter

It would be reasonable to expect all of the above to be a page of argument parsing. The reason it is not is that the command line is where the program's addressing model, its configuration model, its parsing model, and its autostart policy all become visible at once, in a form the user can manipulate. Everything else in this book describes machinery reached *through* one of these decisions. The command line is where they are made, and it is the only surface where a user can get all four of them wrong in a single line of a shell script.

Chapter 48 – Identity: Ids, Indices, and Names

There are three ways to refer to a session, a window, or a pane, and they have different lifetimes, different guarantees, and different failure modes. Appendix B lists them as format variables — `session_id`, `window_id` and `window_index`, `pane_id` and `pane_index` — and Chapter 30 covered the index space and its arithmetic in depth. What has not been laid out is the *identity model* they collectively form, which is the thing any script, any control-mode front end, and any session-restore tool has to reason about correctly to avoid acting on the wrong object.

Three namespaces with different contracts

Ids are server-allocated and unique. Appendix B records their surface form — `pane_id` is documented there as the unique id in the `%N` form — and the sigil is per type. The contract an id provides is the strongest one available: an id names one object, and if that object is destroyed, the id names nothing rather than naming something else.

Indices are positions, and Chapter 30 is the chapter about them. A window index is its slot in a session's index space; a pane index is its position within a window. They are small, human-friendly numbers, and they are *reassigned*. Everything Chapter 30 says about `base-index`, holes, and `renumber-windows` is a statement about how the mapping from index to object changes over time.

Names are user-facing labels. A session has a name; a window has a name, which may be set by the user or derived automatically. Names are searched rather than matched exactly — Chapter 8's target section describes the search: exact match preferred, then prefix, then an `fnmatch` pattern, which is what makes `attach -t wo` find session `work`.

Three namespaces, three contracts, and the ranking is the point: ids are stable and unfriendly, indices are unstable and ordinal, names are unstable and searched. A target string can carry any of them, which means the *same* target syntax can carry three very different guarantees, and the guarantee is invisible at the call site.

The failure that motivates all of this

Consider a script that creates a window, then configures it:

```
ztmux new-window
ztmux send-keys -t :2 'make' Enter
```

This is wrong, and the way it is wrong is instructive. The second command names window index 2 in the current session, on the assumption that the new window landed there. It may not have: the index a new window receives depends on the session's existing occupancy and on `base-index`, both of which Chapter 30 describes. Worse, if some *other* window occupies index 2 — because the assumption was off by one, or because a hook created a window, or because the user was working in the session at the same moment — the command does not fail. It succeeds, against the wrong window, and sends a build command into whatever was running there.

That is the characteristic failure of index-based targeting, and it has three properties that make it much worse

than an ordinary bug: it is silent, it is timing-dependent, and its damage is arbitrary because the payload was written for a different context.

The name-based version has the same shape with a different trigger. Targeting a name resolves by search, so it finds a window with that name if one exists — and if none does, it can still match a *prefix* or a pattern, which means a target intended for one object can land on another that merely starts with the same letters. Chapter 8 presents the search order as the mechanism that makes targeting forgiving for a human at a keyboard. For a script, forgiving means “will find something even when the thing it wanted is gone.”

The id-based version cannot fail this way. A target naming a destroyed pane’s id resolves to nothing and the command errors. That is the entire value proposition: an id turns a silent wrong-object action into a loud no-object failure.

The asymmetry that makes ids the right default for programs

There is a symmetry argument that says all three forms are equivalent because a target string can carry any of them. The symmetry is false, and the reason is that the three namespaces differ in *who can invalidate them*.

An id is invalidated only by destroying the object it names. An index is invalidated by destroying the object, by destroying any *other* object in the same index space — which can shift positions, per Chapter 30’s `renumber-windows` behaviour — or by creating one. A name is invalidated by all of those plus renaming, and window names in particular can change without anyone asking: Chapter 21 noted that `automatic-rename` and `automatic-rename-format` exist precisely so that naming policy is a setting, and the whole point of automatic renaming is that a window’s name tracks what is running in it. A script that stores a window name and comes back to it later has stored a value that a program in that window can change by being replaced.

So the ordering is not a matter of taste. For any reference held across time — across two commands in a script, across a poll interval in a monitoring tool, across a save and restore — the id is the only form whose invalidation condition is the one the holder actually cares about.

The complement is equally firm: for a reference typed by a human, indices and names are correct and ids are hostile. Nobody remembers which numbered id belongs to the pane they are looking at. This is why the program offers all three rather than picking one, and it is why the target resolver of Chapter 8 has to accept all three forms through one syntax.

Getting an id in the first place

The identity model would be academic if ids were hard to obtain. They are not, and the route is the format language — the same route Chapter 11 uses to make the parity suite’s assertions: run the action, then read the result back through a format.

```
ztmux display-message -p '#{pane_id}'
```

That prints the current pane’s id, and a `-t` target makes it print another pane’s. The `list-*` commands take `-F` and can therefore emit ids for a whole set at once — Chapter 43’s session-restore discussion notes that state is dumpable through the `list-*` commands with `-F`, and ids are part of what that dumps.

This is the general shape of every identity-safe script: perform the mutation, immediately resolve the object just created to its id through a format, and use the id from that point on. The window between the mutation and the resolution is the only interval in which the script is exposed to index or name instability, and it is as short as it can be made.

The `#{q:...}` operator of Chapter 16 belongs in the same idiom for the name case: a script that must carry a name rather than an id should quote it on the way out, because names are arbitrary user text and Chapter 47’s three-parser problem applies to every one of them.

Ids under a restart, and why they are not identity in the durable sense

There is a limit to the guarantee, and it lines up exactly with Chapter 43's inventory of what the server forgets.

Ids are allocated by a running server. A restarted server allocates from a fresh sequence, which means an id recorded before a restart is meaningless after it — not stale in the sense of naming a destroyed object, but potentially naming a *different* object, which is the one failure mode ids otherwise rule out. Appendix B's `next_session_id` variable exposes the allocator's cursor, which is the clearest statement possible that this is a per-server counter rather than a durable identifier.

Chapter 43's argument about what is and is not serialisable applies directly. A save-and-restore tool cannot round-trip ids; it must round-trip names, indices, and geometry, and accept that the restored objects are new objects that resemble the old ones. That is why Chapter 43 concludes that the layout string is the one thing that *is* serialisable — a layout string describes shape, not identity, and shape survives a restart in a way identity cannot.

Stated as a rule: ids are the correct handle within a server's lifetime and the wrong handle across it. Names are the correct handle across a restart and the wrong handle within a busy session. There is no handle that is correct for both, and a tool that needs both has to hold both and know which one it is trusting at each point.

Names have their own hierarchy, and it is not the same as identity

One more distinction, because the vocabulary collides. A window has a *name*; a pane has a *title*. They are different things with different owners, and Chapter 6 established the gating: `allow-rename` controls whether a program in a pane may rename its window through the rename escape sequence, and `allow-set-title` controls whether the title-setting OSC sequences take effect. Appendix C lists both as flags, and Chapter 38 lists them among the options that exist to constrain what untrusted pane output can do.

So a window's name has up to three claimants — the user via `rename-window`, the automatic renamer via `automatic-rename-format`, and the program in the pane via the rename sequence if `allow-rename` permits — while a pane's title has one, the program, gated by `allow-set-title`. Counting the surfaces a user is likely to call “the title” gives four: the session name, the window name, the pane title, and the outer terminal's title bar, which `set-titles` and `set-titles-string` (Appendix C) drive from a format.

The identity consequence is the one this chapter is about: none of these are identifiers. All four are display strings; two of them — the window name and the pane title — are writable by a program running inside a pane, subject to the two gating options, and the fourth is derived from a format that can read either of those. Using any of them as a handle means using a string that untrusted output can change. Chapter 38's trust boundary and this chapter's identity model meet exactly here: `allow-rename` is not only a display-hygiene option, it is the option that decides whether a program in a pane can invalidate a name-based target that some script elsewhere is holding.

Chapter 49 – The Platform Layer

A faithful port inherits its target's portability problems along with everything else, and this book has touched that fact in several places without ever collecting it. Appendix F lists `libc.rs`, `osdep.rs`, and `compat/` in one bullet as libc bindings and per-OS shims. Chapter 19 describes a macOS-specific `closefrom` variant. Chapter 23 notes that the tidy pass calls `malloc_trim` where the platform provides it and skips it on macOS. Chapter 3 records that the event loop waits in `select` on macOS and `poll` elsewhere. Chapter 25 observes that a width implementation built on the C library's `wcwidth` would disagree between platforms for the same character. Those are five facts about one subsystem, and the subsystem has never been described as one.

What a platform layer is for, in a program of this kind

The naive model of portability is that a program uses standard interfaces and the standard interfaces behave the same everywhere. Little in a terminal multiplexer fits that model. Several of its core operations sit on an interface that is either non-standard, or standard with divergent behaviour, or standard and absent on some target:

- Allocating a pseudo-terminal and putting a child on the far side of it.
- Discovering what command a process is currently running, for `pane_current_command`.
- Closing inherited descriptors above a threshold before `exec`.
- Returning freed heap to the operating system.
- Multiplexing descriptors and timers.
- Deciding how wide a character is.

The platform layer is the set of shims that give the rest of the program one interface per item on that list. What matters for a *port* specifically is that the C program already made every one of those choices, and made them across decades of running on systems the port's author will never test on. Re-deriving them is not merely wasted work; it is work whose errors are undetectable on the developer's machine by construction, because the whole category consists of things that differ between machines.

The divergence taxonomy

The five facts collected above are not all the same kind of divergence, and separating them is what makes the layer tractable rather than a bag of conditional attributes.

Absent interface. The interface does not exist on some target and the program provides its own or omits the step. `malloc_trim` is the clean instance: Chapter 23 records that it is glibc's and that the macOS allocator does not provide it, so the tidy pass omits that step there. The important property is that the *caller* is unchanged — the tidy timer runs on both platforms and does the same other work — and the platform difference is confined to whether one call happens.

Present interface, divergent behaviour. The call exists everywhere and does not do the same thing. `wcwidth` is the case Chapter 25 names, and its consequence is severe enough to have decided the design: because a C-library width function would return different answers for the same character on two platforms, the width decision cannot be delegated to the platform at all. The program carries its own tables. This is the most dangerous class, because the code compiles and runs everywhere and is wrong somewhere.

Present interface, divergent edge behaviour. The call works the same in the common case and differs at a boundary. The macOS `closefrom` situation of Chapter 19 is this: closing inherited descriptors is a step both

platforms perform, and the divergence appears only in interaction with a raised `RLIMIT_NOFILE`, which is exactly the condition the server creates. This class produces the most confusing bugs, because the symptom appears far from the cause — in that case as `pane_current_command` reporting the wrong value, which is several subsystems away from descriptor handling.

Build-time divergence. No behavioural difference at all, only a difference in how the program is assembled. ztmux's clearest instance of this class is one it no longer has: the C event loop used to link statically on macOS and dynamically elsewhere, each overridable with a feature flag, because a Homebrew dylib brings runtime-path problems a static link does not have. Nothing in the program's behaviour depended on which was chosen — the divergence was entirely in packaging — and removing the C dependency (Chapter 3) removed the whole question, leaving the two feature flags behind as inert vestiges. That is the characteristic fate of this class: build-time divergence is the only kind that can be deleted rather than handled.

Sorting a platform difference into one of these four before fixing it is most of the work, because the correct response differs per class. Absent interfaces want a conditional call site. Divergent behaviour wants the platform interface removed from the design entirely. Divergent edge behaviour wants a ported shim — which is what Chapter 19's fix was, a port of tmux's platform-specific variant rather than an invention. Build-time divergence wants a build script and a feature flag, and wants to stay out of the source.

Two platforms is a qualitatively different claim from one

The parity suite runs on Linux and macOS, and Chapter 11 makes the claim in that form: green on both, with the determinism rules — fixed 80×24 geometry, `LC_ALL=C`, `-f /dev/null` — designed so that the same case produces the same reference bytes on both. Chapter 11 draws the operational conclusion: a case that passes on one and fails on the other is a *platform* bug, isolated by construction.

That deserves more weight than a passing mention, because it changes what the suite is. A suite that runs on one platform tests the program. A suite that runs the same cases on two platforms and requires identical output tests the program *and* the platform layer, and it does so without any case being written about platforms. No case in `parity/cases/` is about `closefrom` or `malloc_trim`. The `pane_current_command` divergence of Chapter 19 surfaced through an ordinary case that passed on Linux and failed on macOS, and the platform layer was implicated by the *shape* of the failure rather than by its content.

This is the highest-leverage property the two-platform requirement has, and it is worth naming as a design principle rather than a testing detail: running an identical deterministic suite on more than one platform gives platform coverage proportional to the whole suite, not to the number of platform-specific cases anyone thought to write. The cases do not know they are testing portability. That is precisely why they catch divergences nobody anticipated.

The cost is the mirror image and should be stated with the same clarity. The determinism rules that make cross-platform comparison possible also *exclude* the values most obviously likely to differ. Chapter 11 forbids `#{host}`, `#{pid}`, socket paths, and wall-clock time from cases, because those differ between the reference and the port. Those same values are where a naive platform bug would be most visible. The suite's design therefore trades away the easy platform signals in exchange for making every remaining case a platform test, and that trade is correct — the easy signals were never the dangerous ones — but it means the platform layer's own surface is verified indirectly, through its effects on ordinary behaviour, rather than directly.

What the two-platform claim does not cover

The honesty Chapter 15 applies to subsystem completeness applies here too, and the statement is short: the evidence covers the platforms the suite runs on.

tmux runs on systems this project does not test. The C program's `compat/` directory exists because of them, and a port that vendors the C source vendors those files too; Chapter 12's anti-drift gate scans

`vendor/tmux/compat/*.c` alongside `vendor/tmux/*.c` when it builds its C index, which means the compat sources are inside the gate's world even where the port has no build for the platform they serve.

That is the correct state to be in and it should not be dressed up as more. The port's claim is Linux and macOS, byte-verified. The port's *structure* — vendored C including compat, a gate that indexes it, a platform layer with the four divergence classes separated — is what would make a third platform a matter of work rather than a matter of redesign. Those are different claims and only one of them has a number attached.

Why the layer is small, and why that is the goal

The final observation is about size. The platform layer is one bullet in Appendix F's module map, next to bullets covering the grid, the command language, and the input parser. That proportion is not an accident of the port's youth; it is the target state, and it follows from the taxonomy above.

Every divergence resolved as “remove the platform interface from the design” — the `wcwidth` decision — shrinks the layer permanently and removes a whole class of possible bugs rather than handling one instance. Every divergence resolved as “conditional call site” leaves a small, local, readable difference. Every divergence resolved as “build-time flag” leaves nothing in the source at all. The layer stays small because the C program already pushed most of its portability problems into one of those three shapes over a very long time, and the port inherits the results.

The alternative — the shape a from-scratch reimplementation would land in — is portability handled at each call site by whoever wrote it, discovered one platform bug report at a time. That is the same argument Chapter 1 makes for porting rather than rewriting, applied to the part of the program where the argument is strongest, because portability knowledge is exactly the kind of knowledge that cannot be re-derived by reading a specification. It exists only as a record of what broke.

Chapter 50 – Scripting the Server

Automation is the use case that separates a multiplexer from a terminal emulator. A user who only ever attaches interactively could be served by a tabbed terminal; a user who builds development environments, CI harnesses, and remote-work layouts out of sessions is using the program as a *runtime*, driven by shell scripts. Every command in Appendix A is available to those scripts, the format language of Chapter 16 is available to read state back, and the whole surface is documented elsewhere in this book. What has not been written down is how to compose it correctly, and the failure modes are specific enough to enumerate.

Chapter 42 covers the other automation surface — control mode, for a program that maintains a live model of server state. This chapter is about the shell-shaped case: a script that runs, does a bounded amount of work through one-shot invocations, and exits. The two have different correctness problems, and this one is where most people start.

The examples below use tmux’s own flag spellings. That is the whole premise of Appendix A’s reference — the ported commands are drop-in compatible, so a script written against tmux’s documented flags is a script that runs against ztmux — but it means the flags in these examples are documented by tmux’s manual rather than by this book, and the per-command flag surface is not reproduced here.

The three questions a script must answer before its first command

Which server? Chapter 47 established that the socket is the address and that there is no discovery. A script that passes neither `-L` nor `-S` acts on the user’s default server, sharing it with the user’s interactive work. Whether that is right depends entirely on intent: a script that sets up the user’s daily environment should use the default server; a CI harness or a build orchestrator should not, because sharing means a `kill-server` from either side takes down both.

Does the server exist? `has-session` communicates purely by exit status (Appendix A), and Chapter 47’s autostart rule explains why it is the right probe: it does not carry the start-server flag, so asking the question cannot change the answer.

What configuration is in effect? A script that relies on `base-index` having a particular value, or on any option’s default, is relying on the user’s configuration file — Chapter 47’s `-f /dev/null` point, in its scripting form. A script that wants determinism must either pass `-f` with a known file, or set the options it depends on explicitly, or use forms of targeting that do not depend on options at all. The third is the best of the three and Chapter 48 describes it: id-based targeting is invariant under every option in the table.

Idempotence, and the shape that gets it

The most common automation task is “make sure this environment exists,” and the most common bug is writing it as “create this environment,” which does the wrong thing on the second run.

The correct shape is a guard on the state, using the command designed for it:

```
if ! ztmux has-session -t dev 2>/dev/null; then
    ztmux new-session -d -s dev
    # ... build the layout ...
fi
ztmux attach-session -t dev
```

Three details in that fragment are load-bearing and each corresponds to something earlier in this book.

The guard uses exit status, not output parsing. Chapter 4 established that the exit status comes back over the socket from the command's own verdict, which makes it the most reliable channel the program offers — it is not affected by the format language, by locale, or by message text.

The stderr redirection is there because “no server” and “no session” both produce a non-zero exit with a message. The script is using the status and does not want the message, which is a distinction Chapter 4's exit-reason discussion makes precise: the message and the status are separate channels set from the same decision.

The `attach-session` is outside the guard. This is the part people get wrong. Attaching is not part of creating; a script that attaches only inside the creation branch works the first time and silently does nothing on subsequent runs, which is the exact inverse of the intended behaviour.

Ordering is guaranteed; completion is not the same thing

Chapter 8's queue discussion establishes something a script author can rely on absolutely: commands within one list run in order, a list queued later runs entirely after a list queued earlier, and an asynchronous command suspends its item and everything behind it. Two one-shot invocations from a shell script are two lists, and the second is queued after the first completes, because the shell ran them sequentially and each waited for its reply.

So this is safe with respect to *server state*:

```
ztmux new-window -t dev
ztmux split-window -t dev
```

The window exists before the split is attempted. What is *not* guaranteed is anything about the program running inside the new pane. Chapter 19 traced the spawn path: the pane's child is forked and `execed`, and the command that created it returns as soon as the server-side work is done. A shell in that pane has not necessarily printed a prompt, read its startup files, or become ready for input.

This is the root of the most-reported scripting bug in this program's history, and it deserves to be stated without hedging: `send-keys` immediately after creating a pane is a race. The keys are delivered to the pseudo-terminal correctly and the server does its job perfectly; whether the program on the far side has started reading yet is not something the server knows or can know. The symptom is that the script works on a fast machine and drops the first characters on a slow one, or works for a shell with a small startup file and fails for one with a large one.

There is no version of this that is fixed by ordering guarantees, because the thing being waited for is not a server operation.

`wait-for`, and the handshake that actually works

The program provides the primitive for this and it is among the least-used commands in Appendix A: `wait-for`, listed there as “block on, or signal, a named synchronization channel.” Chapter 8 places it among the commands that suspend their queue item, which is the mechanism.

A channel is a name. One side blocks on it; another side signals it; the blocked side resumes. Neither side needs to know anything about the other beyond the name. The reason this solves the spawn race and nothing

else does is that it lets the *program in the pane* be the thing that reports readiness, which is the only party that actually knows:

```
ztmux new-window -t dev \  
  'sh -c "ztmux wait-for -S ready; exec ${SHELL}"'  
ztmux wait-for ready  
ztmux send-keys -t dev 'make' Enter
```

The new window's command signals the channel and then replaces itself with the shell; the script blocks until that signal arrives. The `send-keys` now runs after a point the pane's own process defined. The handshake is explicit, it involves no sleeps, and it does not degrade on a slower machine — it takes longer, which is correct.

Two properties of channels are worth knowing before relying on them. Channels are server-global names, in the same way paste buffers are (Chapter 28), which means two independent scripts using the same channel name will interfere; channel names should be generated per run in anything that might run concurrently. And a wait with no signaller suspends its queue item indefinitely, which — per Chapter 8 — means everything queued behind that item waits with it. Chapter 8 is equally clear about what does *not* happen: the item yields rather than blocking the event loop, so the server keeps serving every other client. The symptom of a forgotten signal is therefore a script that never returns and a pending item that never completes, not a frozen server, and looking for the wrong one of those two wastes the diagnosis.

The alternative people reach for first — sleeping — is worth explicitly rejecting. A sleep long enough to be reliable is long enough to be annoying, and it is still not reliable, because the thing it is waiting on has no bound. A sleep is a guess about somebody else's machine.

send-keys is not “type this text”

The command every script reaches for deserves a closer reading than its name invites, because its argument is not a string. Appendix A describes it as sending “keys (or literal input) to a pane as if typed,” and the parenthesis is doing real work: there are two modes, and the default one is the surprising one.

By default the arguments are *key names* — the same names `bind-key` accepts and `list-keys` prints, parsed by the `key_string.rs` half of the bijection Chapter 46 described. That is why `Enter` in a `send-keys` argument produces a newline rather than the five letters, and it is why a word that happens to collide with a key name is at risk of being interpreted rather than typed. The command's arguments enter the pipeline as key codes, which are then encoded to bytes for the pane by `input_keys.rs` under the current options — the same encoding half of Chapter 46's round trip. Nothing about this path is byte-level; the bytes are produced at the end from the key codes, not carried through from the argument.

Three consequences follow that are worth internalising before writing a script that depends on this command.

Arbitrary text is not safe to pass in the default mode. A string containing a word that parses as a key name, or a character sequence the name parser treats specially, does not arrive as written. The literal mode Appendix A mentions exists exactly for this and is the correct choice whenever the argument is data rather than a keystroke — anything interpolated from a variable, a file, or a format expansion.

The bytes that arrive depend on the pane's key-encoding options, not on the script. Chapter 46 established that the encoding half is configured for what the pane's program should see. A `send-keys` of a modified key is therefore a request for a key, and the byte sequence that results is whatever the current configuration says that key looks like — which is the correct design for the interactive case and a source of surprise for a script author who expected to be writing bytes to a pty.

The command has no idea whether anything read what it sent. It puts input into the pane's pseudo-terminal; there is no acknowledgement, no delivery confirmation, and no error if the program on the other side exited a moment earlier. This is the same property the race in the previous section rests on, stated as an interface fact rather than a timing one: `send-keys` is fire-and-forget by construction, so anything a script needs to know about the result has to come back through a different channel — `capture-pane`, a `wait-for` the program itself signals, or the side effect the keys were supposed to cause, observed through a format.

The general lesson is that `send-keys` is a *keyboard emulator*, not an IPC mechanism. Where a script needs to run something in a pane, running it as the pane's command (Chapter 19) is direct, synchronous with respect to spawn, and free of every hazard above. `send-keys` earns its place when the target is an already-running interactive program that has no other input channel, which is a narrower set of cases than its ubiquity in scripts suggests.

Reading state back: formats, not output parsing

Every `list-*` command takes `-F` (Appendix A), and `display-message -p` expands a format and prints it — the parity suite's workhorse, per Chapter 11. Between them, any state the format language exposes, which is nearly all of it per Appendix B, is available as exactly the bytes the script asked for.

This makes output parsing unnecessary, and unnecessary output parsing is a bug waiting to happen. The default output of a `list-*` command is a human-readable line intended to be read by a human; it contains fields the script does not want, in an order that is a presentation choice. A `-F` format containing one variable produces that value and nothing else:

```
ztmux list-panes -t dev -F '#{pane_id}'
```

Three habits follow, and together they are most of what separates a robust script from a fragile one.

Ask for one thing at a time, or use a separator you control. A format producing several values needs a delimiter the values cannot contain, and pane titles and window names can contain nearly anything.

Quote what you interpolate. Chapter 16's `#{q:...}` operator in its shell form exists for this and Chapter 47's three-parser model explains why it is not optional. Any value that originated as user text or as program output — a title, a name, a current path — crosses at least one parser boundary on its way into a shell command.

Resolve to ids immediately. Chapter 48 makes this argument in full; in scripting form it is a one-line rule: the first thing a script does after creating an object is read back its id, and every later reference uses that.

What a script cannot do, and where the boundary is

Two limits are worth stating so that nobody spends an afternoon looking for the command that removes them.

A script cannot observe a pane's *content* except by capturing it. `capture-pane` (Appendix A) dumps visible content or scrollbar as text, and that is the interface; there is no event that fires when a pane's content matches a pattern. A script that wants to react to output is either polling `capture-pane` — which has the problems Chapter 42 lists for polling — or it is using `pipe-pane` (Chapter 27) to tee the pane's output to a process that can watch it, which is the right answer and is a different program shape.

A script cannot make a change atomically across several commands. There is no transaction and there is no lock; the queue orders items but nothing prevents another client's item from landing between two of a script's items. Chapter 8's queue guarantees are about ordering within and between lists, not about exclusion. In practice this is why a script that constructs an environment should do so in its own session, and ideally on its own socket: isolation is available at the addressing level (Chapter 47) and is not available at the command level, so the correct place to get it is the command line.

The through-line

Almost every rule in this chapter is a consequence of something established elsewhere in this book, and that is the useful observation to leave with. Idempotence follows from autostart being per-command. Id-based targeting follows from three namespaces with three invalidation conditions. `wait-for` follows from the queue's suspension model. Formats instead of output parsing follows from the format language reading server state directly. Isolation by socket follows from the socket being the address.

None of these is a scripting feature bolted onto a program designed for interactive use. They are the interactive design, observed from a script's position, which is what "the client is a relay and the server is the truth" (Chapter 4) means when the client is a shell script instead of a person.

Appendix A – The Command Reference

This appendix lists the commands ztmux ports, grouped by what they act on, with each command's canonical name, its alias in parentheses, and a one-line description of what it does. The list is drawn from `CMD_TABLE` in `src/ported/cmd.rs` — an array of ninety command entries — and the per-command files `src/ported/cmd_*.rs`, each a mirror of tmux's corresponding `cmd-*.c`. A command with no alias listed has none. Every entry here corresponds to a real tmux command; the point of the port is that these behave identically, so this doubles as a map of the tmux command surface that ztmux covers.

Session commands

Sessions are the top of the state tree — a collection of windows a client attaches to.

- **new-session (new)** — create a new session (and, on first use, autostart the server) and optionally attach to it.
- **attach-session (attach)** — attach the current client to an existing session.
- **detach-client (detach)** — detach one or more clients from their sessions, leaving the sessions running.
- **has-session (has)** — test whether a named session exists (exit status only).
- **kill-session** — destroy a session and everything in it.
- **rename-session (rename)** — change a session's name.
- **switch-client (switchc)** — point an attached client at a different session, window, or pane without detaching.
- **suspend-client (suspendc)** — stop the client process with `SIGTSTP`.
- **lock-session (locks)** — lock a session, running the lock command.
- **list-sessions (ls)** — list sessions, one per line, formatted by `-F`.
- **choose-client** — interactive chooser for attached clients.
- **choose-tree** — interactive session/window tree chooser.

Window commands

A window is a collection of panes tiled by a layout, with one current window per session.

- **new-window (neww)** — create a new window in the current session.
- **kill-window (killw)** — destroy a window and its panes.
- **rename-window (renamew)** — change a window's name (also affects automatic-rename).
- **select-window (selectw)** — make a window the current window.
- **next-window (next)** — switch to the next window by index.
- **previous-window (prev)** — switch to the previous window by index.
- **last-window (last)** — switch to the most recently current window.
- **move-window (movew)** — renumber a window (move it to another index/session).
- **link-window (linkw)** — link an existing window into another session.
- **unlink-window (unlinkw)** — remove a linked window from a session.
- **swap-window (swapw)** — exchange two windows' positions.
- **rotate-window (rotatew)** — rotate the panes of a window through their positions.
- **respawn-window (respawnw)** — restart the command in a window whose program exited.
- **list-windows (lsw)** — list a session's windows, formatted by `-F`.
- **find-window (findw)** — search window names/titles/content and select a match.

Pane commands

A pane is a single pseudo-terminal running a program, backed by a screen and grid.

- **split-window (splitw)** — split a pane into two, **-h** horizontally or **-v** vertically.
- **kill-pane (killp)** — destroy a pane.
- **select-pane (selectp)** — make a pane active, or move to the pane in a direction (**-L/-R/-U/-D**), or mark a pane (**-m**).
- **last-pane (lastp)** — switch to the most recently active pane.
- **resize-pane (resizp)** — grow or shrink a pane, or zoom it (**-Z**).
- **swap-pane (swapp)** — exchange two panes' positions in the layout.
- **break-pane (breakp)** — promote a pane into its own new window.
- **join-pane (joinp)** — pull a pane from another window into this one as a split.
- **move-pane (movep)** — the **join-pane** machinery targeted as a move.
- **respawn-pane (respawnp)** — restart the command in a pane whose program exited.
- **capture-pane (capturep)** — dump a pane's visible content or scrollback as text (optionally with escape sequences) to a buffer or stdout.
- **clear-history (clearhist)** — drop a pane's scrollback history.
- **pipe-pane (pipep)** — pipe a pane's output to a shell command in real time.
- **display-panes (displayp)** — overlay each pane with its index for keyboard selection.
- **list-panes (lisp)** — list a window's panes, formatted by **-F**.

Layout commands

- **select-layout (selectl)** — apply a preset layout or a saved layout string.
- **next-layout (nextl)** — cycle to the next preset layout.
- **previous-layout (prevl)** — cycle to the previous preset layout.

Buffer and copy-mode commands

Paste buffers are server-global clipboards; copy mode is the grid navigator of Chapter 18.

- **copy-mode** — enter copy mode in a pane.
- **clock-mode** — display the big clock in a pane.
- **list-buffers (lsb)** — list paste buffers.
- **set-buffer (setb)** — set the contents of a paste buffer (named with **-b**).
- **delete-buffer (deleteb)** — delete a paste buffer.
- **load-buffer (loadb)** — load a file into a paste buffer (over the socket, in the client's context — Chapter 17).
- **save-buffer (saveb)** — write a paste buffer to a file.
- **show-buffer (showb)** — print a paste buffer's contents.
- **paste-buffer (pasteb)** — paste a buffer's contents into a pane.
- **choose-buffer** — interactive paste-buffer chooser.

Key-binding commands

- **bind-key (bind)** — bind a key in a table to a command list.
- **unbind-key (unbind)** — remove a binding.
- **list-keys (lsk)** — print the key tables and their bindings.
- **send-keys (send)** — send keys (or literal input) to a pane as if typed.
- **send-prefix** — send the prefix key to a pane.
- **list-commands (lscm)** — print the command list with usage strings.

Option, hook, and environment commands

- **set-option (set)** — set a server/session option (**-g** global, **-s** server).
- **set-window-option (setw)** — set a window option.
- **show-options (show)** — print options and their values.
- **show-window-options (showw)** — print window options.
- **set-hook** — set a hook (a command list run on an event).
- **show-hooks** — print hooks.
- **set-environment (setenv)** — set a variable in the session environment.
- **show-environment (showenv)** — print the session environment.

Server, client, and display commands

- **start-server (start)** — start the server without attaching.
- **kill-server** — terminate the server and all sessions.
- **lock-server (lock)** — lock all clients.
- **server-access** — manage the access-control list for the server socket.
- **list-clients (lsc)** — list attached clients.
- **refresh-client (refresh)** — force a client redraw, resize, or control-mode update.
- **display-message (display)** — show a message (or, with **-p**, print an expanded format to stdout — the parity suite's workhorse).
- **show-messages (showmsgs)** — print the server's recent message log.
- **display-menu (menu)** — show an interactive menu overlay.
- **display-popup (popup)** — show a popup running a command in an overlay.

Control-flow and configuration commands

- **run-shell (run)** — run a shell command, optionally capturing its output.
- **if-shell (if)** — run a command list conditionally on a shell command's exit status.
- **confirm-before (confirm)** — prompt for confirmation before running a command.
- **command-prompt** — open the interactive command prompt in the status line.
- **wait-for (wait)** — block on, or signal, a named synchronization channel.
- **source-file (source)** — load and run another configuration file.

Mode and history commands

- **customize-mode** — enter the interactive options/keys customizer.
- **show-prompt-history (showphist)** — print the saved command-prompt history.
- **clear-prompt-history (clearphist)** — clear the saved command-prompt history.

Appendix B – The Format Variable Reference

The `#{...}` format language (Chapters 9 and 16) exposes nearly every field of the server's state as a variable. This appendix groups the variables by the object they describe. The dynamic per-object variables are resolved from live state at expansion time; the callback variables live in `FORMAT_TABLE` in `src/ported/format.rs`. This is not the entire set tmux defines, but it is the substantial majority the port carries, and it is the surface the parity `.fmt` cases probe. Flag variables (`*_flag`, and booleans like `pane_active`) expand to `1` or `0` (or empty), which is exactly why the `#{?...}` conditional tests them.

Client variables

Describe an attached client — a terminal lending itself to the server.

- `client_name`, `client_tty` — the client's name and controlling tty path.
- `client_pid`, `client_uid`, `client_user` — the client process's pid, uid, and user name.
- `client_width`, `client_height` — the client terminal's size in cells.
- `client_cell_width`, `client_cell_height` — the pixel size of a cell, when known.
- `client_termname`, `client_terminfo`, `client_termfeatures` — the `TERM` value, type, and negotiated feature set.
- `client_session`, `client_last_session` — the current and previous session names.
- `client_key_table` — the client's current key table (e.g. `prefix` while pending).
- `client_prefix` — flag: is the prefix key currently pending?
- `client_readonly`, `client_control_mode` — flags: read-only client, control mode.
- `client_activity`, `client_created` — timestamps.
- `client_flags`, `client_written`, `client_discarded` — flag string and I/O byte counters.

Session variables

Describe a session — the top of the state tree.

- `session_name`, `session_id` — name and numeric id.
- `session_windows` — number of windows in the session.
- `session_attached`, `session_many_attached`, `session_attached_list` — how many clients are attached, whether more than one, and their list.
- `session_created`, `session_activity`, `session_last_attached` — timestamps.
- `session_path` — the session's working directory.
- `session_marked` — flag: does this session contain the marked pane?
- `session_stack` — the window stack (most-recent order).
- `session_grouped`, `session_group`, `session_group_size`, `session_group_list`, `session_group_attached`, `session_group_attached_list`, `session_group_many_attached` — session-group membership and sizes.
- `session_alerts` — pending alerts on the session's windows.
- `session_format` — flag used internally when formatting a session line.

Window variables

Describe a window within a session.

- **window_name**, **window_index**, **window_id** — name, index, and unique id.
- **window_width**, **window_height** — the window's size in cells.
- **window_panes** — number of panes in the window.
- **window_active** — flag: is this the session's current window?
- **window_layout**, **window_visible_layout**, **window_raw_flags** — the layout string, the visible-portion layout, and the raw flag characters.
- **window_flags** — the summary flag string (*** current, *-* last, *Z* zoomed, *!* bell, and so on).
- **window_activity**, **window_activity_flag**, **window_bell_flag**, **window_silence_flag** — activity/bell/silence state and timestamps.
- **window_zoomed_flag** — flag: is a pane zoomed?
- **window_last_flag**, **window_marked_flag**, **window_linked** — last-window, contains- marked-pane, and linked-into-multiple-sessions flags.
- **window_start_flag**, **window_end_flag**, **window_bigger** — used when the window is larger than the client and scrolled.
- **window_offset_x**, **window_offset_y** — scroll offset when bigger than the client.
- **window_cell_width**, **window_cell_height** — pixel cell size.
- **window_stack_index** — position in the session's most-recent-use stack.
- **window_active_clients**, **window_active_clients_list**, **window_active_sessions**, **window_active_sessions_list**, **window_linked_sessions**, **window_linked_sessions_list** — which clients/sessions have this window active or linked.
- **active_window_index**, **last_window_index** — the session's current and last window indices.

Pane variables

Describe a pane — a single `pty` running a program.

- **pane_id**, **pane_index** — unique id (*%N*) and index within the window.
- **pane_width**, **pane_height** — size in cells.
- **pane_left**, **pane_right**, **pane_top**, **pane_bottom** — the pane's edges in window coordinates.
- **pane_at_left**, **pane_at_right**, **pane_at_top**, **pane_at_bottom** — flags: is the pane against each edge? (*pane_at_top/pane_at_bottom* are two of the current parity gaps — see Appendix D.)
- **pane_active**, **pane_last**, **pane_marked**, **pane_marked_set** — active, last, marked, and any-marked-set flags.
- **pane_current_command**, **pane_current_path** — the foreground command and its cwd.
- **pane_start_command**, **pane_start_path** — the command and path the pane was spawned with.
- **pane_pid**, **pane_tty** — the child process pid and the pane's `pty` path.
- **pane_title** — the pane title (set by OSC 0/2).
- **pane_dead**, **pane_dead_status**, **pane_dead_signal**, **pane_dead_time** — for a pane whose program exited (with *remain-on-exit*), the death flag, exit code, signal, and time.
- **pane_in_mode**, **pane_mode** — whether the pane is in a mode (copy/clock) and which.
- **pane_synchronized**, **pane_input_off**, **pane_pipe** — synchronize-panes, input-disabled, and pipe-pane states.
- **pane_fg**, **pane_bg** — the pane's default foreground/background colors.
- **pane_tabs** — the pane's tab-stop columns.
- **pane_search_string**, **pane_unseen_changes** — copy-mode search string and the changed-since-last-visible flag.
- **cursor_x**, **cursor_y**, **cursor_character**, **cursor_flag** — the pane cursor's position, the character under it, and visibility.
- **scroll_region_upper**, **scroll_region_lower** — the pane's scroll region rows.
- **alternate_on**, **alternate_saved_x**, **alternate_saved_y** — alternate-screen state.

- **history_size**, **history_limit**, **history_bytes**, **history_all_bytes** — scrollbar line count, limit, and byte sizes.
- **wrap_flag**, **insert_flag**, **origin_flag**, **keypad_flag**, **keypad_cursor_flag** — the pane's terminal mode flags.
- **pane_format**, **pane_key_mode**, **pane_path** — internal/format helpers.

Mouse variables

Set when a format is expanded in the context of a mouse event.

- **mouse_x**, **mouse_y** — cell coordinates of the event.
- **mouse_button_flag**, **mouse_standard_flag**, **mouse_sgr_flag**, **mouse_all_flag**, **mouse_any_flag** — which mouse-reporting mode is active.
- **mouse_pane**, **mouse_line**, **mouse_word**, **mouse_hyperlink** — the pane, line, word, and hyperlink under the pointer.
- **mouse_status_line**, **mouse_status_range** — status-line row and style range hit.

Server and host variables

Global to the server or the host, and largely *excluded* from parity cases because they are non-deterministic (Chapter 11).

- **host**, **host_short** — full and short hostname.
- **pid** — the server pid.
- **version** — the tmux/ztmux version string (deliberately the port's own, hence excluded from parity).
- **socket_path** — the server socket path (the **ztmux-<uid>** one).
- **uid**, **user** — the server's uid and user name.
- **start_time** — when the server started.
- **server_sessions** — total session count across the server.
- **next_session_id** — the id the next session will get.
- **config_files** — the configuration files that were loaded.

Appendix C – The Option Reference

Options are the typed, scoped configuration values of Chapter 9, defined in `src/ported/options_table.rs` (123 entries) and read through `src/ported/options.rs`. This appendix groups them by their primary scope and notes types and notable defaults where they are load-bearing. Scope determines which tree an option lives in and how it inherits: **server** options are global to the server; **session** options apply per session (and fall back to the global session tree); **window** options apply per window; **pane** options apply per pane and inherit from the window. Remember the caveat from Chapter 15: every option here is *accepted and stored* so a real `.tmux.conf` parses, but not every one is yet wired to behavior — the configuration surface is complete before the behavior behind all of it is.

Server options

Global settings that govern the server process and its interaction with terminals.

- **buffer-limit** (number) — maximum number of paste buffers kept.
- **command-alias** (array) — user-defined command-name aliases.
- **default-terminal** (string) — the `TERM` the server advertises to panes (commonly `screen-256color` or `tmux-256color`).
- **escape-time** (number, ms) — how long to wait after an `Escape` to distinguish it from a function-key sequence.
- **exit-empty** (flag) — exit the server when no sessions remain (a `server_loop` exit conjunct, Chapter 3).
- **exit-unattached** (flag) — exit when no client is attached.
- **extended-keys**, **extended-keys-format** (choice/flag) — the CSI-u / modified-key reporting mode.
- **focus-events** (flag) — pass focus-in/out reporting to panes.
- **history-file** (string) — file for command-prompt history persistence.
- **message-limit** (number) — how many server messages `show-messages` keeps.
- **prompt-history-limit** (number) — command-prompt history depth.
- **set-clipboard** (choice) — whether/how to set the system clipboard via OSC 52.
- **terminal-features**, **terminal-overrides** (array) — the terminfo correction machinery (Chapter 10).
- **user-keys** (array) — user-defined key sequences.

Session options

Per-session behavior, the largest group, covering the status line, keys, and policy.

- **base-index** (number) — the first window index (default 0; many users set 1).
- **default-command**, **default-shell** (string) — the program a new pane runs.
- **default-size** (string) — the size for a detached/unattached session (`80x24`).
- **destroy-unattached** (flag) — destroy a session when its last client detaches.
- **detach-on-destroy** (choice) — what a client does when its session is destroyed.
- **display-time** (number, ms) — how long `display-message` stays up.
- **display-panes-time**, **display-panes-colour**, **display-panes-active-colour** — the `display-panes` overlay.
- **history-limit** (number, default 2000) — scrollbar lines per new pane (Chapter 5).
- **key-table** (string) — the default key table (normally `root`).
- **lock-after-time** (number), **lock-command** (string) — idle auto-lock.

- **mouse** (flag) — enable mouse support.
- **prefix** (key, default **C-b**), **prefix-timeout** (number) — the prefix key and its timeout.
- **renumber-windows** (flag) — renumber windows when one closes.
- **repeat-time** (number, ms) — the window during which a **-r** binding repeats.
- **set-titles**, **set-titles-string** — set the outer terminal's title.
- **status** (choice: off/on/2..5) — status line on and how many rows.
- **status-bg**, **status-fg**, **status-style** — status colors/style.
- **status-format** (array) — the raw per-row status format.
- **status-interval** (number) — seconds between status refreshes.
- **status-justify** (choice), **status-position** (choice: top/bottom) — layout.
- **status-keys** (choice: emacs/vi) — editing keys in the status prompt.
- **status-left**, **status-left-length**, **status-left-style** — the left section.
- **status-right**, **status-right-length**, **status-right-style** — the right section.
- **message-style**, **message-command-style**, **message-line** — the message bar.
- **update-environment** (array) — variables copied from the client on attach (Chapter 19).
- **visual-activity**, **visual-bell**, **visual-silence** — alert display modes.
- **activity-action**, **bell-action**, **silence-action** — alert policies.
- **word-separators** (string) — copy-mode word boundaries (Chapter 18).
- **buffer/editor/set-clipboard-adjacent** helpers — **copy-command**, **editor**.

Window options

Per-window appearance and behavior.

- **aggressive-resize** (flag) — resize a window to the smallest client actively viewing it rather than the session size.
- **automatic-rename**, **automatic-rename-format** — auto-title a window from its active pane's command.
- **clock-mode-colour**, **clock-mode-style** — the **clock-mode** display.
- **main-pane-width**, **main-pane-height** — the main pane size for the main-horizontal/ main-vertical layouts (**main-pane-width** under main-vertical is a current parity gap — Appendix D).
- **other-pane-height**, **other-pane-width** — the non-main panes' size.
- **mode-keys** (choice: emacs/vi) — copy-mode key table selection.
- **mode-style** — the highlight style for selections and menus.
- **monitor-activity**, **monitor-bell**, **monitor-silence** — per-window alert monitoring.
- **pane-base-index** (number) — the first pane index.
- **pane-border-status** (choice), **pane-border-format**, **pane-border-lines**, **pane-border-indicators**, **pane-border-style**, **pane-active-border-style** — pane border drawing (Chapter 20).
- **window-size** (choice: largest/smallest/latest/manual) — how a session sizes to differing clients (Chapter 4).
- **window-status-format**, **window-status-current-format**, **window-status-separator** — the middle of the status line.
- **window-status-style**, **window-status-current-style**, **window-status-last-style**, **window-status-activity-style**, **window-status-bell-style** — window-status colors.
- **window-style**, **window-active-style** — default cell style for inactive/active panes.
- **wrap-search** (flag) — wrap copy-mode search at the ends.

Window-and-pane options

Options that live in a combined scope, settable per window or per pane.

- **allow-passthrough** (flag) — permit the DCS passthrough escape.
- **allow-rename** (flag) — permit programs to rename the window via the rename string (Chapter 6).
- **allow-set-title** (flag) — permit OSC 0/2 title changes.

- **alternate-screen** (flag) — honor the alternate-screen switch (Chapter 5).
- **cursor-colour**, **cursor-style** — the pane cursor appearance.
- **remain-on-exit**, **remain-on-exit-format** — keep a dead pane in place (Chapter 19).
- **scroll-on-clear** (flag) — push cleared lines into scrollbar.
- **synchronize-panes** (flag) — send input to all panes of a window at once.
- **fill-character** (string) — the character used to fill unused pane area.
- **pane-colours** (array) — the pane's palette overrides.

How scope resolution works

When you type a bare `set-option foo bar`, ztmux must decide *which* scope `foo` belongs to, and it does so with `options_scope_from_name` (ported with its C citation `vendor/tmux/options.c:863`): the option's name determines its natural scope from the table, and the command flags (`-s` server, `-g` global session, `-w` window, `-p` pane) constrain it further. This is why `set -g status on` sets the global session option, `setw -g window-status-format ...` sets the global window option, and `set -p allow-rename on` sets it on the current pane. Getting this resolution byte-identical to tmux matters because a config that sets an option in the wrong scope silently does nothing, and tmux users have years of precise expectations about which `set` lands where.

Appendix D – The Parity Methodology and the Current Gap List

Chapter 11 argued that the parity suite is the operational definition of correct. This appendix is the reference companion: how to run the suite, how a case is structured, the determinism rules a case must obey, and — the record that the expansion caught twenty gaps which the port then closed — the now-green cases that each still guard a specific corner.

Running the suite

The runner is `parity/run_parity.sh`. It builds the vendored tmux reference (`vendor/tmux/tmux`, the `next-3.7` line) and a release ztmux if they are missing, then runs every case under both and compares:

```
# per-case OK/FAIL plus totals
bash parity/run_parity.sh
# totals only, for CI
bash parity/run_parity.sh --summary
# emit a machine-readable summary
bash parity/run_parity.sh --json parity/parity_summary.json
# test a debug build instead of release
ZTMUX=target/debug/ztmux bash parity/run_parity.sh
```

Per-case failure detail — both outputs plus a unified diff — lands in `parity/parity_failures.log` (gitignored, truncated each run), so a **FAIL** is always accompanied by the exact bytes that differed. The reference can be overridden with `TMUX_REF=/path/to/tmux`, but the default — and the only reference the numbers are reported against — is the vendored tmux, because version drift against a system tmux produces false diffs (Chapter 11).

Case anatomy

Cases live in `parity/cases/` in two flavors:

- `*.fmt` — a single tmux FORMAT string. The runner expands it with `display-message -p` against a fresh detached session and diffs the two binaries' output. Example: `010_arith_add.fmt` contains `{e|+|:2,3}` and both must print `5`.
- `*.sh` — a shell scenario for multi-command behavior. `$TM` is exported as the binary bound to a private socket; the script runs `$TM <cmd>` lines and prints deterministic output. Example: `100_list_windows_after_neww.sh` runs `$TM new-window` then `$TM list-windows -F '#{window_index}'`.

For each case the runner starts an isolated server per binary (`-L <uniq>`, `-f /dev/null`, fixed 80×24 geometry), runs the case under a `timeout`, captures combined stdout and stderr, kills the server, and compares with a raw byte comparison. One differing byte fails the case.

Determinism rules

Byte comparison only works on deterministic output, so cases must avoid host/time/version/ pid/random state. Forbidden inputs include `{host}`, `{host_short}`, `{version}`, `{pid}`, `{client_pid}`, wall-clock times, and socket paths — all of which differ between the reference and the port (the port's version and socket prefix are deliberately its own) or between runs. The runner pins geometry (80×24), locale (`LC_ALL=C`), and options (`-f /dev/null`) so widths, collation, and defaults are stable. The discipline is to exclude non-deterministic *inputs* rather than normalize non-deterministic *output*, which keeps the comparison exact.

The gaps the expansion caught — and how each was closed (now 1115/1115)

The suite reached 122/122, was expanded to 1115, and the expansion surfaced twenty deterministic failures — each pinned to a real, bounded port gap. Every one has since been fixed against the C, and the suite now reports **1115 of 1115 passing**. The cases stay in the suite as regression guards, per policy. Grouped by the gap the expansion exposed, with the fix that closed it:

- `{!:...}` **logical-not operator** — cases **548–550**. tmux prints `1` for the negation of a false value; ztmux printed empty because the `!` operator was not wired into the format modifier parse. *Fixed*: the `!` arm was added to the modifier `match` in `src/ported/format.rs`.
- `{s/.../.../}` **regex backreferences** — cases **566–568, 572**. `\1/\2` in a substitution replacement were not substituted; on `abcd` a pair-swap that should yield `badc` emitted the literal digit next to the capture (`b2a1d2c1`). *Fixed*: backreference expansion was implemented in the replacement builder (`src/ported/regsub.rs` / the `s` modifier).
- `{p-N:...}` **left padding** — cases **589, 592**. The left-pad form padded to `N` *plus* the content length instead of a total field width of `N`. *Fixed*: the pad is now computed as a total-width target, matching tmux's field-width semantics.
- `{pane_at_top}` / `{pane_at_bottom}` — cases **644–645**. Formatted as Rust `true/false` (`format!("{}", flag)`) instead of the C's `%d` → `1/0`. *Fixed*: the flags now format as integers, as tmux does for the other `pane_at_*` flags.
- `{S:}` / `{W:}` / `{P:}` **loop variables** — cases **676–678**. The capital-letter loop modifiers iterated but did not inject `loop_index` / `loop_last_flag` (real tmux format vars, `vendor/tmux/format.c:4776`), so an inner `{loop_index}` expanded empty. *Fixed*: the two variables are now injected per iteration, as the lowercase loop forms already did.
- **Named buffers** — cases **720–724**. `set-buffer -b <name>` reported “empty buffer name”; named-buffer set/show/list/delete/rename were broken. *Fixed*: the `-b <name>` argument is now wired through the paste-buffer commands.
- `main-vertical` / `main-pane-width` — case **754**. The main-vertical layout ignored `main-pane-width`; the main pane filled the width instead of being pinned to it. *Fixed*: the main-vertical builder now honors `main-pane-width`.

Each was a small, mechanical fix against the C, not an architectural hole — the profile a healthy expansion produces. The authoritative, current record is `parity/PARITY_ROADMAP.md` and the machine artifact is `parity/parity_summary.json` (passed 1115, failed 0); this appendix is a snapshot, and like every number in this book it is exactly as current as the code it was written against.

Why keep the cases forever

It bears repeating because it is counterintuitive: a parity case is an *asset* — even, or especially, after it goes green. It is a byte-exact statement of tmux's behavior with a minimal reproduction attached, and deleting it to shorten the suite would throw away the guard that keeps the corner from silently regressing. The green count means something only because the suite was *made to fail first*: the twenty cases the expansion broke on, now green, are the proof that 1115/1115 is a real number rather than the artifact of a suite too small to fail. A suite that never fails is a suite that has stopped measuring.

Appendix E – The Anti-Drift Gate and the Fake-Fn Categories

Chapter 12 described the anti-drift gate that keeps the port from quietly growing functions tmux does not have. This appendix is the reference detail: the gate’s mechanism, the exact categories the allowlist documents, and how the C-signature citations are generated and checked.

The gate mechanism

The gate is one test, `tests/ported_fn_names_match_c.rs`, run in CI like any other. It has two halves. `collect_free_fns` parses the Rust source character by character, tracking brace depth so it counts only depth-zero free functions (not methods inside `impl/trait` blocks, which correspond to C’s struct-of-function-pointers idiom), skipping braces inside comments, strings (including raw strings), and char literals, stripping `pub/unsafe/async/extern/const`, normalizing raw identifiers (`r#loop` → `loop`), and skipping `mod tests` wholesale. `c_fn_names` builds an index of every function *definition* in `vendor/tmux/*.c`, `vendor/tmux/compat/*.c`, and `cmd-parse.y` by scanning for line-initial identifiers followed by `(` with a brace nearby (a definition, not a `;-`terminated prototype), excluding C keywords, and asserting the index has more than 500 names as a sanity check that the vendored tmux is actually present.

The check: every collected Rust free-function name must be in the C index or in the allowlist. There is **no** trailing-underscore leniency — `foo_` is not auto-accepted as an implementation of C’s `foo`; the underscore convention is exactly how the port marks a split or a macro helper, and those must be listed rather than waved through. On failure the gate prints one greppable line per violation, `FAKE-FN <name> (<file>:<line>) – no tmux C counterpart`, and panics, so a developer sees precisely which functions are unaccounted for.

The allowlist categories

`tests/data/fake_fn_allowlist.txt` holds roughly three hundred accepted exceptions, and its header frames it as “an AUDIT TRAIL, not a free pass,” with the explicit goal to shrink it. The entries fall into documented categories, so a reader sees not just *that* a function is excepted but *why*:

1. **Macro-paired helpers.** C variadic functions (`format_add(fmt, ...)`) are ported as a Rust `macro_rules!` carrying the real C name plus a `_`-suffixed helper the macro forwards to (`format_add_`). Rust macros and functions occupy different namespaces, so the macro keeps the C name and the helper takes the underscore.
2. **C-function splits.** One C function ported as several typed Rust overloads — `xstrdup` as `xstrdup_`/`xstrdup__`/`xstrdup___` — to be merged back into one signature later. These are exactly the deviations the no-underscore-leniency rule is designed to surface.
3. **libc shims.** Wrappers around library functions with no `vendor/tmux` definition at all: the `img` calls, `fnmatch`, `basename`, `dirname`. Real functions — just not tmux’s — so they cannot match the C index and must be listed. The event loop’s API (`event_*`, `evbuffer_*`, `bufferevent_*`, `evtimer_*`) belonged here while it was an FFI surface; the Rust implementation that replaced it lives under `src/extensions/`, which the gate skips, so those names left the list entirely rather than moving within it.

4. **BSD intrusive-data-structure macros.** The red-black tree family (`rb_insert`, `rb_remove`, `rb_foreach`, `rb_min`, `rb_next`, ...), the tail-queue family (`tailq_foreach`, `tailq_insert_tail`, `tailq_remove`, ...), and the list family. These are C *macros* in tmux (no function definition to match) ported as Rust functions because Rust has no equivalent of C's token-pasting intrusive-structure macros. They are a large share of the allowlist, and Chapters 3, 5, 7, and 9 all depend on them (`WINDOWS` is an `RB` tree, `CLIENTS` a `TAILQ`, key tables are `RB` trees).
5. **Genuine Rust glue.** The handful of truly Rust-only helpers — `cstr_to_str`, `assume_init_ref` — that exist because Rust needs them and C did not.

The allowlist is version-controlled, so its growth or shrinkage over time is itself a signal: it should trend toward zero as splits are merged, macros are replaced with safe Rust equivalents (Chapter 14), and shims are consolidated behind thin boundaries. Shrinking the allowlist and shrinking the `unsafe` surface are nearly the same project.

The C-signature citations

The name gate checks names; `scripts/annotate_c_links.py` checks *signatures*. For every Rust function whose name matches a tmux C function, it writes or refreshes the doc-comment citation this book quotes throughout — `/// C `vendor/tmux/<file>.c:<line>`: `<full C signature>`` — including the C function's return type and full parameter list *with names*, so a reviewer can diff Rust parameters against C parameters and catch a renamed, reordered, dropped, or invented argument (a “fake param,” the argument-level analog of a fake fn). There are more than two thousand such citations.

The script is careful about *which* C definition it cites when a name appears in several files: it prefers the C file whose stem matches the Rust file's stem (`grid.rs` → `grid.c`, `cmd_kill_pane.rs` → `cmd-kill-pane.c`, mapping underscores to hyphens); failing that, if the name is defined in exactly one C file it cites that; if it is ambiguous with no stem match, it *skips* rather than guess. It is idempotent — it replaces an existing citation in place rather than duplicating it — so re-running it after a vendored-tmux refresh updates every line number and signature to match the new C. A `--check` mode writes nothing and exits non-zero if any citation is missing or stale, so citation freshness can itself be a CI gate.

Together the name gate, the signature citations, and the allowlist form a structural- fidelity system that parallels the behavioral-fidelity system of the parity suite: names must correspond, interfaces must match, exceptions must be documented and bounded, and all of it is derived from the vendored C rather than a hand-maintained list, so none of it can drift out of sync with the source of truth.

Appendix F – The Source Tree, Module by Module

The chapters walked the subsystems in a reading order; this appendix is the map, grouping `src/` by role so you can find where a behavior lives. Every module named here is a port of the same-named tmux C file (with underscores where the C uses hyphens), carrying its C citations. The `cmd_*.rs` files are catalogued in Appendix A and omitted here except as a group.

Entry points and process model

- **main.rs** — the binary entry; installs the libc-shim global allocator and calls `tmux_main`.
- **tmux.rs** — argument handling, socket-label construction (`make_label`, the `ztmux-<uid>` path), and the client-vs-server decision.
- **proc.rs** — the `tmuxproc/tmuxpeer` process and peer abstraction over `msg`; the event dispatch (`proc_loop`) both server and client run on.
- **tmux_protocol_h.rs** — `PROTOCOL_VERSION` and the `MSG_*` message set (Chapter 17).
- **extensions/event_loop/** — the event loop itself (Chapter 3): `base.rs` for registration, dispatch, timers and signals; `backend.rs` for the readiness call (`select` on macOS, `poll` elsewhere); `buffer.rs` for `evbuffer`; `bufev.rs` for `bufferevent`. It sits under `extensions/` because it mirrors no tmux C file — there is no `vendor/tmux/event.c`, only calls into a library that is now ours.

The server

- **server.rs** — the event loop (`server_loop`), the socket (`server_create_socket`), the accept path (`server_accept`), the global state trees, and the lifecycle (`server_start`, signal handling).
- **server_client.rs** — per-client servicing (`server_client_loop`), client creation, and the client-side of key and redraw handling.
- **server_fn.rs** — server helper functions (marked pane, message log, redraw requests).
- **server_acl.rs** — the socket access-control list (`server-access`).
- **client.rs** — the client half: connect, autostart, the identify handshake, the dispatch table (`client_dispatch_wait/_attached`), exit reasons.
- **control.rs**, **control_notify.rs** — control mode (`-CC`) and its notifications.

Grid, screen, and drawing

- **grid.rs** — the cell grid: creation, line growth, packed/extended cells, scrollbar (`grid_scroll_history`, `grid_collect_history`), reflow, string extraction.
- **grid_view.rs** — the coordinate-translated view used for writing into a region.
- **grid_reader.rs** — the copy-mode motion primitives (Chapter 18).
- **screen.rs** — the stateful view over a grid: cursor, scroll region, modes, tabs, selection, alternate screen, resize/reflow.
- **screen_write.rs** — the mutation API the input parser drives (`screen_write_*`), with the batched collection fast path.

- **screen_redraw.rs** — compositing panes and borders into a client (Chapter 20).
- **attributes.rs** — attribute-name ↔ bitset conversion (**attributes_tostring/ _fromstring**).
- **colour.rs** — color pack/unpack across the 16/256/RGB tag scheme.
- **style.rs** — parsing and applying **#[...]** style strings.
- **format_draw.rs** — rendering an expanded status string with its style ranges into cells.

Input and terminal

- **input.rs** — the VT parser state machine (Chapter 6): states, transition tables, CSI/OSC/ESC/DCS dispatch, SGR.
- **input_keys.rs** — encoding tmux key codes into the byte sequences sent to panes.
- **tty.rs** — the output back end: differential redraw, flow control, cursor/attribute optimization, the output buffer.
- **tty_term.rs** — terminfo capability lookup and overrides.
- **tty_keys.rs** — decoding the outside terminal's keyboard bytes into key codes.
- **tty_features.rs** — the runtime feature registry (truecolor, synchronized output, ...).
- **tty_acs.rs** — the alternate-character-set (line-drawing) mapping.

Layout and windows

- **layout.rs** — the pane-tiling tree, splits, resize, navigation.
- **layout_set.rs** — the five preset layouts and their cycling.
- **layout_custom.rs** — the layout string: dump, parse, checksum.
- **window.rs** — the window object, panes, zoom (**window_zoom/window_unzoom**).
- **session.rs** — the session object and its window list/stack.
- **resize.rs** — whole-window and client resize orchestration.
- **spawn.rs** — creating windows/panes, the pty, and the fork/exec path (Chapter 19).
- **names.rs** — automatic window naming.

The command language

- **cmd.rs** — the **cmd_entry** model, **CMD_TABLE**, command lookup and aliases.
- **cmd_parse.lalrpop / cmd_parse.rs** — the grammar and the generated parser plus the hand-written lexer.
- **arguments.rs** — argument-spec interpretation (**args_has/_get/_count**).
- **cmd_find.rs** — target resolution (**session:window.pane, {marked}**, defaults).
- **cmd_queue.rs** — the command queue, per-item state, and asynchronous continuations.
- ****cmd_*.rs**** — one file per command, mirroring **cmd-*.c** (Appendix A).

Configuration, formats, keys

- **format.rs** — the **#[...]** engine (Chapters 9 and 16); the single largest module.
- **regsub.rs** — regex substitution behind the **s** modifier.
- **options.rs / options_table.rs** — the scoped, typed options system and its default table (Appendix C).
- **key_bindings.rs** — key tables and bindings (add/remove/lookup, reference counting).
- **key_string.rs** — parsing key names (**C-b, M-left**) into key codes.
- **keyc_mouse_key.rs** — mouse key-code mapping.
- **cfg.rs** — loading config files (**start_cfg, load_cfg**) and collecting causes.
- **environ.rs** — the session/global environment and **update-environment**.
- **notify.rs, alerts.rs** — hooks/notifications and the activity/bell/silence alerts.

Copy mode, modes, and buffers

- **window_copy.rs** — copy mode (Chapter 18): motions, selection, search, extraction.
- **window_clock.rs** — clock mode.
- **window_buffer.rs**, **window_client.rs**, **window_tree.rs**, **window_customize.rs** — the interactive choosers and customizer (**choose-***, **customize-mode**).
- **mode_tree.rs** — the shared tree/list UI the choosers are built on.
- **menu.rs**, **popup.rs** — **display-menu** and **display-popup** overlays.
- **paste.rs** — paste buffers.
- **status.rs** — the status line, messages, and the command prompt (Chapter 20).

Utilities and platform

- **utf8.rs**, **utf8_combined.rs** — UTF-8 handling and combining characters.
- **hyperlinks.rs** — OSC 8 hyperlink storage.
- **image.rs**, **image_sixel.rs** — sixel graphics (the one fully-implemented “extra”).
- **job.rs** — background jobs behind **#{shell}** formats and **run-shell**.
- **file.rs** — file I/O over the socket (Chapter 17’s read/write messages).
- **bitstr.rs** — the bit-string used for tab stops.
- **xmalloc.rs** — the checked-allocation wrappers (**xmalloc**, **xstrdup**, ...).
- **libc.rs**, **osdep.rs**, **compat/** — libc bindings and per-OS shims (the macOS **closefrom** fix of Chapter 19 lives in this territory).
- **log.rs** — logging, including the tty output log.
- **structured.rs** — structured-logging helpers.
- **utempter.rs** — utmp/wtmp login records where supported.
- **lib.rs** — the shared struct and constant definitions (**grid_cell**, **screen**, **layout_cell**, the flag sets) the rest of the tree is built on.

Appendix G — Glossary

A compact glossary of the terms this book uses in their tmux-specific senses. Where a term names a source construct, the module is noted.

- **Alternate screen** — a saved-and-restored secondary screen a full-screen program switches to so the shell's scrollbar is untouched when it exits (`screen.rs`, `screen_alter_on/_off`).
- **Anti-drift gate** — the CI test that fails the build if a Rust free function has no tmux C counterpart and is not allowlisted (`tests/ported_fn_names_match_c.rs`, Appendix E).
- **Attach** — connecting a client to a session; involves the identify handshake and fd passing (Chapters 4, 17).
- **Cell** — one character position in the grid, with its character, attributes, and colors; stored *packed* or *extended* (`grid.rs`, Chapter 5).
- **Client** — the thin half of the binary that lends its terminal to the server and relays keys (`client.rs`, Chapter 4).
- **Command queue** — the ordered stream through which every state mutation passes (`cmd_queue.rs`, Chapter 8).
- **Copy mode** — a mode that turns a pane into a navigable, searchable, extractable view of its grid and scrollbar (`window_copy.rs`, Chapter 18).
- **Detach** — ending a client's terminal loan while the session keeps running (Chapter 4).
- **Extended cell** — a cell too rich for the packed form (wide/combining glyph, RGB color, non-default underscore color, hyperlink), stored via a `grid_extd_entry`.
- **Fake fn** — an invented Rust helper with no C counterpart that would inflate apparent completeness; caught by the anti-drift gate (Chapter 12).
- **Format** — a `#{...}` expression evaluated against server state; the configuration and introspection language (`format.rs`, Chapters 9, 16).
- **Grid** — the two-dimensional cell array plus scrollbar that backs a pane (`grid.rs`).
- **Grid reader** — the virtual cursor and its motions over stored grid lines (`grid_reader.rs`).
- **msg** — the OpenBSD message-passing library tmux uses to frame socket messages and pass file descriptors (Chapter 17).
- **Identify handshake** — the burst of `MSG_IDENTIFY_*` messages by which a client describes its terminal to the server on attach.
- **Layout cell** — a node in the pane-tiling tree: a left-right split, a top-bottom split, or a leaf window pane (`layout.rs`).
- **Layout string** — the checksummed serialization of a layout, printed by `{window_layout}` (`layout_custom.rs`).
- **Marked pane** — the server-global pane set by `select-pane -m`, used as an implicit source by `join-pane/swap-pane`.
- **Mode** — a display/key state pushed onto a pane (copy, clock, chooser) that takes over until popped (`mode_tree.rs`).
- **Option** — a typed, scoped configuration value (server/session/window/pane) (`options.rs`, Appendix C).
- **Packed cell** — the compact five-byte form for the common cell (ASCII, palette color, low attributes).
- **Pane** — a single pty running a program, backed by a screen and grid; the leaf of the state tree.
- **Parity suite** — the runner that diffs ztmux against the vendored tmux byte-for-byte; the operational definition of correct (`parity/`, Chapter 11, Appendix D).
- **Prefix key** — the key (default `C-b`) that switches the active key table to `prefix` so the next key is a tmux binding.
- **Reflow** — re-wrapping grid lines to a new width on resize (`grid_reflow_*`).

- **Screen** — the stateful view over a grid (cursor, scroll region, modes) (`screen.rs`).
- **Screen-write** — the mutation API (`screen_write_*`) the input parser drives; the mechanism half of the parser's policy/mechanism split.
- **Scrollbar** — the bounded history of past lines above the visible screen, trimmed by `grid_collect_history`.
- **Server** — the long-lived daemon that owns all state and outlives every client (`server.rs`, Chapter 3).
- **Session** — a collection of windows a client attaches to; the top of the state tree.
- **Source of truth** — the vendored `vendor/tmux/ C`, the reference every module is validated against.
- **Superset** — the goal of being a strict extension of tmux: everything tmux does, exactly, plus additive features (Chapter 13).
- **Target** — the `-t session:window.pane` (or `{marked}`, `-`, relative) argument resolved by `cmd_find.rs`.
- **terminfo** — the terminal-capability database the tty back end consults to emit the right escape sequences (`tty_term.rs`).
- **Window** — a collection of panes tiled by a layout, with one active pane; a member of a session.
- **Zoom** — temporarily making one pane fill the window without touching the layout tree (`window_zoom/window_unzoom`).
- **Activity / bell / silence** — the three background-window alerts, gated by `monitor-activity/monitor-bell/monitor-silence` and delivered per `*-action / visual-*` options (`alerts.rs`, Chapter 21).
- **Control mode** — a text, line-based protocol (`-CC`) by which a program drives the server and mirrors its state from `%`-prefixed notifications (`control.rs`, Chapter 17).
- **Differential redraw** — repainting only the cells that changed since the last frame, minimized further by `tty_cursor/tty_attributes` (Chapter 10).
- **Flow control** — throttling a pane whose output floods faster than the terminal can consume, via `TTY_BLOCK_START/TTY_BLOCK_INTERVAL` (Chapter 23).
- **Hook** — a command list bound to run when a named event fires; the reactive counterpart of a key binding (`notify.rs`, Chapter 21).
- **Key code** — the internal representation of a key plus its modifiers and flags, produced by `tty_keys.rs / key_string.rs`.
- **MSG_*** — the `imsg` message types the client and server exchange over the socket (Chapter 17).
- **Notification** — the server's internal "something changed" broadcast, consumed by hooks and control-mode clients (`notify.rs`, Chapter 21).
- **Pending wrap** — the deferred-wrap state that keeps the cursor on the last column after a full-width write until the next character arrives (Chapter 6).
- **Semantic prompt** — OSC 133 markers a shell emits to delimit prompts and command output, recorded as grid line flags (Chapter 6).

Appendix H – Building ztmux and the Vended References

This appendix collects the build-side facts a reader who wants to compile, test, or audit the port needs: the dependencies, the feature flags, how the C libraries are linked, and how the vended reference tree is kept and refreshed.

Dependencies

ztmux's `Cargo.toml` is deliberately lean, because a faithful port of a C program should not smuggle in a parallel stack of Rust libraries that change behavior. The runtime dependencies are:

- **libc** — the raw libc bindings the port calls directly (the custom allocator, the syscalls, the terminal ioctls).
- **bitflags** — the flag-set types (`grid_attr`, `format_modifiers`, and the rest) that mirror the C's `#define` flag constants.
- **lalrpop-util** — the runtime half of the LALRPOP parser generator (Chapter 8); the build-dependency **lalrpop** compiles `cmd_parse.lalrpop` to `cmd_parse.rs`.
- **memchr** — fast byte scanning used in the parser and format hot paths.
- **num_enum** — deriving the integer $\leftrightarrow$ enum conversions the ported tagged unions need.
- **paste** — token pasting for the macro-generated code.
- **terminfo-lean** — the one non-libc, non-parser dependency doing real domain work: it reads the terminfo database for the tty back end (Chapter 10).

That is the whole list, and it is entirely Rust: since the event loop was reimplemented (Chapter 3) the crate links no C library beyond libc, so there is nothing to `pkg-config`, no `-dev` package to install, and no Homebrew prefix to locate. There is no async runtime (the loop is libevent's model, written out; see Chapter 29 for why not an async runtime), no Rust terminal-emulation crate (the port *is* the emulator), and no UI framework. The dependency list is short on purpose: every library that is not libc, the parser generator, or the terminfo reader is a potential source of behavioral divergence from tmux, and the port keeps that surface minimal.

Feature flags

The `[features]` block exists mostly, as the manifest itself notes, “to make matching upstream’s configuration options easier” — they let a `.tmux.conf` that references a build-time capability parse without error. The flags are `cgroups`, `hyperlinks`, `iutf8`, `ncurses`, `nokerninfo`, `sixel`, `sys_signame`, `systemd`, `utempter`, and `utf8proc`, with `default = ["sixel"]`. The important exception the book has stressed is `sixel`: it is not just an accepted flag, it is genuinely implemented (`src/ported/image_sixel.rs`), and its presence changes an observable behavior — the Device Attributes reply advertises `sixel (\x1b[?1;2;4c` instead of `\x1b[?1;2c`, Chapter 6). The remaining flags are the honest “configuration surface exists, behavior is being wired” story of Chapter 15 in build form.

No C libraries to link

There are none. `build.rs` is eight lines: it generates the command grammar from `cmd_parse.lalrpop` and stops. A build needs a Rust toolchain and nothing else — no `pkg-config` probe, no `libevent-dev` or `libevent-devel`, no `brew install libevent`, no search paths to point at a cross-built static library. `otool -L / ldd` on the finished binary shows the system C library and nothing more.

This is recent, and the shape of what it replaced is worth recording. `build.rs` used to be a hundred and fifty lines of link policy: locate the Homebrew libevent prefix on macOS and link `event_core` statically, ask `pkg-config` for it elsewhere and link dynamically, fall back to a bare `-levent_core`, and — when neither worked — fail the build with a per-distribution install instruction, because the alternative was several hundred lines of linker error that named no package. The release pipeline carried the same weight from the other end: to ship an x86_64 mac binary from an arm64 runner it downloaded a libevent release tarball, configured and built it static for the cross target, and pointed `RUSTFLAGS` at the result; the Homebrew formula declared `depends_on "libevent"`. All of that is gone with the C dependency (Chapter 3).

Two feature flags survive it. `static` and `dynamic` used to choose how `event_core` was linked and are kept only so existing invocations keep working; with no C library in the picture, neither has any effect.

The vendored reference

Under `vendor/` sits one complete, plain-copied, SHA-pinned source tree, documented in `vendor/VENDOR.md`:

- `vendor/tmux/` — the upstream tmux **C source**, the `next-3.7` development line, and the *source of truth*. It is what every ported module is diffed against, what the parity runner builds and runs as its reference, and what the anti-drift gate indexes. Pinning to a specific upstream commit rather than “whatever tmux the system has” is essential because tmux’s behavior drifts between versions. It is read-only by convention: nothing is developed in it, and it is never patched to make a port pass.

It carries its upstream ISC license and its pin metadata. The refresh procedure, per `VENDOR.md`, is mechanical: export a fresh `git archive` of the upstream with no `.git`, rsync it over the vendored path, and record the new `git rev-parse HEAD`. After a `vendor/tmux/` refresh, re-running `scripts/annotate_c_links.py` updates every one of the two-thousand-plus C-signature citations to the new line numbers and signatures, and the parity suite re-measures against the new reference — so a version bump is a controlled, re-verifiable operation rather than a leap of faith.

Appendix I – The Default Key Bindings

tmux ships a default `prefix` key table so that a fresh install is usable before any `.tmux.conf`, and ztmux builds the identical table at server start in `key_bindings_init` (`src/ported/key_bindings.rs`), each binding added by `key_bindings_add` with the same key, note, and command list as tmux. This appendix lists the notable `prefix`-table bindings (the ones reached by pressing the prefix, default `C-b`, then the key). Each carries a `-N` note in the source, quoted here as its description. This is not the whole table, but it is the working set a user meets daily, and it is byte-for-byte tmux's.

Prefix table – windows and sessions

- **c** — create a new window (`new-window`).
- **&** — kill the current window, with confirmation (`confirm-before ... kill-window`).
- **,** — rename the current window (`command-prompt ... rename-window`).
- **.** — move the current window to a new index (`command-prompt ... move-window`).
- **n / p** — next / previous window (`next-window / previous-window`).
- **l** — last (previously current) window (`last-window`).
- **0–9** — select window by index (`select-window -t:=N`).
- **'** — prompt for a window index to select.
- **w** — choose a window from the tree chooser (`choose-tree`).
- **f** — find a window by name/content (`command-prompt ... find-window`).
- **\$** — rename the current session (`command-prompt ... rename-session`).
- **s** — choose a session from a list (`choose-tree -Zs`).
- **d** — detach the current client (`detach-client`).
- **D** — choose and detach a client (`choose-client -Z`).
- **(/)** — previous / next client (`switch-client -p / -n`).
- **L** — switch to the last client (`switch-client -l`).

Prefix table – panes

- **%** — split the window horizontally (`split-window -h`).
- **"** — split the window vertically (`split-window -v`).
- **o** — select the next pane (`select-pane -t:.`).
- **C-o** — rotate through the panes (`rotate-window`).
- **;** — move to the previously active pane (`last-pane`).
- **!** — break the pane into a new window (`break-pane`).
- **m / M** — toggle / clear the marked pane (`select-pane -m / -M`).
- **x** — kill the current pane, with confirmation.
- **z** — zoom / unzoom the current pane (`resize-pane -Z`).
- **q** — display pane numbers for selection (`display-panes`).
- **{ / }** — swap the pane with the previous / next (`swap-pane`).
- **Space** — cycle to the next preset layout (`next-layout`).
- **E** — spread panes out evenly (`select-layout -E`).
- **Arrow keys** — select the pane in a direction (`select-pane -L/-R/-U/-D`).
- **C-Arrow / M-Arrow** — resize the current pane (repeatable).

Prefix table — copy, buffers, and misc

- **[** — enter copy mode (`copy-mode`).
- **]** — paste the most recent buffer (`paste-buffer -p`).
- **#** — list all paste buffers (`list-buffers`).
- **=** — choose a paste buffer from a list (`choose-buffer -Z`).
- **-** — delete the most recent paste buffer (`delete-buffer`).
- **:** — open the command prompt (`command-prompt`).
- **?** — list key bindings (`list-keys -N`).
- **/** — describe a key binding (`command-prompt -kpkey { list-keys ... }`).
- **i** — display window information (`display-message`).
- **r** — redraw the current client (`refresh-client`).
- **C** — customize options interactively (`customize-mode -Z`).
- **t** — display the big clock (`clock-mode`).
- **~** — show the server message log (`show-messages`).
- **C-z** — suspend the current client (`suspend-client`).
- **C-b** — send the prefix key to the pane (`send-prefix`).

The `root` table (bindings that fire without the prefix) carries the mouse bindings and any user additions; the `copy-mode` and `copy-mode-vi` tables carry the navigation and selection keys of Chapter 18. Because all four are built by the same `key_bindings_add` calls as `tmux` and looked up the same way, a user's muscle memory transfers unchanged — which is exactly the compatibility the superset promise (Chapter 13) rests on.

Appendix J — Format Recipes

The best way to internalize the format mini-language (Chapters 9 and 16) is to read real compositions. This appendix works through a few, building from the operators the parity suite pins. Each can be tested directly with `display-message -p`.

A status-right with conditional prefix highlight

```
#{?client_prefix,#[reverse] PREFIX #[default],}%H:%M
```

`#{?client_prefix,THEN,ELSE}` tests whether the prefix key is currently pending. When it is, the `THEN` branch emits a reverse-video `PREFIX` badge (using the `#[reverse]` / `#[default]` style directives that `format_draw.rs` renders); when it is not, the `ELSE` branch is empty. Either way it is followed by `%H:%M`, a strftime time (evaluated only when time expansion is enabled — hence excluded from parity). The recipe shows the two most common building blocks together: a boolean-tested conditional and inline style directives.

A window-status entry with a zoom flag

```
#{window_index}:#{window_name}#{?window_zoomed_flag,Z,}
```

Each window's status entry shows its index and name, and appends a `Z` only when a pane in that window is zoomed — again a `#{?...Z,}` conditional over a flag variable. Set this as `window-status-format` and every window in the middle of the status line gains a zoom indicator with no scripting.

Truncating a long title to fit

```
#{=/20/...:pane_title}
```

The `#{=/N/marker:string}` form limits `pane_title` to twenty columns and, if it had to truncate, appends the marker `....`. The plain `#{=20:pane_title}` truncates silently; the `/.../` variant tells the reader the title was cut. This is how status lines keep a fixed width regardless of how long a program sets its title.

Counting panes and branching on it

```
#{?#{==:#{window_panes},1},[solo],#{window_panes} panes}
```

Reading inside out: `#{window_panes}` is the pane count; `#{==:#{window_panes},1}` is a comparison that yields `1` when there is exactly one pane; and the outer `#{?...}` prints `[solo]` in that case, otherwise the count followed by `panes`. It composes three operators — a variable, a comparison, and a conditional — and every layer is re-expanded by the same engine, which is why arbitrary nesting works.

A per-window loop in a summary

```
#{W:#{window_index}#{?window_active,*, } }
```

The `#{W:fmt}` loop expands `fmt` once per window in the current session. Here `fmt` prints each window's index followed by `*` for the active window and a space otherwise, then a trailing space — producing a compact `0 1* 2` summary of the session's windows. It is the loop modifier of Chapter 16 in a single line. (Note that the capital-letter loop's injected `loop_index/loop_last_flag` are a current parity gap, Appendix D; the per-object variables used here — `window_index`, `window_active` — work.)

Arithmetic in a format

```
used #{e|/|:#{history_size},1000}k of #{e|/|:#{history_limit},1000}k lines
```

Two `#{e|/|:a,b}` divisions turn the raw scrollbar line counts into rounded thousands, so a pane with 5,300 lines of a 20,000-line limit reads `used 5k of 20k lines`. Arithmetic formats are how a status line reports computed quantities — memory, ratios, elapsed counts — without an external shell call, and because `#{e}` is deterministic it is heavily represented in the parity suite (cases 010–017, 200–215).

Appendix K — The Audit Workflow

The book’s central claim is that ztmux’s faithfulness is *checkable*, not merely asserted. This appendix is the how-to: the concrete steps a reviewer takes to verify a claim, using the machinery Chapters 2, 11, and 12 built. None of it requires trusting the author.

Verifying one function against the C

Pick any Rust function — say `grid_scroll_history` in `src/ported/grid.rs`. Read its citation, the doc comment directly above it: `/// C `vendor/tmux/grid.c:<line>`: `<signature>``. Open `vendor/tmux/grid.c` at that line and you are looking at the exact C the Rust claims to implement. Compare the two by eye: the parameters (names and order should match the cited signature), the control flow (loops that increment on `continue`, early returns, the order of side effects), and the constants. Because both trees are committed in the repo, this is a local, offline operation — no fetching, no version guessing. If the Rust diverges from the C in a way the parity suite does not yet cover, you have found either a bug or a missing case.

Checking the whole tree’s structural honesty

Run the anti-drift gate: `cargo test --test ported_fn_names_match_c`. It collects every Rust free function, indexes every C function in `vendor/tmux/`, and fails if any Rust name has no C counterpart and no allowlist entry — printing `FAKE-FN <name> (<file>:<line>)` for each violation (Appendix E). A green run means every function in the tree corresponds to something real in tmux or is a documented exception. Then run `python3 scripts/annotate_c_links.py --check`: it exits non-zero if any citation is missing or stale, so a green run means every ported function’s C citation is present and current.

Measuring behavioral fidelity

Run the parity suite: `bash parity/run_parity.sh`. It builds the vendored tmux reference and ztmux, runs all 1115 cases through both, and reports per-case OK/FAIL and totals. For any FAIL, `parity/parity_failures.log` holds both outputs and a unified diff, so the divergence is the exact bytes. To investigate a single case by hand, expand its format string in both binaries — `vendor/tmux/tmux display-message -p '<fmt>'` versus `ztmux display-message -p '<fmt>'` — and compare. The case is the reproduction (Chapter 11), so debugging starts from a minimal input.

Reading the port’s coverage

To see how much of tmux has a ported counterpart, read the generated `port_report.html` (from `scripts/gen_port_report.py`), which matches Rust functions against the vendored C using the same index the annotation script builds. Because it is derived from the committed C rather than from a hand-maintained checklist, it cannot flatter the port: refresh the vendored tmux, re-run the script, and the report reflects reality. Read it next to the anti-drift gate, which rules out the opposite failure — surface that is present in the Rust but came from no upstream at all — and the two together give the completeness picture: coverage (the report) and integrity (the gate).

The whole loop

Put together, the audit is: read a citation and diff one function against the C; run the anti-drift gate for structural honesty; run the citation `--check` for interface freshness; run the parity suite for behavioral fidelity; and read the port report for coverage. Every step consults a committed reference and produces an artifact, so the conclusion does not rest on the author's word at any point. That is what "faithfulness you can check" means operationally — a fixed sequence of commands, each with a pass/fail or a diff, that a skeptic can run start to finish.

Appendix L – Coexisting with tmux and Operational Notes

ztmux is designed to run alongside a real tmux without collision, and to be diagnosable when something goes wrong. This appendix collects the operational facts that do not belong to any one subsystem chapter.

The socket namespace

ztmux opens its own socket namespace — `ztmux-<uid>` rather than tmux's `tmux-<uid>` — built by `make_label` in `src/ported/tmux.rs`. The default socket is `<dir>/ztmux-<uid>/default`, with the directory created `S_IRWXU` and subject to the same owner-and-permission safety checks tmux performs (the directory must be owned by the caller and carry no world-accessible bits). The `ztmux-` prefix is the one place the path deliberately differs from tmux, and it is what lets a running tmux and a running ztmux coexist on the same machine: two servers, two socket namespaces, two sets of sessions, neither aware of the other. You can attach to your tmux sessions and your ztmux sessions independently from the same shell. A custom socket is selected with `-L <label>` (a named socket in the same directory) or `-S <path>` (an explicit path), exactly as in tmux.

Diagnostics: the panic file and the output log

Two diagnostic aids are worth knowing. First, the server installs a Rust panic hook that, on any panic in the daemon, dumps a backtrace to `server-panic-<pid>.txt` — a small original touch that turns a Rust panic in a detached daemon (which has no terminal to print to) into a diagnosable artifact rather than a silent death. If a ztmux server ever vanishes unexpectedly, that file is where to look. Second, the tty layer can mirror the exact byte stream it sends to a client into an output log (`tty_create_log`), a faithful port of tmux's own debugging aid, so a developer can capture precisely what escape sequences the server emitted — invaluable when a terminal renders something wrong and the question is whether ztmux sent the wrong bytes or the terminal interpreted the right ones wrongly.

Signals the server honors

The server's `server_signal` handler wires the operational signals faithfully (Chapter 3): `SIGINT/SIGTERM` begin an orderly shutdown (kill jobs, save prompt history, exit); `SIGCHLD` reaps pane children; `SIGUSR1` recreates the socket, so a moved or deleted socket can be restored without losing the server; and `SIGUSR2` toggles logging at runtime. Knowing these lets an operator recover a server whose socket file was accidentally removed (`kill -USR1`) or turn on logging for a misbehaving server (`kill -USR2`) without restarting it.

The environment a pane sees

A pane's environment is assembled, not inherited raw (Chapter 19). The `update-environment` option names the variables refreshed from the attaching client into the session environment on each attach — `DISPLAY`, `SSH_AUTH_SOCK`, and the rest — so a program launched in a new window after you re-attach over a fresh

SSH connection sees the *new* connection's values, not the stale ones the daemon was started with. This is why agent forwarding and X display forwarding "just work" across re-attaches, and it is a faithful port of behavior tmux users rely on without thinking about.

Appendix M – A Reading Guide to the Source

For a reader who wants to move from this book into the code, this appendix suggests an order, because two hundred kilobytes of `format.rs` is not where to start. The tree rewards being read in roughly the book's order, with a few practical entry points.

Where to start

Begin with `src/lib.rs`, which holds the shared struct and constant definitions — `grid_cell`, `screen`, `layout_cell`, the flag sets — that every other module is built on. Reading the data model first makes the code that operates on it legible. Then read `src/ported/tmux.rs` and `src/main.rs` for the entry points and the client-versus-server decision, and `src/ported/server.rs` for the event loop that everything hangs off.

Following a subsystem

To understand one subsystem deeply, read its module next to its C origin: open `src/ported/grid.rs` and `vendor/tmux/grid.c` side by side, and use the citations to jump between them. The one-to-one function correspondence means you can read the C for the algorithm and the Rust for the port, and the citation on each function tells you exactly which lines to compare. The `cmd_*.rs` files are the easiest to read in isolation — each is a single command, small and self-contained, mirroring its `cmd-*.c`.

Using the tooling as a map

The generated `port_report.html` is a map of what is ported and what is pending, per function. `git log --stat -- src/ported` shows which modules are moving, which is a good way to find the most actively-worked code. And the parity cases in `parity/cases/` are, collectively, a specification-by-example of the format language and the command surface — reading them is a fast way to learn what behavior is pinned and how the format operators actually compose. Between the citations, the report, the history, and the cases, the source is unusually navigable for a program of its size — which is, after all, the whole point of a port that sells itself on being checkable.

Colophon

This book was written alongside the port it describes, and it is generated from one source. `docs/book.md` is the manuscript; `scripts/book_pdf.sh` renders it to PDF through pandoc and LuaLaTeX with the project's own theme, and `scripts/book_epub.sh` renders the same manuscript — plus this apparatus — to a reflowable EPUB 3. The cover is not hand-placed either: `scripts/book_cover.sh` draws it at 1600×2560 from the same palette the PDF theme uses, so the cover on a storefront and the chapter headings inside the book cannot drift apart.

The display faces are Orbitron and Share Tech Mono; body text is set in Arimo. All three are licensed under the SIL Open Font License 1.1.

Every figure in these pages — parity case counts, module counts, line counts, version numbers — was taken from the source tree at this edition by running the tool that produces it, and is cited to that tool or file so a reader can check it rather than trust it. Where a number could not be verified at the time of writing, the text says so instead of estimating.

Reproducing the Book's Claims

The book is written to be checkable. With the repository cloned, these are the commands behind its central claims.

The port's correspondence to the C, function by function:

```
cargo test --test ported_fn_names_match_c
```

The parity suite — the same inputs through the real tmux and through ztmux, compared byte for byte:

```
bash parity/run_parity.sh --summary
```

The size of the superset tree, which is the part of the source that is deliberately *not* a port:

```
ls src/extensions/*.rs | wc -l
```

The build itself, and the reference tmux the suite measures against, are described in Appendix H.

Also from MenkeTechnologies

ztmux is one program in a stack that is documented the same way — from source, with the numbers cited to the tools that produce them. Companion volumes cover the shell, the terminal emulator, the bytecode VM the language frontends share, and the language ports built on it.

Titles and availability change; the current catalog is published at github.com/MenkeTechnologies.